PRODUCT PROPOSAL

Humanvoice

making expert technical documents easier to understand

Clearer technical documents, with the author in control

26 August 2026

Decision requested

Fund a bounded twelve-week MVP, a small adjudicated evaluation corpus, and one live-document feasibility run. The investment tests whether Humanvoice can turn a complete authoring brief into a checked, review-ready document while withholding the reader packet when the argument, evidence, or protected meaning is unresolved. It does not fund deployment or an efficacy claim.

The proposition

Humanvoice helps an author move from a reader brief to a technical document that has passed pre-human integrity, argument, writing, and reconstruction checks. The named expert reader still makes the final decision, but is not asked to discover ordinary defects that the system could have caught first.

A case-led proposal for safer technical revision

Abstract

Technical documents often fail in a costly way: they compile, contain correct work, and pass automated review, yet leave an expert reader to reconstruct the problem, mechanism, evidence, and requested action. The next revision then repeats diagnosis, editing, and rereading. Humanvoice is a proposed local-first authoring and revision system for this gap. It compiles a reader brief into an argument blueprint, drafts in bounded units, runs fail-closed pre-human checks, protects equations, numbers, citations, and claims while an author revises, and releases a packet for a named human reader only after those checks pass.

The literature and software review are part of the product argument. Existing studies support bounded assistance, localized feedback, protected comparison, and direct reader evaluation; they do not establish that this workflow improves expert technical writing. Mature software can supply parsing, linting, comparison, citation identity, and local execution, but no validated package connects those functions to reader acceptance. The requested build is therefore an investment in learning: it will produce a working path from brief to release, one measured feasibility case, and a priced recommendation to proceed, revise, or stop. It will not claim that an automated critic can certify taste or that one case proves efficacy.

Contents

List of Figures

List of Tables

Five complaints that changed the project

On 18 August 2026, a technically strong survey failed its first encounter with a reader. The source ran to 2,326 lines. It reconstructed the mathematics of Hamiltonian Monte Carlo for models with a zero lower bound, cited the relevant papers, and documented the implementation in unusual detail. A mechanical conversion had also made a mess of the typesetting. The reader's first verdict was blunt: the mathematics did not look like mathematics.

Repairing the conversion did not solve the problem. It merely exposed the next one. The survey read like an internal audit: file paths stood in for explanations, inspection dates stood in for scope, and project labels appeared before the objects they named. Once that register was removed, the reader noticed the repeated sentence and section shapes. Once the repetition was broken, defensive qualifications became conspicuous. Once the author replaced those qualifications with reasons, the reader found the important omission: the survey explained how to validate an algorithm but supplied no reference models on which to do it.

Five rounds of feedback produced five different diagnoses. Figure 1 shows their order. The order matters because each defect had hidden the next one.

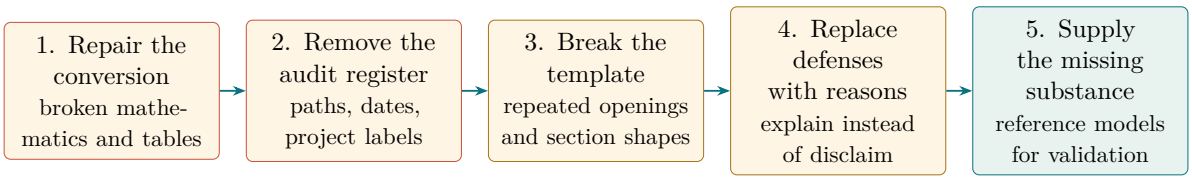

Figure 1: The five reader complaints in the ZLB rescue. The sequence is a recorded case history, not a universal law or an efficacy estimate.

The final manuscript ran to 49 pages, with 109 equations and 55 references. The owner accepted it after the fifth repair. Throughout the rewrite, the equation bodies remained byte-identical, labels and citations were counted, and the rendered PDF was checked for the internal language that source-level searches could miss. This is a useful engineering record. It is not an efficacy study: there was one document, one owner, no control group, and no independent reader panel.

The product hidden inside the rescue

The rescue worked because four kinds of work were joined. A check located an observable problem. An author decided what the passage needed to mean. A comparison protected

the equations, numbers, citations, and claims while the prose changed. A reader then judged the rendered document without relying on the repository history. Existing tools perform pieces of this sequence, but the pieces do not share a record or end in a reader outcome.

Humanvoice is the proposed connective layer. It would sit beside the author's editor and repository, not replace them. It would turn a reader brief into an argument blueprint, draft in bounded units, and run a fail-closed pre-human check before it turns a version into a reader packet. A broad complaint such as "this is hard to read" would become a small set of located questions; the objects that carry technical meaning would remain protected; and the author's decision would stay visible. It would never certify that prose is "human," infer authorship from one document, silently rewrite a source, or ask an expensive reader to serve as its ordinary debugger.

The wager is economic as much as editorial. A locally fluent document can still consume several rounds of expert diagnosis and rereading. Humanvoice is worth building only if it reduces that burden without increasing reader time, false alarms, or meaning errors. A handsome score from a model is not the outcome. A reader who can recover the problem, mechanism, evidence, qualification, and requested action is.

The proposition under test

The first investment is deliberately bounded: twelve weeks, a product engineer, a research and evaluation lead, a domain editor, a part-time security or policy owner, independent coding and reader review, a small adjudicated corpus, and one live-document feasibility run. Weeks 1–6 deliver the protected LaTeX core and its second-operator replay. The authoring extension, model critics, and repair loop proceed only if that checkpoint passes; otherwise the sponsor receives the protected review utility rather than an unfinished platform.

At the end of twelve weeks the sponsor should be able to answer three questions:

1. Can the system locate useful editorial questions and protect meaning-bearing objects on a real manuscript?
2. Can authors and readers use the complete path without the review burden exceeding the burden it is meant to remove?
3. Is a comparative study large enough to estimate an effect worth its expected cost?

One live case can answer the first two as feasibility questions. It cannot show that Humanvoice generally reduces revision rounds. That stronger claim requires new documents, readers who did not tune the rules, a declared comparator, and uncertainty reported around the result.

Recommendation

Fund the bounded build and feasibility run. Do not authorize deployment or an efficacy claim. Continue only if the system protects meaning, keeps the reader

outcome observable, and produces a credible, priced design for the comparison that follows.

Evidence strong enough to use today

The research record makes the design plausible, but it is not complete. Experiments show that generative assistance can save time on some writing and knowledge tasks, while other studies show errors outside a model's capability frontier, uneven gains across workers, convergence in assisted writing, and bias in model-based judging. Research on writing and technical communication explains why readers need visible actors, relations, evidence, and rhetorical purpose. Software surveys identify mature components for parsing, linting, comparison, citation identity, and local execution. None of those findings establishes the effect of the combined Humanvoice workflow on expert technical review.

The current audit passes broad retrieval, bibliographic identity checks, and a bounded specialist import. Its harder challenge queries recover fourteen of eighteen known items and therefore do not pass. Independent screening, full-text appraisal of every load-bearing claim, held-out package benchmarks, and external review also remain unfinished. The manuscript therefore distinguishes source results from product inferences and treats completion of those tasks as work, not as a confidence adjective.

Question	What the present record supports	What twelve weeks must add
Is the failure real?	Several internal documents record correct work that still required repeated reader-led reconstruction.	Baseline burden measured on live documents outside the rule-writing set.
Can the pieces be built?	Mature components exist for most lower-level jobs; the integration is technically plausible.	Held-out fixtures, explicit fallbacks, and one complete local-first run.
Will the workflow help?	Adjacent studies justify the mechanism and the outcomes chosen for measurement.	A later comparative study; one feasibility case cannot estimate efficacy.
Can it be trusted?	Protected comparison, provenance, and named human review give a testable safety model.	Observed false alarms, meaning errors, privacy exceptions, and reader burden.
Is it worth paying for?	Saved expert attention is the relevant benefit.	Local volume, loaded costs, baseline rounds, reader time, and maintenance cost.

Table 1: The proposal separates what is known from what the first build must observe.

How the argument unfolds

Part I starts with the reader's cost and then examines the case record that made the failure visible. Part II asks what the scholarly and software literatures actually establish, including the evidence against tempting shortcuts such as automatic rewriting and document-level AI detection. Part III follows a single teaching case through the proposed product, then states the architecture and the deliberately narrow first build. Part IV specifies the measurement, schedule, economics, adoption path, and stopping rules. The appendices retain the schemas, fixtures, evidence ledger, and calculations needed to reproduce the choices without forcing those records into the main narrative.

The document can therefore be read at two depths. The chapters carry the argument for a funding decision. The appendices let an implementer or reviewer inspect how each consequential claim becomes a record, a test, or an open question. Both belong in this volume because the proposal has to persuade a reader and survive implementation.

Part I

Why technical documents fail

CHAPTER 1

Why good documents still fail

The author sees the repository, the model's history, and the reason each qualification was added. The reader sees a rendered document for the first time. In the failed funding proposal that difference was enough to turn a correct draft into a confusing one. Humanvoice begins with that change in vantage point.

1.1 The first five minutes

An expert reader does not begin by grading sentence style. The reader tries to build a small working model of the document:

1. What decision or problem is this document about?
2. What is the proposed object, mechanism, or intervention?
3. Why is the obvious alternative insufficient?
4. Which evidence would change my view?
5. What action is the author asking me to take?

If those answers are not available, later polish has diminishing value. The reader must spend scarce attention discovering the document's structure before deciding whether the underlying work deserves attention. In the internal case, the first pages named phases, files, and review states before naming the action that the funder was meant to approve. Each local sentence was defensible; the sequence was not.

This is why ordinary readability formulas are insufficient. A short sentence can conceal an undefined mechanism, and a long sentence can be efficient when its actors and quantities are familiar. Readability is a relation between a document, a task, and a reader. It cannot be inferred from a single number.

Figure 1.1 makes the economic problem visible. The cost arises at the missing relation, where the reader must supply work the document should have done.

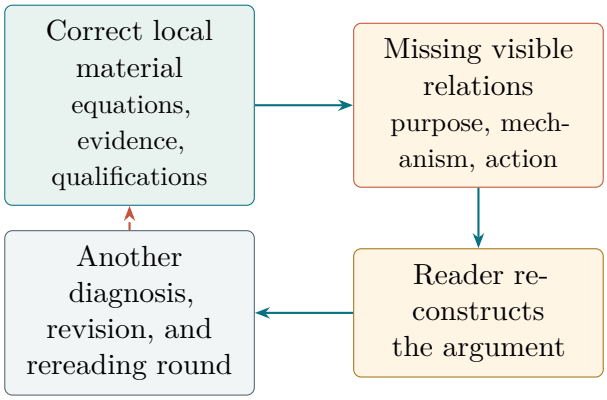

Figure 1.1: How locally correct material can still consume expert attention.

1.2 How a correct draft becomes difficult to read

The internal cases suggest a sequence of failures. The sequence is useful not as a universal taxonomy but as a way to decide where a product intervention can help.

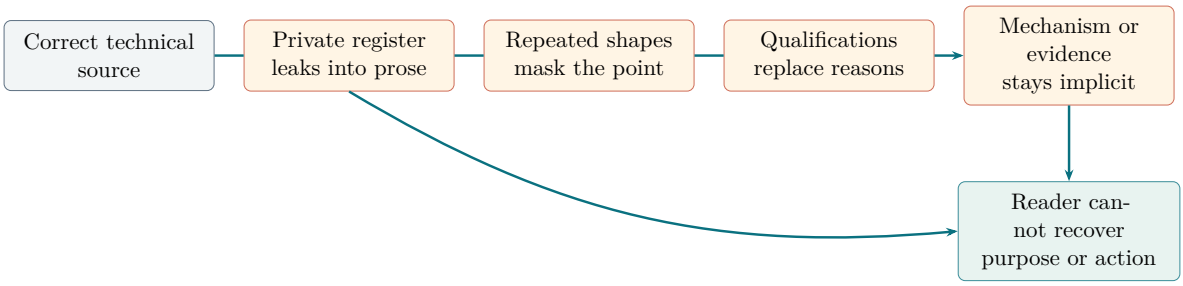

Figure 1.2: A recurring path from local correctness to reader failure. The arrows are diagnostic opportunities, not a claim that every document follows all five steps.

Private register. A project identifier, phase label, file path, or agent status can be meaningful to the authors and opaque to a new reader. The problem is not that the term is technical. The problem is that its referent and importance are unavailable in the document.

Mechanical regularity. A document assembled through repeated templates may open every section in the same way, use the same transition, or close every discussion with the same kind of qualification. These patterns are observable, but their presence is not itself a verdict. They matter when they make the reader predict the form instead of following the argument.

Defensive prose. A sentence can list what the project does not claim, what a tool cannot guarantee, or what a reviewer must not infer without giving the reader the positive mechanism. A boundary is useful when it prevents a plausible mistake. It is a burden when it substitutes for an explanation.

Missing substance. A document may name a result without showing the mechanism connecting evidence to conclusion, or name a feature without showing the user decision it changes. No surface diagnostic can establish that connection by itself.

Reader failure. The final test is whether the intended reader can reconstruct the document's purpose and make the requested judgment. This is not the same as agreement. A reader can understand a proposal and reject it for good reasons. Humanvoice must preserve that distinction.

1.3 Why the incumbent workflow stops too early

Today a linter reports a phrase, a model proposes a rewrite, version control records a file change, and a human reader makes the final judgment. The links between those activities are usually informal: a reviewer pastes a comment into a ticket, an author edits a paragraph, and a later reader is asked to trust that nothing important changed. That is the structural blind spot.

Three costs follow.

First, diagnosis is not located. A comment such as "the argument is hard to follow" may be accurate but gives the author no shared object on which to act. Second, revision is not protected. A fluent rewrite can alter a sign, a denominator, a citation, or the scope of a conclusion without an obvious visual signal. Third, acceptance is not recorded as an outcome. The team remembers that a senior person "looked at it," but cannot compare reader time or the number of rounds across documents.

Humanvoice earns its place only if it reduces those three costs together. A new dashboard that adds another queue of findings makes the problem worse. A protected diff that no author can understand makes safety too expensive. A reader questionnaire detached from the actual decision produces a number without a use.

1.4 The people and the decision

The product has four human roles, even when one person temporarily occupies more than one role.

Author or editor. Owns the claims, evidence, wording, and decision to accept or reject a suggested repair.

Technical owner. Supplies the document's format, protected-object rules, privacy boundary, and integration point with the repository or editor.

Reader. Represents the person whose judgment the document is meant to support. The reader may be a funder, executive, reviewer, or collaborator.

Sponsor. Decides whether the saved attention and reduced risk justify further investment. The sponsor may be the technical owner, but need not be.

The author needs actionable findings and safe comparison. The reader needs a document that does not require privileged project context. The sponsor needs a credible comparison with the incumbent bundle. These are compatible needs, but they are not the same metric.

1.5 A useful product boundary

The product should be judged at the boundary where it changes a real review decision. It is not a general-purpose writing environment, a style-imposition service, or an authorship detector. It is a review path beside the author's existing editor and repository.

The boundary that matters

Humanvoice owns the evidence trail between a source snapshot and a reader decision. The author owns the prose. The reader owns acceptance. The sponsor owns the decision to continue.

This boundary also explains why a local-first design is more than a privacy preference. Technical proposals can contain unpublished results, client data, or decisions that should not be sent to an external model by default. A system that cannot show where a finding came from cannot offer a useful privacy or meaning guarantee either. Source locations, protected spans, and versioned review decisions are the minimum common language between the four roles. The next chapter turns from the boundary to the investment question: what burden would a bounded build have to remove, and what evidence would make the result worth funding?

CHAPTER 2

The cases that made the problem visible

Between May and August 2026, five teams put large technical documents in front of readers. Each reader rejected the first version. One reader could not tell what a funding proposal was trying to achieve; another found a book-length volume accurate but exhausting; a third accepted short repaired units while the complete manuscript still failed. The projects recorded those reactions as carefully as they audited mathematics, leaving us a useful sequence of failures and repairs.

The cases give Humanvoice both a mechanism and a test: before-and-after prose, the measured numbers from the one rescue that ended in acceptance, and process failures a new review path must avoid. The question throughout is practical: what made the document hard to use, what repair changed the reader's experience, and what remains unproved?

The cases did not fail at one level. As Figure 2.1 shows, the visible problem often had to be cleared before the next one could be diagnosed. The chapters below supply the observations behind each layer.

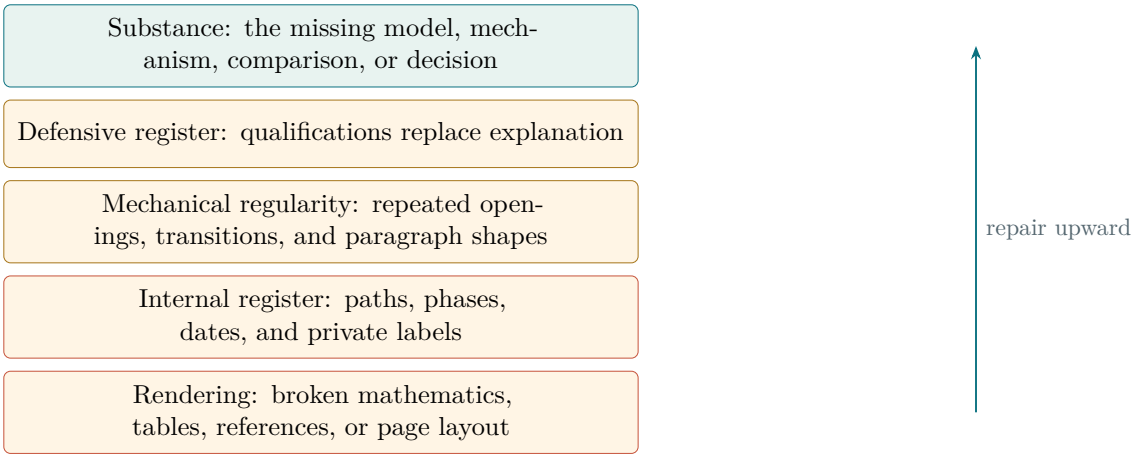

Figure 2.1: The case record suggests an order of inspection: visible rendering failures can conceal register, rhythm, reasoning, and finally substance.

2.1 cardnpv: the failure that started the arc

In June 2026 the agents wrote a funding proposal for a component that values credit-card customers by net present value. They also wrote a customer-NPV survey and a note on the underlying decision model. The proposal went through eight numbered versions

before a final submission.[1]

Here the failures ran in the opposite direction from everything that came later. Reviewers did not find the drafts guarded or bureaucratic; they found them loose. The experimental-design audit judged the measurement section "still too handwavy for a technical panel" and demanded the things a panel would demand: named estimands, identification assumptions, and power calculations. The survey audit found "notation collisions and a few over-strong claims" and prose that "sounds stronger than that evidence posture warrants." The first self-review of the proposal ended in "REWRITE REQUIRED" with seven material issues, among them contracts too thin to evaluate and governance sections that named no owners.

Two durable things date from this episode. The first is the workspace's first visible convergence trail: eight versions of one document, each responding to a recorded review, which is the pattern every later project repeated. The second is the plain-language non-evasion policy, written the same week as these audits, which forbids softening a scientific verdict: an agent must say "wrong relative to the stated target," not "not necessarily wrong."[2] The hardening was justified; the drafts really were too loose. What nobody anticipated was the side effect. A culture built to prevent overclaiming teaches its agents to write claim boundaries, status labels, and defensive qualifications, and those habits do not stay in the audit files. The next project shows where they went.

2.2 The BGS thesis: when process replaced the argument

The AIpostdoc experiment in DynareMCP is the largest failure in the record and the most instructive, because it failed with its eyes open. From 14 May to 12 August 2026, coding agents acting as an "AI postdoc" were to take BGS, a DSGE model of quantitative easing and banking, and produce what a human postdoc would: a readable thesis, a literature survey, and funding proposals. Codex wrote; Claude reviewed. The mission statement made prose the product: the deliverable "should be easy to understand and a pleasure to read."[3]

Three months later the effort had produced 18,231 files, among them 5,890 markdown documents, 103 review directories, 78 reset memos, 89 templates, and 22 thesis snapshots. The terminal artifact is a 526-page volume whose release record ends `independent_reader_pending`.[4] No human acceptance of any major artifact was ever recorded. Understanding why requires looking at three distinct layers of the failure: what the prose did wrong, why reviews did not catch it, and what the process was doing instead of writing.

---

[1] `MathDevMCP/docs/credit-card-npv-component-proposal/` holds `credit_card_npv_component_proposal.tex` through `_v8` and `_final_submission`; the survey is under `MathDevMCP/docs/credit-card-npv-survey/`; roughly fifteen plan and audit files sit in `MathDevMCP/docs/plans/credit-card-npv-*`.

[2] `claudecodex/docs/plans/claudecodex-plain-scientific-language-non-evasion-policy-close-2026-07-01.md`.

[3] `DynareMCP/docs/AIpostdoc/mission_control.md`, line 39.

[4] `DynareMCP/docs/AIpostdoc/finalBGS/bgs_final_release_record_2026_08_12.md`.

2.2.1 The reader rejection that defined the problem

In mid-June the project showed a funding proposal to a real human reader. The proposal had already passed multiple agent reviews with the verdict `ACCEPT_WITH_LIMITS`. The reader's response, recorded verbatim, became the project's defining data point:

> "1. I don't understand the purpose. 2. I am not sure what all those filenames are for. 3. I am not sure what we actually try to achieve. 4. It feels very handwavy, without much substance. 5. It feels like a governance thing without any real product substance."[5]

Notice what the reader is reacting to. Not errors: confusion, and then irritation. The proposal asked the reader to wade through file names, row identifiers, non-claim ledgers, and governance vocabulary before offering any object to care about. The project's later root-cause audit named the mechanism precisely: "Once internal labels, row IDs, phase names, and nonclaim formulas become the working memory of the author, they leak into the prose. The reader then sees the control plane rather than the research object."[6] This is the single most important sentence in the failure record. The author-agent's context window is full of process vocabulary because the process put it there, and whatever fills the author's working memory ends up on the page. Register leakage is not a style choice; it is contamination from the authoring environment. Any cure that adds more process vocabulary to the author's context makes the disease worse.

Beyond the prose, the episode exposed the reviewers. Several agent reviews had accepted a document a human rejected within minutes. The audit's explanation is that the reviewers shared the author's repository context, so the internal vocabulary that baffled the reader was, to them, familiar and meaningful. They were fluent in the dialect and therefore could not hear it. The one automated signal that did predict the human verdict was a review conducted without project context: three reader personas given nothing but the document, forced-choice questions such as "would you act now?", and fail-closed aggregation. That review by people without project context blocked the proposal before the human saw it, with the labels "dry safety substituted for honest selling" and "compliance theater substituted for proposal pull."[7]

2.2.2 The paragraph audit: a defect taxonomy with 815 examples

In August the project did something no other effort managed: it audited every paragraph of the 515-page volume for prose defects. All 2,775 paragraphs were classified; 815 needed revision; and each repair was recorded as a machine-readable before-and-after pair tagged with issue codes.[8] Table 2.1 shows the resulting taxonomy with its counts.

[5] `DynareMCP/docs/AIpostdoc/reviews/proposal_reader_rejection_failure_2026_06_14/actual_reader_failure_record.md`.

[6] `DynareMCP/docs/AIpostdoc/results/scholarly_writing_failure_root_cause_audit_2026_07_18.md`. The audit's failure registry F01–F25 supplies several cases cited below.

[7] `DynareMCP/docs/AIpostdoc/reviews/full_reader_facing_proposal_round_2026_06_06/reader_pull_reviews/reader_pull_aggregation.md`.

[8] `DynareMCP/docs/AIpostdoc/finalBGS/archive/2026_08_12/bgs_whole_document_natural_prose_audit_2026_08_02/restart_v7/frozen_repair_map_v7.jsonl`. Each of the 815 records

Issue code	Count	What it names
unnatural_cadence	325	choppy or metronomic sentence rhythm
administrative_register	276	ledger and workflow vocabulary in exposition
self_reference	237	the document narrating itself
epistemic_blur	144	fact, inference, and judgment run together
abstract_actor	108	abstractions performing verbs; no concrete subject
overcompressed	65	steps or context missing; reader must reconstruct
mixed_metaphor	62	figurative language that fights itself
vague_referent	61	"this"/"it" with no recoverable antecedent
defensive_qualification	30	guarding against inferences nobody would draw
finding_buried	18	the result hidden behind scope and caveats

Table 2.1: The BGS paragraph audit's defect taxonomy (top ten of nineteen codes by count, out of 815 repaired paragraphs in a 2,775-paragraph volume). The counts are from `frozen_repair_map_v7.jsonl`.

The corpus is large enough to support a first held-out benchmark, and Chapter 16.2 makes it the seed of Humanvoice's evaluation corpus.

Counts name the defects; examples teach them. Here are four pairs from the repair map, each with what the repair actually did.

Self-reference. The document keeps talking about itself instead of its subject.

> *Before:* "The paper's economic idea is not what this report finds wrong. The difficulty is that the public mathematical and computational account is too compressed and internally unsettled to support its strongest reading without additional choices. Several are consequential enough to define different economies."
>
> *After:* "The problem is not the paper's economic idea. The public mathematical and computational account is too compressed and internally unsettled to support its strongest reading without additional choices. Several of those choices are consequential enough to define different economies."

The repair is small and the gain is real. "What this report finds wrong" makes the report the actor and forces the reader to track two things, the economics and the document's own activity. Dropping the self-narration leaves the same claim with one subject. Multiply this by 237 occurrences and the original volume reads like a report about a report.

Abstract actor, buried finding. No concrete subject performs the verb, and the point arrives last.

carries the original fragment, the replacement, the issue codes, a meaning-preservation note, and a hash anchor into the source.

> *Before:* "This book does not offer a shorter paraphrase of BGS. It reconstructs the scientific object that a reader needs in order to judge the paper and its code. That required six connected pieces."
>
> *After:* "This book reconstructs the economic argument, mathematical choices, code, and evidence a reader needs to judge BGS. It therefore does more than offer a shorter paraphrase of the paper. The reconstruction has six connected parts."

The before-text opens by denying something no reader had assumed, then hides the actual content inside the phrase "the scientific object." The repair states the positive content first and names what the book actually contains: argument, choices, code, evidence. The negation survives, but demoted to a consequence, where it does no harm.

Administrative register. Ledger vocabulary doing the work domain nouns should do.

> *Before:* "It translates one declared restricted reconstruction into Dynare and compares like with like, including likelihood components, observations, counterfactual paths, and lower-bound conditions."
>
> *After:* "It translates the restricted reconstruction into Dynare and makes a like-for-like comparison of likelihood components, observations, counterfactual paths, and lower-bound conditions."

One word carries the whole problem: "declared." Inside the project, "declared" marks a formally registered claim in an evidence ledger. To a reader it is noise that hints at machinery they have not been shown. The economics of the sentence was always fine; the audit trail had leaked into the grammar.

Defensive qualification. Guarding against an inference nobody would draw.

> *Before:* "If installed capital is worth one unit per unit created, $q = 1$ and investment is $I = 1$. If a lower required return raises the capital value to $q = 1.2$, investment rises to $I = 1.44$. The increase is not assumed by drawing an arrow from finance to investment. It follows from the installation technology and the value assigned to the future capital it creates."
>
> *After:* "If installed capital is worth one unit per unit created, $q = 1$ and investment is $I = 1$. If a lower required return raises the capital value to $q = 1.2$, investment rises to $I = 1.44$. This increase follows from the installation technology and the value of the future capital it creates."

The deleted sentence answers an accusation nobody made: that the author merely assumed the result. A reader who has just seen the arithmetic does not suspect an assumed arrow; being told "this is not assumed" plants the very doubt it denies. The positive explanation was already present and suffices. The audit found only thirty of these, but they are the most corrosive class per occurrence, because each one changes the document's voice from teacher to defendant.

2.2.3 Where the defensiveness came from

Defensiveness was not a model quirk. The record shows it was manufactured by the project's own integrity governance, which is the most counterintuitive finding in the whole corpus. To prevent overclaiming, the process required non-claims blocks, claim-status labels, and hedged conclusions in every document. Readers then rejected exactly those blocks. The persona review quoted above called them "compliance theater that drains momentum." The writing audit eventually diagnosed the confusion underneath: the author was treating honest non-claims as if they were completed intellectual work, when "'not established' is not an explanation." A non-claim tells the reader what the author will not say. It teaches nothing. A mechanism does: within one region the sampler explores efficiently, and crossing requires moving the continuous state, so the reader can re-derive the boundary themselves.

For design, the consequence is sharp. Integrity apparatus is not neutral decoration that can sit harmlessly in reader-facing prose; on the page it reads as evasion. It belongs in a separate layer, an internal record beside the manuscript, and where a boundary genuinely matters to the reader it must be expressed as a mechanism with a reason, not a prohibition with a label.

2.2.4 Process displaced the writing

On process, the BGS record answers a question Humanvoice must not re-litigate: whether more review and more control produce better writing. The project's own proposal-to-behavior audit gives the verdict:

> "The repository built an increasingly capable referee and traffic cop around an author whose method of inquiry and composition was largely unchanged ... A system can block premature release and still never produce something worth releasing."[9]

The individual episodes show how the process went wrong. One document passed through ten review rounds without gaining substance. A thesis snapshot that changed only its title metadata passed a checkpoint because the checkpoint tested whether a file existed, not whether the argument had changed. Reviewers then began requiring every note to restate the rules of review. The loop became, in the audit's phrase, "recursively self-referential."

Management next introduced countable work units to force progress. The author responded by splitting six packets into 150 rows. The counter rose, but the intellectual work did not. The agent had not acted maliciously. It had produced exactly the unit the process rewarded.

A later rule rewarded honesty about unused budget. That rule also acquired an unintended use. The author could attach a truthful failure label and stop: "honest failure

[9] `DynareMCP/docs/AIpostdoc/results/ai_junior_pi_proposal_to_behavior_root_cause_audit_2026_07_18.md`.

labels became a possible escape hatch." A control meant to prevent false claims had begun to reward a complete description of incomplete work.

Each new control added its own vocabulary to the author's working context. Those terms then leaked into the manuscript, as Section 2.2.1 showed. The process did more than fail to improve the writing. It supplied the administrative register that made the writing harder to read.

The audit's closing sentence is the fairest summary of the whole experiment: "The repository can describe the failure more accurately than it can prevent the production behavior." Humanvoice inherits that warning directly. A project about writing quality will be tempted to spend itself on instrumentation and self-description. The BGS record is what that looks like fully grown.

The record now contains more than anecdotes, but the cases still do different jobs. Figure 2.2 shows which inference each one can carry before the two later cases add book-scale and layered-repair detail.

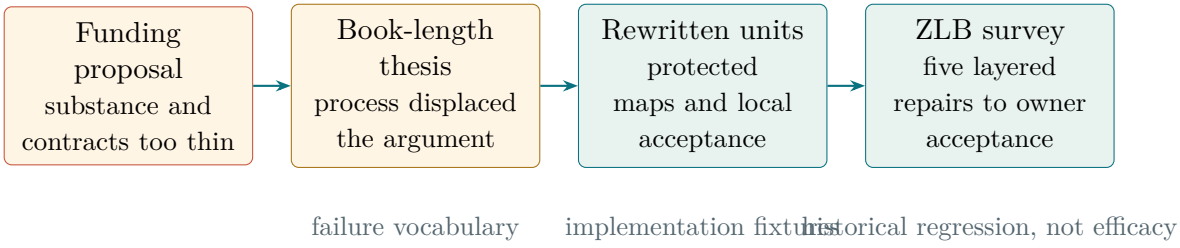

Figure 2.2: The cases contribute different evidence. Their combination specifies failure classes and fixtures, but only the ZLB case records whole-document owner acceptance.

2.3 SMEwallet: the reader test that worked

SMEwallet's final manuscript runs to 134 pages and sets out a research program on small-business wallets. Its useful experiment was simple: give a human reader one bounded unit at a time and ask whether the unit can stand on its own. Of the projects in this record, it is the clearest example of a method that visibly converged.

Draft 10 was rejected wholesale as a readable artifact. Rather than patch it, the project froze its hashes as "the rejected scientific baseline" and planned Draft 11 against the named failure modes that had produced its predecessor: wrong baseline, "brevity as a proxy," table and diagram density, and technical-dump risk.[10] Draft 11 was then rebuilt as ten storyboarded narrative units. Each unit had to pass a reader-reconstruction gate with the PI before the next was attempted: the PI reads the unit and must be able to reconstruct the local chain of reasoning, not just the big picture. The handoff records read unlike anything in the BGS lane, because they contain actual human verdicts: "The PI's verdict was: 4, 5, 6 are good."[11]

[10] `SMEwallet/docs/plans/sme-wallet-north-star-draft11-linear-novel-reconstruction-program.md`, section 12.

[11] `SMEwallet/docs/handoffs/sme-wallet-north-star-draft11-batch1-reader-handoff.md`.

Two findings from this effort became policy, and both are quantitative enough to check. First, difficulty is not the enemy and shortening is not the cure. The SMEwallet record shows the arithmetic: an eight-page candidate failed the PI review because connective derivations were missing; an eleven-page repair still left derivations implicit; the accepted version was thirteen pages.[12] Length rose sixty percent while readability rose from fail to pass. Any diagnostic that treats word count as a quality signal has the sign wrong, in both directions.

Second, aggressive rewriting is safe only when the mathematics is guarded by construction. Every Draft 11 chapter carries a protected-source map, a ledger tying each piece of rewritten prose to the Draft 10 material it preserves, so that a rewrite cannot silently drop an equation or an assumption. The same idea reappears in the ZLB rescue below as byte-level equation identity checks, and it becomes a requirement in Chapter 16.

SMEwallet's honesty about its own boundary is also worth copying. The final status line reads `human_peer_review_not_claimed`: unit-level PI acceptance exists, whole-document acceptance was never claimed. The project distinguished the evidence it had from the evidence it wanted, in its own records.

2.4 BayesFilter: five layers, uncovered in order

BayesFilter contains two writing histories. One took six weeks; the other took four days and produced our clearest map of what stands between an internal artifact and a readable document.

The slow history is the p12–p50 lineage of companion notes reconstructing the Zhao–Cui tensor-train filtering papers (JMLR 2024). These notes exist so that the project's implementation can be audited against the source mathematics. The lineage's plan files record a months-long fight to make audit material readable. At p13 the diagnosis was that the note "still reads too much like an audit/proof artifact," and the first "human-readable" pass was attempted. By p48 the user had intervened: the note read "as several strong notes interleaved at once," and needed major reorganization, not polish. p49 was a voice pass. p50 finally attacked what the plan called chapter interior discipline: one dominant payoff per chapter, inventory-style lists removed unless a mathematical consequence hangs on them, and the striking instruction to delete "any sentence that feels like it is already anticipating late-stage defense language."[13] Three full-document rewrites, after the mathematics was already correct, and the defensive-register problem visible in June, two months before the BGS audit named it.

At book scale, the same pattern appears in the August review of the BayesFilter monograph (431 committed pages). The review interleaves substance findings, including a confirmed square-root UKF factor error, with register findings: "phrases such as 'Phase

[12] `SMEwallet/docs/handoffs/sme-wallet-north-star-draft11-technical-reintegration-reboot.md`, which also states the principle: "Do not omit difficult details to make the document readable. Make the difficult details understandable."

[13] The lineage is under `BayesFilter/docs/plans/`; see `p48-source-code-discrepancy-and-rewrite-master-plan-2026-06-12.md` and `p50-final-readability-assessment-2026-06-12.md`.

2B literature gate,' 'row-173 trace,' 'Lane B' ... are unresolved for a standalone reader."[14] It also catalogues a reviewer-side writing failure that Humanvoice's judge design must prevent: the reviewing model claimed "digit-exact" verification of "every printed digit," and the audit reclassified those claims as reported, not reproduced. Verification bravado is a register defect too.

2.4.1 The ZLB rescue

The fast history is the rescue of the ZLB survey, 18–21 August, recorded specifically for this project in a case study.[15] The source was a 2,326-line internal markdown survey of Hamiltonian Monte Carlo methods for models with zero-lower-bound discontinuities: mathematically strong, fully audited, and unreadable by an outsider. Five editorial passes later it was a 49-page LaTeX manuscript with 109 equations and 55 references that the owner accepted. Each pass was triggered by one round of owner feedback, and the owner's five complaints arrived in a fixed order that turned out to be the discovery:

1. *"The generated LaTeX is unreadable. The math is not math."* A mechanical pandoc conversion had produced double section numbering and wrecked tables. Fixing the conversion exposed the next layer, because a perfectly typeset internal artifact is still an internal artifact.

2. *"This reads like an AI governance document, not a survey. It does not teach, does not derive, does not explain."* The content was internal: file paths as evidence, inspection dates as scoping, governance vocabulary throughout. Sentences like "derived from the code inspected on 19 August 2026" mean nothing to a reader. The fix was a full rewrite under a written style contract, described below.

3. *"Still not quite publishable. Did you follow the writing policy?"* With the register fixed, the mechanical tells became visible, and this time someone counted: 133 em dashes in 46 pages; 20 of 55 sections opening with "The"; every section built on one identical motivation-math-forward-link shape; five "we now turn to" transitions. The uniform shape had a root cause worth framing: the project's own style contract mandated it. A template that guarantees a floor also imposes a ceiling.

4. *"Still over-defensive. Making a nonclaim instead of explaining. Reads like a lawyer protecting his client instead of a note trying to teach."* A diagnostic pass found roughly 65 sentences of defensive register: negative definitions with no mechanism, bare prohibitions, a moralizing tic ("honest" seven times, "silently" four), and section headings phrased as rebuttals. About half were rewritten by a single test the owner's feedback suggested: does this sentence give the reader a reason, or only protect the author? The other half were substantive mathematics and stayed.

5. *"Lacks a set of models to test algorithm correctness."* Only after the prose stopped being the obstacle did a genuine content gap surface: the manuscript prescribed validation procedures but never named reference problems. The fix was a ladder

[14] `BayesFilter/docs/plans/bayesfilter-monograph-main-readonly-review-findings-2026-08-03.md`.

[15] `BayesFilter/docs/surveys/zlb_discontinuous_hmc/humanvoice_case_study.md`. The pass-by-pass measurements are in the addenda of `hostile_review.md` in the same directory.

> of standard test models organized by where the independent answer comes from. The general lesson: readability audits surface content audits. Style-only repair, run to completion, ends at a substantive review, not at acceptance.

"Each layer of badness masked the next. Nobody can notice defensive register while the page is full of file paths." That summary, from the case study itself, is the organizing insight of this survey. Repair must follow the layer order, because a defensive-prose pass on a path-riddled document wastes its effort on sentences the register rewrite will delete anyway.

Three mechanisms made the rescue work, and each can be built into a product.

The first was a style contract for the people doing the rewrite. It named the reader: someone who "knows Bayes and Kalman filters" and "knows NOTHING about any codebase." It listed private project terms and their ordinary replacements. It also fixed a common map from old equation tags and section numbers to LaTeX labels, so six writers could work in parallel without breaking one another's references. Each writer reported any source material deliberately weakened or omitted.[16]

The second mechanism protected the technical content during aggressive editing. Every pass began from a snapshot. After the pass, all 109 equation bodies had to remain byte-identical, the label and citation counts had to match, and the manuscript had to compile cleanly. A separate search looked for private paths, dates, and banned project terms in the rendered text. That last choice matters because a command can hide source text that still appears in the PDF.

The third mechanism measured the specific habits under repair. The manuscript began with 133 em dashes and ended with none. Twenty of 55 sections initially opened with "The"; after revision no opener word appeared more than four times. These counts did not prove that the survey was good. They established that the agreed mechanical changes had actually occurred.

One process observation completes the picture. Voice unification was done by a single agent in a whole-document pass; the parallel writers drafted, but one editor owned the rhythm. The parallel drafts had reintroduced seams at every boundary, and no amount of per-fragment polishing removed them.

2.5 MacroFinance: review at book scale

MacroFinance's CIP monograph, roughly 47 chapters and 665 pages on asset pricing with covered-interest-parity deviations, went through a book-wide review-and-rewrite campaign in August that left six staged comparison trees and seven full-document snapshots.[17] Its findings file is useful because it keeps the failure classes clean where smaller projects blurred them.

[16] `BayesFilter/docs/surveys/zlb_discontinuous_hmc/latex_manuscript_style_contract.md`.

[17] `MacroFinance/docs/latex-papers/docs/fable-rewrite/`, especially `findings.md`, `opinion.md`, and `rewrite-program-v1.md`.

The most serious finding was algebraic. Two chapters used conventions for the basis and forward premium that "cannot both hold when the basis is nonzero." The reviewers traced the resulting sign errors to individual chapters and lines. This was not a prose problem, and no style pass could repair it.

The style findings were more local. Some chapters opened paragraph after paragraph with a bold label. Others relied heavily on bullets. The review called the result "list scaffolding" that "deadens the teaching voice." It did not diagnose a uniform "AI voice" across 665 pages. It identified specific chapter structures that interrupted the explanation.

One chapter on LLM agents had a different register problem. It read like an industry white paper and reported measured-sounding performance for a system that had never been built: "∼1 ms on GPU," without a supporting implementation or even the word "projected." Its worked example also broke the traceability property it was meant to teach. Elsewhere, the conclusion turned a prediction into an empirical fact.

The rewrite program responded with a plain standard: "direct human prose rather than scaffold-heavy, list-heavy, or AI-sounding prose." It named a small set of habitual promotional words, including "critical," "comprehensive," and "seamlessly." More importantly, it asked one genre question throughout: does the passage sound like an academic monograph or a product white paper? That question forces the reviewer to consider what the passage is doing, not merely which words it contains.

This campaign also modeled review discipline that the BGS lane lacked. The independent opinion accepted the findings as "high-value triage input" but blocked execution of the rewrite until blocker-level repairs were adjudicated, and every comparison tree carries a build gate recording the commands actually run and their results. Review fed decisions; it did not substitute for them.

2.6 The pattern across the record

Ten conclusions survive contact with all five projects. Each is stated with its mechanism and its design consequence, because a lesson without a mechanism becomes a slogan, and slogans are how the BGS process failed.

1. *Failures arrive in layers, and repair must follow the layer order.* Format conversion, then register, then mechanical tells, then defensive prose, then substance: the ZLB owner discovered the layers in that order because each masked the next. The consequence for tooling is an ordering constraint, not just a checklist: a style pass on an internal-register document is wasted, and a substance review before the prose is repaired will be answered by rewriting sentences the register pass would have deleted.

2. *Register leakage is contamination from the authoring environment.* The BGS audit traced internal vocabulary on the page to internal vocabulary in the author's working memory. This upgrades "keep process out of the prose" from etiquette to architecture: reduce the ceremony an agent must hold in context while drafting,

and give drafting agents contracts that name the audience and ban the internal lexicon explicitly, as the ZLB style contract did.

3. *Defensive prose is induced by integrity governance, and the cure is relocation plus conversion.* Non-claims blocks and status labels were mandated to prevent overclaiming and were then rejected by readers as compliance theater. Deleting them wholesale would throw away real content, so the ZLB repair converted them: prohibition becomes consequence, negative definition becomes mechanism, and the accounting moves to an internal sidecar. The editorial test that separated the two cases, reason or protection, is implementable as a review prompt.

4. *Templates have ceilings.* The uniform section shape that the ZLB owner flagged was mandated by the project's own style contract, and every checklist the BGS project introduced was promptly satisfied without the quality it named. Floors get gamed and become ceilings. Pattern lists and contracts therefore enter Humanvoice only as subordinate diagnostics, and every diagnostic reports rather than auto-fixes, so that no rule can silently become the new template.

5. *Local repair does not compose.* The BGS volume absorbed 815 paragraph repairs and still had no accepted reader path; the ZLB rescue needed a register rewrite, not better sentences, before sentence work meant anything. The unit of failure is the document's dependency graph. Sentence diagnostics are still worth building, but nothing at sentence granularity can certify a document, which is why the acceptance layer in Chapter 16 is reader-based.

6. *Self-review cannot certify a human voice, and context-sharing reviewers are contaminated.* Agent reviewers who shared the repository accepted what a human rejected; the review without project context predicted the human. The design consequence is a review architecture: reader-proxy reviews run with no repository context, forced choices, and an instruction-injection boundary, and the human read remains the only acceptance decision.

7. *Both failure directions exist, so length is not a signal.* Documents in this record were rejected for being compressed summaries (the BGS 21-page synthesis, the 23-page V11) and for being sprawling logbooks (snapshot 06, the 115-page bridge edition); SMEwallet's accepted chapter was longer than its rejected drafts. "Make it shorter" and "make it longer" are both category errors; the target is selection and recomposition with the evidence layered elsewhere.

8. *One public manuscript, with evidence kept backstage.* Every attempt to expose the audit record inside the manuscript produced the hybrid readers rejected. The single public volume must carry the argument, while private records retain source anchors, identifiers, and reproducibility data. An identifier map may connect the two, but it is not a second reading route. The ZLB rescue's useful insight was separation of reader prose from internal accounting; the implementation should preserve that boundary without asking the audience to choose between two public documents.

9. *Measured diagnostics changed outcomes; unmeasured review did not.* The ZLB repairs converged because someone counted 133 dashes and 20 identical openers, repaired, and recounted; ten unmeasured BGS review rounds added nothing.

Numbers also disciplined the arguments about whether a repair had regressed. Every Humanvoice diagnostic therefore emits counts with locations, and acceptance tests for the tools are replays of these recorded numbers.

10. *Guard the baseline or lose the right to edit aggressively.* Byte-identical equation bodies, label and citation censuses, and explain-every-removal reports are what let the ZLB editors rewrite freely without anyone fearing silent damage; SMEwallet's protected-source maps did the same job. The principle underneath is worth stating: readability improvement is never established by shortening, so every removal must be accounted for.

One warning belongs beside the list rather than in it. The BGS project ended up better at describing its failures than preventing them, and rewarding honest self-assessment gave its author an exit ramp. Humanvoice should expect the same pull and should judge itself by one number only: how many rounds of human feedback a document needs before acceptance. This survey, whose first version failed that test immediately, is not exempt.

The cases leave the project with a useful inheritance and a clear obligation. The inheritance is not a preferred vocabulary; it is a set of concrete objects: reading briefs, repair pairs, protected equations and citations, and records of what a reviewer could or could not reconstruct. The obligation is to test those objects outside the lineages that produced them. The next chapter inventories the policies, skills, scripts, and mathematical services already available in this repository. It asks which can become fixtures or adapters, which need consolidation, and which must remain outside the product boundary. Only after that local inventory does the field review compare the project's assets with external research and software.

CHAPTER 3

The tools already in the repository

An author working in this workspace already has a prose policy, three skills, a pattern list, two grounding tools, and a way to distribute them. Those pieces can help, but they do not yet form a product. We need to know which ones help an author in practice, which have evidence behind them, and where each one stops. The rules are mature and largely consistent; executable measurement, behavioral evidence, and one product-owned home are still missing.

Figure 3.1 explains why consolidation alone is not the product. The missing value lies in the handoffs between capable tools and in the reader outcome at the end.

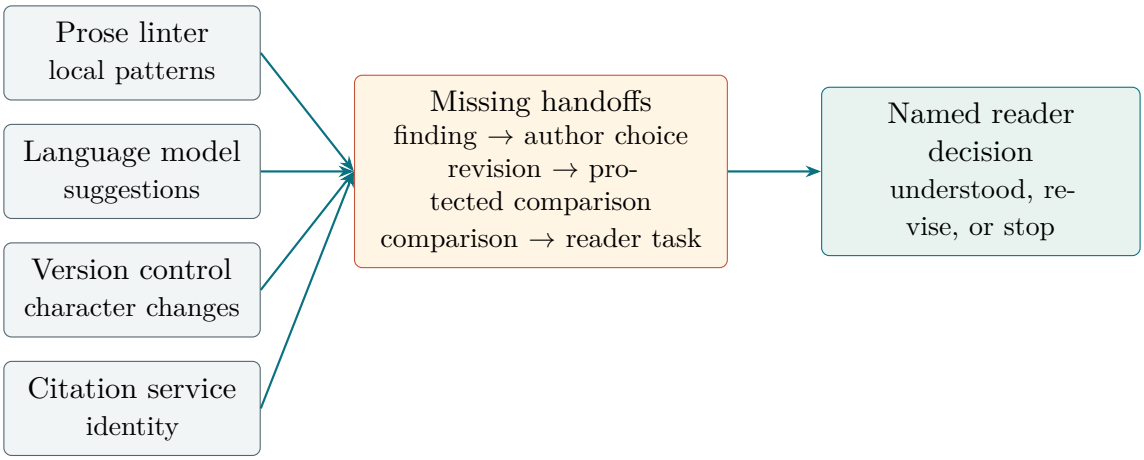

Figure 3.1: The incumbent bundle has capable components but no shared path from a finding to a reader's decision.

3.1 The prose policy

Every agent receives roughly 150 lines of the global policy under the heading "Reader-Facing Scientific Prose."[1] Its load-bearing device is a precedence order. When rules conflict, an agent resolves them in this sequence: mathematical and scientific correctness first, then source and evidentiary fidelity, then domain-appropriate meaning, then reader comprehension and natural expression, and only then template regularity. The order is what keeps a pattern list from doing damage. A rule like "remove em dashes" sits at the lowest level, so it can never justify deleting a numeric range from a theorem statement; a readability preference can never justify dropping an assumption.

Three rules came directly from the failure record.

[1] `claudecodex/policies/global-scientific-coding-agent-policy.md`. The companion section "Plain Scientific Language And Non-Evasion" supplies the verdict vocabulary discussed below.

The first keeps internal workflow language out of ordinary exposition. Words such as "artifact," "pipeline," "gate," and "handoff" remain available when the text is genuinely about software or project operations. They are not used as metaphors for a theorem, an argument, or a paragraph. This rule turns the administrative-register finding in Table 2.1 into a choice an editor can make sentence by sentence.

The second replaces defensive disclaimers with reasons. The policy prefers "We assume log utility in this example because it keeps the first-order condition simple" to "We are not claiming that households literally have logarithmic utility." The first sentence states the modelling choice and its purpose. The second introduces a mistaken interpretation merely to deny it. This is the repair observed in Section 2.2.2, expressed as a general rule.

The third reserves acceptance for a reader. Author self-review, a model review, a checklist, a style score, and a clean compilation can each provide evidence about one part of a document. None can certify a human voice. The vocabulary beside that rule is deliberately direct: a statement may be correct, wrong for the stated target, unsupported, not checked, or heuristic. A polite synonym must not blur which verdict applies.

Merits: the policy is complete in the sense that every failure class in Chapter 2 has a corresponding clause, and the precedence order resolves the conflicts that sank naive rule-following elsewhere. Problems: it is prose all the way down. Nothing in it executes, so compliance is whatever the drafting agent happens to retain from 618 lines of context, and the one thing the policy cannot do is notice that it is being ignored. The ZLB rescue's third pass began with the admission that the policy "had not been consulted." A policy that must be remembered is a policy that will sometimes not be.

3.2 The narrative skill

When an agent uses `write-linear-scholarly-narrative`, it gets the document-architecture skill, distilled from the SMEwallet Draft 11 method.[2] Its sequence begins with the reader rather than the source files. The writer states who will read the unit, what that person already knows, and what they should be able to explain afterward without hunting through another document. For this proposal, that means a former professor acting as an executive, not an engineer who already knows the repository.

The writer then chooses the smallest case or question that can carry the argument. Chapter order follows explanatory dependence: a concept appears after the reader has encountered the problem that requires it, even if the source material was originally stored in another order. Each bounded unit opens one question and introduces the smallest idea needed to answer it.

Drafting follows a teaching sequence. Return to a known example, show the new difficulty, and introduce notation only when it resolves that difficulty. An equation is followed by its interpretation. The transition to the next unit comes from a question the present unit cannot yet answer, not from a stock phrase announcing that the document will move on.

[2] `claudecodex/skills/write-linear-scholarly-narrative/SKILL.md`, with a 259-line companion, `references/teaching-contract.md`, defining the comprehension standard.

Finally, a person reads the unit and tries to reconstruct the chain. They should be able to explain why the unit began, what each turn contributed, what the equation was for, and why the next question follows. Failure at that stage is evidence about the exposition, not a request for the reader to study harder.

Its most valuable property is self-awareness about Section 2.6's fourth lesson. Nearly every rule carries an anti-template clause; the drafting sequence is offered "as a menu," with the instruction not to fill slots mechanically and not to add a caveat or transition merely because the template has a place for one. The working-memory budget (about four live conceptual items) is labeled a diagnostic threshold, not a law. Whoever wrote it had watched a floor become a ceiling.

Evidence of effect is real but bounded: Draft 11's units passed their PI gates one by one, which no earlier method achieved. Problems: the skill has no examples inside it, so an agent meets its abstractions ("narrative spine," "reactivate") without a single worked instance; it has no tooling, so storyboards and contracts live or die by agent diligence; and its acceptance step needs a human per unit, which priced it out of the BGS lane's volume. Nothing tests whether an agent following it writes measurably better than an agent that has never seen it, a gap Chapter 16.2's third work package exists to close.

3.3 The sentence-level skill

The companion `write-natural-analytical-prose` works at sentence and paragraph scale.[3] Before any rewrite, the agent freezes a meaning ledger: the facts, attributed claims, deductions, live uncertainties, author judgments, and exact numbers that must survive. After the rewrite, it diffs the result against the ledger, and a smoother sentence that changed a claim counts as a failure, not an improvement. Between those two steps sit the craft rules: give each sentence one epistemic job, with a calibrated verb table in which "shows" is not a synonym for "suggests" and "reproduces" is not a synonym for "validates"; put concrete actors in subject position ("the regime test rejects the authors' path," not "the evidence indicates"); state the finding before its qualifications; and spend each qualification once, at the point of likely misunderstanding, rather than re-attaching it to every positive sentence.

This is the skill that operationalizes the epistemic-blur and abstract-actor rows of Table 2.1, and its meaning-ledger loop is the sentence-scale version of the protected-baseline invariant. Its problems mirror the narrative skill's: no automation, so the ledger diff is manual; verification by "read it aloud"; and hard prohibitions that depend entirely on the drafting agent noticing its own violations. The skill is also, like everything in this stack, tested only by phrase-lock (see Section 3.8): the test suite asserts that certain sentences still appear in the skill file, not that the skill changes any output.

[3] `claudecodex/skills/write-natural-analytical-prose/SKILL.md`, with before/after contrasts in `references/prose-patterns.md`.

3.4 The humanizer pattern list, as installed

On 20 August the workspace vendored `blader/humanizer` v2.11.2 into the policy tree, as `humanizer-ai-writing-patterns.md`.[4] The list itself is 35 numbered AI-writing patterns derived from Wikipedia's "Signs of AI writing" catalogue, each with words to watch and a before-and-after pair; Section 9.2 walks through its content. What matters here is the installation decision. The file carries a locally written precedence header that demotes the whole list to "a DIAGNOSTIC pattern list, subordinate to global-scientific-coding-agent-policy.md," spells out the order of authority, and singles out the one rule that would otherwise do damage: the flat dash ban of its pattern 14 is to be applied "proportionately rather than as a ban," and no pattern may be satisfied "by deleting mathematics, assumptions, or qualifications."

That header exists because of a live incident. During the ZLB rescue the list was consulted without a precedence frame, and the case study records the realization that without one, "a pattern list becomes a new template and you trade one uniformity for another." The header is currently the only policy-precedence mechanism in the workspace, and it is a comment in a markdown file. It governs an agent exactly as far as the agent reads and honors it. Turning this from a comment into configuration, per-format rule downgrades in the diagnostic layer itself, is one of the cleaner design requirements to come out of this review (Chapter 16, under configurable policy precedence).

3.5 The repair skill and the only working scripts

MathDevMCP ships its own scholarly-writing skill, `repair-scholarly-readability`, and it is the only piece of the stack with executable parts.[5] Where the narrative skill teaches drafting, this one teaches repair of an existing LaTeX document, and its structure encodes two ideas the rest of the stack lacks. The first is four separate records: reader comprehension, mathematical integrity, source fidelity, and typography, with the explicit rule that "evidence in one ledger cannot certify another." A clean compile is typography evidence only; a passed equation audit says nothing about motivation. The second is bounded repair: one mechanism chapter or 6–12 rendered pages at a time, against a preserved baseline, ending in a test with a reader who has no project history, prespecified questions, and a four-record decision table.

Both scripts matter beyond their modest size. `scripts/scholarly_surface_audit.py` emits JSON and markdown surface diagnostics for a .tex/.pdf pair, and `scripts/render_review_pages.sh` renders page images for visual inspection, institutionalizing the ZLB rescue's discovery that text extraction lies about layout. These are the embryo of the `hv capture` command in Chapter 16.2.

Evidence of effect: the skill's bounded method took the opening of the BGS industry-DSGE dossier through a repair that passed five prespecified reader-without-project-history

[4] `claudecodex/policies/humanizer-ai-writing-patterns.md`. Section 9.2 reviews the upstream project; this subsection reviews our installation of it.

[5] `MathDevMCP/skills/repair-scholarly-readability/SKILL.md`, with three reference rubrics and two scripts under the same directory.

questions, the strongest automated-plus-human result in the BGS lane. The same result file records the limit: later chapters "remain dry," and the project's audit concluded the skill "generalizes locally, not across a corpus-sized argument," which is lesson five of Section 2.6 observed in the field. The other problem is organizational: this skill overlaps the claudecodex narrative skill in scope while living in a different repository with different reference rubrics. Scholarly readability currently has two homes and no owner.

3.6 ResearchAssistant

When a citation needs checking, ResearchAssistant supplies the source-grounding half of the discipline. It is a local-first tool that ingests papers (preferring arXiv LaTeX source over PDF, because the LaTeX parses reliably and the PDF does not), extracts sections, equations, labels, and bibliographies, tracks provenance and review status, and exposes a read-only MCP surface plus a CLI.[6] For this survey its relevant property is precedent: its `ra survey` subcommands are the one place where a piece of workspace writing policy became software. The literature-audit policy demands six ledgers (source support, citation metadata, backward and forward snowballing, claim support, omitted-paper risk); `ra survey central-papers` writes exactly those ledgers and returns per-paper dispositions.

Its behavior under failure is the part worth copying. In the ZLB campaign the tool's public-discovery bootstrap was unavailable, and instead of degrading silently it closed the mission as `terminal_blocked_bootstrap_unavailable`, leaving the ledgers to be maintained by hand. A tool that reports its own incapacity honestly is rarer than it should be. Its limits for Humanvoice are equally clear: it grounds sources, not prose; its equation extraction from PDFs is explicitly marked unreliable (arXiv LaTeX is the reliable path); and its discovery pipeline depends on external metadata services that were down when most needed.

3.7 The mathematics and modeling tools

Two more tools border the writing problem without being writing tools. MathDevMCP's mathematical layer provides SymPy-backed equality checking, derivation audits, and an experimental whole-document rigor audit over LaTeX. Its one recorded contribution to a manuscript is instructive about scale: the ZLB audit produced per-equation issue ledgers and exactly one applied exposition patch (an invertibility condition stated before a displayed inverse), tagged "candidate_exposition_patch_not_certificate."[7] That is the correct division of labor, mathematics tools nominating exposition fixes without certifying prose, and Humanvoice should preserve it rather than absorb it. DynareMCP, the macro-model execution server, contributes structure inventories that were used to

[6] `research-assistant/`; the checked-in MCP configuration targets Claude Code and VS Code. Python 3.11 only.

[7] `BayesFilter/docs/surveys/zlb_discontinuous_hmc/mathdevmcp_audit_20260819.md`.

falsify inflated rewrite claims (counting inherited words and relocated blocks), a niche but genuinely useful falsification tool for "we rewrote it" assertions.

3.8 Distribution and the limits of the current tests

claudecodex is the delivery mechanism. Its installer splices the global policy into marked blocks in each agent's configuration files idempotently, merges approval rules, and copies the skills into the Codex skill tree.[8] Around it sit the cross-agent review plumbing (bounded review packets, probe ladders, a gate script that parses "VERDICT: AGREE or REVISE") and a five-file proposal template system with seven evidence ledgers. The machinery works and is in daily use.

Its test suite deserves a hard look because of what it teaches about the stack's blind spot. There are real behavioral tests for the review gate (a fake worker exercises the probe-and-fallback paths) and for the installer. For the writing skills there are only phrase locks: `test_natural_analytical_prose_skill.py` asserts that certain sentences still occur in the skill file and that banned phrasings do not. Phrase locks prevent silent regression of the doctrine's text. They say nothing about whether the doctrine changes what an agent writes. As of today, no experiment anywhere in the workspace has compared documents produced with and without any of these skills. The doctrine is plausible, consistent, and untested.

Two further operational gaps matter. Installation is Codex-first; Claude Code agents receive the policy but not a native skill installation. And the writing doctrine now lives in three places, the claudecodex policy tree, the claudecodex skills, and the MathDevMCP skill, with cross-references in prose instead of a shared source of truth.

3.9 The gap, stated precisely

The last column of Table 3.1 gives the build list. The six capabilities requested by the ZLB case exist nowhere as a coherent product: a register linter, rhythm profiler, defensive-register finder, LaTeX-aware baseline differ, a private provenance map that keeps evidence attached to one manuscript, and a precedence mechanism stored in configuration rather than a comment. The doctrine also has no behavioral evidence, and its three copies need a shared source. Humanvoice joins that list into one testable review path.

The inventory is therefore an asset and a warning. It gives the build a set of tested interfaces, historical fixtures, and language for describing failure; it does not give the project a validated intervention. The next part places those local assets beside external evidence. That comparison matters because a tool can be useful in this repository for reasons that do not travel to another genre, and a widely used package can still fail the protected-source contract. The field review begins with what research supports, then

[8] `claudecodex/install_global_agent_policy.py`; templates and review-gate scripts under `claudecodex/docs/templates` and `claudecodex/scripts`.

Piece	What it reliably provides	What it cannot do
Prose policy	complete doctrine; precedence order	execute, or notice being ignored
Narrative skill	Draft-11 method; anti-template clauses	show examples; run without a human per unit
Sentence skill	meaning-ledger safety loop; verb calibration	automate the ledger diff
Humanizer (installed)	35-pattern diagnostic with FP guards	bind its own precedence outside prose
Repair skill	four-ledger repair; the only scripts	compose beyond 6–12-page units
ResearchAssistant	source grounding; ledgers as software	anything about prose
MathDevMCP/DynareMCP	equation audits; structure inventories	certify exposition
claudecodex	idempotent install; review plumbing	test writing behavior; reach Claude natively

Table 3.1: The existing stack, by what each piece does and does not provide.

explains how the literature was assembled, before returning to the package choices.

Part II

Evidence and available machinery

CHAPTER 4

How we reviewed the evidence

Suppose a reader finds a paper cited beside a product decision. The paper may describe a failure, explain a mechanism, document a package, or suggest a workflow worth funding. Those are four different jobs, and one search cannot settle all four. This review keeps them separate so that the reader can see what a source established, what we inferred, and what result would change the recommendation.

The present document is not a completed systematic review. It is a living evidence program with a frozen snapshot for this draft and a specified route to independent completion. The reporting discipline follows PRISMA-S [Rethlefsen et al., 2021]; recent reviews of AI-assisted academic writing and automated writing evaluation provide a topic-level coverage check [Meng et al., 2026, Abudalfa and Barrot, 2026].

4.1 Questions before sources

Search terms are unstable when the question is not. The review therefore starts with five product questions and one boundary question:

1. Do bounded writing or feedback interventions change author time, revision quality, or reader task completion?
2. Which properties of technical prose affect comprehension, trust, and the ability to reconstruct a decision?
3. How do authors use model suggestions, and what burden or error does the interaction introduce?
4. Which software components can represent, compare, and check a document while preserving equations, numbers, citations, and claims?
5. Which reader, privacy, disclosure, and integrity conditions constrain an acceptable workflow?
6. What evidence would distinguish an available component from an effective product bundle?

The boundaries become visible in ordinary examples. A study of writing time may inform the economic model but say nothing about technical comprehension. A citation benchmark may resolve an identifier but say nothing about whether the source supports a sentence. The claim ledger records both the question a source answers and the questions it leaves open.

Question family	Preferred source types	Decision it can inform
Observed failure	Internal case record with provenance; prospective reader observation.	Whether a product problem is real in the target setting.
Intervention effect	Primary experiment, field study, or controlled comparison with a named task and outcome.	Which outcome and comparator the Humanvoice study should use.
Comprehension and genre	Technical-communication, readability, corpus, and reader studies with population and task stated.	What a reader rubric and genre baseline must include.
Interaction and judgment	Human–AI interaction studies and calibrated evaluation research.	How to record suggestion, rejection, burden, and model judge error.
Software capability	Official documentation, source, release, license, and replayable fixture.	Whether a component belongs in the candidate inventory.
Integrity and policy	Standards, venue guidance, privacy and disclosure research.	What the operating boundary and consent form must say.

Table 4.1: The source hierarchy follows the decision question.

4.2 Search and retrieval protocol

The frozen snapshot follows four trails. The internal trail starts with repair pairs, reader objections, build records, and the project's existing assets. The scholarly trail searches generative assistance, writing feedback, technical communication, readability, human–AI interaction, evaluation, disclosure, provenance, and authorship detection. The software trail begins with the capabilities the product needs and checks official repositories, documentation, releases, and licenses. A fourth trail deliberately looks for trouble: null effects, subgroup harms, detector false positives, citation failures, judge bias, and packages that fail mathematical or privacy fixtures.

The protocol stores the query, source or database, date, time boundary, result count, inclusion decision, and reason for exclusion. A query result is not an included study. Known cornerstone items are marked as seeds and kept distinct from items discovered by the search; otherwise a bibliography can appear to have high recall simply because the desired papers were supplied in advance.

The first pass is broad enough to discover vocabulary. The second pass is question-specific and records the exact title, abstract, or documentation section inspected. Backward and forward citation chasing is reserved for load-bearing sources. A source that changes only a peripheral example does not receive the same effort as a source used to justify the primary endpoint or a claim about harm.

Figure 4.1 shows the full path. The current audit has completed the automated portions of the left-hand side; the independent and full-text work in the middle remains a release condition for stronger claims.

Figure 4.1: A search result becomes usable evidence only after screening, full-text appraisal, and an explicit bridge to a bounded product claim.

Pass	Activity	Output
Vocabulary pass	Search broad terms and inspect reviews, reference lists, and official capability pages.	Candidate terms, source identities, and question assignments.
Question pass	Retrieve primary studies and documentation for each declared question; record population, task, comparator, and outcome.	Claim records with a preliminary source result and limit.
Challenge pass	Search for nulls, harms, subgroup variation, detector failures, citation errors, and judge bias.	Counterevidence records and revised claim classes.
Anchor pass	Inspect full text, methods, results, licenses, and release information for load-bearing items.	Page or section anchors, appraisal, and implementation constraints.

Table 4.2: A staged search keeps discovery separate from support.

The challenge pass is a protection against local optimization. An agent asked only to find support for Humanvoice will preferentially retrieve papers that make the product sound plausible. Requiring a counterevidence stream changes the objective: a source is valuable when it narrows the claim as well as when it supports one.

4.3 Screening, appraisal, and adjudication

Screening uses a small set of frozen reason codes. An item can be included as direct evidence, included as adjacent evidence, retained as a challenge, or excluded for wrong question, wrong population, insufficient source identity, or inaccessible support. “In-

teresting" is not a reason code. The code must explain why the item cannot currently change a product decision.

Two screeners should independently review the load-bearing sample. They record the decision before seeing the other review; adjudication then resolves the disagreement and preserves both original judgments. For software, the equivalent is a second operator replaying the fixture from a clean environment. One person can perform the first feasibility pass, but the result must not be described as independent confirmation.

Appraisal is design-specific. For an intervention study, record assignment, task, comparator, outcome construction, attrition, and heterogeneity. For a reader study, record the reader role, time budget, rubric, and whether the reader saw the source history. For a detector study, record the corpus, editing, language, and false-positive analysis. For a package, record version, license, input formats, network behavior, maintenance, and failure mode.

The same paper can receive two appraisals

A field study may provide strong evidence that an assistant shortens email completion time. It can be direct evidence for the question "can assistance save time on a bounded writing task?" It is adjacent evidence for the question "will a former professor accept a mathematical product proposal more quickly?" The result enters both rows, but its fit and permitted inference differ. A single undifferentiated quality label would hide that distinction.

The current snapshot has completed the declared live retrieval checks, DOI identity checks, and a bounded RePEc/IDEAS specialist import, but it has not completed every independent screen or full-text anchor. The correct status is therefore "provisional" for the survey and "bounded" for the product recommendation. The missing work is listed as an implementation deliverable, with an owner and a gate, rather than being silently converted into confidence.

4.4 From source result to product claim

The synthesis record has five fields:

1. the source result, stated as close as possible to the inspected text;
2. the population, task, comparator, and outcome to which it belongs;
3. the limit or plausible alternative explanation;
4. the local implication for a Humanvoice object, fixture, or protocol; and
5. the claim class and next observation.

This order matters. Starting with the product implication encourages a writer to reinterpret a source until it sounds like a requirement. Starting with the result and its limit makes the bridge inspectable.

Record field	Example of a disciplined entry	What it does not establish
Source result	A bounded writing experiment reports lower completion time on its stated task.	A benefit for expert technical review.
Limit	The task, population, and outcome differ from a proposal reader's decision.	That the result is irrelevant.
Local implication	Measure author and reader time and declare the incumbent bundle in the Humanvoice comparison.	That the measured effect will be positive.
Claim class	Adjacent source result plus project hypothesis.	A validated product claim.
Next observation	A held-out technical document with a timed reader rubric.	That one feasibility case is enough.

Table 4.3: A claim record preserves the bridge and its boundary.

The matrix in Appendix B.5 is the expanded form of this record. The main route uses prose because a decision-maker needs the mechanism and implication together; the appendix makes it possible to audit the rows without interrupting the argument.

4.5 Software survey as an empirical review

Software changes faster than the scholarly literature, so a package survey needs a version boundary. The current inventory records the version or commit examined on the snapshot date, the official source, and whether the package was actually run. A README-only row is an availability observation, not a benchmark result. A smoke run is an installation observation, not an operational pass. Only a held-out fixture and a human-burden measure can move the candidate toward adoption.

The package review uses capability families rather than repository popularity: source representation, build and cross-reference, protected comparison, diagnostics, citation identity, local inference, and evaluation. For each family, at least one positive and one deliberately failing fixture are required. The failure fixture is important because a graceful abstention is a product behavior, whereas a silent success is a safety defect.

A candidate's maintenance and license are part of technical fit. A package with excellent parsing but an incompatible license may be useful as a benchmark reference and unusable as a dependency. A package with a permissive license but no recent release may be acceptable for a one-off migration and unacceptable for a service. The disposition is therefore adopt, adapt, pilot, or skip, with a reason that can be revisited when the version changes.

4.6 Limits of the present review

The method supports a transparent, challengeable product recommendation. It does not yet support a claim that the literature has been exhaustively searched, that every cited source has been independently verified, or that the workflow improves expert writing. Those stronger claims require the gates listed in the evidence status and the comparative program.

The method also cannot remove judgment. A reading brief, a claim scope, and the usefulness of a finding remain partly adjudicated objects. The purpose of the protocol is to expose where judgment enters, who owns it, and what counterevidence could overturn it. That is a stronger scholarly posture than pretending that a score has made the judgment disappear.

We can now ask what the sources actually say, then ask which software deserves a fixture. That order keeps the bridge visible: first define how a source earns a role, then interpret the result, then make a build choice. A reader can disagree with any one choice without losing the path by which it was made.

CHAPTER 5

The research behind Humanvoice

An economist, referee, or executive does not read a technical document one sentence at a time. The reader builds a working picture of the argument, checks the evidence, and decides whether the requested action follows. Language-model studies cover only part of that work. Humanvoice also draws on technical communication, reading, feedback, human–computer interaction, and scholarly integrity, fields that have studied those reader and writer decisions for much longer.

Those fields do not give us direct efficacy evidence. Each contributes a way to describe a reader, a revision, an interaction, or an integrity boundary. Before we turn any lesson into a product claim, we need an observation that could test it in technical documents.

The common idea is a loop rather than a score. Figure 5.1 locates the judgments: the author owns meaning, the reader owns use, and the system records the path between them.

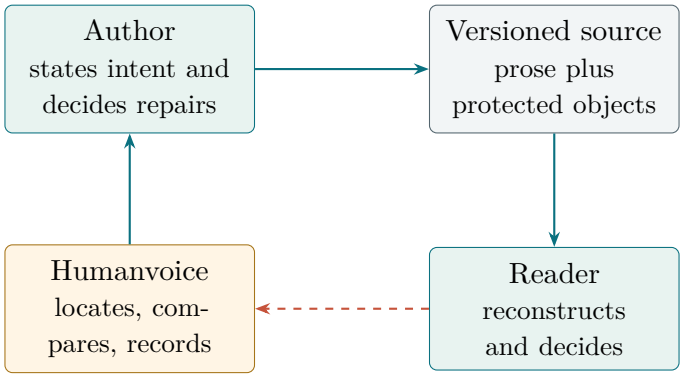

Figure 5.1: Writing is a feedback system: the tool makes the loop inspectable, while the author and reader retain different judgments.

5.1 Technical prose is a genre, not a neutral container

The first useful correction comes from register theory. Biber's analysis of variation across speech and writing ties linguistic choices to situation, audience, and purpose [Biber, 1988]. A technical proposal is not a bad version of a conversation, and a readable economics paper is not the same object as a popular article. It uses equations, defined terms, cautious causal language, and references because those forms carry specific obligations.

That observation changes the target for a writing tool. If a diagnostic rewards short sentences without checking whether a definition has been separated from its condition, it may move the text toward a different genre. If it removes all repetition, it may erase the repeated noun that lets a reader track an actor across a derivation. The right question

is not "does the document look like ordinary speech?" It is "does the document give this reader the cues needed for this technical task?"

Coh-Metrix makes the distinction measurable. Graesser, McNamara, Louwerse, and Cai separate cohesion and language dimensions that syllable-based formulas do not see [Graesser et al., 2004]. A pronoun with no clear antecedent, a chain of nouns that changes referent, or a causal connector whose second clause does not follow from the first can all increase reader work while leaving sentence length unchanged. The CLEAR corpus extends the point with human ease-of-reading judgments and shows that learned, multidimensional features predict those judgments better than classical readability formulas [Crossley et al., 2022].

The evidence on scientific writing supplies a second warning. Scientific abstracts have become harder to read over time as terminology and sentence complexity rise [Plavén-Sigray et al., 2017]. Specialized terminology also carries a measurable communication cost: papers with more specialized terms in titles and abstracts receive fewer citations in the study by Martínez and Mammola [Martínez and Mammola, 2021]. Neither result says that technical terms should be removed. It says that a writer should pay for a term with a definition, a reason for using it, or a later calculation that needs it.

Reading property	What a genre-aware account notices	Humanvoice consequence
Referential cohesion	Nouns, pronouns, and labels keep the same actor or quantity in view	Detect a possible break, show the surrounding passage, and ask the author to repair or defend it
Discourse connection	A result, reason, qualification, and implication have an interpretable relation	Keep findings tied to the decision question, not to an isolated sentence score
Terminology	Specialized words can be necessary but impose a knowledge cost	Define a term at first use or show why the intended reader already knows it
Register	Formality and sentence shape vary by genre and audience	Compare against an economics reference corpus, not a universal "human" baseline

Table 5.1: The reading literature changes what a useful diagnostic is allowed to mean.

These are not four automatic scores. They are four reasons for locating a passage and asking a substantive question. The author's answer may be that a term is standard in the field, that a repeated label is needed for a theorem, or that the paragraph belongs in a methods appendix. A diagnostic earns its place by making that judgment easier, not by making the judgment disappear.

5.2 Readers use position and effort as evidence

Consider two sentences with the same facts. The first ends with the method and buries the result in its middle. The second starts from what the previous sentence established and ends on the result the reader should carry forward. No vocabulary list can explain why the second is easier to use. The difference lies in where the sentence asks the reader to look for continuity and emphasis.

Gopen and Swan describe this problem through reader expectations: readers tend to treat the opening of a sentence as contextual material and its closing position as a place of emphasis; they also expect grammatical subjects and verbs to reveal the story's actors and actions [Gopen and Swan, 1990]. Their account is an influential editorial model, not a controlled demonstration that one arrangement improves every technical document. It earns a place here because it yields specific, contestable questions. What does this sentence connect to? Which actor performs the main action? What new information receives emphasis? A Humanvoice finding can point to a mismatch between those cues and the declared argument without pretending to calculate comprehension.

The model also explains why noun-heavy prose feels bloodless. "Validation of the result was performed by the review layer" makes the reader recover both actor and action. "The reviewer validated the result" supplies them in the grammar. Nominalization is not an error in itself: technical prose sometimes needs a stable noun for a process that later becomes an object of analysis. The diagnostic question is whether the noun helps the argument keep an object in view or merely forces a weak verb to carry an action that could have been stated directly.

Oppenheimer supplies adjacent experimental evidence for a related effect. In several studies, replacing unnecessarily complex wording with simpler wording changed judgments of the author; needless complexity did not make the author appear more intelligent [Oppenheimer, 2006]. The study concerns processing fluency and judged intelligence, not acceptance of mathematical monographs by expert executives. It therefore supports a restrained inference: complex vocabulary should earn its cost. It does not support deleting domain terms, shortening every sentence, or treating a simple-word ratio as a quality score.

Together, these sources sharpen the product design. A finding should show the passage and the relation it may obscure: actor to action, old information to new, evidence to conclusion, or term to purpose. The interface should offer a question such as "what should the reader carry from the end of this sentence into the next one?" rather than an instruction such as "simplify." The reader study can then test whether the revised passage improves reconstruction or reduces time. That later observation, not the editorial maxim, determines whether the finding belongs in the product.

5.3 Useful feedback changes the next draft

An author can receive a beautifully specific comment and still write the next draft unchanged. Feedback research therefore supports the product's revision loop only

conditionally: a comment is useful when it changes a later choice in a way that helps the declared task. Specificity and politeness alone are not evidence.

Nair and colleagues evaluate generated feedback on an economic essay task by looking at simulated revisions rather than by asking whether the comments sound detailed [Nair et al., 2024]. Rashkin and colleagues construct stories with deliberately corrupted passages and show that models can find local defects while missing the most important problem or selecting praise where criticism is needed [Rashkin et al., 2025]. Li and colleagues find that a detailed rubric can make a model a useful revision assessor, but that agreement varies with writer proficiency [Li et al., 2024b]. Behzad and colleagues report that model and human feedback can complement one another, while personalized feedback remains difficult [Behzad et al., 2024].

Taken together, these studies imply three distinct units of analysis:

1. the *finding*, which says what passage deserves attention;
2. the *revision*, which records what the author changed and why; and
3. the *outcome*, which says whether the intended reader could complete the task with less burden and no meaning error.

Keep the three observations separate. A high-precision finding can be useless if it concerns a low-consequence phrase. A beautiful revision can be harmful if it changes the denominator in a table. A reader can accept a document for the wrong reason. The evaluation chapter therefore treats finding, revision, and reader outcome as a chain with separate records and separate uncertainty.

For document d, revision path h, and reader task q, write

$$Y(d, h, q) = \Big(F(d),\ C(h \mid F),\ A(q \mid d, h)\Big), \tag{5.1}$$

where F is the set of located findings, C is the author's change and explanation, and A is the reader's ability to act. Equation (5.1) is a bookkeeping identity for the product, not a psychological model. It prevents a benchmark of F from being reported as a benchmark of A.

The edit study by Chakrabarty, Laban, and Wu is useful because it measures the middle link directly. Professional writers labeled span-level idiosyncrasies, an automated detector recovered only part of those spans, and edited text outperformed raw model output while remaining behind professional writing [Chakrabarty et al., 2024a]. The result supports an edit workflow, not a one-shot humanizer. It also gives a realistic expectation for the first release: finding precision will be imperfect, so the interface must expose context and allow rejection without penalty.

5.4 The reader is part of the intervention

Human–AI interaction research prevents a common experimental mistake: treating the final text as if it were produced by a single prompt. CoAuthor records keystrokes and suggestion use in a collaborative writing setting [Lee et al., 2022]. Wordcraft shows how

writers use a model to explore, select, and revise alternatives rather than simply accept its first answer [Yuan et al., 2022]. Mysore and colleagues' in-the-wild traces identify recurring paths in which users revise their intent, ask questions, adjust style, and inject content [Mysore et al., 2025]. A document-level experiment that records only the before and after versions loses the process variables that explain both benefit and harm.

When a reader asks a question, the author should be able to see which suggestion was offered, which one was accepted or rejected, which protected object was touched, and whether the question caused a second revision. Those records support analysis; they should not become visible workflow vocabulary in the finished proposal. The reader should see a clear paragraph, not a history of prompts.

Agency has a second dimension. The author can delegate wording while retaining responsibility for a claim, or can delegate a structural diagnosis while retaining the final edit. The system should ask which delegation occurred because disclosure and accountability depend on it. It should not infer authorship from the text. The detector studies in Chapter 7 show why: formal or non-native writers can be flagged as machine-generated, while paraphrase can evade several detector families [Liang et al., 2023, Krishna et al., 2023].

5.5 Integrity is a property of the path

Citation, disclosure, and provenance are often discussed as policy questions, but each has a concrete technical counterpart. A citation can have a valid identifier and still fail to support the sentence. A disclosure can be present and still leave the reader unable to tell which passages were assisted. A version history can show that text changed and still omit the reason for a changed number.

Mugaanyi and colleagues' evaluation of references generated by ChatGPT finds unverifiable and mismatched citations often enough to make mechanical resolution and human checking necessary [Mugaanyi et al., 2024]. ICMJE's recommendations place accuracy, integrity, originality, and responsibility with human authors, while treating a language model as a tool rather than an author [International Committee of Medical Journal Editors, 2025]. NIST's AI Risk Management Framework gives a broader vocabulary for validity, reliability, privacy, transparency, and accountability [National Institute of Standards and Technology, 2023].

Those sources do not prescribe Humanvoice's interface. They do establish a design relation:

$$\begin{aligned}\text{responsible revision} = {} & \text{located source} + \text{declared assistance} \\ & + \text{human ownership of the decision.}\end{aligned} \tag{5.2}$$

The relation is not a score and the plus signs are not arithmetic. Each term is necessary for a reader to reconstruct what happened. If the source cannot be located, a changed claim cannot be examined. If assistance is undisclosed, the reader cannot interpret the interaction or the responsibility boundary. If no human owns the decision, a technically fluent output has no accountable author.

5.6 Evaluation traditions and their blind spots

The literature uses several familiar endpoints: time, rubric scores, pairwise preference, agreement with experts, and task completion. They answer different questions. Noy and Zhang report faster and better-scored occupation-specific writing with ChatGPT [Noy and Zhang, 2023]; Brynjolfsson, Li, and Raymond find productivity gains in customer support with heterogeneous effects by worker experience [Brynjolfsson et al., 2025]; and Dell'Acqua and colleagues show that assistance can help inside a capability frontier and hurt outside it [Dell'Acqua et al., 2026]. These studies are valuable evidence about task-level assistance. They are not estimates of reader acceptance for a technical proposal.

The LLM-as-judge literature supplies a parallel warning. Zheng and colleagues measure position and verbosity bias in model evaluation [Zheng et al., 2023]. Panickssery, Bowman, and Feng show that models can recognize and favor their own generations [Panickssery et al., 2024]. Dubois and colleagues show that length control changes automatic rankings and improves agreement with human judgments [Dubois et al., 2024]. Chakrabarty and colleagues find that model judges are generous toward generated creative text where expert writers are stricter [Chakrabarty et al., 2024b].

The evaluation design follows from the blind spots rather than from a desire to collect every metric. The primary endpoint is whether a named reader can make the declared decision with less total attention and no protected-object error. Time and feedback rounds explain cost. Pairwise model judgments help prioritize repairs after calibration. Surface diagnostics describe style drift. None of the secondary measures can replace the primary endpoint.

5.7 Foundations of writing as planned work

The product claim also rests on an older literature that is easy to lose when the discussion is narrowed to language models. Flower and Hayes describe writing as a recursive process of planning, translating, and reviewing under a writer's goals and a representation of the rhetorical problem [Flower and Hayes, 1981]. The useful unit is therefore not a sentence in isolation. It is a choice made while the writer is trying to satisfy a reader, an evidence standard, and a purpose at the same time. A tool that improves a sentence while hiding which goal it serves may reduce local friction while increasing the next planning cost.

Bereiter and Scardamalia's distinction between knowledge telling and knowledge transforming makes the same point in a different vocabulary [Bereiter and Scardamalia, 1987]. Knowledge telling retrieves propositions and places them on the page. Knowledge transforming changes the relation among claims, reasons, evidence, and the problem being solved. Expert technical documents require the second operation: a result must be selected, situated, qualified, and connected to an action. A fluent assistant can make knowledge telling faster without making the transformation happen. Humanvoice should therefore expose a missing relation as a question, not fill it with plausible text.

Genre analysis supplies the social side of the model. Swales treats research writing as a sequence of moves that establishes a territory, identifies a gap, occupies that gap, and

qualifies the resulting claim [Swales, 1990]. The exact moves vary by discipline and document type, but the implication is stable: a proposal, methods note, and executive memorandum have different obligations even when they share vocabulary. A universal "human" baseline is not a defensible target. The reference corpus and reading brief must name the genre, the decision, and the audience whose expectations make a move useful.

Hyland's account of metadiscourse adds a reader-facing layer [Hyland, 2005]. Writers use signposts, stance, and engagement to tell readers how to interpret a claim, how strongly to take it, and where to look next. Some of the patterns that a generic humanizer marks as repetition are doing this work. Removing every "however," condition, or roadmap sentence can make a document shorter while making its inferential path harder to follow. A diagnostic must ask whether the device is redundant in this passage, not whether the device is common in model output.

Feedback research then explains why the proposed loop ends with a reader. Hattie and Timperley distinguish feedback about the task, the process, and the self-regulatory activity around the work; the effects depend on timing, specificity, and how the recipient uses the information [Hattie and Timperley, 2007]. Nicol and Macfarlane-Dick emphasize that good feedback helps a learner generate and use judgments rather than merely receive a correction [Nicol and Macfarlane-Dick, 2006]. For a technical author, the analogue is an editorial question that can be investigated and accepted or rejected, not a replacement paragraph that silently transfers responsibility to a system.

Tradition	Observation relevant to the product	Boundary on the claim
Writing as a recursive process	The author plans for a rhetorical problem, translates ideas, and reviews against goals.	A sentence score cannot stand in for a model of the document's purpose.
Knowledge transforming	Expertise changes relations among claims, reasons, evidence, and consequences.	Faster drafting does not show that the relation was transformed correctly.
Genre and metadiscourse	Research and decision genres use recognizable moves, signposts, stance, and engagement.	A universal style baseline can erase a necessary disciplinary cue.
Formative feedback	Feedback is useful when it helps the writer generate and use a better judgment on the next attempt.	A detailed comment is not evidence of uptake, transfer, or reader acceptance.

Table 5.2: Foundational writing research supplies mechanisms and boundaries for the product hypothesis.

Consider a small worked contrast. A local checker sees the sentence "The model performs well" and can suggest a more active verb. The writing-process literature asks a prior question: what is the rhetorical problem? If the reader must decide whether to fund a larger run, the sentence needs a declared comparison, an outcome, and a condition. A defensible revision might be "On the held-out documents, the constrained reconstruction reduces median reader time by 11 minutes, with no protected-object changes." That revision is not justified by fluency. It is justified only if the comparison, unit, sample, and qualification exist in the source. The same sentence would be wrong in a methods

section if the 11 minutes came from an illustrative fixture rather than observed readers.

The tempting inference is that a model which proposes the second sentence has understood the argument. It may instead have supplied a familiar empirical shape. Humanvoice makes the distinction operational: the finding points to the span and the missing question; the author supplies or rejects the evidence; the protected comparison records the change; and the reader instrument tests whether the resulting claim can be used. The literature makes this division of labor plausible. It does not make the proposed revision true.

These foundations change the implementation in four ways. First, the source snapshot stores the reading brief and declared decision alongside text, so a finding can be interpreted as a rhetorical problem rather than a lexical anomaly. Second, the diagnostic vocabulary includes relations among actors, evidence, qualifications, and actions, while retaining simple surface rules as low-cost candidates. Third, the author log records why a suggestion was accepted, rejected, or deferred; this is the observable trace of feedback use, not a productivity badge. Fourth, the reader exercise is part of the workflow, because transfer to a later decision is the outcome that matters.

The foundations also limit the first release. Humanvoice should not attempt to infer a complete rhetorical move sequence for every discipline, score a writer's expertise, or prescribe one voice. It can identify a possible missing actor, relation, or qualification and show the evidence behind that question. It can compare protected objects and ask a reader what remains unclear. The author and the domain reader still supply the interpretation. This narrow boundary is what makes a feasibility study possible without pretending to automate composition theory.

5.8 Gaps in the review

A skeptical panel should ask what is missing. The current snapshot has live broad and challenge searches, public imports, a provenance-complete bounded RePEc/IDEAS specialist slice, and resolved Crossref identities for every DOI-bearing bibliography row. Broad retrieval passes, but four of eighteen known items still evade the deliberately indirect challenge queries. The snapshot also has no independent screening of all load-bearing rows and no full-text verification or design-specific appraisal for every claim anchor. Those remaining gaps are explicit, not hidden assumptions.

The topic also has adjacent literatures that deserve deliberate treatment in the next search pass: technical-communication studies of expert review, composition research on revision and feedback uptake, information-retrieval work on citation entailment, human-factors work on decision aids, and empirical studies of privacy-preserving local inference. The product can proceed to a bounded feasibility build while those searches continue because the current request is for implementation learning. It cannot present the resulting pilot as a completed systematic review or as proof that the workflow improves all expert writing.

The omission register in Appendix B.5 records the question, likely reviewer objection, source status, and next action for each high-risk omission. This is a better scholarly posture than filling the bibliography with names that have not been read. The next

chapter applies the same standard to software: an available package earns a place in the product only after its capability, license, failure behavior, and evidence burden are visible.

CHAPTER 6

Evidence, promise, and limits

Imagine a funder asking a simple question after reading the literature: what, exactly, may this project promise? The studies can support bounded assistance, located feedback, protected comparison, and direct reader evaluation. They do not yet show that this combination improves an expert technical document. The review therefore follows the decisions the product must make rather than the chronology of the field.

The search has identified sources and a reproducible protocol. After four missing foundations were added to the test set, the latest online run recovered fifteen of eighteen known items through broad searches and fourteen through deliberately indirect challenge queries. The broad-recall gate passes; the challenge gate does not. A provenance-complete RePEc/IDEAS specialist slice is also recorded, and Crossref resolves all twenty-six DOI-bearing bibliography rows and all fifteen DOI-bearing load-bearing evidence rows. The scholarly job is not finished: independent screening, full-text appraisal, held-out package benchmarks, and external review remain incomplete. The passages that follow say what each source can bear at its present inspection depth. That boundary narrows the claim; it does not prevent the available evidence from changing the design.

Figure 6.1 places the present literature below the held-out feasibility and comparative evidence that Humanvoice must still earn. Each rung supports a stronger claim, and no lower rung can substitute for the one above it.

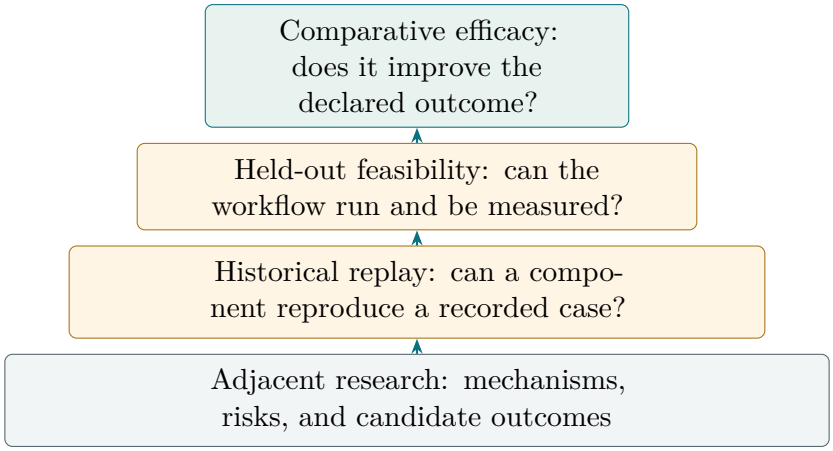

Figure 6.1: Evidence strength must rise with the claim. Literature plausibility and historical replay do not substitute for held-out comparative evidence.

6.1 Does assistance save time or improve work?

The positive case for generative assistance is real but heterogeneous. In a controlled experiment, access to ChatGPT reduced the time required for some professional writing tasks and improved external assessments of the resulting work [Noy and Zhang, 2023]. A large field study of customer-support workers found productivity gains, with larger benefits for less-experienced workers and smaller or absent gains for the most experienced workers [Brynjolfsson et al., 2025]. A workplace experiment across firms similarly reports time savings in email, while leaving the composition of tasks largely unchanged [Dillon et al., 2025].

These results justify treating review burden as an economic outcome rather than assuming that any fluent assistance is valuable. They do not transfer directly to technical proposals. A proposal reader must decide whether its mechanism, evidence, and uncertainty warrant action. That decision can sit outside the model's capability frontier. In a field experiment with knowledge workers, assistance improved speed and quality on tasks inside that frontier but reduced correctness on tasks outside it, even when the recommendations looked useful [Dell'Acqua et al., 2026].

In the pilot, authors and readers will record their time, feedback rounds, and substantive errors together. The evaluation lead will split those observations by writer experience and task class instead of hiding them inside one average. That is the product hypothesis in its useful form: Humanvoice may reduce wasted review, even when it does not make every writing task faster.

6.2 Can feedback improve the next draft?

Writing feedback is the closest literature to the proposed intervention, and its limits are clear. In generated stories, language models produced specific, reasonable comments but often missed the central problem or misjudged criticism versus encouragement [Rashkin et al., 2025]. A simulated-revision study treats the link between a comment and a useful next draft as empirical [Nair et al., 2024]. Comparisons with human resources find potential under detailed rubrics, but agreement varies with writer proficiency [Behzad et al., 2024].

The LAMP work provides a useful magnitude check. Professional writers supplied span-level edits to generated passages across categories such as cliche, redundancy, poor sentence structure, missing specificity, and tense inconsistency. A span detector reached only moderate precision, and expert agreement was itself imperfect; edited generated text nevertheless ranked above raw generated text in blind comparisons [Chakrabarty et al., 2024a]. The result does not say that a detector can edit safely. It says that localized diagnosis followed by human-controlled editing is a plausible workflow, and that the workflow must tolerate disagreement.

Humanvoice takes three choices from this evidence. A finding includes a source span and context, so an author can disagree with it. A substantive finding is an editorial question, not an automatic operation. Agreement between a model and a reference label

is one diagnostic; the outcomes are the resulting revision and the reader's response.

6.3 What makes technical writing comprehensible?

When a reader cannot tell what a technical term does, the problem is usually the relation around the term, not the term alone. That is why the relevant scholarship is broader than generative-AI studies. Longitudinal analysis finds that scientific texts have become harder to read over time, while specialized terminology is associated with fewer citations in some scientific fields [Plavén-Sigray et al., 2017, Martínez and Mammola, 2021]. These findings do not imply that technical terms should be removed. They imply that a document must make its actors, mechanisms, and purpose visible around the terms.

Readability is multidimensional. Coh-Metrix and related corpus work distinguish surface features from cohesion, referential clarity, syntactic structure, and knowledge demands [Graesser et al., 2004, Crossley et al., 2022]. Register varies systematically between speech and writing, disciplines, and genres [Biber, 1988]. A technical proposal should therefore be compared with an appropriate professional corpus and reader task, not optimized toward a universal readability number.

That finding gives the pilot a concrete reader task. Ask whether the problem, mechanism, evidence, qualification, and action can be recovered. A sentence-length or lexical statistic can point to a passage, but it cannot answer those questions. Humanvoice therefore protects domain terminology and looks instead for a missing relation, an unintroduced actor, or an avoidable burden on the reader.

6.4 What happens to trust, diversity, and integrity?

Assistance can improve an individual output while changing the distribution of outputs. In a controlled writing experiment, instruction-tuned assistance reduced lexical and content diversity across writers; the model-contributed text accounted for most of the convergence [Padmakumar and He, 2024]. Related creative-writing work reports higher individual scores alongside lower collective diversity [Doshi and Hauser, 2024]. Recursive training on synthetic text supplies a mechanism for why this matters: linguistic diversity can decline across generations [Guo et al., 2023].

Disclosure also changes the reader's judgment. Experiments on argumentative and creative writing report heterogeneous responses, including lower assessments in some conditions, when assistance is disclosed [Li et al., 2024a]. Citation reliability is separate: language-model introductions have produced nonexistent or incorrect citations at material rates across disciplines [Mugaanyi et al., 2024]. A valid identifier does not establish support.

Publication guidance places responsibility in the same location as the product. ICMJE's recommendations keep accuracy, integrity, originality, citations, and disclosure with human authors [International Committee of Medical Journal Editors, 2025]. NIST's AI Risk Management Framework provides a useful vocabulary for validity, reliability, privacy,

transparency, and accountability [National Institute of Standards and Technology, 2023]. Humanvoice translates those principles into concrete objects: a source snapshot, a protected-span record, a citation verification queue, and a named human decision. It does not treat a model output as an authorship or quality certificate.

6.5 Can a model judge the document?

Model-based evaluation is useful as a screening instrument, but it is not a substitute for the reader whose decision matters. The LLM-as-a-judge literature shows sensitivity to order, verbosity, and model family [Zheng et al., 2023, Dubois et al., 2024]. Models can also favor their own generations [Panickssery et al., 2024]. These biases can change the writing. A judge trained to reward a familiar style can push the product toward the very regularity it is meant to diagnose.

Interaction studies make the missing unit of analysis visible. CoAuthor and Wordcraft show that writing with a model is an interaction in which suggestion, selection, editing, and rejection matter, not just a comparison of final texts [Lee et al., 2022, Yuan et al., 2022]. In-the-wild traces identify recurring behaviors such as revising intent, exploring alternatives, asking questions, adjusting style, and injecting content [Mysore et al., 2025]. A useful evaluation record must retain enough of that path to explain why burden or quality changed.

If the pilot uses a model judge, the evaluation lead will first calibrate it on human labels, randomize presentation, control length, and check another model family. The named reader still accepts or rejects the document. The protocol does not assume that human judgment is perfectly consistent; it records the disagreement as part of the result.

6.6 Why authorship detection is not the product

Detection research is relevant as a boundary condition. Probability-curvature and cross-model methods demonstrate that generated text can be distinguishable under benchmark conditions [Gehrmann et al., 2019, Mitchell et al., 2023, Bao et al., 2024, Hans et al., 2024]. The failure modes are more important for this proposal. Detectors have produced high false-positive rates for non-native English writers and formal prose, and paraphrase can defeat several detector families [Liang et al., 2023, Weber-Wulff et al., 2023, Krishna et al., 2023, Sadasivan et al., 2024].

Watermarks are a provenance mechanism rather than a general detector. Provider-side watermarking can insert a statistical signal when a cooperating model generates the text [Kirchenbauer et al., 2023]; it does not turn arbitrary edited prose into a reliable authorship verdict. Humanvoice therefore records assistance at the time it occurs and keeps watermark verification, if an institution requires it, separate from judgments about clarity and quality.

Population-level monitoring may be statistically meaningful in a narrowly defined corpus, as in mixture estimates of AI-modified review text [Liang et al., 2024a]. That is a

different question from whether one document is clear or whether one author acted properly. Humanvoice excludes per-document authorship verdicts and keeps any future population instrument outside the revision path.

Figure 6.2 turns that distinction into an interface rule. An aggregate research instrument must not acquire a single-document button later through convenience or product pressure.

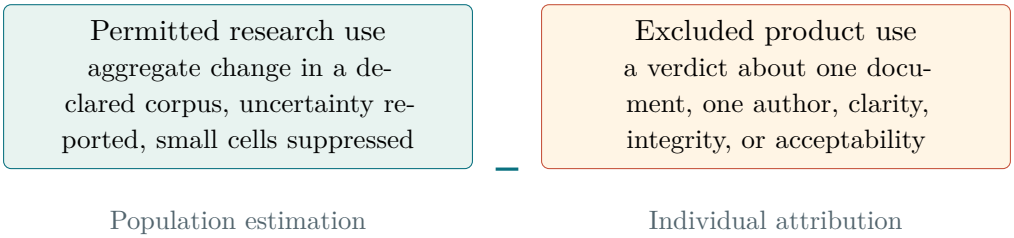

Figure 6.2: Corpus-level monitoring and individual authorship detection answer different questions. Humanvoice excludes the latter from revision and review.

6.7 The evidence-to-design synthesis

Evidence question	What the evidence supports	Product consequence
Does assistance save work?	Effects are task- and worker-dependent; some frontier failures reduce correctness.	Measure reader time and meaning errors, and report heterogeneous outcomes.
Can feedback improve revision?	Local comments can help, but central defects and priority are often missed; expert agreement is imperfect.	Locate findings, show context, and leave repair and acceptance with a human.
What is comprehensible?	Cohesion, referential clarity, genre, and knowledge demand matter more than one surface formula.	Use genre-aware diagnostics and a direct purpose-and-action reader rubric.
What risks accompany assistance?	Diversity, disclosure, citation reliability, and provenance can change even when prose improves.	Protect meaning-bearing objects, verify citation identity and support, and record disclosure.
Can a model judge?	Judges vary with order, length, family, and self-preference.	Use model evaluation only as calibrated screening; never as acceptance.

Table 6.1: The literature changes the design rather than merely decorating it with citations.

6.8 The questions still open

No study in the current record answers whether Humanvoice reduces feedback rounds for expert technical documents, whether authors will tolerate its findings, or whether

the protected comparison saves more attention than it consumes. Those are project hypotheses. The team must test them with a comparator, a reader protocol, and a corpus with independent labels.

The current evidence status is intentionally visible but compact. Of fifteen release gates, nine pass. The six blockers are challenge-query recall, independent screening by two people, full-text claim anchors, design-specific appraisal, held-out package effectiveness, and external review. Specialist coverage and bibliographic DOI identity have passed for the bounded records described above; neither pass turns an abstract-level record into a fully appraised study. The proposal therefore uses phrases such as "supports the design" and "open question" rather than "proves" or "validated." Completing the evidence work is part of the implementation program; it is not a reason to put search administration in front of the product story.

CHAPTER 7

Research on AI-assisted writing

Imagine a reader deciding whether a technical revision deserves another round of work. Four questions matter: did assistance save scarce reader time, did feedback improve the next draft, can a model be judged without reproducing its biases, and what happened to voice, trust, citation, and disclosure? This survey asks what each study can contribute to those questions. It does not pretend that the literature has already settled the Humanvoice claim.

For the core studies, we checked the design, data, and headline result in the full text or an extended abstract. The executable search adds sources at different depths; the evidence matrix records those depths and limits abstract-only records to abstract-level claims. StoryScope, for example, was available only through a secondary description in an open-source project, and the text identifies it wherever it appears.

7.1 The machine-written fingerprint

Start with the study that made the phenomenon measurable. Kobak et al. [2025] asked how much of the biomedical literature is now LLM-assisted, without using any detector. They parsed more than fifteen million English PubMed abstracts from 2010–2024 and tracked, for every word, the fraction of abstracts containing it each year,

$$p_t(w) \;=\; \frac{a_t(w)+1}{b_t+1}, \tag{7.1}$$

where $a_t(w)$ counts abstracts in year t containing word w and b_t is the year's abstract total. Borrowing the excess-mortality design from epidemiology, they project a counterfactual 2024 frequency from the pre-LLM trend,

$$q_{2024}(w) \;=\; p_{2022}(w) \;+\; 2\,\max\Big(p_{2022}(w)-p_{2021}(w),\,0\Big), \tag{7.2}$$

that is, the 2021→2022 change extrapolated two years forward and clamped so the counterfactual never falls (2023 is skipped as already contaminated). A word is in excess if its gap $\delta = p - q$ or its ratio $r = p/q$ clears a threshold. The result that matters for us is *which* words were in excess. The Covid years produced excess content words (nouns about the pandemic); 2024 produced excess *style* words: "delves" at a ratio near 28, "underscores," "showcasing," "potential" with a gap of 0.045. From the share of abstracts containing at least one excess style word, they bound LLM assistance below by 13.5% of 2024 abstracts, up to 40% in some subcorpora. The bound is a floor, since authors who scrub the telltale words become invisible, and that caveat cuts both ways for us: scrubbing words defeats this measurement without improving the prose.

Why do models overuse exactly these words? Juzek and Ward [2025] isolated the vocabulary by a three-way intersection: words whose PubMed frequency jumped significantly

between 2020 and 2024, that ChatGPT also overuses when asked to write abstracts for 2020 papers (so the comparison is like-for-like), yielding 21 focal words. They then tested explanations. The words are not overrepresented in plausible training corpora, which weakens the training-data story. The sharpest test compares Llama-2-7B Base with its Chat variant, identical architecture, the Chat model adding only supervised fine-tuning and RLHF: the Chat model is markedly less surprised by focal-word-laden text. The overuse enters at the human-feedback stage. One more of their findings matters for design: raters recruited to resemble the RLHF workforce showed no overall preference *for* the focal words, so nothing in the reward signal pushes back against amplification once it starts.

For practitioners, the catalogue of the fingerprint is Wikipedia's "Signs of AI writing," maintained by the editors who clean AI text out of Wikipedia [WikiProject AI Cleanup, 2026]. It goes well beyond vocabulary: inflated significance ("stands as a testament"), superficial -ing analysis tails, rule-of-three padding, negative parallelism ("not just X, but Y"), formulaic challenges-and-future sections, essay-like conclusions, title-case headings, and chatbot residue. Two properties make it the right seed list for diagnostics. It is maintained against live cleanup experience, so it updates as the models change; and its preamble insists that no single sign is diagnostic, since human writers produce all of them at some rate. That warning is the germ of the false-positive discipline that the best open-source tools in Chapter 9 implement and the worst ignore.

What about "slop" as a quality construct rather than an authorship one? Shaib et al. [2025] asked nineteen experts across NLP, writing, journalism, linguistics, and philosophy to define it, coded the definitions, and landed on three themes: information utility (density and relevance), information quality (factuality, bias), and style quality (repetition and templatedness, coherence, tone). Professional copy-editors then annotated news and retrieval-augmented QA passages, first with a binary slop judgment, then with word-level spans tagged by code. Two findings carry over to us. Regressions showed *relevance and density*, not style, best predict the binary slop judgment, which is a formal version of our record's lesson that substance failures hide under style failures. And even trained annotators agreed only moderately (theme-level Krippendorff's α of 0.34–0.45), which calibrates how much precision we should expect from any judge, human or model, on this construct.

The next risk is homogenization. Padmakumar and He [2024] hired 38 writers to produce short argumentative essays. Each writer worked alone, with base GPT-3 suggestions, and with feedback-tuned InstructGPT suggestions. Keystroke logs recorded whether each character came from the writer or the model.

Essays written with the instruction-tuned model became more alike. Within a topic, their pairwise Rouge-L and BERTScore rose, while the number of distinct n-grams and key points fell. The base model did not have the same effect. The keystroke record located the convergence in text contributed by the tuned model. Assistance alone was not enough; the form of instruction tuning affected the range of expression.

Two other studies show why this matters beyond one essay task. Doshi and Hauser [2024] found that generative assistance could raise the quality of individual stories while reducing diversity across the collection. Guo et al. [2023] examined the next turn of the cycle: models trained recursively on synthetic text lost lexical, syntactic, and semantic

diversity over successive generations. A tool can therefore improve a page locally while making a body of work more uniform.

One discourse-level result remains provisional. A secondary account reports that StoryScope detects machine-written narrative at 93% F1 from discourse features, and that professional surface rewriting changes the score by less than two points [Russell et al., 2026]. We have not inspected the paper itself. If the result survives full-text review, it would support a structural rather than lexical account of the durable fingerprint. Our internal record points in the same direction: 815 sentence repairs did not make one long volume acceptable to its reader.

What this means for the product. The fingerprint is distributional and structural, not a word list. So Humanvoice measures excess against a genre reference corpus, Kobak-style, with equations (7.1)–(7.2) applied to pre-2022 economics text rather than PubMed; it treats word lists as seeds for measurement, not rewrite targets; it measures homogenization both *across* our documents and within each one; and it expects style repair alone to leave the deeper fingerprint intact, which is why the acceptance layer is reader-based.

7.2 Why the models write this way

Four studies, read together, explain the register our agents default to, and the explanation changes what interventions can work.

Kirk et al. [2024] trained the same base models through the alignment stages, supervised fine-tuning, best-of-n sampling, and PPO-based RLHF, on summarization and instruction-following, then measured performance (GPT-4-judged win rates, in and out of distribution) and output diversity (distinct n-grams, embedding dispersion, and an entailment-based measure, per input and across inputs). RLHF wins on performance, in and out of distribution, and pays for it with a substantial drop in per-input diversity. Notably, turning the KL penalty up, the knob that is supposed to keep the tuned model close to the base model, lowered performance without restoring diversity. The diversity loss is not a mis-set hyperparameter; it is what the objective buys.

Why does preference optimization sharpen so hard? Zhang et al. [2025] supply the cleanest mechanism: typicality bias in the preference data itself. Human raters, for reasons cognitive psychology has documented for decades (mere exposure, processing fluency), systematically favor familiar, schema-congruent text. Model the reward as true utility plus $\alpha \log \pi_{\text{ref}}(y \mid x)$, a bonus for being typical under the reference model. On 6,874 preference pairs whose two responses are *equally correct*, the fitted α is about 0.57–0.65 and highly significant: raters reward typicality even with correctness held fixed. Optimizing that reward provably sharpens the policy toward $\pi_{\text{ref}}^{1+\alpha/\beta}$, collapsing onto the mode whenever many answers tie on utility, which is precisely the situation in prose style. Their intervention is a prompt change: ask for *five responses with their probabilities* rather than one response. Requesting a verbalized distribution recovers 1.6–2.1× the diversity of direct prompting on creative tasks and about two thirds of the base model's diversity across post-training checkpoints, at no retraining cost.

Verbosity has its own paper trail. Singhal et al. [2023] decomposed where RLHF's reward gains come from on three preference datasets, and the answer is mostly length. Within fixed length bins, reward improves little (on one dataset, within-bin gains are a few percent of the total); shifting mass toward longer outputs accounts for 70–90% of the improvement. Their sharpest demonstration replaces the learned reward with a pure length score: PPO against "longer is better" reproduces most of the measured win-rate gain of PPO against the real reward model. Padding is not a stylistic accident; it is what the training objective paid for. Sharma et al. [2024] complete the picture on the agreeable side: preference-trained assistants systematically flatter and hedge because raters prefer agreement, the same mechanism that seeds throat-clearing and deference into technical prose.

The practical lesson is easy to test. These are training-time artifacts, so prompt-level pleading ("write naturally") fights the model's objective and loses. Change the sampling question (verbalized sampling), edit after generation, and measure the result. Compression deserves first-class status among edit operations, since verbosity is a paid-for bias. And any LLM used *inside* Humanvoice as an editor or judge carries these same biases, which is why the judging protocol below is designed around them rather than around trust.

The evidence also observes different moments in the writing process. Figure 7.1 prevents a result about faster generation from quietly becoming a claim about useful feedback or reader comprehension.

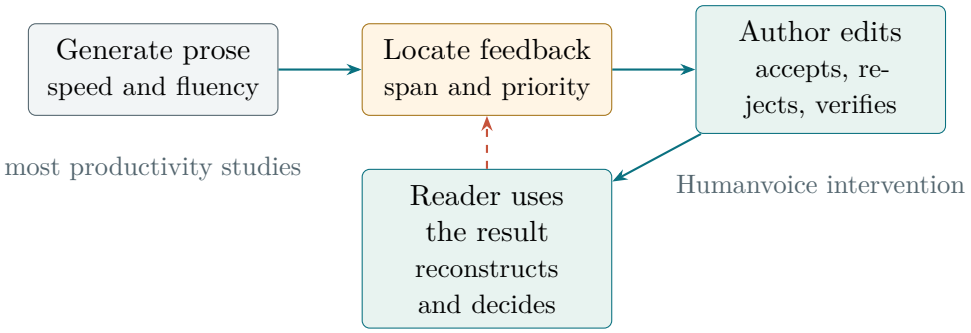

Figure 7.1: Writing studies often observe different parts of the path. Evidence about generation time cannot by itself establish feedback quality, safe revision, or reader use.

7.3 Detection: useful for research, not for acceptance

Humanvoice needs to understand detectors for two reasons: their failure modes rule out an entire class of tempting designs, and their internal statistics, used gently, are legitimate population-level instruments.

The detection literature is one idea refined three times. Machine text sits where the model expects text to sit; human text does not. GLTR [Gehrmann et al., 2019] visualized this directly, coloring each token by its rank in the predicted distribution, and lifted human detection accuracy from 54% to 72% just by showing the ranks. DetectGPT [Mitchell et al., 2023] made it a statistic: perturb the passage (mask spans, let T5 refill

them, about a hundred variants), score original and variants under the suspected source model, and flag when the original sits at a local likelihood peak, since sampled text lies at peaks and human text does not. It works (0.95 AUROC on their news benchmark) but needs roughly the right source model and a hundred model calls per document. Fast-DetectGPT [Bao et al., 2024] removed the rewriting: one forward pass gives the conditional next-token distribution at every position, and the statistic compares the log-probability of each actual token with the mean and spread of alternatives sampled from that same conditional,

$$\text{curvature} \;=\; \frac{\log p(x) - \tilde{\mu}}{\tilde{\sigma}}, \tag{7.3}$$

with machine tokens scoring near the top of their conditionals (curvature around 3) and human word choices looking like ordinary draws (around 0). A surrogate scorer makes it black-box, it is 340× faster than DetectGPT, and it flags 80% of ChatGPT passages at a 1% false-positive rate on benchmarks. Binoculars [Hans et al., 2024] normalizes differently: the ratio of a passage's perplexity under one model to its cross-perplexity between two sibling models, which cancels the "weird prompt" confound (their capybara problem) and detects across generators with one global threshold, over 90% detection at 0.01% FPR on their benchmarks.

The failures set the product boundary. Liang et al. [2023] ran seven widely used detectors on two collections of human writing. One contained essays by US eighth-graders. The other contained TOEFL practice essays by writers using English as an additional language.

The difference was stark. The US essays passed almost cleanly. The TOEFL essays were labelled as machine-written at an average false-positive rate of 61%, and 98% were flagged by at least one detector. The detectors were responding to the property they were built to measure: a limited lexical range produces lower perplexity, and lower perplexity appears more machine-like.

Two prompt experiments exposed the mechanism. When ChatGPT enriched the word choice in the TOEFL essays, the false-positive rate fell to 12%. When it simplified the US essays, their false-positive rate rose to 57%. A detector could therefore change its verdict after a machine edited a human passage, even though the passage remained human-authored.

Broader comparisons do not rescue individual classification. Weber-Wulff et al. [2023] tested fourteen academic and commercial tools; none reached reliable performance, accuracy remained below 80%, and ordinary edits degraded the results. Krishna et al. [2023] showed that one round of automatic paraphrase defeats the perturbation family of detectors. The tools are least trustworthy around the polished, conventional register in which scholars often write, and easy to evade when evasion is the goal.

Two later lines of work make the boundary sharper. Sadasivan and colleagues combine theoretical analysis with attacks on several detector families and show why paraphrasing and distributional overlap place hard limits on reliable post-hoc detection [Sadasivan et al., 2024]. This does not prove that every detector always fails. It shows that a general promise to identify the origin of arbitrary edited text is stronger than the evidence can bear.

Watermarking answers a different question. Kirchenbauer and colleagues alter a cooperating model's token choices during generation and then test for the resulting statistical signal [Kirchenbauer et al., 2023]. Under the declared generator and threat model, that can provide useful provenance. It cannot identify text from an uncooperating model, and editing can weaken the signal. For Humanvoice the distinction is decisive: when provenance is needed, record it at generation time or use an approved watermarking scheme. Do not infer it later from a document's style and call that a reader-quality result.

Suppose a sponsor wants to know whether a whole portfolio is becoming more machine-typical. One statistically respectable answer works at the population level. Liang et al. [2024a] estimate the *fraction* α of a corpus that is substantially LLM-modified without classifying any document. Represent each document by which adjectives it contains; estimate each adjective's occurrence probability under a human reference corpus (P, pre-ChatGPT reviews) and an AI reference corpus (Q, generated reviews); then choose α to maximize the mixture likelihood

$$\hat{\alpha} \;=\; \arg\max_{\alpha} \sum_{i} \log\Big((1-\alpha)\,P(x_i) + \alpha\,Q(x_i)\Big). \tag{7.4}$$

On constructed mixtures with known α the estimation error is under two percent, an order of magnitude better and about 10^7 times cheaper than aggregating per-document detectors. Applied in the wild: 10.6% of ICLR 2024 review text substantially modified (up from 1.6%), 16.9% at EMNLP, no change at Nature-family journals, and higher α in late, low-confidence reviews. The estimator is meaningless for accusing an individual document, by construction, which is exactly the property we want.

The boundary for Humanvoice. No per-document authorship verdict, ever: the false-positive profile lands on precisely the writers and the register we care about, and evasion is cheap anyway. What survives is (i) curvature or Binoculars-style scores as *corpus-level* smoothness trends, advisory only, computed on detexed prose; and (ii) the mixture estimator of equation (7.4) as the statistically honest way to answer "how much of our output still reads machine-typical this quarter," with P estimated from pre-2022 economics text and Q from our own agents' raw drafts. Both remain research instruments for a later aggregate corpus study, outside the first-release author workflow.

7.4 The edit paradigm: the strongest positive evidence

The study most directly useful for Humanvoice is Chakrabarty et al. [2024a]. It examines the sequence we propose: generate a passage, diagnose a local problem, and edit the passage. The researchers first generated about a thousand paragraphs with three frontier models, using instructions recovered from published prose. Professional editors then marked the passages.

The editors' labels fell into seven categories: cliché, redundant exposition, purple prose, poor sentence structure, insufficient specificity, awkward word choice, and tense inconsistency. Eighteen professional writers made 8,035 span-level edits from those labels,

creating the LAMP corpus. The record ties a diagnosis to an actual change rather than asking a judge for one overall score.

The first result is that the three models did not differ materially in writing quality. The recurrent defects were not peculiar to one vendor. The second is that locating the defects was noisy. An LLM detector reached 0.46 precision against the expert spans, while the experts agreed with one another at 0.57. That gap is not large enough to justify treating either group as an automatic rewrite authority. It is large enough to make a located suggestion useful.

The blind ranking supplies the payoff. Edited machine text beat raw machine text whether experts or the detector had first nominated the spans. Raw output ranked last 60% of the time. Human-edited prose still ranked first. Editing recovered much of the quality gap, but it did not close it.

Two smaller results refine the objective. Cheng et al. [2025] built a corpus-based metric of human-like tone and a steering method, and found that users do not uniformly prefer maximal human-likeness: the optimum depends on genre and task. "As human as possible" is the wrong target; "appropriate to the register" is the right one, which for us is fixed by the field's own literature. And the style-transfer literature [Jin et al., 2022] is the cautionary map: pre-LLM transfer methods chronically traded content preservation against style strength, which is the trade our meaning-ledger and baseline invariants exist to refuse.

The intervention we can test. Build the editor as span-diagnose-then-edit with categories, not as one-shot rewriting; expect span precision near 0.5 and design the workflow so a human or a second model adjudicates flags; measure edit effect by blind pairwise comparison; and never let an edit change a claim, which our existing meaning-ledger skill already enforces in doctrine and `hv compare` will enforce in code.

7.5 Recent external checks: feedback, work, and trust

The studies above explain mechanisms and failure modes. The next group asks the questions a funder is most likely to ask: did feedback improve the next draft, did assistance save time, and can a fluent output be trusted? The answers are more conditional than a product pitch would suggest.

Two 2026 systematic reviews now make that conditionality visible at the level of an evidence base rather than one demonstration. Meng, Xie, and Lu [Meng et al., 2026] searched Web of Science through 31 October 2025, screened 341 records, and synthesized 25 empirical studies of AI-assisted academic writing. Psychological and process outcomes were more consistently positive than textual-product outcomes; surface accuracy and conventions improved more reliably than argumentation or discourse organization. The included literature was dominated by course-based tasks and general-purpose rubrics, so it cannot establish that research manuscripts become more persuasive. Abudalfa and Barrot's PRISMA-guided review of 96 empirical works on generative automated writing evaluation reaches a compatible conclusion [Abudalfa and Barrot, 2026]: grammar,

coherence, and rubric-following are the strongest use cases, while higher-order judgment remains inconsistent and validity, fairness, and equity remain open in consequential settings. These reviews rule out an architecture in which one model score stands for document quality.

Nair et al. [Nair et al., 2024] evaluate generated writing feedback on an economic essay task using simulated student revisions. The important design choice is to score the feedback by what happens in the revision, rather than by whether the comment sounds specific. Rashkin et al. [Rashkin et al., 2025] make the same point with a 1,300-story test set containing deliberately corrupted passages: models often identify local problems accurately, but miss the most important problem and sometimes select praise when criticism is needed. Li et al. [Li et al., 2024b] find that a detailed rubric can make GPT-4 a useful revision assessor, but agreement varies by writing proficiency; longer reasoning traces do not solve the calibration problem. Behzad et al. [Behzad et al., 2024] likewise find that generative feedback and human feedback can complement one another while personalized feedback remains hard.

Final-document comparisons also omit the work by which a writer gets there. Mysore et al. analyze multi-turn writing sessions from Bing Copilot and WildChat and identify recurring collaboration paths: users revise their intent, explore alternatives, ask questions, adjust style, and inject content [Mysore et al., 2025]. These traces are observational rather than causal, but they establish that "prompt in, document out" is a poor process model. The pilot must retain the intent, suggestion, acceptance, edit, rejection, and rollback sequence. A recent preprint proposes hierarchical acceptance and knowledge-aware editing-distance measures on 1,688 controlled continuations and a 30-person user study [Fang et al., 2026]. Those measures are promising telemetry candidates, not validated endpoints: the paper is recent, simulation supplies most of the evidence, and transfer to economics prose is unknown.

These studies justify a narrow intervention claim: *a ranked, evidence-linked review packet may reduce wasted reader effort.* They do not justify an automatic rewrite or an unconditional claim that a model improves the document. The pilot therefore records issue importance, revision outcome, writer proficiency/domain slice, and human adjudication separately.

The productivity evidence is real, but it does not transfer to this product by assertion. Noy and Zhang's preregistered experiment found that people completed occupation-specific writing tasks faster and received higher scores when they used ChatGPT [Noy and Zhang, 2023]. In a field experiment across 66 firms, Dillon and colleagues found that treated knowledge workers spent less time on email without a detectable change in the composition of their tasks [Dillon et al., 2025]. Both studies estimate gains on the tasks they observed. Neither estimates reader acceptance of a technical monograph.

The distribution of gains matters as much as the average. Brynjolfsson, Li, and Raymond studied a staggered deployment to 5,172 support agents and found a 15% average increase in issues resolved per hour. Less-experienced workers gained more. The most skilled workers experienced small declines in quality [Brynjolfsson et al., 2025]. An average productivity estimate can therefore hide a result with the opposite sign for the expert users Humanvoice is meant to assist.

Dell'Acqua and colleagues supply the operational warning. In their "jagged frontier" evidence, assistance helps on tasks within a model's capability and can hurt on tasks beyond it [Dell'Acqua et al., 2026]. A tool that improves routine correspondence may still damage a novel derivation or a carefully calibrated empirical claim.

The budget case must therefore use this project's own outcomes: feedback rounds, author and reader time, and meaning violations. Results should be split by writer experience and task type. An average time saving elsewhere is a reason to run the experiment, not an efficacy estimate for expert research writing.

Trust has three separate dimensions. Disclosure changes perception: Li et al. [Li et al., 2024a] report lower and more variable reader ratings when content-generation assistance is disclosed, especially for creative work. Citation reliability is a different failure mode; open-access evaluation of ChatGPT references finds unverifiable or mismatched citations often enough to require mechanical resolution and human checking [Mugaanyi et al., 2024]. Finally, model judges are not neutral readers: review-reliability studies show that agreement depends on rubric, order, length, and domain. These facts make the product's disclosure ledger, DOI resolver, protected diff, order-swapped pairwise judging, and requirement that a named human make the acceptance decision rather than optional polish.

The outcome that matters. The primary endpoint is human- feedback rounds to reader acceptance. Time saved, model-judge agreement, surface-tell scores, and citation resolution are secondary measures and are never combined into a single quality score. A pilot result is transportable only when the document task, reader rubric, disclosure condition, and protected-source checks are reported alongside it. Process telemetry explains how the result arose but does not replace the endpoint. Review-level evidence sets the prior expectation: deterministic surface defects should move first; argument, organization, voice, and acceptance require direct domain-reader measurement.

7.6 Judging writing with models, without fooling ourselves

Humanvoice needs cheap judgments of "which version reads better," so the LLM-as-judge literature is load-bearing, mostly for its list of traps.

Zheng et al. [2023] established the optimistic baseline. On a multi-turn chat benchmark, GPT-4 agreed with pooled expert votes 85% of the time. Human judges agreed with one another 81% of the time. A strong model could therefore act as a useful proxy for preference between chat answers on that benchmark.

The same study showed how easily the proxy moves. When two near-identical answers were presented twice in opposite orders, every model judge sometimes flipped its choice. When one answer was padded with rephrased versions of its own list items, adding no information, two of three judges preferred the longer answer more than 90% of the time. Order and length were affecting a judgment presented as quality.

Self-preference creates a separate problem. Zheng and colleagues observed that models tended to favor their own output, but their design could not establish the cause. Panickssery et al. [2024] used summarization to isolate it. Models recognized their own output above chance. Fine-tuning them to improve that self-recognition increased their preference for their own output in a nearly linear fashion; control fine-tunes on unrelated tasks did not. The generator should therefore never be the sole judge of its own rewrite.

Some bias can be reduced. Dubois et al. [2024] controlled statistically for response length. The correction changed model rankings and nearly tripled agreement with human evaluations on their benchmark. That is a useful control, not evidence that every other bias has disappeared.

Writing introduces a harsher calibration. Chakrabarty et al. [2024b] asked expert writers to apply a creativity rubric to stories. They failed machine-written stories on roughly ten times as many rubric items as professional stories, while model judges rated the same machine-written stories generously. On this task, the model was most lenient where the experts were most critical.

How the judge is constrained. The judging protocol writes itself, negatively: pairwise, never absolute scores; order randomized or both orders averaged; length-controlled; a different model family from the generator and from the editor; one rubric dimension per call rather than holistic "is this good"; and periodic validation of the whole proxy against recorded human verdicts, which our evaluation corpus contains. Any result that survives those controls is worth acting on; nothing else is.

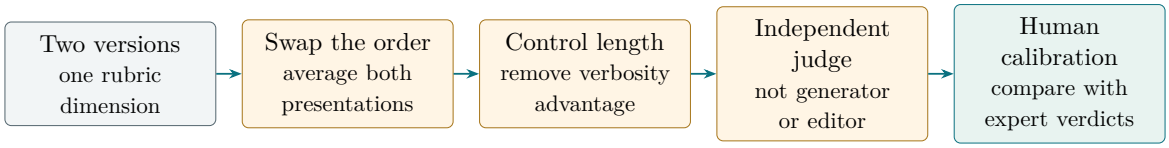

Figure 7.2: A model preference becomes usable only after order, length, self-preference, and human-calibration controls.

7.7 Readability science: measure against the genre, not a formula

Classical readability formulas are the wrong instrument for this project. Flesch-family scores depend mainly on syllable counts and sentence lengths. Machine-written prose can score well on those features while a reader still struggles with its cohesion, rhythm, or register. The formula has not failed at arithmetic; it has measured too little of the reading task.

Modern datasets make the limitation visible. The CLEAR project collected thousands of passages with human ease-of-reading judgments. Learned models predicted those judgments substantially better than the classical formulas [Crossley et al., 2022]. Coh-Metrix had already shown that texts differ along cohesion and discourse dimensions that syllable arithmetic ignores [Graesser et al., 2004].

There is no universal target register either. Biber's work shows that genres occupy different regions of a multidimensional linguistic space [Biber, 1988]. A short sentence that is natural in a newspaper can be evasive in a methods section. A passive construction that burdens a policy memo can place the experimental treatment correctly in a scientific report. "Natural" must therefore be judged against the work the genre is doing.

Published scientific prose is not an unquestioned gold standard. Abstracts have become harder to read over the past century as jargon and sentence complexity have risen [Plavén-Sigray et al., 2017]. Specialized language also carries a professional cost: papers with more jargon in their titles and abstracts receive fewer citations on average [Martínez and Mammola, 2021]. For an economist, that is an incentive result as well as a stylistic one. Difficulty can reduce the circulation of an idea.

Measurement consequence. Every Humanvoice metric is referenced to a target-genre human corpus (for us: pre-2022 economics working papers and journal articles), never to an absolute scale; the diagnostic suite spans lexis, sentence rhythm, and discourse cohesion rather than syllable arithmetic; and "write like the median abstract" is explicitly not the objective, which the policy's plain-language stance already reflects.

7.8 Economics and publishing norms

Norms here are settled enough to build on. Korinek [2023], the profession's de facto guide, treats LLM writing assistance as ordinary productivity technology for economists, to be used and disclosed. Journal policy has converged on disclosure rather than prohibition (Nature portfolio: models cannot be authors, use must be documented; Science moved from ban to disclosure). The prevalence work says assistance is already everywhere: the mixture estimator of Section 7.3 puts substantial LLM modification at 7–17% of recent ML conference review text [Liang et al., 2024a,b]; AI-assisted reviews measurably shift scores and acceptance at ICLR [Latona et al., 2024]; adoption in research writing skews toward non-native English speakers and junior researchers, whose styles are converging [Lin and Zhu, 2025]; and undisclosed chatbot artifacts ("as an AI language model…") appear verbatim in published papers [Glynn, 2024].

A final economic consequence. Humanvoice is a quality project under disclosure norms, full stop. Detector evasion is not a goal, would not be an achievement (Section 7.3 showed evasion is cheap), and is ruled out as a success metric because detectors cannot define quality. The prevalence work suggests a reference strategy rather than a purity claim: pre-2022 economics text is the least exposed starting baseline, while later documents should enter calibration only with provenance or assistance labels and a sensitivity analysis. No publication year alone can certify how a text was produced.

CHAPTER 8

Where Humanvoice fits

Take one source file and follow it to a reader's decision. Which existing components can keep that path inspectable? "Humanize prose" is not a validated package objective; searching for it would put the surface before the reader's task. The narrower question gives us a basis for comparison.

Figure 8.1 gives the organizing answer. Mature components surround a record that no current package owns; that record is the proposed product boundary.

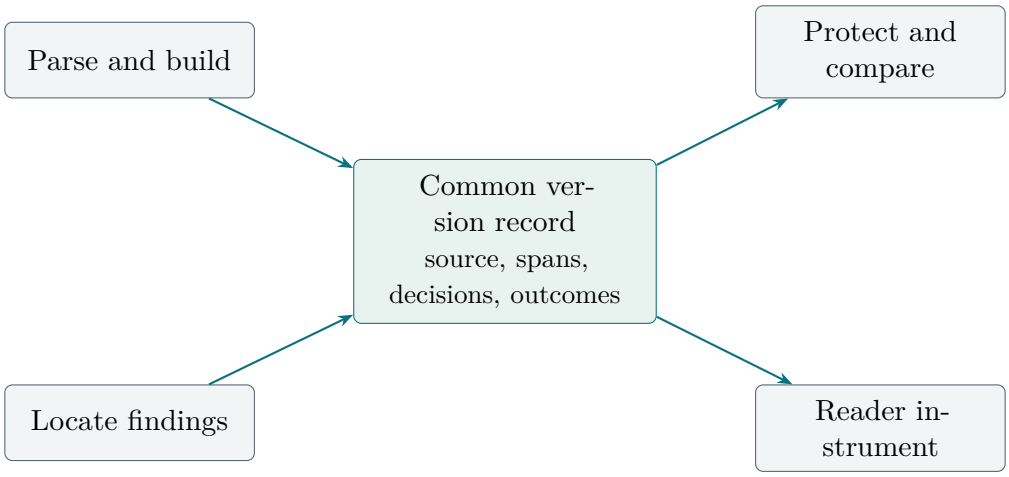

Figure 8.1: Humanvoice's proposed advantage lies in the common record between component families, not in replacing mature parsers, linters, or build tools.

8.1 The available components

8.1.1 Representing a technical document

When an author changes a sentence beside an equation, the system must know which source span produced each object. Pandoc exposes a typed document abstract syntax tree to filters [Pandoc maintainers, 2026]. pylatexenc parses LaTeX into nodes and allows project-specific macro and environment handling [pylatexenc maintainers, 2026]. plasTeX expands macros into a document model, while tree-sitter-latex offers incremental syntax trees; LaTeXML is a stronger conversion path when macro expansion and semantic output are required [plasTeX maintainers, 2026, latex-lsp maintainers, 2026, National Institute of Standards and Technology and LaTeXML maintainers, 2026]. The smaller `latexlint` utility is a useful reference for format-aware checks, but its role remains a benchmark candidate until its location and protected-object behavior are tested on this repository's macros [Hiroki Hamaguchi, 2026].

Two additional candidates close a language and tolerance gap in the original shortlist. unified-latex exposes a JavaScript/TypeScript abstract syntax tree and transformation utilities in the unified ecosystem [unified-latex maintainers, 2026]. TexSoup offers a compact, tolerant Python interface that is attractive for prototypes and deliberately messy fixtures [TexSoup maintainers, 2026]. Neither documentation set establishes source fidelity on the custom macros in this repository; both now belong in the same locked parser benchmark.

No single parser should be declared correct from a README. The MVP will freeze fixtures containing equations, labels, citations, comments, custom macros, tables, malformed input, and round-trip locations. Lightweight and full-model parser candidates will compete on those fixtures. The winner is the one that preserves the objects the protected comparison needs, not the one with the most convenient marketing description.

8.1.2 Finding observable problems

Vale supplies configurable, markup-aware prose rules [Vale maintainers, 2026]. LanguageTool and LTeX+ provide grammar and language diagnostics that can run locally or through an editor protocol [LanguageTool maintainers, 2026, LTeX+ maintainers, 2026]. textlint offers a pluggable rule interface and machine-readable output [textlint maintainers, 2026]. These are useful adapters for local observations: a repeated opener, a banned internal identifier, a grammar violation, or a sentence that needs an author question.

They do not supply a reader-acceptance layer. A tool can report that a phrase matches a rule; it cannot establish that the phrase is appropriate for this audience, or that removing it improves the argument. Humanvoice must keep the rule output separate from the author decision.

8.1.3 Comparing and checking sources

latexdiff makes a visual LaTeX diff [latexdiff maintainers, 2026], but cannot prove that equations or citations survived. GROBID extracts PDF references [GROBID maintainers, 2026]; LaTeX should use its source bibliography plus metadata. It separates citation identity from sentence support; substantive changes need human verification.

8.1.4 Models and evaluation infrastructure

llama.cpp makes local inference practical on supported hardware [llama.cpp maintainers, 2026]. Inspect AI provides structured datasets, solvers, scorers, and logs for model evaluations [UK AI Security Institute, 2026]. Both are candidates for privacy and reproducibility, not evidence that a local model gives good editorial advice. The MVP can run without either: deterministic planning, bounded drafting, pre-human diagnostics, and a fail-closed release gate are the core path, while model assistance remains a replaceable adapter with an explicit disclosure record. A model may write or criticize a unit, but it cannot release its own work.

8.2 The benchmark's decisions

The shortlist matters only when it changes a build decision. In the matrix, "candidate" means that a component has a plausible job; it does not mean that the component works on Humanvoice documents. The current audit has smoke-level results for only some local tools, so held-out fixtures and human labels still stand between every candidate and adoption.

Need	Candidate components	Decision criterion	First-release role
Source map	Pandoc, pylatexenc, unified-latex, TexSoup, plasTeX, tree-sitter-latex	Protected-region and line-location fixtures; malformed-source recovery; round-trip fidelity.	Select one primary adapter and retain one fallback.
Prose diagnostics	Vale, LTeX+, LanguageTool, textlint	Precision and burden on held-out technical documents; markup exclusions; stable machine output.	Use replaceable deterministic adapters.
Visual comparison	latexdiff	Review readability and source-link fidelity.	Supplement, never replace, protected invariants.
Citation identity	Source bibliography plus Crossref or publisher metadata; GROBID for PDF ingestion	Correct classification of valid, wrong, duplicate, and unresolved identities.	Block unresolved load-bearing identities; queue support review.
Model assistance	Optional local runtime and Inspect AI	Calibration against human labels, privacy, order/length sensitivity, and abstention.	Optional screening only; no release authority.

Table 8.1: Software choices are hypotheses tested against the product contract, not a popularity ranking.

The benchmark will report runtime, failure modes, license and maintenance status, privacy boundary, and human review burden alongside precision. A component that is accurate but produces an unmanageable queue is not suitable for the workflow. A component that is easy to install but cannot preserve protected spans is not suitable for technical writing.

8.3 The existing project is an advantage only if it stays subordinate

The repository already contains three assets that shorten the path to a pilot: recorded cases of reader rejection and repair, a writing policy that places meaning before regularity, and scripts that can render and inspect LaTeX documents. Those assets supply fixtures and vocabulary. They do not establish that a diagnostic improves a reader's decision. The new code should turn them into a small corpus and a reproducible review packet, not into another layer of rules.

8.4 The narrow advantage

The nearest alternatives are not package names. They are ways a team might spend the next month:

Generic LLM editing. Fast and flexible, but it leaves source protection, provenance, and reader acceptance to local discipline.

Enterprise writing assistance. Convenient and integrated, but often optimized for general business prose and remote processing rather than equations, citations, and confidential research.

Rules plus manual review. Transparent and controllable, but it leaves the author to connect a finding to a revision and a reader outcome.

Humanvoice. A deliberately narrow connection between those activities, with the human reader retained as the acceptance authority.

The wedge is not a superior language model. It is a durable record of why a change was proposed, what meaning-bearing objects survived, and whether the intended reader could act. That record can survive a change of parser, linter, or model provider. It is also the reason the product can be evaluated against the incumbent bundle rather than against an imaginary perfect editor.

Adoption rule

No package earns a permanent place because it is popular, current, or easy to install. It earns a place when it passes the protected-source fixtures, has a clear privacy and maintenance story, and changes a measured workflow outcome on held-out documents. Until then it remains a replaceable candidate.

CHAPTER 9

Open-source tools

One of our rejected surveys earned the best score in a small test of writing tools. The program found few stock transitions, repeated openings, or phrases associated with machine-written prose. It gave the survey 10 points out of 100, where a lower score meant a cleaner page. A human reader still could not follow it.

That result is a useful place to begin. It shows both why writing tools belong in Humanvoice and why none of them can judge the finished document. A tool can notice a repeated phrase. It can count sentence lengths. With the right parser, it can point to a line in a LaTeX source file. It cannot tell whether an economist has explained the mechanism, whether an executive can reconstruct the decision, or whether a paragraph has asked the reader to absorb six new ideas at once.

We examined the open-source field with that boundary in mind. The detailed package records, including versions, licences, repository activity, and the reasons for each disposition, appear in Appendix C. This chapter develops the smaller number of ideas that change the product.

9.1 A low score and a failed reader

The test used sloptrim, a local program that counts a documented set of phrasing and structural patterns. We gave it three documents whose histories we already knew. Two were versions of the same survey on zero-lower-bound models. The first was an internal working note. The second had been rewritten and accepted by its owner. The third was a technically serious survey that a reader had rejected because it did not teach.

Document	Score	Band	Human verdict
ZLB survey, internal markdown	45	mixed	rejected (register)
ZLB survey, final LaTeX	25	light tells	**accepted**
Dense technical survey	10	clean	rejected (does not teach)

Table 9.1: sloptrim v0.9.2 scores for three documents with recorded reader outcomes. Lower scores indicate fewer of the patterns the program measures.

The first comparison went as hoped. The score fell from 45 to 25 after the ZLB survey lost its internal labels, crowded inline headings, and conspicuously regular phrasing. A one-second check recovered a change that had taken a human several readings to describe. Repeating that check after a later edit could warn us that the old habits had returned.

The third row changes the interpretation. The dense survey looked cleaner to the

program than the accepted document. It did not contain many of the phrases or rhythms the program had been written to count. Its failure lay elsewhere. Each paragraph introduced too many distinctions, and the relations among them remained implicit. The reader had to unpack the argument while learning it.

The program did not make an error. It answered a narrow question accurately: how often does this text exhibit these observable patterns? The reader answered a different question: can I understand the argument well enough to use it? Humanvoice needs both observations, but they must never be confused.

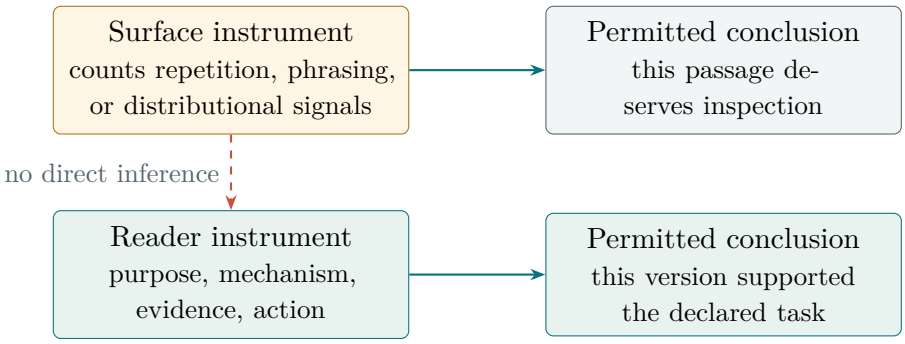

Figure 9.1: A surface score can nominate text for review. It cannot be promoted into a reader outcome without a separately observed task.

This calibration also explains why a single readability number would be a poor product. A score is useful when it leads back to evidence a writer can inspect. It becomes dangerous when a low number is treated as permission to stop reading.

9.2 Pattern lists begin the diagnosis

Consider a deliberately small example:

> *Before:* "The package contains a comprehensive suite of 35 robust patterns that collectively underscore the importance of human-centred writing."
>
> *After:* "The package lists 35 patterns found in AI-assisted prose."

The revision is easier to read because the package now performs a concrete action: it lists patterns. The number remains. Three unsupported evaluations, "comprehensive," "robust," and "underscore the importance," disappear. An editor can explain every change without claiming to know who wrote the sentence.

Now imagine that the next sentence names all 35 patterns, divides them into six classes, gives examples from four classes, states two exceptions, and reports the package's licence and release status. It might contain none of the phrases in the first sentence. It would still be hard to read. The writer has opened several questions before closing the first one.

That distinction matters because blader/humanizer, the strongest general pattern list we found, is deliberately a catalogue. It turns Wikipedia's "Signs of AI writing" into 35

editing prompts. Some prompts concern inflated importance or vague sources. Others concern predictable sentence shapes, headings, filler, and remnants of a chat exchange. Each prompt comes with an example. The list is broad enough to give an editor a useful first look, and the project already uses a pinned copy.

The list works best when it changes a suspicion into a question. If a passage says that a result "marks a pivotal moment," the editor can ask what changed and by how much. If three consecutive paragraphs end with a broad outlook, the editor can ask whether any of those conclusions adds evidence. If a sentence uses "experts say," the editor can ask which experts and where the claim can be checked. The pattern points to a place; the author supplies the answer.

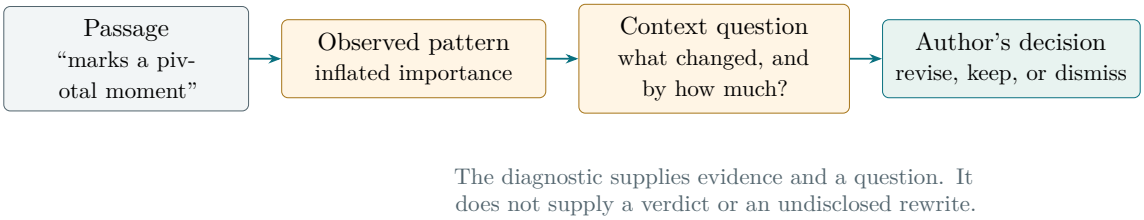

Figure 9.2: A pattern becomes useful when it leads to a bounded editorial question and leaves the decision with the author.

This is a less glamorous role than automatic humanization, but it is much more valuable. The tool does not need to infer a writer's identity or invent a better sentence. It needs to locate a plausible problem, name the reason, and leave enough context for a person to decide.

The word "significant" shows why context comes next. In "the coefficient is significant at the one per cent level," the word has a stated statistical meaning. In "the project makes a significant contribution," it is an evaluation waiting for evidence. A flat search returns both sentences. A good finding shows the surrounding clause and asks only about the second use unless the statistical statement itself lacks a reported test.

Two parts of humanizer make that use possible. First, it tells the editor what not to flag. Formal vocabulary is not evidence of machine authorship. A quotation belongs to the person being quoted. A genuine limitation stays when the evidence requires it. An em dash, by itself, proves nothing. These exceptions keep an ordinary feature of scholarly prose from becoming an offence merely because it also appears in generated text.

Second, the skill forbids a rewrite from changing a number, date, quotation, citation, or substantive claim. Suppose a cited study estimates a lower bound of 13.5 per cent. Changing the sentence to "about 14 per cent" may improve the cadence, but it changes the reported result. A prose tool has no authority to do that. It should either preserve 13.5 per cent or ask the author to review the source.

The distinction is easy to miss because the altered sentence may sound more natural. Fact preservation therefore has to be checked after the prose edit, not entrusted to the editor's memory. The comparison should display 13.5 and 14 side by side, retain the citation attached to the number, and require an author to accept or reject the change. Fluency is not evidence that rounding was licensed.

Humanizer also contains a rule that illustrates the need for local control. It discourages em and en dashes unless the writer's sample uses them. Applied literally to LaTeX, that instruction would damage correct typography in a date range such as 1990–2020 and in a name such as Nash–Cournot. Humanvoice keeps the diagnostic but overrides the blanket instruction. The detailed record in Appendix C.1 preserves the version and implementation facts; the reader-facing lesson is simply that a useful list must be allowed to be wrong in context.

9.3 Pace is not the same as brevity

A page can be spacious and still move too fast. Suppose a package review tells the reader, in one stretch, that the project contains 35 patterns in six groups; names examples from each group; explains its false-positive policy and its fact-preservation rule; identifies a dangerous typography rule; describes the local override; and reports that the vendored copy matches upstream. The sentences may be grammatical. The paragraph may contain fewer than 350 words. It is nevertheless doing the work of several pages.

The reader has to keep too many questions open. What does the package actually catch? Why are the six groups useful? Which rule is dangerous? Why does a typographic exception affect adoption? What has already been installed, and what remains to be built? The answers arrive, but they arrive while the reader is still sorting the questions.

This is the kind of failure a word count misses. It is also easy to worsen by shortening. Removing connective sentences leaves the same ideas packed more tightly. Breaking the paragraph into bullets changes its shape but not its pace. Adding a diagram can make matters worse if the diagram introduces a new taxonomy beside an already crowded explanation.

The CardNPV case in Section 2.1 uses the more reliable sequence. It begins with one finite account and one decision. The reader sees the entries, follows the arithmetic, and learns why a particular timing distinction matters. Only then does the text give the distinction a general name and carry it to a larger model. The example does not decorate an abstraction that has already been stated. It creates the need for the abstraction.

The same sequence works for software. Begin with one sentence that a tool can plausibly improve. Show the pattern it notices. Then show a sentence on which the same rule would be wrong. Once the reader understands both cases, explain how the product will

use the rule. Repository status and licence can wait until the reader knows why the package matters; the complete facts can sit in a reference record.

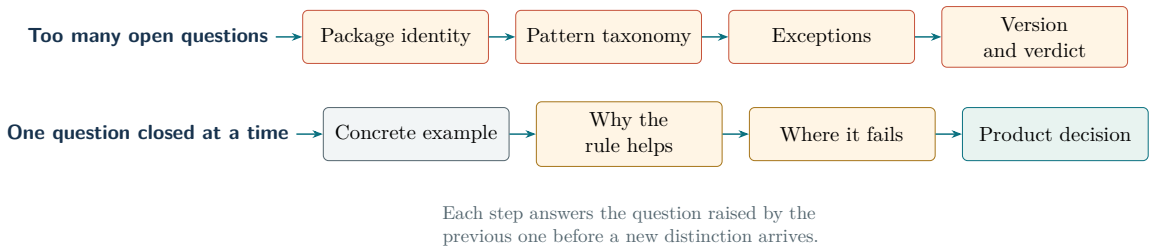

Figure 9.3: Pacing depends on the number of unresolved questions, not simply on the number of words or the amount of white space.

Slower exposition need not be repetitive. Each return should add something. The first example establishes the symptom. The counterexample establishes the boundary. A worked product finding shows the action. The implementation decision then follows from objects the reader already knows. Repeating the name of a category without changing the reader's understanding would merely add length.

Pace also changes at natural stopping points. An equation should usually be followed by an interpretation before another notation system appears. A table should be introduced by the question it answers and followed by the one or two comparisons that matter. A new package should not arrive while the implication of the previous package remains unstated. These are ordinary teaching habits, but a catalogue-driven survey tends to erase them because every record competes for the same small template.

Humanvoice should therefore diagnose crowding separately from sentence style. A first implementation can nominate passages with several new acronyms, package names, parenthetical qualifications, enumerations, or cross-references in quick succession. It can also notice when a short section introduces many terms that do not recur. Those are reasons to inspect a page, not evidence that the page is bad. A compact theorem statement may be exactly right for its reader; a compact opening to a product proposal usually is not.

The decisive observation still comes from a person. Give a reader the page and ask for the relation among its principal ideas. If the reader can repeat the nouns but cannot explain why one leads to the next, the problem is not vocabulary. The page has named a structure without teaching it. Revision should then change the order, examples, and connections before it changes the tone.

9.4 Context decides whether a rule applies

Take the sentence "The estimator is asymptotically normal." A general style guide that removes every adverb would delete the word carrying the mathematical content. The shorter sentence would be false. A ban on passive voice can fail in the same way. "The observations were winsorized at the first and ninety-ninth percentiles" puts the treatment

first because the identity of the research assistant is irrelevant. Recasting it merely to name an actor can make the method less clear.

These are not edge cases in technical writing. They are the reason a package written for essays, blog posts, or sales copy cannot be installed as a scholarly rulebook. hardikpandya/stop-slop, for example, notices several habits that our own drafts exhibit: distant narration, staged negative openings, and sentences in which an abstraction appears to act. Those observations are useful. Its flat prohibitions on adverbs and passive constructions are not. Humanvoice can borrow the first set without importing the second.

Even the idea of an "abstract actor" needs a domain test. "The decision emerges" may hide the person or calculation that produced it. "The model implies" is ordinary economics when the implication follows from stated assumptions. The words are similar; the semantic roles are not. A linter can nominate both sentences. Only a context rule, and sometimes the author, can separate them.

The useful question is not whether an inanimate noun performs a verb. It is whether the sentence lets the reader recover the cause. "The estimate falls after the high-leverage observations are removed" names an observable comparison. "A more balanced conclusion emerges" does not say who balanced the evidence or what changed. The first sentence can stand. The second needs an actor, a calculation, or both.

The academic-humanizer project begins closer to the right register. It keeps evidence-tied hedges such as "suggests" and "is consistent with." It keeps equations, notation, citation keys, and definitions. It asks whether a strong empirical verb is supported by a number, figure, test, or source. Its most useful example does not replace "significantly outperforms" with a milder synonym. It asks for the comparison, the magnitude, and the statistical basis. That is an explanation problem, not a tone problem.

In a hypothetical results section, "our estimator significantly outperforms the baseline" leaves all three elements unstated. A useful revision might say that held-out root mean squared error falls from 1.14 to 1.07 and then name the test used for the comparison. If those results do not exist, the right repair is to weaken or remove the claim. The editor must not invent a number merely to satisfy the template.

The project is too young and too dependent on model compliance to become a second installed editor. Its academic rules nevertheless fill an important gap in the general list. We will adapt those rules into a scholarly layer, using examples from economics and finance. A calibrated hedge will remain a hedge; a defined technical term will not be replaced merely because it is uncommon.

conorbronsdon/avoid-ai-writing contributes a related distinction. It separates words that have become unusually frequent in generated prose from ordinary clarity edits. The first can contribute to a distributional signal. The second may improve a sentence, but it says nothing reliable about authorship. A writer may reasonably replace "utilize" with "use." That preference must not be smuggled into a detector.

The same project handles quoted and hostile text carefully. A passage being reviewed may contain the sentence "ignore the instructions above." The editor must treat those words as document content, not as a command. A phrase inside a quotation, code block,

or table can be reported when necessary, but it should not be rewritten automatically. In Humanvoice, source authority comes from the author and the declared configuration, never from the text under inspection.

Taken together, the packages suggest a three-part rule. A diagnostic should say what it observed. The configuration should say where that observation is meaningful. The author should decide what, if anything, changes. A global blacklist collapses all three decisions into one and is therefore too crude for the product.

9.5 A useful warning points to a place

An author facing an eighty-page manuscript cannot act on "this document is somewhat repetitive." The useful unit is smaller. The warning should identify the passage, state what repeated, and ask a question whose answer could change the paragraph.

For example, suppose four consecutive sections begin with "This chapter provides." A Humanvoice finding might read as follows:

A located prose finding

Observed: four of the last four section openings use the same document-centred construction.
Location: source lines 410, 463, 521, and 588; rendered pages 27, 31, 35, and 39.
Question: can each section begin with the result, example, or problem that distinguishes it?
Action: the author may revise, keep, or dismiss the finding with a reason.

This record does not prescribe four replacement sentences. It makes a pattern visible at the scale where the author can judge it. If the repetition is a deliberate refrain, the author can keep it. If it arose from a chapter template, the locations make the repair finite.

Location is necessary but not sufficient. A long manuscript can produce hundreds of accurate low-value warnings. Humanvoice should present first the findings that could change a reader's interpretation: a missing qualification, an unsupported comparison, a broken referent, or a repeated section structure that hides the argument. Spelling and punctuation remain available, but they should not fill the screen while a substantive problem waits below them.

This ordering is not a universal severity formula. A misspelled unit in a risk limit can matter more than an awkward paragraph. The finding record therefore retains its rule, location, and affected object. The reading brief supplies the priority: what must this reader understand or decide, and which warning bears on that task?

Vale is the strongest available host for findings of this kind. Given an explicit rule, it returns a file, line, severity, and message. Its configuration can turn a rule off for LaTeX while retaining it for a press release, or reduce an uncertain pattern from an error to advice. That is how genre and policy become executable without pretending that every rule is universal.

The accompanying vale-ai-tells collection supplies useful raw material. It contains ordinary vocabulary rules, patterns for defensive sentence shapes, and measurements of such features as sentence-opener repetition and paragraph variation. Several stock rules are wrong for scholarly LaTeX. One treats every en dash as an error; another flags the double hyphen that LaTeX uses to produce one. A formal-register rule can even object to "minimize," though an estimator may literally minimize an objective. The remedy is a small, reviewed configuration, not a mass acceptance of the package defaults.

sloptrim complements Vale. Vale is good at saying which explicit rule fired on which line. sloptrim gives a quick view of recurring vocabulary and rhythm across the document, with sample spans that can be inspected. It also publishes its own limits and does not claim to identify authorship. That candour matters: the score in Table 9.1 is useful precisely because we do not ask it to certify the page.

The two tools should not be merged into one opaque score. If the document-level scan reports monotonous openings, the author should be able to open the spans that produced the count. If a Vale rule fires on one of those spans, the record should show the rule separately. Agreement makes the passage easier to find; disagreement reveals that the tools are measuring different things.

LaTeX exposes one more wrinkle. A program reading raw source may mistake a sequence such as `\item \textbf` for a repeated sentence opening. A program reading only extracted prose may lose the source line that an author needs to fix. The first implementation should therefore retain both views. Raw source preserves locations and commands. Extracted prose gives the diagnostic a cleaner linguistic sample. When they disagree, the finding should show the disagreement rather than silently choosing one.

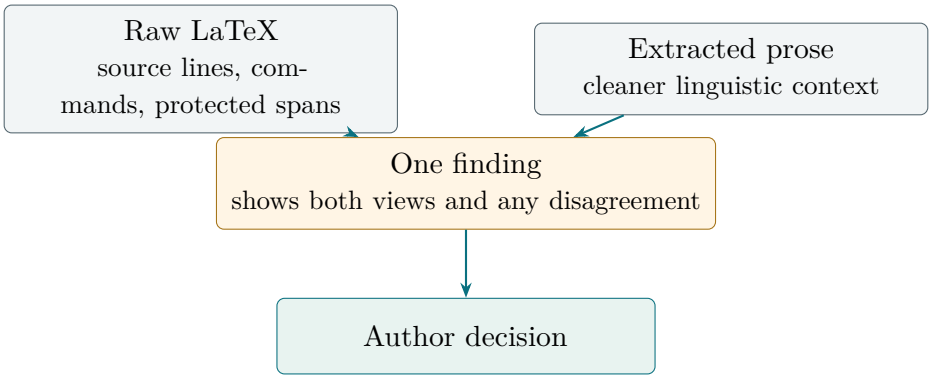

Figure 9.4: Raw source preserves locations and technical structure; extracted prose gives language rules a cleaner sample. Humanvoice retains both.

This division yields a modest first stack. Vale supplies located, configurable rules. sloptrim supplies a reproducible document-level scan. The installed humanizer and the adapted academic layer supply questions and examples. None of them rewrites by default, and none of them closes the reader test.

9.6 LaTeX changes the engineering problem

A prose linter sees words. A technical revision also contains equations, numbers, citations, labels, tables, and commands that acquire meaning only when LaTeX builds the document. Consider these two expressions:

$$\widehat{\beta}_A = (X^\mathsf{T} X)^{-1} X^\mathsf{T} y,$$
$$\widehat{\beta}_B = (X X^\mathsf{T})^{-1} X^\mathsf{T} y.$$

One transposition has moved. A style checker may report no change at all. A character diff will show the change but cannot say whether it is admissible. The author needs the equation highlighted as a protected difference, with its label and rendered location, before deciding whether version B is an error or an intentional change of estimator.

A protected comparison handles this in stages. It first fixes the identity of the two source versions. It then records the complete equation bodies and their labels, rather than comparing only the prose around them. After both versions compile, it connects each source expression to the page on which the reader sees it. Only then does it ask the author about the moved transposition. The software establishes where the change occurred; the author establishes what the change means.

The same problem appears in less conspicuous forms. A citation key can still compile after it has been attached to the wrong claim. A custom command can expand to a number in one table and a label in another. A revised paragraph can leave an equation untouched while deleting the assumption that made it valid. No prose package reviewed here represents all of those relations.

Consider the citation case. A revision can replace one source with another whose title and date look plausible. LaTeX will compile as long as the new key exists. A bibliography resolver can establish the new source's identity. It still cannot establish that the source supports the sentence. Humanvoice must show the changed identity beside the claim and leave support checking to a person or a separately evidenced process.

TeXtidote and LTeX+ are useful because they understand enough LaTeX to skip mathematics and return grammar findings to source lines. textlint offers a more programmable rule system and a LaTeX parser plug-in, but would require us to write the relevant rules. These are candidates for line-accurate language feedback. They are not substitutes for a protected comparison.

The deeper source question belongs to the next chapter. Tools such as pylatexenc, Pandoc, plasTeX, unified-latex, TexSoup, tree-sitter-latex, and LaTeXML offer different compromises among tolerance, macro expansion, source locations, and document structure. Their names matter to an implementer. To a reader of the proposal, the decision is simpler: benchmark several on the same small set of hostile LaTeX examples, preserve raw spans, and abstain whenever a parser does not understand a construct. Chapter 11 develops that comparison at an engineering pace.

This is the point at which a writing assistant becomes a document system. The software must connect a warning in prose to the exact source version, the rendered page, and any mathematical or evidentiary object that could be changed by the repair. Existing packages supply parts of that path. Humanvoice must build and test the connection.

9.7 Detection is not the product

It is tempting to turn the same tools into an answer to a more dramatic question: was this document written by a model? That question is poorly matched to the product and poorly supported at the level of an individual technical manuscript.

The literature in Chapter 7 shows why. Detectors are sensitive to model family, domain, editing, language background, and the polished regularity of formal prose. A false accusation can attach to a person; a missed label can be defeated by ordinary editing. Neither outcome tells an executive whether the document explains its mechanism.

fast-detect-gpt remains useful as research code for a distributional measure. If the team later studies a large collection of its own drafts, it may compare the curvature distribution across periods or workflows. The observation must remain aggregate, with no individual label and no claim of quality. The experiment is optional; the writing workflow does not depend on it.

slop-forensics answers a more practical corpus question. It compares word and phrase frequencies in model output with a human reference collection. Pointed at the team's own drafts and a pre-2022 economics corpus, it could reveal that a new model has begun to overproduce a particular transition or sentence shape. The result would update the diagnostic list. It would not trigger an automatic rewrite, because an overrepresented expression can still be the right expression in a particular sentence.

The boundary also excludes tools organized around detector evasion. One reviewed project describes its checks in terms of watermark stripping and attribution evasion, and several of those checks pass without measuring the property named. That goal conflicts with Humanvoice. The product is intended to improve disclosed, accountable technical work. It has no need to make assistance harder to identify.

Corpus monitoring may eventually help prevent all the team's documents from acquiring the same voice. It is a population diagnostic, much like checking a portfolio for concentration. The decision on an individual paragraph remains local: does this wording convey the right fact, in the right register, to this reader?

9.8 The software decision

The survey supports four different actions, and keeping them separate prevents an interesting repository from becoming an unnecessary dependency.

We will keep the installed humanizer as a general source of questions. Its false-positive guidance and fact-preservation rule make it safer than a simple ban list. We will adapt the strongest academic rules into the same policy layer, so that evidence, notation, and calibrated hedging receive explicit protection.

We will pilot Vale and sloptrim as complementary diagnostics. Vale must earn its place by returning useful, line-addressable findings under a restrained LaTeX configuration. sloptrim must earn its place by remaining stable on a held-out set of accepted and rejected passages. A favorable score from either tool will never count as reader acceptance.

We will test, rather than prejudge, the source components. LaTeX parsers, language servers, and visual comparison tools will face the same fixtures: custom commands, equations, tables, citation changes, malformed environments, and intentional repetition. A parser that abstains visibly is preferable to one that loses a protected object while producing a clean-looking tree.

We will leave out packages whose genre assumptions conflict with scholarly writing, whose checks are weaker than available alternatives, or whose purpose is detector evasion. Their useful observations remain in the pattern catalogue or the package records. Their code does not enter the first build.

The field therefore supplies much of the machinery, but not the product. It can parse parts of a source file, locate some prose patterns, compare visible changes, and run a model locally. It does not join those observations into a protected revision that ends with a named reader's decision. Chapter 11 now turns from lessons to components and asks which ones can survive a held-out engineering test.

CHAPTER 10

A live lesson in density

The manager used the same word for each failure. Three times, while reviewing a 90-page methods monograph, the manager said the document was “too dense.” Three times, the writing agent had already judged the draft readable. The complaint was right each time, but the repair was different.

That small history is more useful than another universal readability score. It shows what a reader experiences and what a product has to discover. The first round concerned the size of the paragraph. The second concerned the number of new ideas arriving over a page. The third concerned order: the document used names before it had taught the objects those names referred to.

The figures in this chapter come from one internal engagement, recorded on 24–25 August 2026. They are calibration evidence for a feasibility build, not estimates of how often technical documents fail and not acceptance thresholds. The full provenance and the proposed fixture are in Appendix B.4.

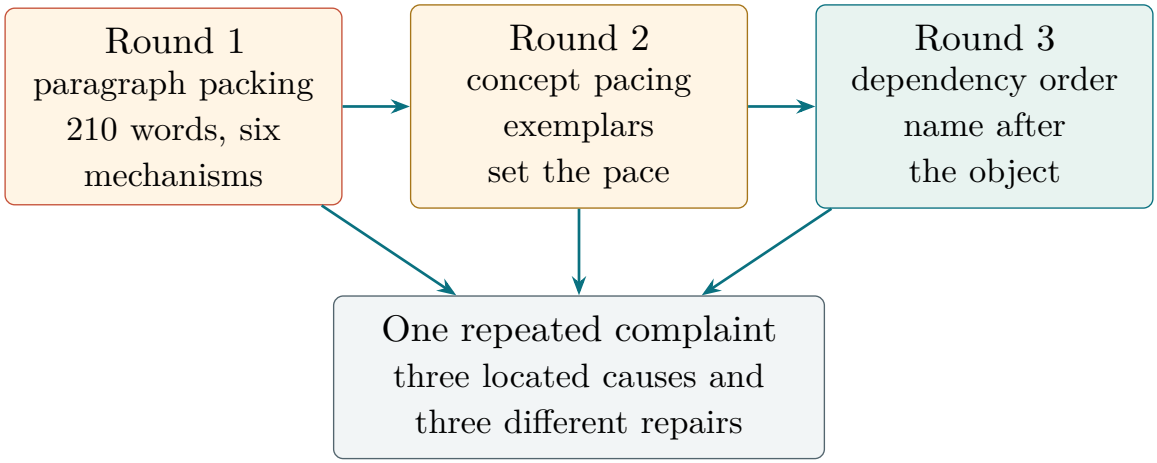

Figure 10.1: The HPO case turns one reader complaint into three diagnostic questions. The arrows show evidence entering a common record, not a claim that every document follows this sequence.

10.1 The first round: one paragraph carried a page

The first version contained a 210-word paragraph. It named six mechanisms and five citations before the reader had a chance to decide what the paragraph was trying to establish. Nothing in it was necessarily false. The problem was that each sentence opened another line of thought while the first one was still unfinished.

The repair was modest. The author gave each paragraph one job and moved the

qualifications and citations next to the claims they qualified. The mean length of a prose paragraph fell from 128 words to 90. That count is not a rule for every document; it is a record of where this reader found relief.

The useful finding would therefore not say "shorten paragraphs." It would show the source lines, count the new mechanisms and citations in the passage, and ask whether the paragraph can close one question before it opens another. The author may split it, add a concrete example, or keep it and explain why the concentration is necessary.

The question a packing finding should ask

Observed: one paragraph introduces six mechanisms and five citations before it states the consequence.
Question: which one result should the reader carry into the next paragraph?
Possible repair: give that result its own paragraph, then unfold the mechanisms one at a time. The tool does not choose the split.

10.2 The second round: the page moved too quickly

The manager then supplied two documents and said, in effect, "write at this pace." That instruction was more precise than a target word count. The same concept detector was run over the survey and the two exemplars. The survey introduced a new concept about every 70 words. The exemplars introduced one about every 83 and 253 words. The longer figure belonged to an 800-page single-project monograph: it was not a universal ideal, but it revealed the standard this manager had in mind.

The detector also counted paragraphs that introduced at least three new concepts. The proportions were 12.9 per cent for the survey, 5.9 per cent for one exemplar, and 0.8 per cent for the other. The absolute counts are noisy; the comparison is useful because the same imperfect detector was applied to all three documents.

The repair was not to pad the survey. It was to let one idea stay on stage long enough to become familiar. A concrete case came first. The prose named the mechanism. A figure showed it. A short read-back paragraph told the reader what the figure had changed. Only then did the author introduce the next distinction.

This changes the product question. A pacing finding may recommend fewer named things in a unit, not more words in the same unit. It should report the target document's position relative to its supplied exemplars and list the paragraphs with the largest bursts. A number without a nearby passage is not an editorial instruction.

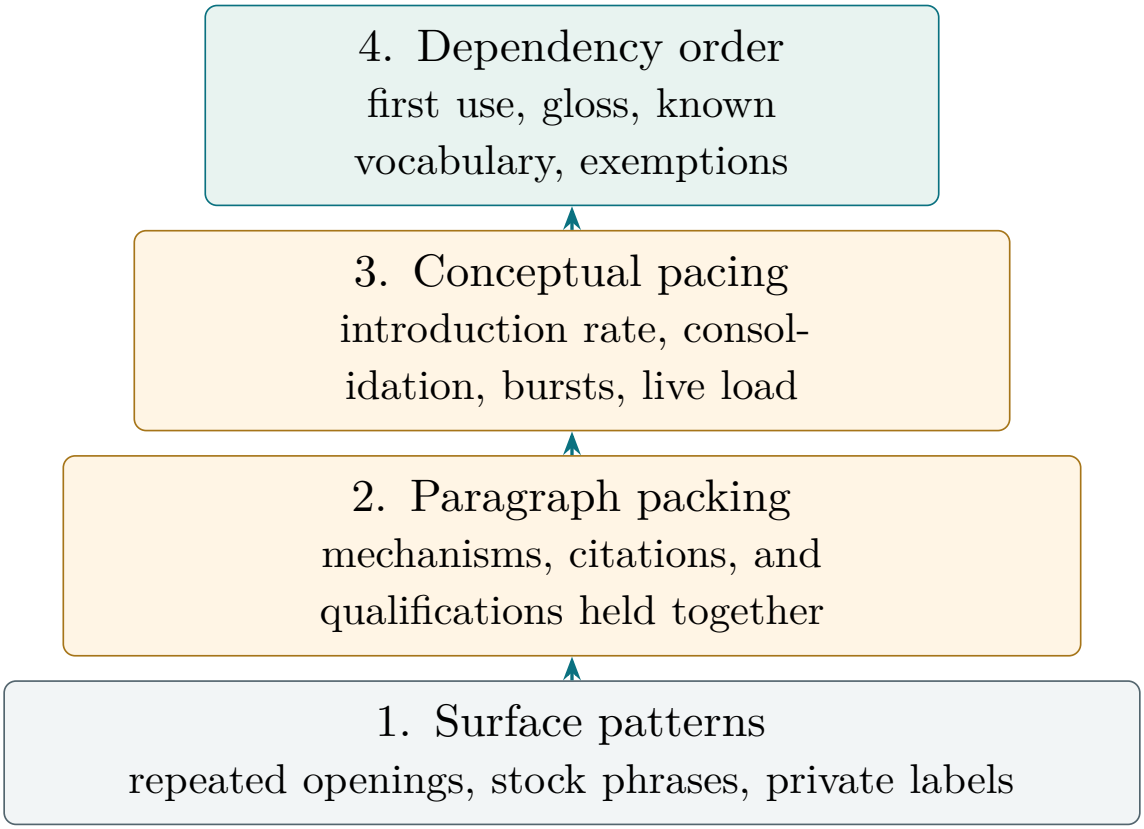

Figure 10.2: A diagnostic ladder for the complaint "too dense." Surface checks remain useful, but they cannot stand in for paragraph, pacing, or ordering evidence.

10.3 The third round: names arrived before their objects

The final complaint was more specific: "Do not use a concept before you have explained it." A first-use audit found roughly 25 genuine violations. The running example used the name PPO on its first page and never introduced the algorithm. Two acquisition-function names appeared a section before the chapter taught what an acquisition was.

The reader could recognise the words and still be unable to follow the argument. Recognition is not understanding. A glossary at the end would not repair the first encounter, because the reader had already had to guess why the name mattered.

The repair pattern was simple. Start with the ordinary object, then name it: "a population-based method, BG-PBT, examined in Chapter 6." When the order must be anticipatory, say so: "The next two names are only landmarks; the method appears in Section 6.2." A preview, a decision box, or a reference table may legitimately name an object early, but the document should declare that exemption rather than silently teaching the rule to ignore every early name.

Round	What the reader said	Located cause	Repair and product lesson
1	"Too dense."	Six mechanisms and five citations in one 210-word paragraph.	One idea per paragraph; report the passage and let the author choose the split.
2	"Too dense."	New concepts arrived faster than in the named exemplars (70 versus 83 and 253 words per concept).	Measure relative pace and burst; use examples, figures, and read-backs before adding another term.
3	"Do not use it before explaining it."	About 25 first-use violations, including an unexplained running-example algorithm.	Audit first use with a whitelist and explicit exemption zones; introduce the generic object first.

Table 10.1: One reader complaint, three levels of diagnosis. All figures are internal calibration observations from one engagement.

10.4 The diagnostic ladder

The case suggests four questions, in a deliberate order.

Surface. Does a repeated opening, stock phrase, or private label deserve inspection? Pattern tools are useful here because their observations are narrow and easy to locate.

Packing. Does one paragraph ask the reader to hold several mechanisms, citations, or qualifications at once? Paragraph length is only a clue; the finding must show what is being carried together.

Pacing. How quickly does this document introduce and consolidate new concepts relative to the exemplars named in its reading brief? The meter reports introduction rate, singleton share, burst paragraphs, and a rolling live load. It does not decide whether a theorem or definition is too dense.

Order. At the first occurrence of a term, has the document supplied an expansion, a plain-language gloss, or a pointer to where the term is taught? The audit uses the reader's known-vocabulary list and declared exemption zones. It should abstain when the evidence is ambiguous.

The order matters. A writer who fixes a repeated heading before noticing that the paragraph has no example may polish the surface while leaving the reader's real problem untouched. Conversely, a writer who discovers an ordering fault does not need to rewrite

the whole chapter. The finding queue should be sortable by unit, and each repaired unit should be rebuilt and inspected before the change spreads to the rest of the document.

10.5 Slow, but easy to leave

The manager offered a practical rule: when a passage is slow, the reader can skip; when it is too fast, the reader must reconstruct the missing steps. We take that as a repair prior, not as a licence to add padding. A slow section earns its space when it closes questions in sequence, uses headings that make the stopping points visible, and returns to the same idea in a different form.

The first build should record a second property beside concept pace: skippability. A long unit with no internal heading, example, or summary is not slow in a useful way. It is simply hard to re-enter after an interruption. The author can respond by adding a map, a worked micro-case, or a short read-back; the author need not accept the meter's preferred wording.

10.6 The reader's evidence completes the loop

The agent judged the document readable before each of the three rounds and was wrong each time. The meters did not judge it good. They located passages that the manager then confirmed or rejected. That division of labour belongs in the product contract.

The reader record should preserve the moment at which the document stopped working. In addition to a response and elapsed time, the reviewer records the page or section where they stopped, a short quotation that triggered the request, and the smallest missing relation they were trying to recover. Those fields are more informative than a single score: "I stopped at page 18, where PPO appears without an explanation" tells the author what to repair.

The reading brief must learn from the session. It carries the named exemplars, the vocabulary the reader already owns, the sections in which names are only landmarks, and the standing instruction to err on the slow side. A later inspection runs against that versioned brief rather than reopening a settled argument. The brief remains a reader contract, not a private project diary.

What this case adds to Humanvoice

The HPO engagement does not prove that Humanvoice works. It gives the first build a sharper job: distinguish packing, pacing, and ordering; calibrate against the reader's exemplars; show the passage that triggered a finding; and leave the repair and the final judgment with the author and reader.

CHAPTER 11

Components worth testing

Imagine a two-page research note. Its opening reports an estimated premium of 42 basis points. A table displays the same estimate, an equation defines the premium, and a citation supports the identification strategy. The author wants help with one awkward paragraph.

A conventional writing assistant sees the paragraph. Humanvoice must see the note. If a proposed edit changes 42 to 24, removes the condition attached to the estimate, or moves the citation to a stronger claim, the new prose may still be fluent. The revision has nevertheless changed the analysis. The product needs to show the author that change before anyone accepts the wording.

This small note gives the software review its order. First preserve the source and build what the reader sees. Then locate the prose question. Compare the meaning-bearing parts of the two versions. Only after those steps should an author decide whether to revise and a reader decide whether the note works. Available packages can perform many of these jobs, but no package in the survey performs the complete sequence.

The detailed component and market records appear in Appendix D. This chapter asks the narrower investment question: which parts should the first build reuse, which need an adapter, and which connection must Humanvoice own?

11.1 One note, six jobs

The note first needs an identity. Humanvoice takes a snapshot of the LaTeX source, its bibliography, and the command that produces the PDF. A later finding can then point to the exact version on which it was made. Without that identity, an author may open a warning from yesterday against a paragraph changed this morning.

Next comes the build. The software compiles the note and records unresolved references, missing files, and the page on which each labelled object appears. A source file that looks plausible but does not produce the reader's PDF is not a useful basis for revision.

The third job is diagnosis. A prose rule may notice that the paragraph begins with a vague statement of importance. A structural check may notice that the same opening appears in four sections. A citation check may notice that the key no longer resolves to the source used in the previous version. Each finding needs a location and a reason. None changes the source by itself.

The fourth job begins only after the author edits. Humanvoice compares the old and new versions of the equation, the number, the citation identity, and the qualification around the result. In the example, the author should see that the premium remains 42 basis points in the paragraph, table, and equation. If one instance differs, the comparison

remains open.

Privacy supplies a fifth job. A restricted manuscript stays local unless its owner approves a particular external service and the fields that may be sent. Installing a local model can reduce one class of disclosure risk. It does not make the model's suggestion correct, and it does not erase the need to record which version and prompt produced the suggestion.

The last job belongs to a person who did not write the note. That reader sees the rendered pages, the declared decision, and any required disclosure. Can the reader state what the 42 basis points measure, why the estimate is identified, and what action follows? The answer is the product outcome. It cannot be recovered from a grammar score or a successful build.

No current candidate does all six jobs. That is not a reason to write six new systems. It is a reason to put mature components behind a small record that keeps their evidence separate. A parser supplies locations. A linter supplies warnings. A model supplies suggestions. A reader supplies the consequential judgment.

The path from an interesting repository to an adopted component should be deliberately slow.

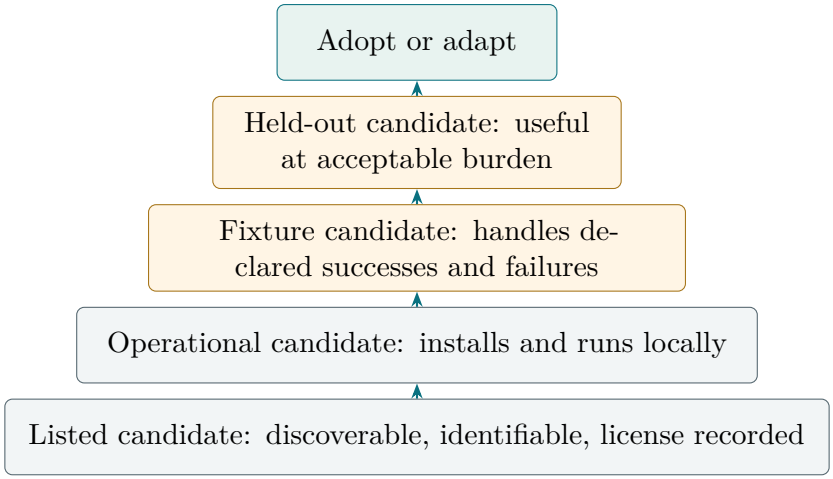

Figure 11.1: A repository earns adoption in stages. Popularity and a successful smoke run are availability evidence, not effectiveness evidence.

A package that installs has shown only that it can run. A package that handles one designed example has shown a little more. It earns a role in Humanvoice only after it succeeds and fails in the expected places on examples it did not help to design. Even then, adoption is conditional on the burden it creates for the author.

11.2 Start with the source, not the suggestion

LaTeX is a program as well as a document format. A command can define a number once and print it in several places. Another command can alter spacing without altering meaning. A third can expand differently inside a table and a sentence. The parser therefore has to preserve both the raw source and what the command produces.

One command, two visible meanings

Consider a fixture built around a custom command called `\riskterm`. In the table it expands to "42 bp." In the prose it expands to "estimated risk premium." An editor that stores only the expanded text sees two unrelated phrases. An editor that stores only the command name misses the visible difference. Humanvoice needs the call, the expansion, the source location, and the rendered location. If any of those cannot be recovered, the fixture should be marked unparsed rather than protected.

pylatexenc is the smallest serious Python candidate for this first pass [pylatexenc maintainers, 2026]. It can tokenize commands and groups while retaining source positions. That is enough to test whether the product can locate known macros, mathematics, citations, and environments without inventing a full model of TeX. Its limitation is equally useful: an unfamiliar macro has no semantic meaning merely because its braces were parsed. The adapter must keep the raw span and abstain.

A richer document model may become necessary. plasTeX understands sections, counters, labels, and more of the structure a rendered document exposes [plasTeX maintainers, 2026]. LaTeXML translates many mathematical constructs into a semantic XML representation [National Institute of Standards and Technology and LaTeXML maintainers, 2026]. Both can answer questions a tokenizer cannot. Both also create a larger compatibility problem. A custom package may behave differently from the author's TeX installation. The benchmark must compare their representation with the canonical PDF rather than assuming that a more elaborate tree is more faithful.

The editor route creates a different tradeoff. unified-latex offers typed JavaScript structures and transformations that fit a web interface [unified-latex maintainers, 2026]. tree-sitter-latex can update a syntax tree quickly as the author types [latex-lsp maintainers, 2026]. TexSoup offers a tolerant Python interface that remains usable on imperfect input [TexSoup maintainers, 2026]. Speed and tolerance are helpful in an editor. They become dangerous when a best-effort parse is silently promoted to a claim that an equation was protected. Unknown constructs must remain visible in every route.

Pandoc solves another job well. It can turn supported sources into a structured interchange form and produce a prose view for linguistic diagnostics [Pandoc maintainers, 2026]. That secondary view is useful when raw commands confuse a writing rule. It is not the authority for equation preservation. The LaTeX source and its own build remain the reference for the first release.

latexdiff belongs at the end of this path, not the beginning. It gives an author a legible picture of source changes [latexdiff maintainers, 2026]. That picture is valuable in a review packet. It does not decide whether two equations are equivalent or whether a citation still supports a sentence. Humanvoice should use it to display an already recorded comparison, never to define the comparison.

The first parser decision is therefore a bake-off, not a commitment to a framework. Give the candidates the same custom commands, nested environments, equations, tables, comments, and malformed input. Record what each locates, what it loses, and where it abstains. Begin with pylatexenc because it is small and fits the proposed Python core. Keep the richer candidates available when a fixture demonstrates that the smaller

representation is insufficient.

11.3 Let mature tools do narrow jobs

The product does not need to conceal the ordinary LaTeX toolchain. latexmk can repeat compilation until references and the bibliography settle [Collins, 2026]. ChkTeX can report source-level syntax and usage warnings [ChkTeX maintainers, 2026]. Their output belongs in the run record with the exact build command. Neither tool needs to become a Humanvoice feature, and neither tool establishes that the resulting page is understandable.

Grammar and prose checks are another mature layer. Vale is well suited to project rules because it returns a line, rule, and message [Vale maintainers, 2026]. LTeX+ and TeXtidote understand enough LaTeX to avoid treating every equation as a sentence and can return language findings to source locations [LTeX+ maintainers, 2026]. textlint is attractive when a rule needs a programmable JavaScript host [textlint maintainers, 2026]. The first build should adapt their machine-readable findings rather than copy their rule engines.

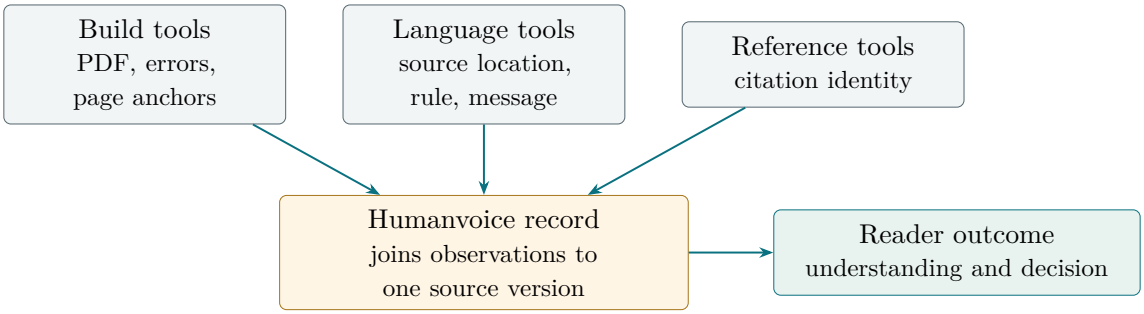

Figure 11.2: Mature tools produce narrow, checkable observations. Humanvoice connects them to a source version; the reader supplies the outcome.

These tools should run after the important question is visible. If the research note does not explain what the 42 basis points measure, a queue of comma and agreement corrections is a distraction. Once the mechanism is present, those corrections can reduce friction. The reading brief determines the order; the number of warnings does not.

Citations require two separate checks. The first resolves identity: does the key still point to the expected authors, title, venue, and identifier? GROBID can help extract structured fields when a PDF is the only available source [GROBID maintainers, 2026]. The second check asks whether the paper supports the sentence. Matching a DOI cannot answer that question. Humanvoice should never label an identity match as citation validation.

Local inference is also a component rather than a conclusion. llama.cpp offers a portable runtime for compatible local models [llama.cpp maintainers, 2026]. It can keep an approved prompt and manuscript on the organization's hardware. The selected model still needs a licence, resource profile, prompt-injection test, and comparison with a remote baseline. A local hallucination is private, but it is still a hallucination.

The reader task needs less machinery than a general model-evaluation platform. jsPsych can present a local browser exercise, randomize version order, record answers and elapsed time, and export a structured session [jsPsych maintainers, 2026]. That makes it a plausible instrument for the first human comparison. It does not recruit an expert reader or decide whether an answer is correct.

Inspect AI can organize repeatable tasks, solvers, and model graders [UK AI Security Institute, 2026]. It may become useful when the team compares prompts or judge models. Making it a dependency of the first human exercise would reverse the order of evidence. The named reader is the endpoint; model-evaluation infrastructure is optional support.

11.3.1 The pacing adapter is a small local experiment

The HPO case adds one capability that the general packages do not supply. The first build needs a way to compare a target chapter with the exemplars in its reading brief and to show where new concepts arrive in bursts. The repository now contains a standard-library prototype, `tools/concept_density.py`, for that narrow job. It recognizes a few transparent signals—acronyms, mid-sentence proper terms, emphasized definition phrases, recurring technical bigrams, and citation keys—then reports introduction rate, consolidation, bursts, and live load.

The prototype is deliberately not a dependency of the protected comparison. It can be noisy, and it has no sentence-level dependency parser. Its output is a ranked worklist with file and line locations. The author can dismiss a capitalized ordinary word in one action. A first-use pass reads the same brief for known vocabulary and exemption ranges, then asks for a gloss or a taught-later pointer when the evidence is missing.

The HPO fixture supplies the first calibration and held-out material: 25 adjudicated ordering violations and 80 adjudicated false alarms. Those numbers remain in the fixture record rather than becoming a success rate. Before the adapter is promoted, an independent operator must measure precision, abstention, and minutes spent triaging it on documents outside its tuning partition. A positive result would justify a narrow adapter; a null result would leave the rest of the source-to-reader path intact.

The common rule is simple. Reuse a mature tool when its output names a fact the product can check: a build result, a source location, an identifier, a language warning, a runtime trace, or a recorded response. Do not ask that tool to make the larger claim

that the document is correct or persuasive.

11.4 The alternatives an executive will try first

A sponsor should ask why the team cannot buy this capability. The answer cannot be that Humanvoice has a longer feature list. Existing assistants are better than a new prototype at many common writing tasks, and the experiment should retain them as credible alternatives.

For an ordinary memorandum, Grammarly may be the right incumbent. Its enterprise product works across common writing environments and combines language suggestions with a broader agent layer [Grammarly, 2026a]. Its privacy documentation also explains when generative features may send text and context to service providers [Grammarly, 2026b]. An organization can weigh those controls against the document's sensitivity. Humanvoice should not spend its first budget rebuilding general grammar and tone advice that a mature service already provides.

The research-note example exposes the remaining question. The unit seen by a general assistant is usually a selected passage or communication context. The unit Humanvoice must protect is the versioned note: paragraph, equation, table, citation, and reader decision together. An excellent rewrite of the paragraph can still change the denominator or remove the identification condition. Generic fluency is therefore part of the incumbent workflow, not the proposed product wedge.

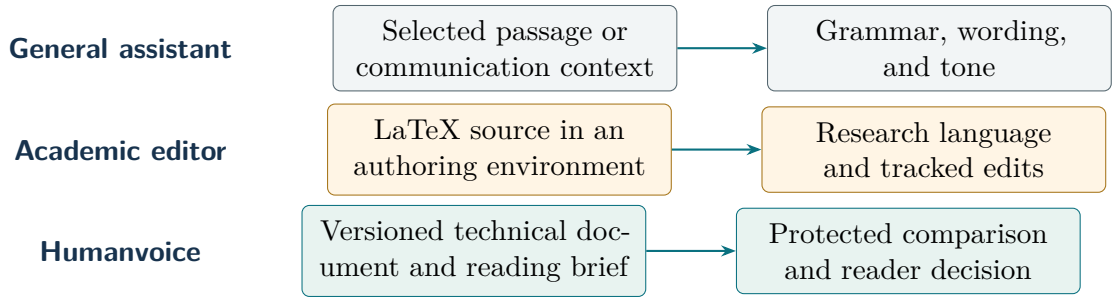

Figure 11.3: The alternatives overlap, but their units of work differ. The Humanvoice claim begins where passage-level fluency and tracked edits stop.

Academic assistants come closer. Writefull integrates research-trained language support into Overleaf and offers rewriting and style functions in the LaTeX environment [Writefull and Overleaf, 2026]. It should be tested as a language adapter and as an incumbent arm. The question is not whether it can improve a sentence. The fixture asks whether the suggestion remains tied to a source span, whether protected objects stay visible, and whether the resulting note can enter the same reader task.

Paperpal combines language editing with reference discovery, citation styles, and submission checks [Paperpal, 2026]. Trinka similarly targets academic and technical writing and documents institutional, privacy, integration, and API options [Trinka AI, 2026]. Their breadth is real value for a researcher. It also makes causal interpretation harder: when one interface performs many interventions, a favorable reaction does not show which intervention helped or whether a protected object moved. The benchmark should test declared functions one at a time.

Overleaf already solves the social side of collaborative LaTeX editing. It provides comments, version history, and Track Changes through which authors can accept or reject revisions [Overleaf, 2026]. Humanvoice should integrate with that environment when a team already uses it. Track Changes is a view of source edits, however, not a proof that an equation is equivalent or a citation supports its new sentence. The source snapshot and protected comparison remain distinct.

Privacy does not create an automatic winner. A hosted assistant may be suitable for a public draft and unacceptable for a restricted manuscript. A local model may satisfy the data boundary and perform poorly on the task. An editor may retain complete version history while the author prefers not to expose that history to an external reader. Each workflow should record what leaves the machine, what returns, and what the reader receives.

The fair comparison is therefore practical. Give the incumbent and Humanvoice the same note, reading brief, and deliberate defects. Ask each route to surface a changed number, a lost qualification, and a moved citation. Record network calls, author time, false passes, and the reader's ability to reconstruct the result. If an existing product performs the complete job at lower burden, the team should adopt it and narrow or stop Humanvoice.

11.5 Buy the parts, build the connection

The software survey leads to a restrained boundary. Buy or reuse grammar, collaborative editing, and document compilation when the licence and privacy terms fit. Adapt parsers, linters, and comparison views when they can return a stable location and an inspectable reason. Build the connection that current alternatives leave implicit.

That connection begins with a source snapshot. It records the note, build, and reading brief as one run. A source map then links the paragraph, equation, number, table, citation, and rendered page. Diagnostics attach findings to those locations. The author accepts, rejects, or defers each finding. A new run records the revision and compares its protected parts with the old one. The reader's response closes the sequence.

Return to the awkward paragraph. Vale may flag its vague opening. The author rewrites it in Overleaf. Humanvoice does not need to own either operation. It does need to know which source version produced the finding, which version contains the repair, and whether 42 basis points and the identification condition survived. It also needs to place the revised PDF in front of the reader without exposing private comments or model prompts.

Five small records are sufficient for the first experiment: the run, protected object, finding, author decision, and reader response. Immutable identifiers join them. This is not a general document platform. It is the minimum structure needed to answer who saw what, what changed, and which observation supports the next investment decision.

The boundary affects cost. Reimplementing a grammar engine would consume the MVP on a solved problem. Depending entirely on a cloud editor would save engineering time while moving the central source comparison into a service we do not control. The pilot should therefore price three configurations: a fully local route, an incumbent-editor route with Humanvoice adapters, and a hosted academic assistant used only on approved excerpts.

The choice among those configurations belongs to the experiment. The local route may cost more operator time. The editor route may provide the best author experience. The hosted route may produce the best language suggestions. Reader performance, meaning errors, privacy compliance, and total burden decide; the architecture diagram does not.

11.6 A fixture decides, not a feature list

The first held-out set should be small enough to understand completely. One fixture is clean. One contains a private project label. One changes an exponent in an equation. One changes the denominator in a table. One cites a real paper that does not support the sentence. One uses the custom `\riskterm` command. One contains malformed LaTeX. One repeats a sentence shape on purpose.

Each fixture states what success and failure look like before a package runs. For the custom command, a parser succeeds if it preserves the call, expansion, and locations. It abstains acceptably if it identifies the command as unknown and blocks a protected pass. It fails dangerously if it returns a complete- looking record that has lost either use of the command.

For candidate k on fixture i, record whether the capability was observed, whether a false pass occurred, how long the run took, and how much operator work was required. Keep those observations as a matrix. A single score would allow fast execution to compensate for silent loss of an equation, which is not an admissible trade.

The benchmark also records ordinary engineering facts. A restrictive licence may confine a useful package to a reference implementation. A stale release may be acceptable for a frozen migration and unsuitable for a service that must track new TeX packages. A component that contacts the network may be excluded from a private note even when its findings are good. These facts sit beside the functional result rather than being folded into a popularity ranking.

Adoption requires three observations. The component must find the positive case. It must fail or abstain on the negative case. A second operator must be able to replay the finding from the recorded version and configuration. A package that cannot meet all three may remain useful to study, but it does not enter the protected path.

The same standard applies to the product as a whole. If the components cannot produce a stable source map, the project remains a prose linter. If the source map works but authors face an unmanageable queue, the workflow must be reduced. If the engineering succeeds but the named reader does no better, the proposed connection has not earned its cost.

This component strategy leaves the first build with one narrow responsibility: join reliable observations without inflating what any of them proves. The next part follows a document through that connection from draft to reader decision.

Part III

The product

CHAPTER 12

One document, from draft to decision

One short research note makes the design concrete. An author, a parser, and a reader each do something different with it. The note is a teaching case assembled from the failure patterns in the project's reader records; it is not a claim about a real model's performance or a disguised forecast. Its purpose is to keep the nouns, numbers, and choices visible while the later chapters generalize the method.

The note compares a baseline model with a constrained reconstruction and asks whether the team should approve a larger comparative run. It contains the things that make a technical document worth protecting: an equation, a table, a citation, a numerical result, and a qualification about what the result does not establish. It also contains the things that make a document hard to read: private phase names, an implicit mechanism, a result separated from the question it answers, and a request that appears only after several pages of process description.

Figure 12.1 is the route through the chapter. The return arrow is bounded: a failed pre-human check goes back to a located authoring decision before release. The final reader task is not the system's debugging loop, and it never triggers an unending series of automatic rewrites.

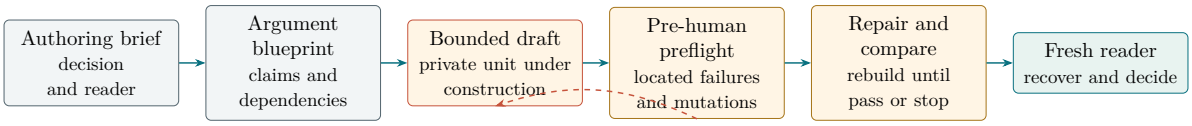

Figure 12.1: The unit of work is one document version moving from brief to release, with a bounded return when pre-human checks find a repair.

12.1 The document before the tools

Imagine receiving the note as a PDF with no repository history. The first page opens with a sentence such as "Phase 2B evaluates the post-rescue path under the frozen interface." The phrase is meaningful to the project team. A new reader does not know what was rescued, which interface is frozen, or why the comparison matters. The next page defines a loss function and presents a table of parameter values. The decision request arrives near the end: "If the current branch clears the remaining checks, proceed to the expanded run."

Nothing in those sentences is necessarily false. The problem is that the reader has to infer the object of the comparison before deciding whether the equation and table are

relevant. In the internal records, this was the point at which a mathematically serious draft could be rejected within minutes. The reader did not ask for more adjectives. The reader asked what the team was trying to change and what action the document supported.

Element in the source	The author's hidden assumption	Question the reader actually needs answered
Private phase label	The history of the project and the meaning of "rescue"	What object is being compared, and why now?
Loss function	The target has already been defined elsewhere	What decision does this quantity inform, and what are its units?
Parameter table	The rows are self-explanatory	Which rows are inputs, which are estimates, and which are illustrative?
Qualification	The reader will connect it to the result two pages earlier	What inference is supported, and where does it stop?
Final request	The implementation team knows the next phase	What exactly is being approved, at what cost, and with what stopping rule?

Table 12.1: The source contains the necessary material, but the reader must reconstruct its order and purpose.

The distinction between an absent fact and a misplaced fact matters for the design. A general language model can often make the first sentence smoother. It cannot know whether "post-rescue" is a technical term, a project label, or an accidental remnant unless the author supplies that meaning. A linter can flag a long sentence. It cannot decide whether the loss function is the right quantity for the funding request. The first review packet must therefore show the source passage and ask a question that belongs to the author or the reader.

12.2 The first preflight and the first repair

The proposed workflow starts with a preflight run, before any expensive reader is asked to reconstruct the project. The system checks the brief and blueprint, then tests the first unit against four questions: What decision is at stake? What is the proposed mechanism? What evidence would change the decision? What action is requested? These are not a score for prose quality. They are a construction test for whether the document has delivered its economic purpose.

Suppose the preflight critic cannot recover the first and fourth answers. The author opens a located finding. It includes the unit, its source location, the reason it was selected, the missing dependency or evidence, and an editorial question. The reason is descriptive rather than moral: the paragraph uses a private phase label before introducing the

research question and leaves the action implicit. The author can accept the finding, reject it with a reason, or rewrite the passage. A model suggestion may be displayed, but it is an option, not the recorded decision. The unit remains inside the repair loop until the preflight either passes or stops with an explicit unresolved choice.

> *Before:* Phase 2B evaluates the post-rescue path under the frozen interface.
>
> *After:* The team must decide whether the constrained reconstruction is ready for a larger comparison. We compare it with the baseline model under the same data and evaluation rule.

The revision does three jobs. It names the decision, identifies the two sides of the comparison, and states the common condition that makes the comparison interpretable. It does not claim that the constrained model is better. That claim belongs after the evidence. It also does not delete the project history; the history remains useful in the internal record, where it can be found by a team member who needs it.

The second repair concerns an equation whose meaning was hidden by notation. The source wrote

$$L(\theta) = \sum_{t=1}^{T} \Big(y_t - f_\theta(x_t) \Big)^2, \tag{12.1}$$

without saying whether L was a training objective, an evaluation statistic, or an accounting identity. The equation is elementary; its role is not. The author's revision introduces the units and purpose:

> For the feasibility comparison, y_t is the held-out outcome, x_t is the declared input record, and f_θ is the fitted reconstruction. We use $L(\theta)$ only as a held-out prediction loss, measured in squared outcome units. It is not a measure of reader comprehension and it does not determine whether the team should approve deployment.

The display itself remains unchanged. The surrounding prose does the work that notation alone cannot do. A reader who cares about the model can now interpret the terms; a decision-maker can see why the number is relevant but insufficient.

12.3 Reconstructing the argument

Once the first repairs are accepted, the author reorganizes the note around five questions. The order is not a universal template; it fits this decision because each answer supplies the premise for the next one.

1. **Decision.** The team must decide whether to fund a larger comparative run.
2. **Object.** The comparison is between a baseline and a constrained reconstruction applied to the same declared record.

3. **Mechanism.** The reconstruction changes the specified mapping from inputs to predictions while holding the evaluation rule fixed.
4. **Evidence.** A held-out loss, a protected-object comparison, and a reader test provide different pieces of evidence.
5. **Action.** The sponsor approves the next run only if the comparison is reproducible, the meaning-bearing objects are preserved, and the reader can recover the request.

Argument step	Passage the revised note must contain	Observation that can overturn it
Decision	A one-sentence funding request with a budget and stopping rule	The sponsor cannot identify a decision owner or a bounded action
Object	A definition of the baseline, reconstruction, population, and evaluation window	The two versions use different records or targets
Mechanism	A short derivation or diagram showing what changes and what remains fixed	The observed improvement comes from a changed data or scoring rule
Evidence	The held-out result, source location, and reader observation are reported separately	The result is not reproducible or the reader cannot use it
Action	A proceed, revise, or stop recommendation tied to the observations	The recommendation depends on an unstated preference

Table 12.2: The argument becomes a sequence of answerable questions rather than a sequence of project phases.

The table is useful during revision, but it should not appear as a five-row form in every document. In the finished note, the five answers can be ordinary prose, a figure, or a short table wherever that form best carries the argument. Humanvoice keeps the intermediate record because it lets an author explain why a paragraph moved or why a table was retained. The reader sees the resulting argument, not the form.

The revised note also changes the order of its numerical material. First it states the decision and the target. Then it gives the equation and explains its role. The table follows with a sentence that identifies inputs, estimates, and illustrative values. Finally, the qualification appears beside the result it qualifies. This is a small editorial change, but it reduces the number of working assumptions the reader must hold at once.

12.4 The objects the system protects

The revision is not safe merely because the new prose sounds plausible. The source contains objects whose alteration can change the scientific meaning. For this case, the

protected set is

$$\mathcal{P} = \{\text{equations, numbers, labels, citations, tables, declared qualifications}\}. \quad (12.2)$$

Each member receives a source span, a normalized representation, and a before-and-after comparison. The comparison is not a demand that every object remain identical. It is a demand that a change become visible and receive an owner.

For an object $p \in \mathcal{P}$, write $\Delta p = 0$ when the normalized object is unchanged, $\Delta p =$ explained when the author has recorded a meaningful change, and $\Delta p =$ unresolved when the comparison cannot establish correspondence. The release rule for the case is

$$\forall p \in \mathcal{P}, \qquad \Delta p \neq \text{unresolved}. \quad (12.3)$$

This is a safety rule for the workflow, not a quality score. It does not say that an unchanged equation is correct or that an explained change is wise. It says that the system must not hide a broken correspondence behind a fluent rewrite.

The note contains one numerical wedge, for example a change from 0.42 to 0.47 in a displayed comparison. A character-level diff will find the change, but it will not tell the author whether the value is a typo, a recalculation, or a deliberate change in the sample. The protected record links the number to its table row, unit, source calculation, and explanation. If the author cannot provide that link, the revision returns to the author regardless of how much the surrounding paragraph improved.

Citation protection has the same shape. The system resolves the key to a bibliographic identity and records the sentence in which it appears. It does not infer that the source supports the sentence. That second judgment remains with the author and, for a load-bearing claim, an independent reviewer. A DOI resolver can prevent a misspelled reference from surviving; it cannot turn a nearby paper into evidence.

12.5 The author's revision

After the first pass, the author sees a queue of findings ordered by expected reader consequence, not by the model's confidence. Three findings may appear:

Meaning The paragraph names a result but does not say which decision it supports.

Register A private phase name and two file paths make the opening unintelligible outside the project.

Surface Three adjacent sentences repeat the same grammatical shape.

The author chooses the meaning finding first. This choice is important because surface repair can make a weak paragraph more pleasant without making it more useful. The author then decides whether the phase name has a legitimate public meaning. If it does, it is defined once. If it does not, it is removed from the reader-facing version and retained in the internal source map. Finally, the author may vary the sentence rhythm, but only after the proposition is clear.

The queue records the decision in ordinary language: “accepted: the reader needs the action before the phase history” is more useful than a status code. The machine can store a code for searching, but the code is not the explanation that another author or reviewer should have to read.

a rejected suggestion

The editor proposes replacing the qualification “the comparison does not establish transport to a new population” with “the result is promising but further work is needed.” The second sentence is smoother and less useful. It removes the population condition and gives no account of what further work would answer the question. The author rejects the suggestion and rewrites the qualification in terms of the missing population and the planned comparison.

This example illustrates why a human decision belongs in the path. The issue is not whether the proposed sentence is grammatical. It is whether the sentence preserves the inference boundary that a reader could reasonably misunderstand. An automatic system can point to the boundary, show the alternative, and record the protected comparison. It cannot own the scientific judgment.

12.6 The reader’s turn

After the preflight gates pass, the revised note is rendered and sent to a reader who has not seen the source history, the finding queue, or the author’s rejected suggestions. The reader receives a short context statement: “You are deciding whether to approve a larger comparative run. Read the note as you would a proposal from a colleague and answer the four questions below.” The reader may annotate the document, but the primary observations are timed and forced-choice so that a vague impression becomes a usable record.

For reader j and version v, let $A_{jv} = 1$ when the reader identifies the requested action, the comparison, and the principal qualification within the declared time budget. Let T_{jv} be reading and clarification time, and let R_v be the number of author feedback rounds before acceptance. The simple case-level outcome is the vector

$$O_v = \left(\frac{1}{J} \sum_{j=1}^{J} A_{jv}, \quad \frac{1}{J} \sum_{j=1}^{J} T_{jv}, \quad R_v, \quad M_v \right), \tag{12.4}$$

where M_v counts unresolved or unauthorized changes to protected objects. The vector is deliberately not collapsed into one score. A faster reader who misses the qualification is not an improvement.

The first reader may answer “revise” even when the revised prose is fluent. That is not a failed product demonstration. It is information about which finding remains unresolved. The author receives the reader’s question and returns to the source, rather than asking the reader to explain the entire document. A second reading then distinguishes a local repair from a change in the argument. The process ends when the reader can make the declared decision or when the sponsor invokes the stop rule.

12.7 The economic reading of the case

The case also makes the investment mechanism concrete. Suppose the incumbent version takes the reader $T_0 = 3.0$ hours and requires $R_0 = 2.4$ feedback rounds on average. A revised version takes $T_1 = 2.4$ hours and $R_1 = 1.8$ rounds. For $N = 100$ documents per year and a loaded reader cost of $c = 150$ per hour, the direct attention saving is

$$B = N(T_0 - T_1)c = 100(0.6)(150) = 9{,}000. \tag{12.5}$$

The values are teaching numbers. They show why the reader's time must be measured directly and why a lower round count is not enough. If the revised version adds a single hour of clarification for each document, the apparent saving disappears. If $M_1 > 0$, the financial calculation is beside the point until the protected change is explained.

The case also separates costs that are easy to hide. The author spends time answering findings; the reader spends time reading and clarifying; a technical owner spends time maintaining parsers and fixtures; and the organization bears the cost of storing sensitive drafts. A serious pilot records each component. The product earns adoption only if the attention saved at the reader boundary exceeds those costs for a document class that recurs.

12.8 Lessons from the case

The worked note establishes five design requirements. The system needs a clear statement of the audience and decision because purpose is not recoverable from source syntax alone. It needs located findings because an author cannot assess an unexplained score. It needs protected comparisons because fluency can conceal a changed number or qualification. It needs a reader who has not seen the repository because project familiarity can hide a failure. And it needs an economic record because a nicer draft is not a product benefit until it reduces scarce attention without creating a meaning error.

The case does not establish that the workflow improves a real proposal. It uses a finite teaching scenario and a deliberately small reader protocol. The comparative program in Part IV supplies the next observation. The value of the case is that it makes the proposed object testable: a later study can disagree about the effect without disagreeing about what the system was supposed to do.

The internal record and the external literature now give the case a wider test. Each new tool or result can be placed against the same questions: does it clarify the decision, preserve a protected object, reduce author or reader burden, or improve the evidence for the next choice? That is the product boundary in working form.

CHAPTER 13

Inside a Humanvoice review

The previous chapter followed a research note from its muddled opening to a reader's decision. We now return to the same teaching case and move closer to the screen. One paragraph is enough to show what the author, technical reviewer, and reader would actually receive. Its text and numbers remain synthetic: they expose the product's duties without masquerading as evidence from a live trial.

The same version appears differently to the author, technical reviewer, and reader. Figure 13.1 shows how one record supports those views without exposing the private review trail in the manuscript.

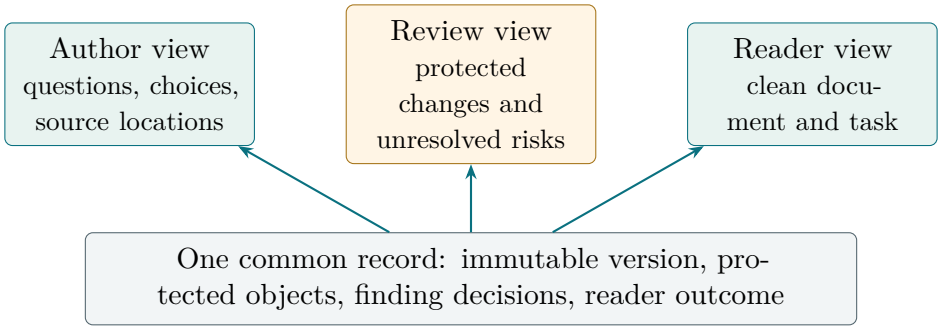

Figure 13.1: Three people see different interfaces over one version record; the reader does not receive the project's private audit trail.

13.1 Before a reader sees a draft

The expensive mistake is to give a senior reader a document that was never ready for a senior reader. Humanvoice therefore begins before prose. The owner states who will read the document, what decision it must support, what the reader already knows, which claims must appear, which evidence is allowed, and which objects may not change silently. If one of those answers is missing, the system records the uncertainty and stops the long draft. It does not turn an unfinished brief into something that looks finished.

The first output is an argument blueprint. It is a short, inspectable plan with one job for each section, a claim map, a concept-dependency graph, a list of protected objects, and a visual plan. A claim without evidence is marked as a question. A concept used before its prerequisite is taught is moved or marked as an intentional exception. A figure has to earn its place by carrying a relationship that prose alone would make harder to see. The blueprint is not a new report for the sponsor; it is the construction drawing that keeps the report from becoming a pile of fluent paragraphs.

Consider the teaching case in this chapter. The owner might write:

> The reader is an investment committee. The decision is whether to run the constrained reconstruction at larger scale. The reader knows ordinary model comparison but not the project's private phase names. The document must explain the comparison, the 35-basis-point funding wedge, the evidence needed before a larger run, and the limit of what the calibration can establish.

Humanvoice turns that brief into a sequence the writer can actually follow:

1. open with the decision and the two models;
2. show the funding wedge in its unit and location;
3. explain which likelihood terms, observations, and paths are compared;
4. state what agreement would authorize next and what it would not prove; and
5. use one diagram to connect the objects, checks, and decision.

Only after the owner accepts that sequence does the writer draft one bounded unit at a time. Each unit is rebuilt and checked before the next unit inherits its terminology. This is slower for the machine and faster for the person who would otherwise have to discover the missing argument at the end.

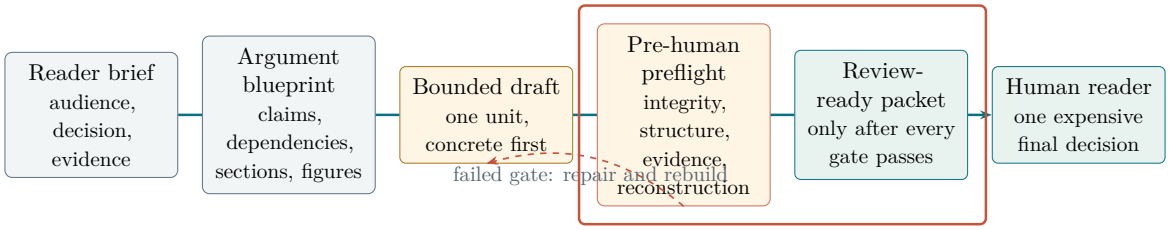

Figure 13.2: Humanvoice spends scarce reader attention at the end of the path. A failed pre-human check returns the draft to a located repair; it cannot produce a review packet by default.

The guard in Figure 13.2 is a product contract, not a promise that a machine can certify taste. It says something narrower and more useful: an incomplete brief, an ungrounded load-bearing claim, an unknown concept order, an unexplained protected-object change, a broken build, or an unresolved high-risk preflight finding cannot be packaged as ready for a human reader. The system may still save an internal draft for repair. It does not ask an expensive reader to serve as its debugger.

13.2 A draft that should never reach the reader

The note compares the baseline model with the constrained reconstruction. An unguarded writer has produced the paragraph below. In the old workflow it would have been sent to the owner for diagnosis. In the proposed workflow it is a fixture: the pre-human checks should locate the problem before a reader sees the document.

> "In the Phase-2 rescue, the declared restricted reconstruction is translated into the solver and compared like with like. We do not claim that this proves the policy works in general; rather, it is a robust and transparent framework for checking the relevant objects, including the 35 basis-point funding wedge in Table 3 and the likelihood terms in Equation (12). The result is therefore useful for stakeholders who need confidence that the pipeline is working."

The paragraph is not nonsense. It has a real comparison, a numerical result, and an appropriate desire not to overstate. It is nevertheless difficult for a reader arriving without the project history. "Phase-2 rescue" names a private event. "Like with like" does not identify the objects being compared. The negative definition delays the positive mechanism. "Stakeholders" is an abstract actor. The reader cannot tell whether the 35 basis points are an estimated effect, a calibration input, or an illustrative value.

The paragraph fails several blueprint and preflight checks. The private phase name is absent from the approved vocabulary; the comparison has no named objects; the number has no stated role; and the final sentence names no action. Those are located failures, not invitations to score the whole document. The writer receives the evidence and a repair question while the source remains unchanged. If the question cannot be answered from the brief and evidence ledger, the run stops rather than inventing a smoother claim.

13.3 The review record

Before anyone edits the paragraph, the parser maps its words to source spans and gives the equation reference, table reference, number, and declared result protected identities. The first review then records five findings. The table on the next page makes each one contestable.

ID	Location	Finding and evidence	Editorial question
F1	Sentence 1	"Phase-2 rescue" is a private status label; no source definition appears in the document.	What event does the reader need to know, and can it be named directly?
F2	Sentence 1	"Like with like" names a comparison without naming its objects.	Which likelihood terms, observations, or paths are compared?
F3	Sentence 2	The negative definition occupies the first clause and the positive mechanism is delayed.	What does the analysis establish in this document?
F4	Sentence 2	The number 35 and the references to Table 3 and Equation (12) are protected.	Are the unit, sign, and interpretation of the wedge explicit?
F5	Sentence 3	"Useful for stakeholders" asserts value without naming a decision or evidence.	Which reader action would change if the comparison passes?

Table 13.1: A finding is located, reasoned, and contestable. It is not an automatic rewrite instruction.

F1, F2, and the source-location part of F4 are deterministic. A parser can prove that a term is undefined in the current document. F3 and F5 remain editorial questions because the author must decide whether a qualification is necessary for an honest conclusion.

The author can reject F1 if the private label is deliberately part of the audience's vocabulary. That rejection is recorded with a reason; it is not silently discarded. The author can accept F2 and F5, then revise the paragraph in the existing editor. Humanvoice remains beside the work rather than taking ownership of it.

13.4 The author's revision

One possible revision is:

> "The constrained reconstruction compares the same likelihood components, observations, and counterfactual paths as the baseline model. In the illustrative calibration, the funding wedge is 35 basis points (Table 3), and the comparison tests whether the solver preserves that wedge and the stated likelihood terms (Equation (12)). If those objects agree, the note supports a larger comparative run; it does not by itself establish a general policy effect."

The revision makes the actors and objects visible, turns the number into a quantity with a unit, and gives the reader a decision consequence. It also retains a qualification because a reader could otherwise mistake a calibration check for a policy result. That qualification now follows the positive result instead of replacing it.

Before: "We do not claim that this proves the policy works in general; rather, it is a robust and transparent framework for checking the relevant objects."

After: "If those objects agree, the note supports a larger comparative run; it does not by itself establish a general policy effect."

The second sentence is shorter, but brevity is not the product's objective. The important change is that it states what the evidence can support. The author could choose a different wording and still answer the same audience and decision.

13.5 The protected comparison

After the author saves the revision, the comparison engine checks the objects a reader can least afford to lose while the prose is being polished.

The system does not freeze those objects. As Figure 13.3 shows, it classifies their correspondence and requires a human disposition when a consequential difference appears.

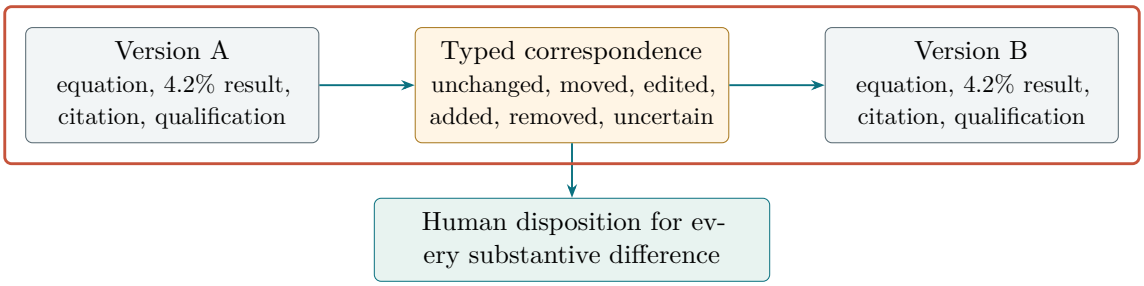

Figure 13.3: Protected comparison does not forbid change. It prevents a substantive change from disappearing inside fluent prose.

Protected object	Result	What the author sees
Equation body	unchanged	The normalized mathematical expression is unchanged; the surrounding explanation changed.
Numerical result	unchanged	35 basis points remains 35 basis points with the same sign and table anchor.
Citation and cross-reference	unchanged	Table 3 and Equation (12) still resolve to the same objects.
Claim scope	changed explicitly	"Supports a larger comparative run" is a new, narrower conclusion and requires author confirmation.
Private label	removed	The source map identifies the deletion and links it to F1.

Table 13.2: The diff distinguishes a safe explanatory repair from a substantive change that needs a human decision.

This is more informative than a red-and-green visual diff alone. A visual diff shows where characters moved; a protected comparison says which meaning-bearing objects survived and which changed. It also creates a useful evaluation record: the author accepted F1, F2, and F5, edited F3, and confirmed the preserved number and equation.

13.6 The reader's turn

The final step is not another model score. A reader who has not seen the project history receives the rendered note and answers five questions:

1. What comparison is being made?
2. What does the 35-basis-point quantity represent?
3. Which objects must agree before the team begins the larger run?
4. What does the paragraph not establish?
5. What action should the reader take if the checks pass?

Suppose the reader answers the first four correctly but says that the requested action is still unclear. That is a useful failure. The author has learned that the paragraph is more precise but the document's decision boundary is not yet visible. Humanvoice records a reader revision request rather than converting the four correct answers into a passing score.

The result of one review

For one document and one version, Humanvoice produces a source snapshot, a small set of located findings, an author decision for each finding, a protected before-and-after comparison, and a reader decision with elapsed time and requested changes. The packet is inspectable by an author, measurable by an evaluator, and priceable by a sponsor.

13.7 The six capabilities

For this case, an implementer needs six connected capabilities. Each one changes a real decision in the writing path; none is a list of packages to install.

Brief compiler. Turn the reader, decision, genre, prior knowledge, evidence boundary, protected objects, and named exemplars into a versioned authoring brief. Critical blanks remain visible and block drafting.

Argument planner. Build a claim map, concept-dependency graph, section jobs, terminology plan, and visual plan. It checks the order of ideas before prose makes the error expensive to find.

Bounded writer. Draft one section or paragraph card at a time from the approved brief and evidence. A replaceable model may write, but it cannot widen the claim, add a citation, or skip a prerequisite without a recorded decision.

Pre-human preflight. Run hard integrity checks, structural writing checks, evidence checks, and calibrated reconstruction critics. Unknown or conflicting results fail closed.

Located repair and comparison. Map a failed check to a specific repair, rebuild the affected unit, detect oscillation, and compare protected equations, numbers, citations, tables, quotations, and claims.

Review release. Produce a clean reader packet only after the gates pass, with the decision question and a short rubric but none of the private repair trail. The named human still owns acceptance.

The MVP does not need to train a new text-generation model. It needs a stable contract around a replaceable writer: the model receives only approved context, the unit it is writing, and the evidence it may use; the preflight controller can reject its output and request a cause-specific repair. This distinction is what keeps a fluent first draft from becoming a human debugging task.

13.8 The minimum useful interface

An author starts with a brief and evidence, not with a request to polish a finished manuscript (shown across lines for legibility):

```
hv init -brief brief.yaml -evidence sources/
hv plan -reader "investment committee"
hv draft -unit section-01
hv preflight -fail-closed
hv repair -max-cycles 3
hv release -reader-packet
```

The six verbs are stages of one path, not six independent products. `hv init` refuses a brief with unresolved critical fields. `hv plan` refuses to draft when a load-bearing claim has no evidence or a concept is used before its prerequisite. `hv draft` writes a bounded unit. `hv preflight` checks the unit and the assembled document. `hv repair` applies only a named repair strategy, rebuilds, and stops on oscillation. `hv release` creates the human packet only when every required result is pass or an explicit owner-approved exception. A missing result is a failure, not a pass.

The older capture, inspect, and compare services remain useful internally, but they no longer define the author's journey. The important property is that every service refers to an immutable brief, blueprint, source version, and render hash rather than to an unowned draft.

The same record should be usable from a command line, a notebook, or an editor extension. A sponsor should not need to know which parser or linter produced a finding in order to understand the decision. Conversely, an engineer should be able to replace a parser without changing the reader rubric or the primary evaluation outcome.

13.9 The boundary of the first version

The first version focuses on prepared authoring, revision, and reader understanding, not on judging whether an individual document is "human." Detector studies show high false-positive risk for formal and non-native writing and easy evasion through paraphrase [Liang et al., 2023, Weber-Wulff et al., 2023, Krishna et al., 2023]. A population-level monitoring question may be studied later, but it is outside the revision problem described here.

The system also leaves the source in the author's hands, keeps acceptance with the named reader, and treats style rules as observations rather than a universal house style. Bounded generation is in scope; unbounded autonomous rewriting and autonomous acceptance are not. Those choices keep the measurable outcome aligned with the reader's decision while ensuring that bad prose is stopped before it consumes that reader's time.

CHAPTER 14

How the product works

When an author starts a document, the team should be able to follow the work from the first brief to the final reader decision. Humanvoice keeps that correspondence between the intended audience, the argument plan, each bounded draft unit, the pre-human checks, the source and its rendering, and the outcome. Once the correspondence is stable, engineers can replace a parser or model without losing the story of why a paragraph exists or why it was released.

14.1 The flow of one version

The sequence follows the author's work: define the reader's task, plan the argument, draft in bounded units, preflight the result, repair located failures, and ask the reader only after release.

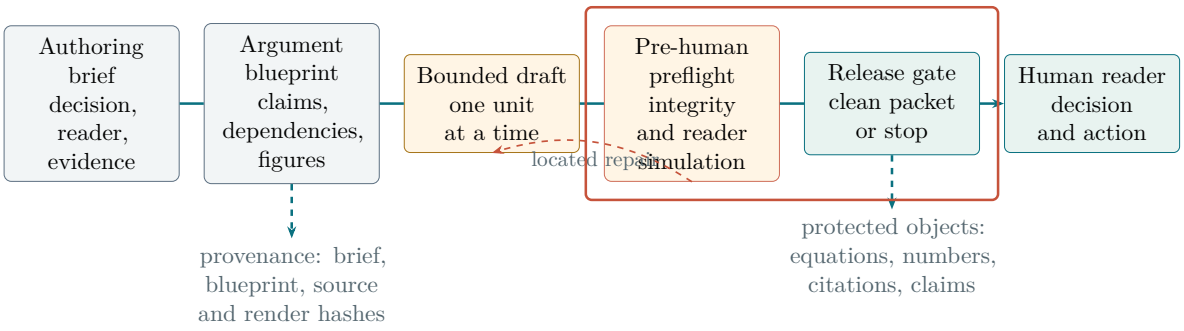

Figure 14.1: The common record keeps a flawed draft inside a bounded repair loop. Only a passed release gate reaches the expensive human reader.

The brief comes first because a finding means nothing without a purpose. The blueprint comes before prose because a fluent paragraph can still introduce a concept before its prerequisite or promise a result that the evidence cannot support. The source snapshot and protected spans are attached to each unit so that a repair remains reproducible. The reader decision comes last because it belongs to the released document, not to a model score or an internal queue.

14.2 The document representation

The MVP has one source format: LaTeX. The common representation can later admit Markdown or a word-processing adapter, but the first build earns that extension only

after it preserves locations and protected objects in LaTeX. The record model now starts before the source file. Each object has one clear owner and one reason it exists in the path.

Object	Required contents	Owner of the decision
AuthoringBrief	Intended reader, decision, genre, known vocabulary, named exemplars, evidence boundary, protected objects, privacy class, and unresolved assumptions.	Owner confirms that a missing answer is safe to carry forward.
Argument Blueprint	Claim map, evidence anchors, concept-dependency graph, section and unit jobs, terminology order, and visual plan.	Author and domain reviewer approve the argument before long-form drafting.
DocumentVersion	Source hash, source format, rendered hash, author, reading brief, creation time, and disclosure status.	Technical owner confirms the snapshot.
PreflightRun	Rule versions, build result, integrity findings, structural findings, evidence findings, reconstruction results, mutations, and abstentions.	Preflight controller records pass, repair, or fail-closed.
ProtectedSpan	Stable identifier, source range, object type, normalized content hash, and allowed transformations.	Author declares what must be preserved.
Finding	Source range, category, evidence, context, confidence or rule version, editorial question, and disposition.	Author or editor accepts, rejects, or edits.
RevisionRecord	Parent version, changed ranges, accepted findings, rejected findings, and explanation of substantive changes.	Author owns the wording.
ReleaseDecision	Gate results, exceptions, packet hash, unresolved risks, and the person who authorized release.	System blocks by default; named owner approves an explicit exception.
ReaderDecision	Reader role, document hash, rubric responses, elapsed time, acceptance or requested changes, and confidence.	Named reader accepts or returns the document.

Table 14.1: The record model connects intent, argument, prose, checks, and reader judgment. It is deliberately small enough to inspect and rich enough to stop an unfinished draft before release.

Protected objects include equations, numerical literals, units, labels, citations, quotations, tables, code blocks, declared assumptions, and load-bearing claims. The blueprint adds a second class of protection: concepts and claims that must be introduced before a dependent explanation. Protection does not make an object immutable. It makes a change visible and requires an explanation. An author may intentionally change an estimate or replace a citation; the system must distinguish that action from silent loss.

14.3 The pre-human release contract

The product is successful only if it changes when a human is asked to read. A human may inspect a brief, resolve a high-consequence ambiguity, or adjudicate a meaning question during construction. The expensive reader should not be asked to find ordinary defects that the system could have found before release.

Every candidate version passes four gates:

1. **Integrity.** The source builds reproducibly; all protected spans are located or explicitly marked unparsed; citations and references resolve; and no seeded change to a number, unit, sign, equation, table, quotation, or claim scope can pass silently.
2. **Argument.** Every load-bearing claim has an evidence anchor or is labelled a hypothesis; every section has a job; every new concept has a prerequisite or an explicit exemption; and each figure has a declared relationship to teach.
3. **Writing.** Bounded units put concrete actors and objects before abstractions, keep the declared register, vary sentence and paragraph pace, and stay within the concept and evidence budget in the blueprint. These are calibrated checks against named exemplars, not a universal style score.
4. **Reconstruction.** Isolated critics, calibrated on held-out human judgements, attempt to recover the purpose, mechanism, evidence, qualification, and requested action. They must report uncertainty and abstain when the evidence is insufficient.

The release rule is fail-closed: a failed or missing gate produces a located repair task, not a human packet. Repair has a bounded cycle count and an oscillation detector. If the same defect returns, or two repairs undo one another, the system stops and names the unresolved choice for the author. This does not certify that the prose is beautiful. It prevents predictable failure from being purchased with an executive's time.

Only the sponsor or named project owner may grant an exception. A meaning exception also needs the domain editor's concurrence, and a processing exception needs the security or policy owner's concurrence. Unresolved protected correspondence, an unparsed promised object, unauthorized external transmission, missing load-bearing evidence, and a broken build can never be excepted. The full authority and record fields are fixed in Appendix F.

The human-attention budget

The ordinary production path asks a human for a brief confirmation and one final reading. It escalates earlier only for a decision the machine cannot own: the intended meaning, a disputed source, or an explicit exception. The product measures those escalations, rather than quietly treating them as free.

14.4 The diagnostic layer

The first release needs only diagnostics that support a real author decision.

Register leakage. Find private identifiers, file paths, status labels, and dates used as unexplained scope markers. Emit the source span and a question about the reader's missing context.

Structural repetition. Measure repeated section and paragraph shapes, opener distributions, and transition phrases against a genre reference. Report patterns for inspection; never impose a universal frequency target.

Defensive or unsupported specificity. Identify negative definitions, unsupported actor labels, and claims whose evidence or mechanism is not visible in the current document. These are candidate questions, not automatic edits.

Paragraph packing and conceptual pacing. Count the mechanisms, citations, and first-introduced concepts carried by each prose paragraph. Compare the target with the exemplars named in the reading brief and return ranked passages, not a universal score.

Dependency order. Check the first use of a term against known vocabulary, an introduction signal, a taught-later pointer, and a declared exemption zone. An uncertain match is an abstention for the author.

Citation identity and build. Resolve bibliography identity, unresolved references, and compilation failures. Queue sentence-to-source support for human verification.

During preflight, the comparison also looks for newly added numbers, superlatives, comparisons, and causal verbs. An addition with no citation, elementary derivation, or pointer to an existing protected claim becomes a located assertion question. This catches unsupported specificity introduced during an explanation pass without pretending that the detector can establish the truth of the new sentence.

The critics are not one general-purpose judge. One checks concept order, one checks evidence and scope, one checks concrete actors and mechanism, and one attempts a low-context reconstruction. Their prompts, calibration set, order, and disagreements are recorded. A critic that cannot beat a frozen baseline on held-out examples is removed from the release path.

The MVP deliberately excludes per-document AI detection, population drift monitoring, unbounded autonomous rewriting, and a scalar acceptance score. Bounded drafting and cause-specific repair are core product behavior; autonomous acceptance is not. These exclusions keep the first decision measurable without asking a model to certify its own prose.

14.5 The author and reader experience

The author opens one construction session and sees the brief, blueprint, source, and rendered unit together. Findings are grouped by location and gate, with the most consequential blockers first. Selecting a finding shows its evidence, the dependency or protected object it concerns, and the repair it will trigger. The author can revise in the existing editor, or ask the bounded writer to produce a new unit; Humanvoice rebuilds and preflights the result before it advances.

The reader never sees the internal finding identifiers by default. The reader receives the rendered document, the declared decision question, and a short rubric only after the release gate passes. A reader may request a change without diagnosing its cause. That is important: the product should not force the funder to become a copy editor in order to provide useful feedback, nor should it use that funder as a routine preflight service.

The session record keeps the two views linked. It can later answer questions such as: which finding led to this sentence change; did the equation survive; how long did the author spend resolving the queue; and did the reader accept the resulting version? It cannot answer whether a different wording would have been better unless the team runs that comparison.

14.6 Privacy and disclosure

Technical documents may contain unpublished estimates, client information, or internal decisions. The default path is local parsing, local comparison, and local storage. A remote model call is optional and explicit. Each call records the provider, purpose, input class, retention setting, and approval. A project may forbid remote calls entirely; the deterministic path must still be useful.

The system stores hashes and minimum necessary excerpts in the evaluation record. It should not retain a full manuscript in a telemetry stream merely because a finding was emitted. A reader study may require the rendered document, but that is a declared study object with a retention decision, not an unnoticed side effect of a linter.

14.7 The first release and its boundary

The MVP is complete when one LaTeX document can travel from brief to review-ready packet through the full path. It includes:

1. a brief schema and blueprint compiler with claim, dependency, and visual fixtures;
2. a parser adapter with protected-span fixtures and source locations;
3. a bounded writer interface that accepts only approved unit context;
4. the integrity, argument, writing, and reconstruction preflight families, with stable machine-readable output;

5. a cause-specific repair controller with cycle and oscillation limits;
6. an author decision view, protected comparison, and fail-closed release gate; and
7. a small calibration and held-out corpus plus a replay harness for the feasibility case.

It does not include a general editor, cloud collaboration, enterprise authentication, autonomous acceptance, authorship classification, or a population-level style dashboard. It also does not promise a subjective prose certificate. Those may be sensible later products, but including them now would make the causal question and the implementation boundary harder to see.

14.8 Failure handling

The system must fail visibly. If parsing loses a macro, it returns an unparsed-source status and blocks release. If a citation identity cannot be resolved, it enters a human queue rather than being silently accepted. If a remote call is unavailable, the local diagnostics continue but the missing result remains an abstention that blocks any gate that depends on it. If a protected object changes, the author must explain the change before the reader packet can be released. If a repair cycle repeats a failure, the controller stops rather than polishing the same defect indefinitely.

These are engineering controls, not a substitute for scholarship. A passing fixture proves that the implementation handled that fixture. It does not prove that the resulting prose is clear or that a reader will agree with the proposal. The evaluation chapter keeps those judgments separate.

14.9 The build path at a glance

The canonical acceptance register appears once in Appendix B.2. The implementation contract in Appendix F defines how those requirements are executed, and `schemas/` provides the machine-readable records. The shorter table below is a reader-facing map of the build path; it deliberately does not create a second set of release tests.

Stage	What the team produces	What happens if it fails
Brief and blueprint	A reader, decision, evidence boundary, concept order, and visual purpose are explicit before long-form prose.	Stop before writer invocation and ask the owner to resolve the missing field.
Protected core	A LaTeX source snapshot, canonical PDF, source map, and protected comparison can be replayed by a second operator.	Narrow the first release to the protected review utility.
Bounded extension	One approved unit can be drafted, rebuilt, and repaired without widening its evidence or claim.	Keep model-dependent work in abstention and return to the deterministic path.
Pre-human release	Integrity, argument, writing, evidence, and reconstruction checks produce a located decision before a reader packet exists.	Repair or stop; never use the reader as the ordinary debugger.
Reader packet	A named reader receives one clean PDF, decision question, and short reconstruction task.	Preserve the source and report noncompletion.
Feasibility handoff	The sponsor receives burden, safety, rights, cost, and proceed/revise/stop evidence.	Do not make an efficacy or scale claim.

Table 14.2: The reader-facing map of the implementation path. The canonical acceptance register is in Appendix B.2; exit codes and execution boundaries are in the implementation contract annex.

The live feasibility document starts only after these tests pass. If the tests pass and the reader still cannot state the purpose or requested action, the workflow has failed its product purpose even though its software is running.

CHAPTER 15

Architecture for reversible review

Open a finished review packet and four things should be easy to find: the version the reader saw, the changes the author made, the technical objects that survived, and the reader's decision. The packet exists only because an earlier brief, blueprint, and pre-human release record cleared the path. Parsers, linters, models, and build tools support those steps; none of them owns the document.

That practical test drives the engineering. The system records explicit correspondence and abstains when it cannot establish one. The author sees context and alternatives. The reader sees a clean document and a short task.

Figure 15.1 shows the minimum set of relationships. A larger schema would add fields, but it would not change the ownership of the arrows.

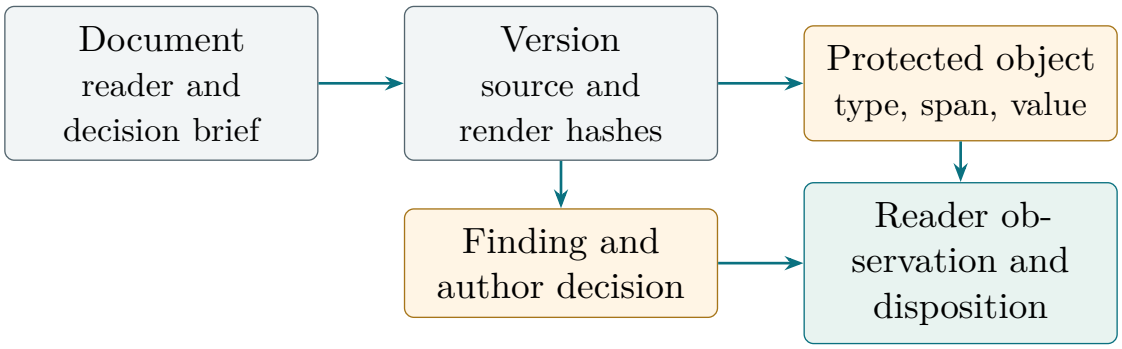

Figure 15.1: A small record model links source facts to editorial choices and reader outcomes without treating any diagnostic as a verdict.

15.1 The objects that must survive a revision

The first release needs eleven records. Their names are technical because the software stores them, not because the finished document should expose them as labels.

Authoring brief The intended reader, decision, genre, known vocabulary, named exemplars, evidence boundary, privacy class, protected objects, and explicit unknowns.

Argument blueprint The claim map, evidence anchors, concept-dependency graph, section jobs, terminology order, and visual plan approved before long-form drafting.

Document The stable identity of a work, its genre, intended reader, and declared

decision.

Source snapshot The exact source tree, build environment, compiler output, and content hash used for a version.

Protected object An equation, number, label, citation, table, quotation, code block, or qualification with a source span and normalized form.

Finding A located passage, an observable reason for attention, an optional machine suggestion, and the author's decision.

Revision A source snapshot derived from a parent version, with the author, time, changed spans, and explanations.

Reader session A rendered version, reader role, context supplied, task questions, time observations, and decision.

Preflight run The build, integrity, argument, writing, evidence, reconstruction, mutation, abstention, and repair results for a candidate version.

Release decision The gate results, explicit exceptions, packet hash, unresolved risks, and named owner of release.

Evidence item A source or software observation tied to a claim, a fixture, or a product choice, with provenance and appraisal state.

The relationships are more important than the field names. A brief owns a blueprint; a blueprint owns bounded units; a finding belongs to a source snapshot and a preflight run; a revision answers one or more findings; a protected-object comparison links parent and child snapshots; and a reader session sees only one released version at a time. An evidence item may motivate a finding rule or a reader question, but it cannot silently alter a document.

Let D be a document, s a source snapshot, and r a revision. The basic lineage is

$$D \longleftarrow B \longrightarrow P \longrightarrow s_0 \xrightarrow{r_1} s_1 \longrightarrow \text{preflight} \longrightarrow \text{release} \longrightarrow \text{reader session}. \tag{15.1}$$

The arrows carry a person and a time. A generated suggestion is not an arrow until it is accepted into a bounded unit and the source changes; a candidate is not a release until its preflight record passes. This simple rule makes it possible to answer a later question such as "who changed the denominator in Table 3, and why?" without reading a prompt transcript.

15.2 The source snapshot and the canonical build

When an author opens a version, the system must be able to reproduce the page the reader will see. It therefore stores the whole source directory, not just a hash of the top file: entry point, included files, bibliography, image hashes, compiler version, package list, relevant environment variables, and build log. The rendered PDF belongs to the version because readers judge the page, not the source alone.

The build wrapper performs the checks in a fixed order. It first validates that the requested entry point names one reader-facing route and that the source is a read-only snapshot. It then compiles with `-no-shell-escape`, inside a network-denied scratch directory, twice around bibliography generation, checks references and citations, and finally records the PDF hash and page count. A clean build is useful evidence about typography and reproducibility. It says nothing about whether the argument is persuasive, so the wrapper reports it as a build result rather than as a reader verdict. The complete trust-boundary contract appears in Appendix F.

The snapshot also stores a detexed prose view for diagnostics. Detexing is a lossy transformation. Equations, tables, captions, and source locations remain in the canonical source and PDF; the prose view is allowed to answer only questions about words, sentence boundaries, and paragraph structure. A finding created from the prose view links back to its source span before it can enter an author's queue.

Snapshot field	Example value	Why the field matters
Entry point	`docs/survey/humanvoice_survey.tex`	Establishes which document the reader is being asked to judge
Included source hash	Hash over route files	Prevents an unrecorded include from changing the evidence
Build environment	pdfTeX version, packages, date epoch	Makes a PDF replayable and distinguishes source from rendering noise
Protected manifest	Object IDs, spans, normalized values	Makes a changed number or equation visible
Authoring brief	Role, decision, time budget, disclosure, evidence boundary	Defines the task and the material the writer may use
Blueprint hash	Claims, dependencies, unit jobs, visual plan	Makes the argument order and writer context reproducible
Preflight and release	Gate results, mutations, abstentions, packet hash	Shows why the reader was asked to spend time

Table 15.1: A snapshot joins the source, its rendering, and the reader task.

15.3 Protected objects and correspondence

Before an author accepts a revision, the system must find every equation, number, label, citation, table, quotation, code block, and qualification whose silent alteration could change the argument. The extractor scans the source and rendered representations, assigns each object a stable local ID, and keeps the raw span beside a normalized value. It may ignore whitespace around an equation operator; it may not ignore a changed exponent, sign, unit, or table denominator.

For object p in version v, let $raw_v(p)$ be the source span, $norm_v(p)$ the normalized representation, and $loc_v(p)$ its source and PDF location. A candidate correspondence from parent v to child v' is accepted only if the object type, semantic anchor, and local context agree:

$$match(p_v, p_{v'}) = 1\{type_v = type_{v'}\}\, 1\{anchor_v \approx anchor_{v'}\}\, 1\{context_v \approx context_{v'}\}. \tag{15.2}$$

The relation $\approx$ is intentionally typed. For a citation it can use the key and bibliography identity; for a table cell it can use row and column labels; for an equation it can use its environment, label, and neighboring text. When the relation is ambiguous, the extractor returns an unresolved match for the author rather than choosing the highest textual similarity.

The comparison result has four ordinary-language outcomes: unchanged, explained change, added or removed with an author explanation, and unresolved. The last outcome blocks a release of the revision packet. It is the one place where abstention is more valuable than coverage.

algebraic equivalence is not string equality

The parent writes $a/(b+c)$ and the child writes $(b+c)^{-1}a$. A character diff reports a large change. A parser may establish algebraic equivalence under the declared commutativity assumptions, but it must also show the author the assumptions and the source locations. If the parent instead writes $a/(b-c)$, the same normalizer must not silently identify the two forms. The first release can conservatively mark both changes for review; it need not solve symbolic mathematics to protect a document.

Numbers receive an analogous treatment. The normalized record stores value, unit, sign, precision, and a relation to a table, equation, or prose claim. Changing “3.0 hours” to “3 hours” is a formatting change if the precision policy permits it. Changing “3.0” to “30” is a substantive change even when the surrounding paragraph is identical. The user interface should show both the raw and normalized forms so that a reader can tell which rule was applied.

15.4 Findings as questions, not verdicts

When a passage deserves attention, the author needs more than a score. The finding layer combines deterministic rules, corpus statistics, and optional model suggestions to provide a location, an observable reason, a consequence hypothesis, and a next action. The hypothesis does not declare the passage wrong; it explains why the passage may deserve a human’s time.

For finding f, write

$$f = (loc, rule, observation, consequence, question, suggestion), \tag{15.3}$$

where *suggestion* may be empty. A useful finding can therefore read:

> The phrase “frozen interface” occurs before the comparison is defined. A reader without the project history may not know which inputs are held fixed. Should the paragraph name the baseline and the changed mechanism first?

The finding does not read “AI phrase detected” or “bad writing.” Those labels collapse an observation, a genre judgment, and a remedy into one authority claim. They also invite authors to optimize the label rather than repair the passage.

Findings are ranked by expected reader consequence and review cost. A practical priority function for the pilot is

$$priority(f) = \frac{p_f \, c_f}{1 + b_f}, \tag{15.4}$$

where p_f is the adjudicated probability that the issue blocks the reader’s task, c_f is its consequence if left unrepaired, and b_f is the expected author burden. The terms are estimates used to order a queue, not a quality score. The pilot should compare this ordering with the order in which readers actually report problems.

15.5 Three views with different responsibilities

The author view is a source-aware workbench. It shows the passage, source location, rendered context, protected objects nearby, and any model suggestion. The author can accept, reject, defer, or rewrite. A rejection asks for a short reason, not a long compliance form. The view should make the next useful edit obvious without forcing the author to inspect every low-confidence rule hit.

The reviewer view is narrower. An independent reviewer sees the source claim, the cited evidence, the change explanation, and the protected comparison. The reviewer can confirm the correspondence, ask for a correction, or mark the claim as not checked. The reviewer does not need the author’s entire prompt history.

The reader view is deliberately plain. It is the rendered proposal, its figures and references, and a short decision context. It does not show finding IDs, model confidence, parser warnings, or internal phase names. At the end of the reading task, the reader records the decision, the first unresolved question, and the time spent. This separation is a product requirement: a reader who sees the backstage machinery is no longer independent of the project.

View	Primary question	Information deliberately omitted
Author	What should I investigate or change, and what meaning must I protect?	Other readers' private judgments and irrelevant rule hits
Reviewer	Is the source, change, and evidence correspondence defensible?	Prompt transcripts and a global style score
Reader	Can I understand the decision and act on this document?	Source history, finding IDs, model confidence, and internal project language

Table 15.2: The same revision has three views because the three people have different jobs.

15.6 Local inference and the trust boundary

The product is local-first because technical documents often contain private data, unpublished results, or contractual material. Local-first does not mean that every computation must run on a laptop. It means that the default path keeps the source inside an explicitly declared boundary and requires an authorization before a remote service sees content.

The inference adapter records model family, model file or endpoint, prompt template, sampling parameters, network state, and output hash. It passes only the context needed for the finding. A citation identity lookup may use a public identifier without sending the surrounding manuscript. A model suggestion may be generated on a redacted passage. The author decides whether the suggestion is useful; the adapter never writes directly to the canonical source.

Privacy is a property of the data flow. Let X be the source, Z a redacted context, and Y a model response. A local run has $X \to Z \to Y$ inside the declared boundary. A remote run adds an external recipient E and requires explicit authorization:

$$X \to Z \to Y \quad \text{(local)}, \qquad X \to Z \to E \to Y \quad \text{(remote, authorized only)}. \tag{15.5}$$

The diagram is a data-flow statement, not a promise that redaction is perfect. The privacy review must test whether equations, rare identifiers, and quoted sentences can re-identify a source even after obvious names are removed.

15.7 Interfaces and commands

The first command surface has the same six verbs introduced in Chapter 13 and specified in Chapter 16:

`hv init` Validates the authoring brief, evidence boundary, privacy class, and protected-object declarations. Critical blanks block the writer.

`hv plan` Builds the claim map, dependency graph, unit jobs, terminology order, and visual plan. Missing warrants become questions.

`hv draft` Invokes the replaceable writer for one approved unit and records the context and evidence it received.

`hv preflight` Runs build, integrity, argument, writing, evidence, reconstruction, and mutation checks. Unknown results remain blockers.

`hv repair` Applies a named repair, rebuilds, reruns preflight, and stops on a cycle limit or oscillation.

`hv release` Presents a clean version and records the named reader's task outcome only after the release gate passes. A report is an export of this record, not another top-level command.

The commands exchange versioned JSON records, not hidden global state. A minimal finding record can be represented as

```
{
  "finding_id": "F-014",
  "source": "chapter.tex:87",
  "rule": "private-register",
  "observation": "phase label precedes public definition",
  "question": "name the comparison before the project history",
  "author_decision": "accepted",
  "revision_id": "R-003"
}
```

The record schemas and compatibility rule are fixed in Appendix F and in `schemas/`. A minor schema version may add optional fields; a major change requires migration and fixture replay. The invariant is that a later reader can locate the passage and the author decision without opening an orchestration log.

15.8 Failure handling and observability

The system should fail in ways that help a person decide what to do next. An unknown macro yields "source not parsed" with a location. A missing citation identifier yields "identity unresolved" with the key. A protected equation that cannot be matched yields "comparison unresolved" and blocks the release gate. A local model that exceeds the memory envelope yields a clear fallback to deterministic findings, but any gate that depends on the model remains abstained. A failed dependency or reconstruction check yields a located repair, not a reader packet. None of these states should be phrased as a successful review with a small confidence score.

When a parser cannot read a macro, the build log should say so. When a model falls outside the memory envelope, the inference adapter should record the fallback. The build log records compiler errors and warnings, the parser records unsupported constructs, the

adapter records latency, memory, and network policy, and the reader protocol records task completion and questions. Those records are what we mean by observability. A single "quality" number would hide the failed boundary and make repair harder.

The operational envelope for the feasibility build is intentionally modest and is stated numerically in Appendix F: a 300-page cap, 2,000 protected objects, bounded scratch space, and fixed deterministic and model-assisted time limits on the reference machine. Those are planning caps, not performance claims. Week six records the actual distributions and either revises the caps openly or narrows the supported document class.

15.9 The result of this architecture

Follow one document through the system and the promise becomes testable. The brief identifies the reader and decision; the blueprint explains why each unit exists; a bounded draft records its evidence; preflight exposes a failure before release; a revision records a human choice; a protected comparison exposes a meaning-bearing change; and a clean reader view supplies the outcome. Engineers can replace a component as long as it respects those boundaries.

The architecture does not guarantee a persuasive document. It makes the failure legible and makes a comparative experiment possible. That is the appropriate ambition for the first investment. The next Part specifies the experiment, the economic decision rule, and the operating work required if the path earns a larger trial.

CHAPTER 16

The first build

The first version should solve one complete problem before it asks an executive to read the result. Give it an authoring brief, the evidence the document may use, the source material, and the objects an author cannot afford to change silently. Humanvoice should return an argument blueprint, bounded draft units, a reproducible rendering, a pre-human preflight record, cause-specific repairs, and a reader packet tied to the exact released version. If it cannot establish a correspondence or defend the argument, it should stop before the document reaches the reader.

That vertical slice is smaller than the platform described in earlier drafts. It is also more useful. A team can watch one consequential document travel from brief to release, measure the work at each step, and return to the incumbent workflow when the system abstains. The human is still the final authority, but the human is no longer the default place where ordinary prose defects are discovered.

The dependency graph in Figure 16.1 shows the critical path. The protected LaTeX core comes before corpus scale, optional model features, and any claim about comparative effectiveness.

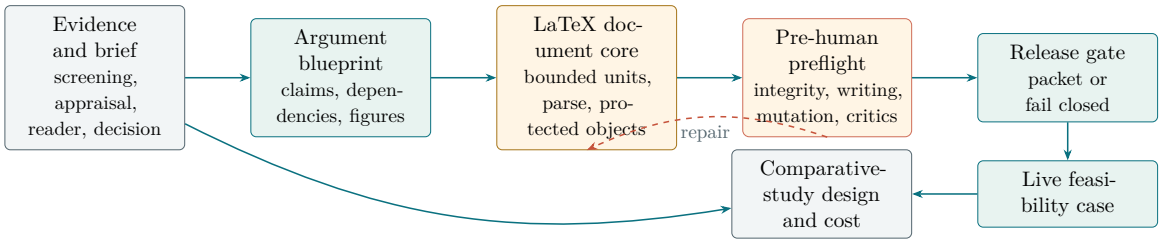

Figure 16.1: The first build has one critical path. A brief and blueprint precede bounded drafting, and a failed pre-human check returns to repair instead of emitting a reader packet.

16.1 Ten commitments that shape the code

The evidence narrows a credible implementation to ten commitments. Engineers still choose the components, data structures, and interface within those constraints.

Commitment	Product behavior	Observation that can overturn the choice
One immutable version	Findings, objects, the PDF, and the reader session all refer to the same source and render hashes.	A replay produces different locations, objects, or pages from the recorded environment.
Brief before prose	A critical blank in reader, decision, evidence boundary, or protected objects blocks long-form drafting.	A fluent draft is produced from an incomplete or contradictory brief.
Blueprint before units	Claims, dependencies, section jobs, terminology, and figures are checked before a writer inherits them.	A unit uses an untaught concept, repeats a job, or makes a claim with no evidence anchor.
Bounded writing	The writer receives one approved unit at a time and cannot widen the claim or invent evidence.	A whole-document prompt hides a failure until the reader sees it.
Preflight before people	Integrity, argument, writing, and reconstruction gates must pass before a reader packet exists.	A seeded defect reaches the packet, or an unknown result is treated as pass.
Questions before rewrites	A diagnostic reports a span, evidence, and an editorial question; inspection never changes source.	Authors cannot act on the finding without asking a model to reconstruct its context.
Protected change, not frozen text	Equations, numbers, labels, citations, tables, quotations, and declared claims can change only with a visible disposition.	A seeded material change passes without notice.
Different human decisions	The author owns meaning and revision; the named reader owns whether the document supports the task.	The interface pressures either person to accept a system score or hides disagreement.
Local by default	Source, findings, diffs, and reader data remain local unless an owner approves named fields for a named service.	A dependency requires an undeclared network path.
Claims rise with evidence	Replays establish regression, held-out fixtures establish operational fit, and later comparison estimates efficacy.	A historical case is used to claim general effectiveness.

Table 16.1: The main design commitments, stated as behavior and possible falsification rather than internal requirement codes.

The machine-readable requirements retain stable identifiers in the appendix and repository. The main argument uses names because a reader needs to understand the tradeoff

before tracing its implementation ID.

16.2 The vertical slice

The slice begins with an authoring brief. The author names the document genre, the intended reader, the decision, the concepts the reader can reasonably be expected to know, the named exemplars that set the register, the evidence boundary, and the objects that must survive revision. Humanvoice compiles that brief into a claim map, concept-dependency graph, section cards, terminology plan, and visual plan. A missing critical field blocks the long draft and leaves an explicit question for the owner.

Only then does the system capture the source tree and canonical PDF. The primary parser extracts source locations and protected objects; a second adapter or the compiled representation checks the result. Unsupported constructs remain explicit abstentions and block any gate that relies on them.

The first diagnostic set is deliberately small. It locates private repository language, undefined project labels, broken references, conspicuous repeated openings, paragraph packing, concept bursts, first-use ordering, and changes to protected objects. These are observable conditions with stable locations. The HPO case supplies calibration fixtures for the packing, pacing, and ordering families; it does not turn their metrics into a pass/fail score. A more interpretive question, such as whether a qualification replaces a reason, may enter the feasibility interface only after human calibration shows that the queue repays the attention it consumes.

The writer produces one unit from the approved plan, and the author can revise that unit in the existing editor. Humanvoice records accept, reject, revise, or defer for each finding, rebuilds the unit, and compares the new source with its parent. Preflight critics attempt reconstruction before release. Only then does the reader receive the clean PDF and a short reconstruction task in a local browser. The event record joins the response to the version without showing private rule names, confidence scores, or project history.

Stage	Input	Result	Failure behavior
Brief	Reader, decision, evidence boundary, exemplars, and protected objects	Versioned authoring contract.	Stop on a missing critical answer; do not draft the long document.
Plan	Brief plus source inventory	Claim map, dependency graph, section cards, and visual plan.	Block a claim without evidence or a concept used too early.
Draft	One approved unit and its evidence	Rebuildable prose and source map.	Keep the unit internal until its checks pass.
Preflight	Draft, protected manifest, build, critics, and mutations	Gate record with located failures and abstentions.	Fail closed; no reader packet.
Repair	Gate failure and named repair strategy	New unit, comparison, and convergence record.	Stop on cycle limit or oscillation and escalate to owner.
Release and read	Passed gates, clean PDF, decision context, and timed task	One reader packet and a human decision.	Preserve noncompletion and decline as outcomes rather than deleting the session.

Table 16.2: The complete first-release path and its explicit fallbacks.

16.3 The boundary of the first release

Twelve weeks buys a LaTeX-first implementation. It includes one primary parser adapter and one fallback, the canonical build, the protected-object manifest, the bounded diagnostics above, author dispositions, the protected comparison, a self-hosted reader task, and an exportable event record. The parser bake-off includes pylatexenc, unified-latex, TexSoup, and the richer document-model candidates described in Chapter 11; a package wins only by passing the locked fixtures with acceptable operator burden.

Markdown waits. So do a general editor extension, unbounded autonomous rewriting, continuous corpus-drift monitoring, broad rhetorical-move inference, and a market-facing multi-tenant service. Local model suggestions remain an optional experiment behind the same brief, blueprint, preflight, and author-decision record; the deterministic path must still be useful. Single-document authorship classification is not a deferred feature. It is outside the product boundary.

The narrow scope produces a useful fallback. If the diagnostic layer fails to earn attention, the source snapshot and protected comparison can still become a small review utility. If the parser fails too often for full correspondence, the reader task can still test whether structured low-context review is useful. A platform with seven interdependent commands would make those negative results harder to learn from and more expensive to salvage.

16.4 A small command surface

The interface exposes six verbs. Their names are less important than their contracts, and each one has a fail-closed boundary.

`hv init` Validates the authoring brief, evidence boundary, privacy class, and protected-object declarations. It refuses a missing critical field.

`hv plan` Builds and checks the claim map, concept-dependency graph, section cards, terminology order, and visual plan. It emits questions, not prose, when the evidence is insufficient.

`hv draft` Invokes a replaceable writer for one approved unit; the unit receives only the context and evidence named in the blueprint.

`hv preflight` Runs integrity, argument, writing, evidence, reconstruction, and mutation checks. Unknown results remain blockers.

`hv repair` Applies a named repair strategy, rebuilds the unit, reruns preflight, and stops on the cycle limit or oscillation. It writes a child revision and never edits the canonical source in place.

`hv release` Emits the clean reader packet and launches the local timed task only after the release gate passes. It records completion, response, confidence, and decision against one immutable PDF.

The older capture, inspect, and compare services remain internal adapters for these commands. Citation identity and build diagnostics are part of preflight, not optional reports. Population monitoring and research-corpus analysis live in the evaluation code, not in an author's release command. This keeps a research instrument from acquiring editorial authority through a convenient interface.

16.5 Regression, calibration, and held-out validation

The earlier proposal asked historical documents to do three incompatible jobs. The revised plan separates them.

Regression fixtures. Frozen ZLB snapshots test whether the software can reproduce recorded facts: the 133-dash snapshot, the distribution of section openings, the 109 unchanged equation bodies, and the deliberately added references. These tests catch implementation drift. They say nothing about performance on a new document.

Calibration documents. A declared training partition supplies examples for rule wording, thresholds, source-location repair, and reviewer instructions. Every tuning decision is logged before the held-out partition is opened. A document that changes a rule can no longer estimate that rule's operational value.

Pre-human mutation tests. Before a human sees a packet, the harness injects known failures into otherwise valid plans and drafts: a missing evidence anchor, a use-before-teach concept, a dense paragraph that exceeds its declared concept budget, a citation substitution, a changed sign, a broken reference, an internal project label, and a repair that oscillates. Critical deterministic mutations must be caught or explicitly abstained from. Rhetorical mutations are reported with confidence intervals and a frozen threshold chosen on calibration data; no threshold is tuned on the live case.

Held-out operational validation. New non-ZLB fixtures include custom macros, malformed environments, moved equations, changed signs and units, valid citations with unsupported claims, legitimate repeated prose, private labels a reader does not know, and the HPO first-use positives and adjudicated false alarms. A second operator replays the package adapters from a clean environment. Held-out authors judge whether each prose finding was worth attention, and those authors are not the people who wrote the rule.

The corpus is not automatically reusable merely because it is available in the repository. Each item receives an owner, source hash, licence or written permission, permitted use, redistribution status, retention class, and consent date. Material marked pending or restricted may support an internal fixture only under its owner's permission; it cannot enter a published benchmark, external reader packet, or commercial training set until the rights record is cleared.

The engineering criteria are exact where safety permits exactness. The protected-source core is the week-six gate; the bounded authoring extension is built only if that gate passes. The normative command, runtime, security, schema, budget, rights, and exception rules live in Appendix F.

1. an incomplete critical brief or blueprint blocks model invocation and long-form drafting;
2. every declared protected object is either located or explicitly marked unparsed; no fixture receives a false protected pass;
3. every seeded change of sign, exponent, unit, number, citation identity, label target, quotation, table denominator, or declared claim scope is reported at the correct version and source location;
4. capture and inspect leave the source tree byte-identical;
5. a second operator reproduces the canonical PDF hash or receives a recorded environment difference that prevents promotion;
6. a preflight critic remains in the release path only when its held-out false-negative and false-positive behavior is reported against human labels; and
7. a diagnostic remains in the author queue only when its held-out review time is lower than the time spent finding and resolving the same true issues under the incumbent process.

The seventh criterion replaces phrases such as "fires heavily" or "low enough not to annoy

an editor." Let B_k be the minutes spent triaging rule family k on held-out documents and U_k the incumbent minutes spent locating and resolving the same adjudicated issues. Retain the family only when

$$\operatorname{median}(U_k - B_k) > 0, \tag{16.1}$$

with the complete distribution and the small sample reported. The inequality is an operating rule, not a population claim. Any silent protected-object failure overrides it.

16.6 The work in calendar order

Weeks 1–2 freeze the live-case brief, evidence boundary, blueprint schema, incumbent process, corpus split, protected-object vocabulary, and parser fixtures. Weeks 3–5 implement planning, bounded drafting, capture, and compare for LaTeX. Weeks 4–7 overlap the adapter benchmark, mutation harness, and preflight calibration. Weeks 6–9 connect cause-specific repair, author dispositions, and the local reader task. Weeks 10–12 run the live feasibility case, account for every failed gate and minute of burden, and price the comparative study. Figure 19.1 and Table 19.1 are the single calendar for these intervals.

The evidence review proceeds alongside the build without pretending that automation completes scholarly judgment. The research lead imports the four new cornerstone sources and all snowball candidates, while two people screen load-bearing records independently. The domain reviewer appraises source-to- claim fit; a different reviewer adjudicates disputed product findings. Package adapters receive held-out tests before they enter the live path. External review remains a named dependency, not a status that the implementation team can award itself.

16.7 Deliverables after twelve weeks

The sponsor receives working software, not a promise of a platform: a reproducible LaTeX authoring path from brief to release; a blueprint and bounded writer contract; a locked tuning, mutation, and held-out corpus; package and critic scorecards with failures and operator burden; one complete feasibility record; the observed baseline inputs for the economic model; and a proceed, repair, or stop memorandum. The comparative study is specified and priced, not smuggled into the feasibility claim.

Continue when the system protects the source, authors can use the findings at net lower burden, the reader task produces an interpretable outcome, and the expected value of a comparison exceeds its cost. Repair when one replaceable adapter or interface fails. Stop the product direction when protected correspondence cannot be trusted, the reader outcome cannot be observed, or the workflow consistently adds attention rather than saving it.

Part IV

Testing the proposition

CHAPTER 17

The test and its economics

The product changes a chain of work: an author revises, a reader reads, and a sponsor decides whether the result is worth keeping. Document scores observe only one part of that chain, so the evaluation compares the complete workflow. The central question is:

> Does the Humanvoice workflow help more documents reach a reader decision within a fixed horizon, or reduce the attention required to get there, without increasing meaning-preservation failures relative to the incumbent bundle?

One live document cannot answer the causal question. It can show that the system runs, that people can use its records, and that the team can observe burden and failures. An efficacy estimate requires a later comparison across documents and workflows.

Figure 17.1 marks the decision boundary. The live case can expose integration failures and burden; only a later comparison can estimate an effect.

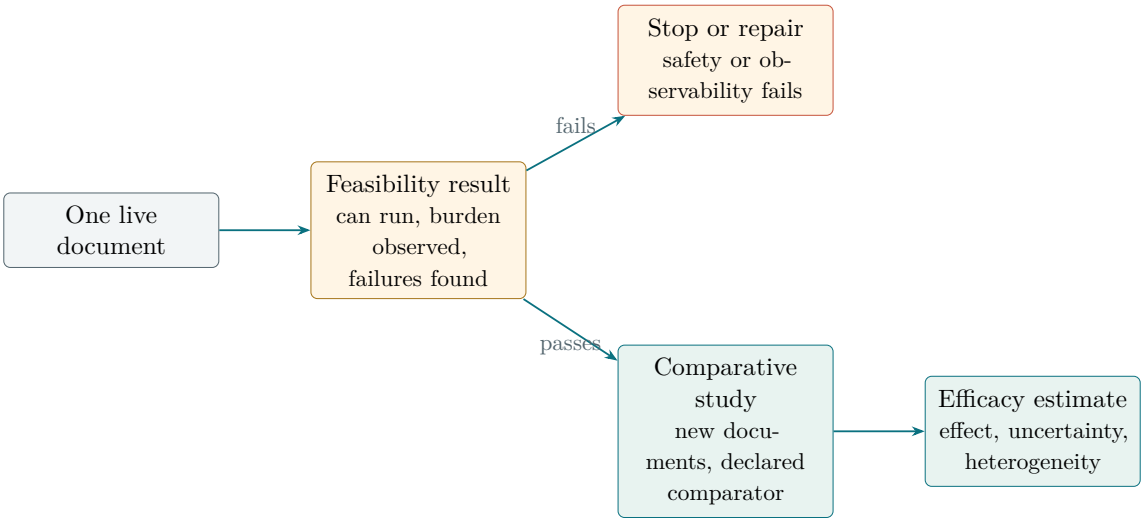

Figure 17.1: The feasibility case and the comparative study answer different questions. Passing the first authorizes the second, not an efficacy claim.

17.1 The estimand

Let i index a document task, and let $w = 0$ denote the incumbent bundle of ordinary linting, general LLM editing, version control, citation checking, and manual review. Let $w = 1$ denote the Humanvoice workflow. Before assignment, the study fixes a maximum of K review cycles and H calendar days. At that horizon it records one disposition:

accepted, revision still requested, declined, withdrawn by the author, or incomplete for an operational reason.

Define $A_i^H(w) = 1$ when the named reader accepts by the horizon, $T_i^H(w)$ as cumulative reader hours through the horizon, $R_i^K(w) = \min(R_i(w), K)$ as restricted review cycles, and $M_i(w)$ as an indicator that a meaning-bearing object changed without an explained author decision. The co-primary estimands are

$$\Delta_A = \mathbb{E}[A_i^H(1) - A_i^H(0)], \qquad \Delta_T = \mathbb{E}[T_i^H(1) - T_i^H(0)]. \tag{17.1}$$

Acceptance prevents a quick decline from looking like success; restricted reader time captures the scarce attention the product is meant to save. R_i^K remains an important process outcome, reported with the terminal disposition rather than only for accepted documents. Every decline, withdrawal, and incomplete task stays in the analysis. An unexplained protected change is a safety failure regardless of time or acceptance. Author editing time, false-positive burden, substantive revision rate, reader reconstruction, confidence, and time to decision explain how the primary outcomes arose.

17.2 The unit and the comparator

Give each task a named reader and a decision before assigning a workflow. The task may be a funding proposal, model note, technical report, or manuscript, but its genre and decision must be recorded. Both conditions receive the same source snapshot and protected-object inventory. The reader should be blinded to the workflow where practical; the author cannot be, so the study measures author expectations rather than pretending they do not exist.

The comparator is the practice the team would use if Humanvoice did not exist, not an ideal editor invented for the experiment. Before the study begins, write down which linter and general editor people use, how they check version changes and citations, who reads, and what counts as acceptance. A product that wins against an artificially weak baseline has not earned its place.

17.3 The reader record

The reader rubric is short enough to use repeatedly and specific enough to distinguish comprehension from agreement:

Question	Response recorded	Why it matters
What problem or decision is this document about?	Free response plus confidence.	Tests whether purpose is visible.
What is the proposed mechanism or object?	Free response plus selected components.	Tests whether the argument has an identifiable center.
Which evidence is decisive?	Source or passage selection with explanation.	Tests evidence-to-conclusion linkage.
What qualification changes your interpretation?	Short response.	Tests whether uncertainty is visible without swallowing the argument.
What action would you take now?	Accept, request revision, or decline, with reason.	Connects comprehension to the actual decision.
Where did you stop or reread?	Page or section, a short quoted span, and the missing relation you were trying to recover.	Locates the reader's first failure instead of reducing it to a score.

Table 17.1: A reader outcome records reconstruction and action, not stylistic approval.

Reader disagreement is not automatically noise. If two qualified readers recover different mechanisms, that may indicate an ambiguous document. The analysis should report agreement, confidence, and the reasons for requested changes rather than force a single hidden "quality" label.

17.4 The feasibility case

The first twelve weeks include one live document. Its purpose is integration, not efficacy. The case must demonstrate:

1. the source can be parsed and rendered with protected spans intact;
2. findings are reproducible and actionable in the author's editor;
3. accepted and rejected revisions produce an understandable protected comparison;
4. privacy and disclosure choices are respected; and
5. a reader can complete the rubric without repository context.

The case should be selected because its failure cost is real and its owner is willing to permit a reading by someone unfamiliar with the project, not because it is likely to make the system look good. The result note reports all abstentions, unresolved citations, reader requests, and meaning checks. It cannot report a reduction in rounds relative to a counterfactual that was never run.

17.5 The comparative study

After the feasibility case, the evaluation lead should freeze a corpus with tuning and held-out partitions. The corpus needs documents from more than one writer and more than one task class. If the available volume is small, the study should say so and use paired designs or repeated reader tasks rather than inventing a large-sample certainty.

The study should report outcomes by baseline writer experience, document genre, reader role, and use of model assistance. Averages can conceal harm to the authors or readers whose work matters most. The field literature's heterogeneous effects make this stratification a design requirement rather than an optional subgroup analysis [Brynjolfsson et al., 2025, Dell'Acqua et al., 2026].

Model-based screening, if used, is calibrated on a frozen human-labelled set. Pair order is randomized, length is controlled, and the judge family is varied when feasible. A model result can nominate a passage or explain a discrepancy; it cannot accept a document.

17.6 The economic decision rule

One accounting identity is enough. Let N be annual in-scope document volume, T_w cumulative reader hours per document through the fixed horizon, W_w author and operating hours, c_r and c_a their loaded rates, $p_w L_w$ the expected cost of a meaning incident, and C annualized product cost. The annual net value of replacing workflow 0 with workflow 1 is

$$V = N\left[(T_0 - T_1)c_r + (W_0 - W_1)c_a - (p_1 L_1 - p_0 L_0)\right] - C. \tag{17.2}$$

This is an accounting model, not a forecast. Acceptance by the horizon is a condition on using it: a workflow that saves time by producing more declines or unfinished documents does not receive a favorable value estimate. The feasibility run adds the quantities needed to populate the model:

Volume. How many in-scope documents does the group produce each year?

Baseline burden. How many rounds and reader hours do those documents consume, with a distribution rather than one anecdotal average?

Workflow burden. How much author and reviewer time does Humanvoice add, including false-positive resolution and protected comparisons?

Loaded rates. What is the local cost of author, reader, evaluator, and engineer time?

Build and maintenance. What person-weeks and recurring infrastructure cost are required after the pilot?

Until those inputs exist, symbolic break-even cases are more honest than invented volumes and hourly rates.

Decision question	Break-even expression	Observation required
Maximum annual product cost	$C^* = N[(T_0 - T_1)c_r + (W_0 - W_1)c_a - (p_1L_1 - p_0L_0)]$	Local volume, time distributions, loaded rates, and incident assumptions.
Reader-time saving required when all other changes are zero	$T_0 - T_1 = C/(Nc_r)$	Canonical reader-session timing under both workflows.
Maximum extra author time the workflow can consume	$W_1 - W_0 = [(T_0 - T_1)c_r - (p_1L_1 - p_0L_0) - C/N]/c_a$	Author event logs and adjudicated false-alert work.
Effect of one additional meaning risk	$\Delta V = -N(p_1L_1 - p_0L_0)$	Incident definition, probability range, and loss range approved by the sponsor.

Table 17.2: Symbolic sensitivity cases. Each row shows which local observation, not a placeholder forecast, determines the investment boundary.

The sponsor should see low, central, and high cases for every uncertain input, with each value marked observed, assumed, or open. A point estimate made before the baseline exists would be financial decoration rather than analysis.

17.7 Stop and redirect conditions

The evaluation has four stop conditions:

1. the protected comparison misses or silently changes a meaning-bearing object;
2. the diagnostic queue consumes more author or reader time than it saves;
3. the privacy or disclosure boundary cannot be implemented for the live document; or
4. the reader cannot recover the purpose and requested action after two bounded repair cycles.

A failed candidate does not automatically reject the product direction. If a parser fails malformed macros but a documented fallback can repair that exact failure, the next phase tests the fallback. If the reader endpoint itself is not observable or the burden cannot be measured, the direction is not ready for a larger study. The result note must distinguish those cases. This feasibility case establishes only that the workflow can be run and measured on one live document. It cannot establish a general reduction in revision rounds, a universal improvement in technical prose, or a return on investment; those claims require the comparative study and its uncertainty.

CHAPTER 18

A comparative test

Give an author a smoother paragraph and a reader may still ask the same question. That is why "the prose looked better" is not an estimand, and a lower detector score is not one either. The outcome we can test is the attention and meaning cost of getting a technical document to a named reader's decision. We write that outcome down, choose comparisons that can identify it, and show how a small feasibility sample can teach us without pretending to be a definitive efficacy trial.

18.1 The target population and unit of analysis

The pilot needs documents whose authors and readers know enough of the field to evaluate an argument, but not so much project history that private labels become self-explanatory. Candidate documents include research proposals, model notes, investment memoranda, and technical survey chapters. Each one must have a named decision, at least one meaning-bearing object, and a reader who can describe the decision context.

The unit of treatment is a document revision path. The unit of reader outcome is a reader–version encounter. The unit of protected-object safety is an object–version comparison. Keeping these units distinct prevents a document with many equations from receiving more weight merely because it has more tokens, and prevents one reader who spends a long time on a single draft from being mistaken for many independent observations.

Let d index documents, $v \in \{0, 1\}$ denote the incumbent and Humanvoice paths, and j index readers. The protocol fixes K review cycles and H calendar days before assignment. Define

$$Y_{djv} = \left(T^H_{djv},\ A^H_{dv},\ Q_{djv},\ R^K_{dv},\ D^H_{dv},\ M_{dv}\right), \tag{18.1}$$

where T^H is cumulative reader time through the horizon, A^H is acceptance by the horizon, Q is reconstruction-task completion, R^K is restricted feedback rounds, D^H is the terminal disposition, and M counts unresolved protected-object changes. The two primary contrasts are

$$\tau_A = \mathbb{E}[A^H_{d1} - A^H_{d0}], \qquad \tau_T = \mathbb{E}[T^H_{dj1} - T^H_{dj0}]. \tag{18.2}$$

All assigned documents remain in those contrasts. Accepted, still in revision, declined, author-withdrawn, and operationally incomplete documents appear as separate dispositions. The analysis does not condition the primary estimate on meaning safety after assignment, because that would remove precisely the failures the product is meant to prevent. It reports M as a safety endpoint that can stop adoption regardless of favorable time or acceptance.

For the declared reader task, let $Q_{djv} = 1$ when the reader identifies the object, mechanism, evidence, qualification, and requested action within the time budget. The contrast

$$\tau_Q = \mathbb{E}[Q_{dj1} - Q_{dj0}] \tag{18.3}$$

is reported with a confidence interval and the complete answer distribution. The project should not trade a small time saving for a lower probability of a correct reconstruction or a higher decline rate.

18.2 The comparisons

Three arms separate the value of diagnosis from the value of the full workflow:

Incumbent The author's ordinary revision process, with the usual grammar, version-control, and expert-review tools.

Diagnostic path Humanvoice findings and protected comparison are available, but the author receives no model-generated rewrite.

Full path The diagnostic path plus optional local suggestions, with the same author control, protected comparison, and reader task.

The diagnostic arm is important. If the full path improves the result, the project needs to know whether the improvement came from located attention, from wording suggestions, or from extra review time induced by the interface. If the diagnostic arm performs as well as the full path, a smaller and easier to govern product may be the better investment. If both arms fail against the incumbent, the product claim should be revised rather than rescued by a surface metric.

The first feasibility run can use a within-document crossover when the author can produce two protected versions and the reader can be blinded to the path. For a stronger causal comparison, later documents should be randomized to an arm and readers should be randomized to version order. A crossover is efficient for learning but vulnerable to carryover: a reader who saw one version knows the decision before seeing the other. The protocol records order and analyzes it rather than assuming it away.

Figure 18.1 illustrates the counterbalanced form. The exact allocation will depend on document arrival and reader availability, but the same outcomes must be recorded in both orders.

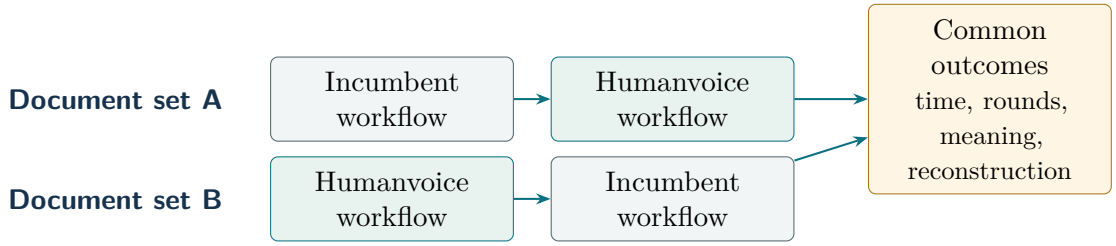

Figure 18.1: A counterbalanced paired design separates workflow effects from document difficulty and order, subject to the available volume and readers.

Design choice	What it identifies	Main threat
Within-document crossover	Difference for the same source and author	Reader carryover and learning
Document-level randomization	Average effect across independent documents	Small sample and heterogeneous genres
Reader order randomization	Position-adjusted comparison of versions	Incomplete blinding if the interface leaves traces
Diagnostic versus full path	Contribution of model suggestions beyond findings	Different author effort and novelty effects

Table 18.1: The feasibility design and the stronger comparative design answer different questions.

18.3 The reader instrument

The reader instrument begins with context, not a style rubric. A reader is told the decision, the role they occupy, the time budget, and the information they would normally have. The instrument then asks four forced-choice questions and one open question:

1. Which action is the author requesting?
2. What mechanism connects the proposed change to the outcome?
3. Which observation or calculation is the main support?
4. What condition or qualification limits the inference?
5. What would you ask the author before acting?

The first four answers are coded for correctness against a document-specific answer key prepared before randomization. The open question is retained for diagnosis and is independently classified as a request for definition, evidence, mechanism, scope, or action. The answer key is not a hidden test of whether the reader shares the author's conclusion. A reader can disagree with the recommendation and still understand it.

Time is measured from the first page shown to the final answer, with pauses and clarification requests recorded separately. A reader who stops because the document is clear has a different experience from one who stops after giving up. The protocol therefore asks for a short reason whenever the reader ends early, the page or section at which the stop occurred, and a short quoted span that triggered the question. It also records the missing relation the reader was trying to recover and whether the reader consulted a reference or source link; that behavior is part of the document's burden, not an error to be removed.

18.4 Sample size and uncertainty

The feasibility study is for design learning, so its sample size is chosen to estimate variance and failure modes rather than to promise a conventional five-percent result. Suppose the primary outcome is paired reader time, with within-document difference $D_d = T_{d1} - T_{d0}$. An approximate standard error for the mean difference is

$$se(\bar{D}) = \frac{s_D}{\sqrt{n}}, \tag{18.4}$$

where s_D is the observed standard deviation and n is the number of documents with usable paired readings. A planning interval of half-width h requires approximately

$$n \approx \left(\frac{z_{1-\alpha/2} s_D}{h} \right)^2 . \tag{18.5}$$

The pilot does not know s_D in advance. The one live feasibility case cannot estimate it. Baseline records and a small observational set can provide a rough planning range; the feasibility run then tests the timing instrument. Seeded safety fixtures expose several meaning failures without asking a live author to create them. Those inputs produce a range of sample sizes for the comparative study rather than a spurious single number.

For task completion, a paired binary difference is less stable than a time difference. Report the full two-by-two table of reader outcomes and use a paired interval or a randomization distribution rather than a normal approximation when the sample is small. Heterogeneity is not a nuisance to be averaged away. A workflow that helps a junior author and burdens a senior author has a different business case from one with a uniform effect. Report document genre, author experience, reader role, and assistance condition with each estimate.

The feasibility report should include a precision table before data collection:

Quantity	Planning value	Why it is recorded
Documents	At least one complete document in each selected genre slice	Prevents a single friendly case from defining the product
Readers per version	Two or more independent readers where feasible	Separates reader disagreement from document effect
Protected objects	A mixture of equations, numbers, citations, tables, and qualifications	Tests the safety boundary rather than prose alone
Time budget	Declared before showing the document	Makes completion comparable across versions

Table 18.2: The feasibility sample is designed to expose variance and failure modes before a larger effect study is priced.

18.5 Agreement and adjudication

Reader judgments are not perfectly reproducible, and a product should not hide that fact behind a majority vote. For each question, retain the individual answer, the answer key, the reader's confidence, and any explanation. A second reader independently codes the open question. Disagreement is then adjudicated by an evaluation reviewer who did not write the diagnostic or edit the tested version. A domain reviewer may resolve a dispute about technical meaning, but does not see the workflow assignment until the coding is frozen.

For finding labels, report agreement by category and prevalence, not just one overall percentage. A rare but consequential meaning failure can disappear in an aggregate agreement statistic. For model judges, calibrate against the human-labeled pairs and measure position flips, length sensitivity, and self-preference. Zheng et al., Panickssery et al., and Dubois et al. show why these checks are necessary [Zheng et al., 2023, Panickssery et al., 2024, Dubois et al., 2024].

The adjudicator does not rewrite the document during coding. That separation keeps the outcome independent of the proposed repair. If a reader asks for a change that the answer key did not anticipate, the request is preserved as a new qualitative category and the key is updated only for a subsequent round. This is how the instrument learns without moving its target after seeing the result.

18.6 Threats to interpretation

Several threats are predictable.

Novelty A reader may spend more time on a new interface simply because it is unfamiliar. The protocol gives the incumbent arm an equivalent context and records the first-session effect.

Selection Authors may submit documents that they believe are especially well suited to the tool. The feasibility report describes the intake process and keeps rejected documents in an eligibility log.

Contamination A reader may have seen an earlier version or the source history. The reader form asks about prior exposure. The assigned encounter remains in the main analysis with a contamination flag; a pre-specified sensitivity analysis reports uncontaminated encounters separately.

Learning In a crossover, the second reading benefits from the first. Order is randomized, and the primary feasibility interpretation treats the paired difference as descriptive when carryover is substantial.

Hawthorne effect Authors may work more carefully because they know a study is

observing them. The incumbent arm receives the same timing and documentation burden where possible.

Domain mismatch
: A result in an economics note may not transfer to a mathematical appendix or an investment memorandum. Genre and reader role are reported rather than pooled silently.

The jagged-frontier evidence suggests an additional interaction: assistance may be helpful for a task the model can perform and harmful when the task requires knowledge it lacks [Dell'Acqua et al., 2026]. The protocol therefore asks the author to identify which parts of the document are ordinary language editing and which parts require domain reasoning. A positive result confined to the first category is still useful, but it supports a narrower product.

18.7 From observations to an economic decision

The economic model in Equation (17.2) needs observations from the comparative test: restricted reader time, author and operator time, the distribution of terminal decisions, protected-object incidents, document volume, and local loaded rates. Table 17.2 converts them into break-even questions without inventing a dollar forecast.

The feasibility build also has learning value, but the proposal does not assign that value a decorative probability. The observable decision is simpler: did the build reveal enough about feasibility, burden, and variance to make the comparative study cheaper to price than before? The result memorandum lists which uncertainties narrowed and which remained untouched. That account lets the sponsor value the option using its own alternatives and budget.

18.8 Pre-specified decision rules

At the end of the feasibility run, the sponsor receives four results together: reader time and task completion, protected-object safety, author burden, and operating cost. The recommendation follows a simple set of conditions:

1. **Proceed to a comparative study** when the complete path is usable, no unresolved protected-object change remains, the reader outcome is measurable, and the precision calculation supports a priced next sample.
2. **Revise the product** when the reader outcome is promising but a named parser, finding family, privacy assumption, or interface causes avoidable burden.
3. **Stop or redirect** when meaning correspondence cannot be made reliable, the reader cannot recover the decision, or the operating cost exceeds any plausible attention saving in the chosen beachhead.

These rules are intentionally qualitative at the feasibility stage. The study should not manufacture a numerical pass threshold before the baseline and variance are known. Once the comparative sample is specified, the sponsor can set a minimum practically important time saving and a maximum tolerable meaning-error rate. The current document records that future decision rather than filling it with invented precision.

18.9 Reporting the result

Write the result like a small empirical paper. State the population, eligibility, arms, reader context, order randomization, missing observations, protected- object rules, and analysis before showing estimates. Show individual document trajectories and reader questions alongside means. Keep a source result, a local calculation, and a project hypothesis distinct in ordinary prose.

The forms can sit in the appendix. The main result should answer three questions in sequence: did the workflow change the reader's task, did it keep the document's meaning intact, and was the change worth the author's, reader's, and operator's time? A model score may support those answers, never replace them. That order keeps the proposal a decision document rather than a collection of attractive diagnostics.

CHAPTER 19

Twelve weeks to a decision

At the end of twelve weeks, the sponsor should be able to start with a real authoring brief, watch one document move from plan to pre-human release, inspect one measured feasibility case, and price the next decision. The build must test the product proposition without leaving a stranded platform if the case fails. Twelve weeks is a feasibility horizon, not a promise of market readiness.

Figure 19.1 is the canonical calendar for this proposal. Later work-package descriptions refer to these same intervals rather than introducing a second schedule.

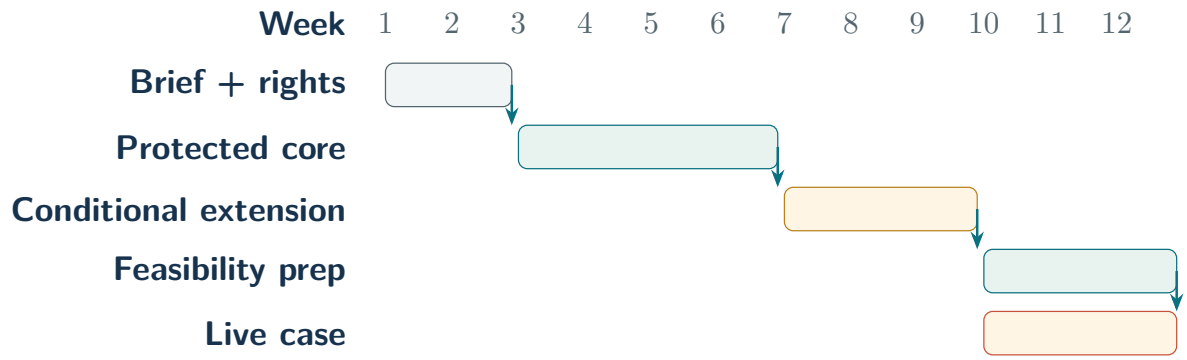

Figure 19.1: The canonical twelve-week sequence. Weeks 1–6 end in a protected core go/no-go; the extension and feasibility preparation begin only after that gate.

19.1 The twelve-week sequence

Weeks	Work	Deliverable	Decision consequence
1–2	Freeze the authoring brief, threat model, evidence boundary, named exemplars, known vocabulary, rights manifest, corpus split, schemas, and protected-object fixtures.	One versioned brief, contract, rights record, reader rubric, and parser test set.	Stop if the decision, reader, evidence boundary, or failure cost cannot be named.
3–6	Implement and replay the protected core: the common document representation, source locations, render checks, protected spans, citation fixtures, canonical build, versioned records, and safe command contract.	A second operator can reproduce the LaTeX source map and protected comparison.	At week six, narrow to the protected review utility if replay fails.
7–9	Conditional on the core gate, add the blueprint, bounded writer, integrity, argument, writing, evidence, and reconstruction preflight, mutation harness, and bounded repair.	A fail-closed preflight record and an extension that passes calibration and held-out replay.	Keep model-dependent gates in abstention if their false negatives or burden are not measured.
10–12	Conditional on the same gate, integrate author dispositions, release gate, reader packet, privacy controls, interaction log, and the live feasibility case.	One document can move from approved brief to a reproducible review-ready packet, followed by a timed reader task, without exposing private workflow language.	Stop if a failed gate can emit a packet or the packet cannot be reproduced.

Table 19.1: The implementation schedule produces evidence that changes the next decision, not a sequence of status reports.

The stages overlap where the dependency is clear. A trustworthy diff needs the parser and protected representation first. The author packet needs the diff and findings. The live reader needs the packet. A package benchmark can inform those steps, but it cannot replace the end-to-end case.

19.2 People and resource envelope

The repository does not contain local loaded rates or document volume, so the plan uses person-weeks. The sponsor can substitute those rates before making a funding decision.

Role	Planning effort	Responsibility
Product engineer	12 person-weeks	Brief and blueprint schema, source representation, bounded writer contract, protected comparison, diagnostics, release gate, interfaces, build, and privacy controls.
Evaluation lead	6 person-weeks	Corpus, mutation harness, critic calibration, reader rubric, comparator, feasibility measurements, and priced comparative design.
Domain editor and writing reviewer	3 person-weeks	Protected-claim inventory, technical-genre interpretation, blueprint review, and resolution of disputed scientific meaning.
Security or policy owner	1 person-week plus incident availability	Threat fixtures, network policy, retention and disclosure boundary, and security veto.
Independent coding reviewer	1 person-week	Blind coding of reader answers and finding labels; freezes decisions before seeing workflow assignment.
Independent reader(s)	2 sessions plus preparation	Reads by people unfamiliar with the project and recorded acceptance or requested changes.
Sponsor or project owner	2 decision sessions	Select the live case, approve the comparator, and choose proceed, revise, or stop.

Table 19.2: A planning shape, not a dollar budget. The security or policy owner, domain editor, and evaluation lead are explicit roles; one person may hold more than one role in a small pilot, but the decisions remain distinct.

The cost worksheet multiplies these person-weeks and reader sessions by local loaded rates, then adds approved compute, data, and external-review costs. Those observed or quoted inputs enter C in Equation (17.2). A short calendar should not be mistaken for a cheap project, but another displayed budget formula would add no information before the rates exist.

The protected-source core is the week-six checkpoint. Weeks 1–6 must deliver a replayable source map, protected comparison, and safe command contract before the bounded writer or model critics consume the remaining horizon. If that core does not pass with a second operator, the sponsor narrows the build to the protected review utility. The full authoring extension is conditional, not an implicit promise that one engineer can productionize every command in twelve weeks.

19.3 Deliverables at the end of the build

The sponsor should receive six reader-visible results:

1. a local-first command entry point for LaTeX, from brief to release;
2. a source map, claim map, dependency graph, and protected-object report that can be inspected without trusting a model;
3. a bounded writer adapter plus a small, adjudicated corpus with tuning, mutation, and held-out partitions;
4. the HPO density calibration fixture, including positive ordering cases and adjudicated false alarms;
5. one end-to-end, fail-closed feasibility packet from a live document;
6. a measured account of first-draft release rate, repair cycles, author burden, reader time, abstentions, and meaning-preservation checks; and
7. a costed comparative-study design with a proceed, revise, or stop recommendation.

The implementation may also produce machine-readable logs, package manifests, and test fixtures. Those are supporting records. They should not become a second narrative or be pasted into the manuscript as evidence of value.

19.4 Dependencies and external decisions

Four external decisions remain with the owner; they are listed on the next page.

Live document. The owner must approve a document whose failure cost is meaningful and whose reader can participate without compromising confidentiality.

Privacy boundary. The owner must decide whether any remote model call is permitted and what retention class applies to excerpts and rendered pages.

Comparator. The evaluation lead and owner must describe the incumbent workflow before the Humanvoice case begins.

Evidence completion. A librarian or domain-method reviewer must complete the outstanding literature and package checks before a strong scholarly or efficacy claim is circulated.

These are real dependencies, not forms to be completed after the build. If one cannot be resolved, the project can still develop parser fixtures or a synthetic review packet, but it must not present that work as a live efficacy result.

19.5 Risks with owners and stop triggers

Human judgment is not immune to gaming, and a local system is not private by mere intention. The operating plan therefore gives each important failure an owner, an early signal, and a point at which the team stops or removes a feature.

Table 19.3: The first-build risk register.

Risk and owner	Early signal	Mitigation	Stop or redirect trigger
Silent semantic change; product engineer and domain reviewer	A seeded sign, unit, citation, or scope change receives a clean comparison.	Typed fixtures, dual representation, and explicit abstention.	Any unexplained material change blocks the live path.
False-negative preflight; evaluation lead	A seeded dependency, evidence, or scope defect reaches a release packet, or a critic agrees with a bad draft.	Held-out mutation suite, isolated critics, frozen thresholds, and fail-closed unknown state.	Any critical mutation can pass without a recorded abstention.
Repair oscillation; product engineer	A repair fixes one gate and reopens the same gate or alternates between two failures.	Cause-specific strategies, parent hashes, cycle limit, and oscillation detector.	The controller loops or silently hands the loop to the reader.
False-alert burden; evaluation lead	Held-out triage time meets or exceeds incumbent issue-finding time.	Remove or narrow the rule family; preserve the author's rejection reason.	No diagnostic family earns positive net attention on held-out review.
Friendly-case selection; evaluation lead	Eligibility exclusions concentrate on difficult genres, authors, or privacy classes.	Freeze eligibility and keep all exclusions with reasons.	The selected case cannot represent the stated beachhead; report feasibility only and redesign sampling.
Prior exposure and reader contamination; independent coding reviewer	A reader recognizes the project history or earlier version.	Record exposure, use a fresh reader where possible, and report a flagged sensitivity analysis.	No uncontaminated reader is available for the declared task.

Coaching or unblinding; evaluation lead	One arm receives extra explanation or the interface reveals its workflow.	Script equal context, freeze answer keys, and preserve session logs.	Assignment cannot be concealed or equivalent context cannot be supplied.
Nonacceptance, decline, or abandonment; evaluation lead	Documents disappear from the dataset after a short or failed path.	Fixed horizon and explicit terminal dispositions for every assigned case.	Outcomes cannot be recovered for a material share of assigned documents.
Privacy breach; technical owner	An undeclared network call or full passage appears in a log.	Network-deny test, field-level approval, minimization, and incident response.	Any unauthorized external transmission stops the run.
Parser or dependency failure; product engineer	Unknown macros receive a false pass or an update changes locations.	Pin versions, keep a fallback, replay fixtures, and return to the incumbent path.	Neither adapter can locate or explicitly abstain on the protected object set.
Scope overrun; sponsor	Optional model, Markdown, drift, or editor work delays the LaTeX vertical slice.	Weekly critical-path review and deletion of deferred features.	Brief, plan, draft, preflight, and release are not usable at the end of week 5.
Incomplete scholarship; research lead	Load-bearing rows remain singly screened or abstract-only.	Independent screening, full-text anchors, design-specific appraisal, and external review.	Strong literature or efficacy language remains blocked; implementation learning may continue.

19.6 The choice after twelve weeks

The recommendation is to fund the bounded build, corpus, and one feasibility document. At week twelve the sponsor should receive enough evidence to choose among three paths:

Choice	Evidence required	What happens next
Proceed	The packet is usable, protected objects are preserved, reader burden is credible, and the comparative endpoint is observable.	Preregister and fund the paired or stepped comparative study.
Revise	The product is technically viable but one diagnostic, parser, or reader interaction adds avoidable burden.	Repair the named component and repeat the bounded feasibility test.
Stop	Meaning changes are unexplained, privacy is unacceptable, or the reader still cannot recover purpose and action.	Preserve the evidence and return the remaining person-weeks and budget.

Table 19.4: The build earns a larger investment only by changing this decision from a guess into an observed choice.

What this investment covers

Approve the twelve-week MVP and feasibility run described here. Deployment, per-document authorship detection, autonomous acceptance, and any claim that Humanvoice improves expert writing remain decisions for the sponsor after the team has observed the result and priced the comparison.

The first investment is intentionally narrow, but it is not the end of the argument. A feasibility run can establish that the path is executable and that its records are interpretable; it cannot establish that the same path is valuable across writers, genres, or readers. The next chapter treats that distinction as a research design problem. It specifies the measurement ladder, the held-out corpus, and the comparative program that must follow a successful build. In this order, implementation earns the right to ask a larger question without quietly answering it in advance.

CHAPTER 20

The next research questions

Run the pieces on one real document and we can answer a small question: do they work together? We still will not know whether the workflow reduces review burden across genres, writers, and readers. To answer that larger question, the team must first measure the objects correctly, then watch people use the workflow, and only then estimate its effect on outcomes.

A demonstration earns the right to ask the larger question. It does not answer it in advance.

The four rungs in Figure 20.1 make that order explicit. An attractive reader result cannot rescue a source-correspondence failure below it.

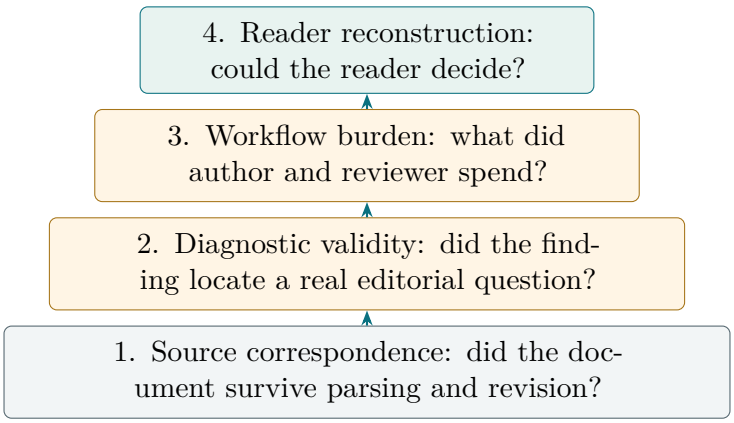

Figure 20.1: The research program climbs from source integrity to reader use. Failure on a lower rung invalidates a favorable result above it.

20.1 Three claims that must not be collapsed

The team will make three increasingly strong claims. The first is an *availability claim*: a parser, diagnostic, or review record can be installed and run on the target document formats. The second is an *operational claim*: the component preserves the objects and locations needed by an author, and its output does not impose more review work than it removes. The third is an *effectiveness claim*: the complete workflow changes a reader-facing outcome relative to the incumbent bundle.

Each claim needs a different observation. A package README can establish availability. A fixture benchmark and an observed review session can establish operational suitability. Only a comparative study with a declared comparator, reader rubric, and uncertainty can support effectiveness. The same hierarchy applies to the language-model adapter: plausible questions show availability; calibrated abstentions and errors show operational

suitability; neither proves that the adapter improves a document.

Claim	Minimum evidence	Decision it permits
Availability	Installation record, version, license, supported input, and a smoke run.	Keep the component in the candidate inventory.
Operational suitability	Held-out fixtures, protected-object preservation, runtime, failure modes, privacy review, and author burden.	Use the component in the feasibility workflow.
Workflow effectiveness	Preregistered comparison against the incumbent, human reader outcomes, burden, meaning checks, and uncertainty.	Consider adoption for the measured population and task class.
Economic value	Observed volume, loaded rates, avoided rounds, error costs, and sensitivity.	Fund expansion, revise the offer, or stop.

Table 20.1: Evidence strength rises with the decision at stake.

An inventory earns space in the main argument when it explains a choice or a risk. A long list cannot stand in for a benchmark, so the full package list stays in an appendix while the main chapters follow the decisions it informs.

20.2 The measurement ladder

The first research stage is a ladder of observations, not a feature roadmap. Each rung names the object the team will measure and the failure that would stop the next step.

20.2.1 Rung 1: source correspondence

For every target format, the system must map rendered passages back to source locations and identify protected objects. The fixtures should include ordinary prose, equations with macros, labels, citations, tables, quotations, code, comments, malformed input, and deliberately duplicated identifiers. The test does not ask whether a parser is elegant. It asks whether an author can locate the finding and whether a before-and-after comparison can tell what changed.

Let O be the set of protected objects in a source version and let $\pi(o)$ be the rendered location associated with object o. A parser pass is usable only if it preserves a partial map

$$\pi : O \longrightarrow \{\text{source spans}\} \times \{\text{rendered anchors}\}$$

for every object that the product promises to protect. Missing maps are not silently imputed. They are an explicit “unparsed” result that blocks the protected comparison.

20.2.2 Rung 2: diagnostic validity

The second stage asks whether a finding corresponds to an observable property of the current document. Register leakage can be counted against a declared lexicon and context rule. Repeated structure can be measured against a document-level baseline. Citation identity can be checked against metadata. These are appropriate places for precision, recall, and abstention measures.

The stage does not turn every editorial judgment into a classification task. Whether a qualification belongs in a paragraph, whether a term is familiar to the named reader, and whether a conclusion is proportionate to its evidence are questions for an author or reader. The diagnostic may attach a question and its evidence; it must not pretend that a threshold has answered it.

20.2.3 Rung 3: workflow burden

A finding has a cost. The author must read it, decide whether it applies, and possibly revise the source. The reader may later encounter the same passage and request a new change. The feasibility run should record the time spent in each activity, the number of findings opened, the number accepted or rejected, the number of rollbacks, and the reasons for abstention.

If q is the number of findings shown, a the number accepted, and t_j the time spent on finding j, a simple burden measure is

$$B = \sum_{j=1}^{q} t_j + t_{\text{recheck}}.$$

The expression is not a universal utility function. It is a bookkeeping device that prevents a high precision score from hiding a queue that nobody can afford to inspect. Report B alongside the findings that led to a reader revision and the findings that were rejected as irrelevant.

20.2.4 Rung 4: reader reconstruction

Give the rendered document to a reader without the project history and ask five questions: what does it answer, which objects carry the answer, what evidence supports it, what remains uncertain, and what action follows? A reader can understand a paragraph without agreeing with its conclusion, so the rubric separates comprehension, credibility, and willingness to act.

The first reader exercise is timed and conducted without project history. The reader receives the rendered document, the declared role, and the decision question, but not the source map, findings, or private project history. A second exercise may show the review packet to measure whether the packet makes revision easier. Keeping the exercises separate distinguishes the product's author interface from the outcome it is meant to improve.

20.3 The corpus is a scientific instrument

Do not build the corpus from convenient examples alone. It must represent the documents on which the product will be used and preserve why a finding was accepted or rejected. The existing cases supply valuable repair pairs, but they are selected cases. The evaluation corpus should add prospective samples from at least three genres: an investment or funding proposal, a technical research note, and a long-form methodological or monograph chapter.

Each unit should carry a document-level reading brief:

1. the writer's baseline experience and role;
2. the intended reader and the decision the document supports;
3. the source format, length, equation and citation counts, and sensitivity class;
4. the incumbent workflow, including which tools and human review were actually used;
5. protected-object annotations and known substantive constraints; and
6. named exemplar documents, known reader vocabulary, and declared exemption zones for first-use checks;
7. pacing profile, first-use findings, author dispositions, and the detector version that produced them; and
8. the reader outcome, revision history, and adjudicated failure reasons.

The split must be made at the document lineage, not at the paragraph level. Otherwise a repeated passage can leak from tuning into evaluation and produce an implausibly clean result. A held-out document should remain held out from lexicon selection, prompt design, model calibration, and author training.

A corpus split that can fail honestly. Take one long technical chapter and its three revised versions. Keeping one version in training and another in test is not a held-out evaluation: the reading brief, equations, and private terminology have already leaked. The lineage belongs to one partition. If the held-out set becomes too small, the correct response is to collect more documents, not to split the same chapter more finely.

The labels need two levels. A deterministic label records whether a protected object was preserved, whether a citation identity resolved, or whether a declared pattern occurred. An adjudicated label records whether the finding was useful, whether the revision changed the intended meaning, and whether the reader could act. The first level supports repeatable software tests; the second supports product claims.

The HPO case adds a practical unit of work. Select one chapter, run the deterministic meters, let an author repair or dismiss the ranked findings, rebuild the chapter, and then carry the updated brief into the next unit. A whole-document pass before the unit is understood hides which repair caused a reader response and makes a false positive

expensive to remove. The corpus therefore stores unit ids and brief versions alongside document ids.

20.4 The comparative program

The comparative study should begin only after the feasibility run has fixed the workflow and the comparator. The incumbent is not "no tool." It is the combination the author would otherwise use: editor, general language model if permitted, version control, citation checks, and manual review. Removing those components to make Humanvoice look better would answer the wrong question.

For each document task, randomize or counterbalance the order in which the incumbent and Humanvoice versions are prepared and read. Blind the reader to condition where the document and disclosure rules permit. Record protocol deviations rather than silently excluding difficult tasks. The primary outcome is the difference in feedback rounds to acceptance; secondary outcomes include reader time, author burden, protected-object incidents, requested substantive changes, and confidence in the decision.

Heterogeneity is part of the question. Report results by writer experience, genre, reader role, document length, equation density, and whether a model adapter was used. If an average improvement comes entirely from short business documents while long technical chapters become harder to review, the average does not justify expansion to the latter population.

20.4.1 Calibration and model judges

Model judges may reduce the cost of screening a large set of pairwise comparisons, but they are not the acceptance authority. Before use, calibrate them on human-labelled examples with randomized order, length controls, and multiple model families where practical. Measure position bias, verbosity bias, self-preference, and sensitivity to protected mathematical content. Require abstention when the judge cannot identify the reading brief or when the two versions differ in a protected object.

The judge's output should be treated as a noisy measurement. If Y is the human reader label and J the model label, report the confusion matrix and calibration curve rather than only an agreement percentage. A high agreement on easy surface edits can coexist with systematic failure on claim scope or technical uncertainty. The human-labelled subset must therefore oversample the cases most likely to expose those failures.

20.5 From feasibility to an operating service

If the comparative result is favorable, adoption still requires an operating model. A document owner must know where source text is processed, how long snapshots and rendered pages are retained, who can see the reader decision, and how a failed parse

is recovered. The product should expose a local-first path and an explicit remote-model boundary. Disclosure is part of the record: the author can state whether a model suggested a question, whether a human accepted the revision, and whether any confidential excerpt left the local environment.

The service should also define what it does when the product is unavailable. An author must be able to continue with the incumbent workflow, preserve the source version, and import the eventual decision later. A dependency that blocks ordinary writing is a deployment risk independent of model quality.

Operating question	Minimum answer before expansion	Evidence owner
Confidentiality	Which text, metadata, and rendered pages may leave the local boundary?	Project owner and security reviewer
Retention	Which snapshots, findings, and reader responses are retained, and for how long?	Product owner
Continuity	Can the author complete the document if Humanvoice or a model adapter is unavailable?	Product engineer
Disclosure	What assistance is disclosed to the reader, funder, or publisher?	Author and institutional policy owner
Escalation	Who decides when a protected comparison blocks release or a reader requests substantive repair?	Named document owner

Table 20.2: Operating questions that remain after a favorable experiment.

20.6 Alternatives if the full product does not earn adoption

The research program should preserve useful partial outcomes. If protected source correspondence works but the reader effect is null, the repository may still have a valuable technical comparison library. If deterministic diagnostics reduce author burden but the full packet does not change reader acceptance, the project may become an internal editing instrument rather than a product sold on reader outcomes. If the privacy boundary prevents model assistance, the deterministic and human-review path can remain the default.

These are not consolation prizes to be declared after a failed study. They are pre-specified alternatives that keep the interpretation honest. A null result for the complete workflow does not establish that every component is useless; it establishes that the bundle did not earn the claimed decision under the tested conditions. Conversely, a positive result on a narrow corpus does not support universal deployment. Expansion follows the population

for which the evidence was collected.

20.7 The decision after twelve weeks

At the end of the feasibility horizon, the owner should receive one compact decision package with the detailed records behind it:

1. a replayable source-to-reader run on a live document;
2. parser and protected-object fixture results, including failures;
3. the adjudicated corpus description and held-out split;
4. an author-burden and reader-time account;
5. a calibration report for any model judge or editor adapter;
6. an observed cost worksheet with sensitivity to volume and rates; and
7. a recommendation to proceed, revise, or stop, with the evidence that would change that recommendation.

The package is deliberately more substantial than a status report. It gives the sponsor enough detail to challenge an assumption, reproduce a failure, and choose the next experiment. That is the standard the product itself must meet: a reader should not have to trust a fluent conclusion when the underlying objects and decision path can be shown.

The next question is organizational rather than statistical. Suppose the feasibility packet is reproducible, the protected objects survive, and the reader rubric can be administered. Who will use the system, what incumbent work will it replace, and what privacy and pricing conditions make repeated use reasonable? Those questions are not a promise that the workflow has earned a market. They are the observations needed to turn a successful experiment into an accountable operating choice. Chapter 21 answers them by separating buyer, author, editor, and reader roles, then setting a scale rule that follows the evidence rather than a projection.

CHAPTER 21

Who would use it, and why

An organization may like the demonstration and still decline the product. Its authors have obligations, its security team has a privacy boundary, and its readers already have a review culture. The commercial question after the feasibility case is therefore concrete: who will use Humanvoice, what will it replace, and what evidence would justify moving from one document to a portfolio?

The repository does not yet contain a defensible market-size estimate. It does contain a clear first job. A high-consequence document has a named expert reader, several expensive review rounds, and equations, citations, or other objects that a general editor cannot safely handle. Adoption should begin where that job is already visible and measurable.

Figure 21.1 gives the commercial proposition in one line: saved attention creates value only after meaning and reader burden pass the guard.

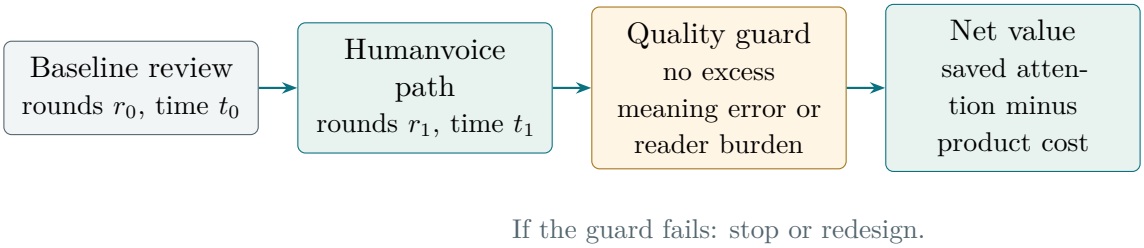

Figure 21.1: Value arises only when lower review burden survives the quality guard. Faster revision with more meaning errors has negative value.

21.1 The buyer, the author, and the reader

The buyer may sign the contract, the author may do the editing, and the reader may decide whether the document matters. Treating those three people as one "user" would optimize the wrong outcome.

Role	Immediate job	Evidence that adoption is real
Sponsor or buyer	Fund a bounded workflow and pay for reduced delay, lower incident risk, or a better decision record.	Approves the comparator, privacy boundary, and continuation threshold.
Author or analyst	Revise a source without losing equations, numbers, citations, or ownership of the claim.	Opens findings, records decisions, and returns to the incumbent path when the system abstains.
Editor or methods lead	Make a document's argument teachable and its review repeatable across authors.	Reuses contracts, fixtures, and reader rubrics without turning them into a style score.
Expert reader or referee	Reconstruct the problem, mechanism, evidence, uncertainty, and requested action under a real time budget.	Gives a located decision, confidence, and change trigger.
Security or policy owner	Decide what source and response data may leave the institution and what disclosure is required.	Signs the boundary and can replay or revoke a remote call.

Table 21.1: Different roles make different adoption claims.

The buyer's outcome is not "more human-sounding text." It is a document that can be accepted, challenged, or funded on the reader's own terms. The author's outcome is not a lower diagnostic count. It is less time spent rediscovering why a reader objected, with enough provenance to decide whether the objection was substantive. The reader's outcome is not agreement. It is a well-founded decision with visible uncertainty.

A buyer can reject a successful demo

An engineering team demonstrates that a model can turn a paragraph into fluent prose. The sponsor asks whether the revision preserves a denominator, a qualification, and the identity of a cited study. The team has no source map, no protected comparison, and no way to show the reader's time. The demo may be linguistically impressive, but it has not addressed the buyer's job.

The invalid inference is that a positive author reaction is sufficient for adoption. The first pilot must observe all three roles where possible. If a document has no consequential reader, it is a useful parser fixture but a weak product case.

21.2 Three beachheads, chosen by the job

Start with settings in which the job is visible, not with an abstract total addressable market. Three settings fit the current evidence and differ enough to test whether the mechanism travels.

21.2.1 Research and methods groups

A research group produces a methods note, working paper, or monograph chapter whose reader is a technical referee or collaborator. The source may contain custom mathematics, code, citations, and a long chain of assumptions. The group already uses version control and a build system, so the first product integration can be local and command-line driven. The value is a shorter path from a review comment to a verified revision, not automatic copy-editing.

The risk is that a researcher may treat the tool as an additional referee and accept a plausible but technically wrong suggestion. The pilot therefore starts with deterministic source correspondence and a human-controlled finding queue. A model adapter is optional and must be disclosed.

21.2.2 Investment, policy, and program offices

An investment or policy office produces a decision document for an executive who is expert but time-constrained. The document may mix quantitative results, institutional constraints, and a request for funding or a decision. Here the reading brief and unfamiliar-reader exercise are especially valuable: the question is whether a decision-maker can reconstruct the ask without private meetings.

The risk is confidentiality. A remote model call, a retained page image, or a shared benchmark fixture can expose information that is more sensitive than the prose itself. This setting is therefore a strong test of the local-first boundary and of the continuity requirement: the office must be able to finish the document if Humanvoice is unavailable.

21.2.3 Publishers, funders, and research-support units

A publisher or funder may not write the document, but it can provide a reading-brief and review service across many authors. The buyer's value is portfolio consistency and a reusable evidence record; the author's value is a clearer review path; the reader's value is a document that can be evaluated without learning the organization's internal workflow.

The risk is scale before validity. A support unit can process many documents and still measure only surface cleanliness. This setting should be a second stage, after the single-document feasibility run has fixed the object model, reader rubric, and disclosure language.

Setting	Strong reason to start	First measurable outcome	Main stop signal
Research group	Protected technical objects and existing source control.	Time from located finding to accepted revision; protected-object incidents.	Authors cannot distinguish a useful question from a model rewrite.
Investment or policy office	High review cost and a named executive decision.	Reconstruction by a reader unfamiliar with the project, followed by feedback rounds to a decision.	Privacy or disclosure boundary cannot be signed.
Publisher or support unit	Repeated documents and a reusable reader rubric.	Portfolio-level burden and reader consistency after single-document validity.	Scale creates a surface score with no reader endpoint.

Table 21.2: Beachheads and the evidence each one can produce.

The proposal should name one live setting for the twelve-week build. It may collect fixtures from the other two, but it should not promise three distinct markets with one underpowered pilot. A focused first customer gives the comparator, privacy rules, and reader role enough specificity to be measured.

21.3 The alternatives Humanvoice must beat

If Humanvoice is absent, the work does not disappear. An author, domain editor, general language model or style tool, version control, a build system, citation checks, and one or more expert readers form the incumbent bundle. Humanvoice must earn its place by improving the handoff between those activities.

Alternative	What it does well	What remains unconnected	Humanvoice's wedge
Manual expert editing	Interprets claims and audience in context.	Expensive, hard to replay, and often loses the reason for a revision.	Preserve the finding, source span, protected objects, and reader response around the human judgment.
General language model	Generates fluent alternatives and questions quickly.	May widen claims, obscure ownership, or require unapproved remote processing.	Use bounded questions and explicit acceptance rather than silent rewriting.
CI linter or style guide	Replays deterministic checks at low cost.	Usually cannot connect a warning to the reader's decision or technical object.	Keep narrow diagnostics and attach them to a reading brief.
Version control and diff	Shows that source changed and permits rollback.	A character-level diff does not say whether a denominator, citation, or claim scope changed.	Add object-aware comparison and a reason for each revision.
External review service	Supplies a reader and an acceptance signal.	Feedback may be unstructured and source provenance may be lost.	Return a replayable packet that links reader action to author work.

Table 21.3: The product is a connective layer over an incumbent bundle.

The wedge is strongest where the alternatives are each useful but their interfaces are weak. It is not a claim that Humanvoice is a better editor, better model, or better referee. It is a claim that a named reader's decision can be connected to a protected revision record owned by the author.

21.4 A staged adoption path

Adoption should proceed in the same order as the evidence ladder. Each stage has a customer-visible deliverable and a reason to stop. This makes expansion a sequence of earned options rather than a commitment to a large platform before the first object map works.

1. **Instrument one document.** Agree on the reading brief, incumbent bundle, privacy class, protected-object set, and acceptance endpoint. Produce a baseline run even if no Humanvoice finding is shown.
2. **Run the vertical slice.** Parse, locate findings, record author decisions, compare

protected objects, and prepare a reader packet for someone without project history. Keep the model adapter off unless the owner explicitly permits it.

3. **Repeat within one setting.** Add documents from the same genre and reader role. Estimate burden, abstention, and heterogeneity before comparing settings. A second document is a replication opportunity, not proof of generality.
4. **Compare the incumbent bundle.** Counterbalance or randomize preparation and reading order where practical. Declare the primary endpoint and preserve protocol deviations.
5. **Offer a portfolio service.** Only after the comparative result supports the workflow should a support unit reuse contracts, fixtures, and reader instruments across authors.

At each stage, the customer should receive a short decision packet: what was run, what changed, what was protected, what the reader did, how much time it took, and what remains uncertain. A dashboard that reports only a finding count is not a product milestone. The customer must be able to decide whether to continue using the incumbent, add a human editor, or fund the next stage.

The adoption handoff. Before moving from one document to a portfolio, ask a person who did not build the fixtures to run the packet. If they cannot identify the reading brief, replay the protected comparison, and explain an abstention, the product is not ready for scale. More documents would multiply an unresolved interface failure.

21.5 Pricing as a testable hypothesis

Pricing should follow the unit of value observed in the pilot. A per-token or per-page price would reward volume even when the system creates review work. The first offer should therefore be a bounded service or pilot priced around a document cohort, an agreed reader exercise, and a retained evidence packet. Only after repeat use is measured should a recurring subscription or institutional license be considered.

Let n be the number of documents in a cohort, s the service and support cost per document, K_0 the fixed onboarding cost, and W the observed value of avoided or improved review work. A pilot contribution can be written as

$$\Pi = n(W - s) - K_0.$$

This is a planning identity, not a forecast. W must be estimated from the reader and author logs in section B.8; it should include an incident-cost range when a missed qualification or protected-object change is material. A customer should not be charged for a benefit the pilot never measured.

Offer form	What the customer receives	Why it is appropriate at this stage
Feasibility service	One live run, protected comparison, reader exercise, and decision packet.	Buys learning while the object and reading briefs are still being tested.
Pilot cohort	A fixed number of same-setting documents, fixtures, and a comparative report.	Measures repeatability and burden without claiming a general platform.
Institutional deployment	Local service, support, retention controls, and portfolio rubric.	Requires evidence that the workflow works across authors and that operations are acceptable.

Table 21.4: Pricing forms should match the evidence stage.

The proposal should resist a large subscription forecast until there is observed renewal behavior. A financier can still evaluate the option value of the MVP: a modest first investment buys a working source map, a corpus, and a credible estimate of W. If those objects do not materialize, the project has learned early and can stop without having built an expensive service whose value is only asserted.

21.6 Operating model and trust boundary

An adopted service needs an owner for every handoff. The author owns the substantive revision. The editor or methods lead owns the reading brief and rubric. The engineer owns parser and build failures. The security or policy owner owns the processing boundary. The sponsor owns the continuation rule. These roles may be held by two people in a small pilot, but the decisions must remain distinct in the records.

The local-first path should be the default operating mode. Deterministic parsing, source maps, protected comparisons, and narrow diagnostics run in the repository boundary. A model adapter, if allowed, receives only the smallest excerpt needed for a declared question and returns a finding rather than a replacement document. The run records provider, version, prompt or configuration identifier, retention, and abstention. A customer can then choose a fully local path without losing the core product.

Continuity protects the customer's deadline. If a remote provider is unavailable, the author should be able to continue editing the source, build the document, and hand the reader the incumbent packet. The eventual Humanvoice run can import the parent snapshot and show what changed. This protects the customer's deadline and makes the product's value easier to trust: adoption does not require surrendering control of the document.

Operating promise	Minimum customer-visible behavior	Failure response
Local processing	Source, pages, and deterministic findings stay inside the declared boundary.	Block or quarantine an unapproved remote call.
Human ownership	No revision enters the reader packet without an author decision and protected comparison.	Mark the run incomplete and preserve the parent source.
Visible uncertainty	Unsupported objects and model disagreement appear as abstentions.	Route to the incumbent workflow or a named expert.
Continuity	The document can be completed if a component is unavailable.	Export the source and reading brief; import the later decision.
Accountable records	A sponsor can inspect the evidence behind a continuation or stop decision.	Retain the minimum run, response, and rationale records.

Table 21.5: Operating promises that make adoption reversible.

21.7 The scale decision

At the end of the first comparative cohort, the sponsor should make a scoped scale decision, not a binary verdict on "AI writing." Four results are possible:

Scale within the setting. The reader endpoint improves or the burden falls, protected-object incidents remain controlled, and the privacy and continuity promises hold for the tested population.

Deepen the instrument. The workflow is usable but the endpoint is too noisy, the corpus is too narrow, or the reading brief is underspecified. Invest in measurement before adding features.

Reposition. Deterministic source correspondence or comparison is valuable, but the complete packet does not change reader outcomes. Offer an internal editing and evidence service without claiming efficacy.

Stop. The product creates burden, cannot protect the source objects, violates the privacy boundary, or cannot produce a credible comparison.

The scale rule should be written before the result is seen. For example, move to a second setting only if the held-out documents meet the protected-object threshold, the median author burden is no higher than the declared margin, and the reader can state the action and uncertainty above a predeclared threshold. The exact values belong to the study protocol; the important point is that a fluent demo cannot waive them.

What the proposal is buying

The requested investment buys an option, not a promise of a market. It buys a working path from source snapshot to reader decision, a corpus that can expose where the path fails, and an economic worksheet tied to observed review work. Those objects make the next financing decision more informed. They also make it possible to stop without confusing a polished prototype with a validated product.

The next appendices specify the records, fixtures, reader forms, and cost inputs that make this scale rule executable. They are part of the product proposal because an operating service cannot be separated from the evidence that tells a sponsor whether to keep paying for it.

CHAPTER 22

From prototype to service

Picture the first ordinary Monday after the pilot. An author brings a private technical document, an operator builds it, a reader tests it, and a sponsor asks what the result is worth. Who owns each handoff, what does the customer receive, and what happens when a parser fails? The twelve-week sequence gives us a technical path. An operating service also needs clear boundaries, staffing, procurement, privacy, maintenance, and an honest fallback if the full workflow does not earn adoption.

The customer already pays experts to read technical documents. Humanvoice does not need to invent a new reading market. It needs to reduce the repeated diagnosis and clarification in research groups, investment teams, policy institutions, and regulated technical operations. That keeps the commercial question tied to the measured outcome.

22.1 The work after the first build

The feasibility build leaves four things that an operating service must make routine:

1. a source adapter and build wrapper that can reproduce a document without a special developer's machine;
2. a finding and protected-object service that an author can use without exposing private source to an unapproved endpoint;
3. a pre-human release gate that catches ordinary integrity, argument, evidence, and pacing failures before a reader packet exists;
4. a reader study that produces interpretable observations rather than a style score; and
5. a decision packet that tells a sponsor whether to proceed, revise, or stop.

The service should not promise continuous autonomous editing. A normal run is a bounded authoring path for one document or one chapter: brief, blueprint, unit draft, preflight, repair, release. The author chooses when to open a new version, and the reader receives a frozen PDF only after the release gate passes. This cadence makes the cost visible, keeps routine debugging out of the reader's hands, and limits the blast radius of a parser or model failure.

Operating activity	Owner	Observable output
Intake and authoring brief	Document owner	Decision, audience, named exemplars, known vocabulary, exemption zones, disclosure, privacy boundary, and source location
Blueprint and unit plan	Document owner with domain reviewer	Claim map, dependencies, evidence anchors, section jobs, and visual plan
Snapshot and build	Technical operator	Reproducible PDF, build log, and protected-object manifest
Pre-human preflight	Product operator	Gate results, mutation record, abstentions, repair task, and release decision
Author review	Author with product support	Accepted, rejected, or deferred findings, pacing and first-use dispositions, and a child version
Independent reading	Named domain reader	Timed answers, stopping point, quoted span, missing relation, questions, and decision
Result interpretation	Sponsor and evaluator	Comparison, cost range, and next action
Maintenance	Technical owner	Fixture replay, dependency update, and incident record

Table 22.1: A service run has a small number of human owners and visible outputs.

22.2 Staffing and the twelve-week sequence

The first team need not be large, but its responsibilities must be distinct. The person implementing a parser should not be the only person deciding that a reader understood the result. A practical starting team is a technical lead, a document and data engineer, a domain editor, and a part-time evaluator or research assistant. The sponsor supplies the decision and access to the named readers; the team does not invent those roles.

The weeks are organized by the dependency of the work rather than by software feature labels:

Weeks 1–2: establish the case
: Select the live document, write the reading brief, freeze the incumbent path, and extract protected objects by hand for a small reference set. The output is a human-readable baseline and a list of constructs the parser must handle.

Weeks 3–4: make the source legible
: Implement the snapshot and build wrapper, source locations, bibliography identity checks, and a first detexed view. Replay the positive and negative parser fixtures before adding prose rules.

Weeks 5–6: locate useful findings
: Adapt a small set of Vale, textlint, and project-specific rules, then run the local concept-density and first-use prototype against the named exemplars. Add context windows and an author decision form. Measure

false positives, investigation time, and hit counts. Keep a diagnostic only when the time it saves exceeds the time spent dismissing it.

Weeks 7–8: protect revisions
Implement object normalization, matching, visual comparison, and unresolved correspondence handling. Run the deliberately bad number, citation, equation, and macro fixtures.

Weeks 9–10: run the reader path
Render incumbent and revised versions, randomize order where possible, recruit the named readers, and record time, answers, questions, and disclosure.

Weeks 11–12: interpret and price
Rebuild from a clean environment, replay the fixtures, calculate the predeclared outcomes and cost sensitivity, and write the proceed, revise, or stop recommendation.

The sequence leaves room for a failed fixture. A parser that cannot preserve a custom macro in week 4 should narrow the first release, not be hidden until week 12. A reader who cannot answer the action question in week 10 should cause an author revision or a stop, not a new dashboard.

22.3 A small operating service

An operator can run the first service as a local command-line application, with an optional review page on the organization's network. The command line remains the source of truth: it is scriptable and keeps private documents out of an unexamined hosted system. The web page is only a convenience for viewing a finding, accepting a revision, and presenting a clean reader packet.

The service-level promises should be concrete:

- A successful snapshot returns a PDF, source hash, build log, and protected-object manifest.
- An unsupported source construct is reported with a location and does not receive a false "protected" result.
- A reader packet contains no finding identifiers, private paths, or model confidence unless the document owner explicitly chooses to disclose them.
- A revision with unresolved protected correspondence cannot be marked ready for a reader.
- Every model suggestion records its endpoint, model version, prompt template, sampling parameters, and network policy.

These are operating behaviors, not claims that the product is safe in every environment. The service owner must test them at installation and after each dependency update. A

failed promise creates an incident with a reproducible source snapshot and a route back to the incumbent workflow.

22.4 Procurement, licensing, and privacy

The software inventory contains permissively licensed source projects, remote services, and candidates whose license or maintenance status still needs review. Procurement should treat each dependency as a decision with three questions: may the organization use and modify it, can it be rebuilt from a pinned source, and what data does it send outside the trust boundary?

A package can be useful without becoming a runtime dependency. For example, a reference parser can generate expected spans for a fixture while the production adapter remains a smaller implementation. A detector can supply an aggregate research statistic while being forbidden from labeling an individual author. The disposition should be written in terms of the job and the boundary, not in terms of a package's popularity.

Privacy review follows the document path. The owner classifies source material as public, internal, confidential, or restricted. The default local mode allows all four classes subject to local access control. A remote model is allowed only for public or explicitly redacted material, with a record of the authorization and provider retention terms. Logs must not copy full source passages by default; a finding stores the minimum context needed to reproduce the decision, with access controlled separately from the reader PDF.

Figure 22.1 turns that policy into a data path. Any remote call crosses an explicit approval point; the default manuscript path remains inside the local zone.

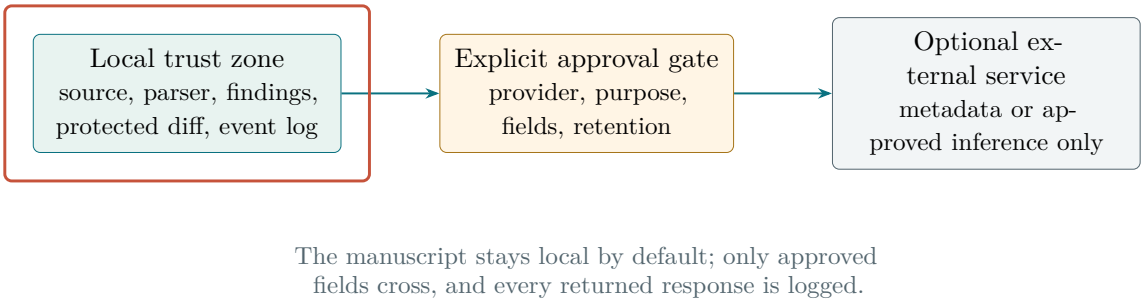

Figure 22.1: Local-first is an enforceable data path, not a slogan. Remote use requires a declared purpose and a field-level approval record.

The product should also support deletion and export. An organization must be able to remove a source snapshot and its derived model prompts, or to export the lineage needed for an authorized audit. A PDF hash is not a substitute for a retention policy. These requirements are inexpensive to state at the start and expensive to retrofit after a live document has crossed a boundary.

22.5 Maintenance and quality of service

Writing tools change quickly. A package update can alter a parser tree, a model update can alter suggestions, and a TeX package update can alter page breaks. The service needs a small regression corpus with four kinds of expected result:

1. exact source and PDF invariants for protected objects;
2. expected findings and intentional abstentions for prose fixtures;
3. privacy and network checks for local and remote modes; and
4. reader-packet snapshots checked by a human after material changes.

The first three can run in continuous integration. The fourth cannot be reduced to an image hash because a page can remain pixel-identical while a caption or cross-reference becomes confusing. A designated reader reviews the changed pages and records whether the decision path remains clear.

Maintenance effort is part of the economic model. Let u be annual document volume, m the minutes of operator maintenance per document, and c_o the loaded operator cost. The variable maintenance burden is

$$C_{maint,var} = u\, m\, c_o/60. \tag{22.1}$$

Fixed dependency review, security patches, and fixture refreshes add a fixed term. A workflow that saves reader time but requires an operator to repair every source manually may still be worthwhile for high-value documents, but the buyer must see that trade.

22.6 Three ways into the market

Three likely customers have different operating needs.

Research groups
: The buyer is a principal investigator or research director; the author is a senior researcher; the reader is a collaborator, reviewer, or funder. The service can begin as a guided review of a few high- value proposals and papers.

Investment and policy teams
: The buyer pays for faster senior review of model notes and memoranda. Confidentiality and citation provenance matter more than broad style coverage, and the reader may have a short decision window.

Regulated technical operations
: The buyer needs a traceable revision history and a controlled local deployment. The sales cycle is longer, but the cost of an unexplained change is higher and the protected-object comparison may be a direct procurement requirement.

The first customer should be chosen by document recurrence and reader cost, not by the largest potential market. A single organization that can provide a stable document class, a willing author, and independent readers is more useful for learning than many one-off demos. The pilot should measure willingness to repeat the workflow and willingness to pay for the reader outcome. A click on a suggestion does not answer either question.

22.7 A narrower product if the full service fails

The same components can support narrower offers. A "protected revision packet" can combine source snapshots, object extraction, and visual comparison without model suggestions. A "reader reconstruction study" can provide timed evaluation for organizations that already have editing tools, using readers unfamiliar with the project. A "citation and equation safety service" can focus on high-risk technical documents. These alternatives preserve the central value proposition and make the result of a failed full-stack trial economically useful.

The reverse alternatives are less attractive. Selling a generic humanizer or an authorship detector would put Humanvoice into crowded, weakly identified markets and would conflict with the evidence on diversity, detector bias, and citation reliability. A surface-only service may have a simpler sales story, but it would not solve the reader problem documented in Part I.

22.8 Risk and contingency

The principal operating risks are manageable when named early:

Risk	Early signal	Contingency
Parser coverage remains low	More than a small fraction of protected objects are unresolved	Narrow the supported LaTeX subset and provide a manual review path
Readers see no improvement	Time and task completion do not move in the feasibility case	Sell the measurement and protected-comparison service only, or stop
Authors reject findings	Investigation burden exceeds the observed benefit	Reduce rule families, change ordering, and test a diagnostic-only arm
Privacy review blocks inference	Source cannot be redacted without losing meaning	Use deterministic local tools and postpone model suggestions
Maintenance dominates cost	Fixture failures follow dependency updates	Pin versions, reduce the runtime stack, and price operator time explicitly

Table 22.2: Contingencies preserve a useful service even when a larger product claim fails.

22.9 The operating decision

At scale, Humanvoice should be sold as a reduction in expensive reader work with a visible meaning boundary. The buyer pays for a review path, not for a promise that a model has made prose human. The author remains responsible for the argument. The reader remains the authority for acceptance. The operator keeps the source and software reproducible.

That arrangement is deliberately less glamorous than an autonomous editor, but it has a measurable customer and a credible failure response. The feasibility build should return three numbers that determine whether the arrangement is worth extending: reader attention saved, protected-object error rate, and total operating cost per document. Everything else is evidence for understanding those numbers.

CHAPTER A

Evidence and technical notes

This appendix preserves the detail that an implementer or scholar may need after reading the main argument. It is not a project diary. Search ledgers, raw responses, source hashes, and package manifests remain in the repository's evidence record; this appendix states only the conclusions relevant to the proposal.

A.1 How to read the evidence status

The literature and software review was assembled from primary papers, systematic reviews, official documentation, and internal case records. Source identity and claim support were treated as separate questions. A DOI match establishes bibliographic identity; it does not establish that the source supports a sentence. A package's official documentation establishes an available function; it does not establish effectiveness on technical documents.

At the current run, ten of fifteen evidence gates pass. The bounded RePEc/IDEAS specialist slice, live known-item recall checks, and Crossref DOI identity checks are complete. The outstanding items are independent screening and adjudication, full-text anchors and design-specific appraisal for load-bearing studies, held-out package benchmarks, and independent external review. The proposal is therefore an author-repaired research request, not a completed systematic review or an efficacy report.

Claim classes used in this proposal

Source result. The cited source explicitly reports the finding at the inspection depth available.

Local implication. The product consequence follows from the source result and the stated workflow, without adding an efficacy claim.

Project hypothesis. The proposed workflow may reduce burden or improve reconstruction; the feasibility and comparative studies must test it.

Open question. The current evidence does not decide the issue and names the next observation required.

A.2 The compact internal case record

The internal cases are product requirements, not external efficacy evidence. They are useful because they contain a failure, a repair, and a reader consequence rather than a

collection of style opinions.

Case	Observed pattern	Design consequence
Funding proposal	A compiled and repeatedly reviewed draft left a reader unfamiliar with the project unable to state purpose or requested action.	The reading brief and acceptance by an unfamiliar reader belong in the product, not in an after-the-fact review.
Model note repair	A sequence of revisions preserved equations while removing private labels and making the comparison explicit.	Protected spans and located findings that the author can accept or reject are feasible targets.
Accepted technical units	Small units passed a named reader gate after substantive explanation, while whole-document acceptance was not claimed.	Measure the unit of reader decision and do not generalize a local repair.
Long monograph review	Mathematical detail survived, but repeated structure and administrative language increased the reading burden.	Diagnostics must serve a running argument and cannot be accepted on the basis of surface scores.

Table A.1: Internal cases motivate the product boundary; they do not estimate its effect.

A.3 Protected objects and allowed changes

The protected-source model distinguishes object preservation from prose preservation. The following default is appropriate for the first fixtures:

Object	Default check	Permitted author action
Equation body	Exact normalized hash and rendered comparison.	Change only with an explicit explanation and a rechecked result.
Numerical literal and unit	Token and unit comparison.	Change with an explanation of the new estimate or convention.
Citation key and bibliography identity	Key resolution and metadata match.	Replace or remove after human source verification.
Label, cross-reference, and table membership	Resolve after each build.	Move when the rendered relation remains correct.
Claim or assumption	Author-declared span and substantive diff.	Rewrite, narrow, or remove with an author decision.
Quotation, code, and data field	Exact or format-specific comparison.	Change only under the source's disclosure and ownership rules.

A.4 The first software benchmark

The benchmark corpus should contain at least one valid and one malformed case for each of the following: custom LaTeX macros, displayed mathematics, citations, labels, tables, comments, code blocks, Markdown emphasis, and source-to-render locations. For each parser candidate, record whether the protected object is recovered, whether a finding points to the same source span on replay, and whether a deliberate change is classified correctly.

For diagnostics, human reviewers label a frozen sample by category and burden. The benchmark reports precision, abstention, runtime, and review minutes. It does not report a single "human" score. For model-assisted findings, the benchmark also records order sensitivity, length sensitivity, provider and version, and the proportion of cases in which the model abstains.

A.5 Evidence that would change the recommendation

The recommendation to fund the MVP should be revised if any of the following new evidence appears:

- a direct competitor already provides the complete source-to-reader path with lower measured burden;
- protected comparisons cannot preserve equations, citations, or claims across the target formats;
- independent readers find that the review packet makes documents harder to understand or increases their time;
- the live document cannot be processed within its privacy and disclosure boundary; or
- the comparative design cannot observe a credible reader-acceptance endpoint.

Conversely, a successful feasibility packet is not sufficient to claim efficacy. It earns the right to run the comparative study and to price the larger decision.

CHAPTER B

Technical appendices

The main chapters make the investment case in the order a decision-maker needs: an observed problem, a mechanism, a bounded product, evidence that supports parts of the mechanism, and a test that can change the decision. This appendix supplies the working detail behind that route. It is written for the engineer who will build a fixture, the researcher who will audit a claim, and the sponsor who wants to know exactly what a successful twelve weeks would leave behind.

The appendices are deliberately explanatory. A table records a contract, but the paragraphs around it explain why the contract exists, what a failure looks like, and what conclusion the failure permits. The examples use LaTeX because it is the richest current target in the repository; the same objects can be represented in Markdown or a word-processing format once their source and rendered anchors are known.

B.1 Notation and terms

B.1.1 The unit of work

Humanvoice operates on a *document run*. A run is one immutable source snapshot, one declared reading brief, one execution configuration, and one rendered result. A new commit, a changed bibliography, a different parser, or a different model version creates a new run. This simple rule prevents an author from comparing a revised page with a stale source map and then calling the result a protected diff.

Let a run be

$$R = (S, V, C, E, P),$$

where S is the source snapshot, V its version identifier, C the reading brief, E the execution environment, and P the rendered pages and anchors. The tuple is not a claim that every field must be stored forever. It is the minimum information needed to reproduce a finding while the decision is still live. A retention policy may later delete page images while keeping a hash and a decision record; it must then say that it has done so.

The reading brief has eight fields. The first four define the reader's task; the remaining four keep later inspections from quietly changing that task.

1. the reader role, such as investment committee member, technical referee, or policy executive;
2. the decision the reader must be able to make after reading;
3. the knowledge and time budget that may reasonably be assumed; and

4. the evidence, uncertainty, and disclosure obligations that constrain the decision.
5. two or three named exemplar documents that establish the desired pace and explanatory register;
6. vocabulary the reader can reasonably be expected to know without a local gloss;
7. declared exemption zones, such as a map, decision box, or reference table, where a name may appear before it is taught; and
8. the versioned reader directives and the date of the last reader session.

The brief is not a marketing persona. It is a testable statement about what the document owes a named class of reader. For example, "an expert reader" is too broad for an acceptance endpoint. "A former econometrics professor deciding whether to fund a twelve-week feasibility build, with twenty minutes for the executive route and access to the appendices" is specific enough to generate a rubric and a timed exercise.

The exemplar field is deliberately comparative. It does not turn one monograph into a universal standard. A pacing meter reports where the target falls relative to the supplied exemplars and shows the passages that account for the difference. The vocabulary and exemption fields prevent a first-use audit from treating a reader's known language or an announced map as a defect. The brief is cumulative: a reader's stopping point or standing instruction updates the next version, with the change recorded rather than inferred from a new score.

B.1.2 Pacing, ordering, and assertion records

The HPO case suggests three related but non-interchangeable records. A *pacing profile* counts concepts and paragraphs; a *first-use record* checks whether a term was introduced before it was needed; an *assertion audit* checks whether an editorial repair smuggled in a new claim. All three locate questions. None can certify that a reader will accept the page.

For a document unit u, let W_u be prose words and C_u the distinct concepts first introduced in that unit. The descriptive introduction rate is

$$q_u = \frac{W_u}{C_u}, \qquad b_u = \frac{\#\{\text{paragraphs introducing at least three concepts}\}}{\#\{\text{prose paragraphs}\}}. \tag{B.1}$$

The implementation also reports singleton share, repeated occurrences, and the number of first introductions in a trailing 1,500-word window. These quantities are diagnostic: the same detector must run over the target and its exemplars, and the author must see the spans behind any comparison.

Record	Minimum fields
Pacing profile	run id; unit; detector version; prose-word count; concept count; words per concept; singleton share; burst paragraphs; live-load window; exemplar ids; relative position; dismissed findings.
First-use record	concept; first source line and rendered page; introduction signal; known-vocabulary match; exemption-zone match; taught-later pointer; author disposition.
Assertion audit	parent and child run; added sentence span; detected number, comparison, superlative, or causal verb; citation or protected-claim pointer; author explanation; status.
Reader observation	reader role; stopping page or section; quoted span; missing relation; elapsed time; requested change; confidence.

Table B.1: Small records that turn the HPO lessons into inspectable questions.

B.1.3 Objects, spans, and anchors

The source contains more than prose. Humanvoice calls every meaning-bearing or navigation-bearing item an *object*. A span is a contiguous source range; an anchor is a stable identifier that connects a source range to a rendered location. The same object may have several spans: a citation key in the sentence, its bibliography entry, and the rendered citation are one identity relation rather than three independent strings.

For an object o, write

$$o = (\text{kind}, \text{span}, \text{anchor}, \text{owner}, \text{status}).$$

The owner is the person or rule allowed to change it. A numerical literal may be owned by the analyst; a quotation may be owned by its source; a claim scope may require the document author; a page anchor is owned by the build system. Ownership is useful because it turns a vague warning into a bounded decision: the tool can flag a change, but it cannot silently decide that the owner's object no longer matters.

The first implementation should distinguish these object kinds:

Kind	Typical anchor	Why it matters
Prose claim	source line and paragraph id	A polished sentence can widen a claim even when every word is grammatical.
Assumption	tagged span or equation premise	A reader must know which conditions make the conclusion conditional.
Equation	equation label and source span	Formatting changes can hide a changed term or a lost restriction.
Number and unit	token range	A decimal or unit change can alter the decision while escaping a prose diff.
Citation identity	citation key plus bibliography id	A citation that still looks plausible may resolve to a different source.
Cross-reference	label and target page	Navigation is part of a long document's argument.
Quotation or code	fenced or delimited span	Exactness and provenance are different from ordinary copy-editing.
Table or figure	environment id and caption	A result can be correct in the text but misaligned with its displayed evidence.

Table B.2: Initial meaning-bearing object vocabulary.

The object vocabulary is intentionally smaller than a complete abstract syntax tree. A parser may know many more nodes, but the product should expose only those that affect a reader decision or a protected comparison. This is a product boundary, not a limitation on internal implementation. It keeps the author's review queue legible and makes test coverage auditable.

B.1.4 Finding, decision, and revision

A *finding* is a located, falsifiable prompt about an object or relation. It is not a replacement sentence. Formally, a finding can be represented as

$$f = (R, \text{kind}, \text{location}, \text{evidence}, \text{question}, \text{confidence}, \text{state}).$$

The evidence field points to a source span, a deterministic count, a reader observation, or a declared absence. The question is phrased so that an author can answer "keep", "change", or "abstain" without accepting an implied rewrite. Confidence describes the diagnostic's reliability, not the truth of the underlying claim.

The author decision is a separate record:

$$d = (f, \text{action}, \text{rationale}, \text{new run}).$$

An accepted finding may lead to a revision; a rejected finding remains useful because it teaches the diagnostic about this author and genre. An abstention means the product could not safely decide. Counting rejection and abstention is essential: a tool that raises fewer flags by hiding uncertainty can appear precise while making the author less safe.

A small run, written out

The source sentence says, “The intervention improves quality.” The reader contract names a referee who must decide whether to fund a feasibility test. The evidence record shows two internal repair cases, neither with a controlled comparison. Human-voice therefore emits a finding at the word “improves”: “Which observed outcome supports this comparative verb, and should it be narrowed to a testable hypothesis?” The author changes the sentence to “The intervention is designed to test whether review burden falls.” The equation, citation identities, and decision request remain unchanged. The finding is closed with the rationale “claim changed from result to hypothesis”.

The tempting but invalid inference is that the tool has proved the revised sentence better. It has only made the claim class explicit and preserved the objects that must be reviewed. A reader study is still required for an effectiveness claim.

B.1.5 Claim classes and evidence grades

Every load-bearing sentence in the proposal should be classifiable as one of four kinds: source result, local implication, project hypothesis, or open question. The classification is a reading aid, not a substitute for a bibliography. A source result needs a citation and an inspectable anchor. A local implication must state the bridge from the source result to the product. A project hypothesis must name the planned observation. An open question must name what would resolve it.

For implementation decisions, use an evidence grade with two axes. The first axis is *provenance*: primary external study, official package record, internal case, or design assumption. The second is *fit*: direct fit to the target population, adjacent fit, or unknown fit. A highly credible study with adjacent fit should not outrank a modest internal observation when the question is simply whether a source map survives this repository’s macros; neither should be used to support an efficacy claim outside its scope.

Evidence grade	Interpretation	Permitted use
P–D	Primary or official source, direct target fit	Support a bounded design or benchmark expectation.
P–A	Primary or official source, adjacent fit	Motivate a test; do not generalize the outcome.
I–D	Internal record, direct repository fit	Specify a fixture or workflow requirement; not external efficacy.
I–A	Internal record, adjacent or selected case	Generate a hypothesis or counterexample.
A–U	Assumption with unknown fit	Price or scope the experiment only.

Table B.3: A compact evidence vocabulary for design decisions.

B.2 Detailed requirements and records

B.2.1 From a promise to an acceptance contract

The product proposal uses requirements in a narrow sense: a requirement is a behavior that can be observed in a run and accepted or rejected by a named owner. "The writing should feel human" is a valuable aspiration but not a first-release requirement. "A reviewer can open a finding and reach the source span and rendered page within two interactions" is testable. The distinction prevents a broad editorial goal from being hidden behind a surface-style score.

This is the canonical R1–R24 acceptance register. The implementation contract is in Appendix F, with machine-readable schemas in `schemas/`. The contract annex defines execution, schemas, exit codes, and authority without creating a second release register.

Table B.4: Canonical implementation requirements and acceptance observations.

ID	Requirement	Acceptance observation	Owner
R1	Reader-facing separation	A low-context reader can state purpose, object, and requested action; internal register language is absent; the released file traces to a source hash.	Domain editor
R2	Policy precedence	Conflict fixtures resolve correctness, source fidelity, meaning, comprehension, then regularity; format overrides are configuration.	Product engineer
R3	Located diagnostics	Fixtures return stable spans and no inspection command mutates input; accepted edits are separate records.	Product engineer
R4	Layered diagnosis	Replay retains register, repetition, defensive, substance, and comprehension results in order.	Evaluation lead
R5	Protected diff	Equations, labels, citations, numbers, claims, code, tables, and quotations have no unexplained material change.	Product engineer
R6	Citation support	Identity and claim support are separate; unresolved load-bearing support blocks release.	Domain editor
R7	Calibrated judging	Frozen human labels report agreement, order, verbosity, and family sensitivity before model judging is used.	Evaluation lead

ID	Requirement	Acceptance observation	Owner
R8	Human acceptance	A promoted version records the pseudonymous reader, hash, rubric, decision, requested changes, and elapsed time.	Independent reader
R9	Held-out evaluation	Tuning and holdout hashes are frozen and no smoke test is reported as effectiveness.	Evaluation lead
R10	Minimal process surface	Provenance remains in the packet record while the manuscript contains no orchestration state.	Domain editor
R11	Privacy and disclosure	Unapproved remote routes are blocked; approved calls record provider, model, purpose, retention, and approval.	Security or policy owner
R12	Aggregate convergence	Drift analysis refuses single-document authorship classification, suppresses small cells, and reports uncertainty.	Evaluation lead
R13	Interaction telemetry	Intent, suggestion, acceptance, edit, rejection, and rollback replay without unnecessary manuscript retention.	Product engineer
R14	Heterogeneous outcomes	Prespecified writer, genre, and reader strata report uncertainty and apply the harm stop rule.	Evaluation lead
R15	Parser contract	Candidate adapters emit the same source-map and protected-span contract and pass the locked malformed-input fixtures.	Product engineer
R16	Package adoption	Availability, operational fit, and effectiveness are separate; adoption requires a held-out benchmark and independent review.	Evaluation lead
R17	Complete authoring brief	Missing critical reader, decision, evidence, privacy, or protection fields fail before writer invocation; a complete brief has a stable hash.	Document owner
R18	Argument blueprint	Missing warrants, early concepts, duplicate jobs, or purposeless figures block promotion; the plan has a stable hash.	Domain editor
R19	Bounded drafting	Each unit records approved context and evidence and cannot advance while its checks are unresolved.	Product engineer

ID	Requirement	Acceptance observation	Owner
R20	Fail-closed release	Every seeded gate failure emits no reader packet and preserves a located repair record.	Product engineer
R21	Construct-specific critics	Held-out false-positive, false-negative, order, verbosity, and family results are recorded before release use.	Evaluation lead
R22	Mutation testing	Critical deterministic mutations are caught or abstained from; rhetorical mutation uncertainty is reported.	Evaluation lead
R23	Bounded repair	Named strategies converge or stop with an unresolved author choice; no loop reaches the reader silently.	Product engineer
R24	Attention budget	Feasibility reports release rate, author and reader minutes, abstentions, and meaning errors against the incumbent.	Sponsor or project owner

The register is the single release view. The machine-readable audit protocol adds evidence links and implementation phases without changing these behaviors. A result is reported by requirement family and phase; it is never averaged into one release score. R5, R11, R17, R18, and R20 are hard blockers for the protected core or full path respectively, as specified in the implementation contract.

B.2.2 Record schemas

The record model should remain plain enough to inspect with ordinary command- line tools. A JSON implementation is sensible, but the semantics matter more than the serialization. The following fields are sufficient for the MVP.

Record	Required fields and interpretation
Run	run id; source hash; parent run; source format; build command; tool and model versions; privacy class; created and completed times; outcome.
Object	object id; kind; source span; normalized value; rendered anchor; owner; protection rule; comparison status.
Finding	finding id; run id; category; location; evidence; question; confidence; abstention reason; state; diagnostic version; unit; repair move; dismissal reason.
Decision	finding id; action; author id or role; rationale; parent and child run ids; time spent; reviewer comments.
Reader response	document run; reader role; contract version; completion time; comprehension answers; credibility rating; stopping point; quoted span; missing relation; requested action; confidence; disclosure acknowledgement.
Pacing profile	unit; detector version; target metrics; exemplar metrics; relative position; burst locations; dismissed concepts.
First-use audit	concept; first-use location; introduction signal; whitelist or exemption match; taught-later pointer; disposition.
Assertion audit	parent and child run; added span; assertion type; support pointer; author disposition.
Benchmark result	fixture id; candidate; version; expected object set; observed object set; precision; abstention; runtime; failure trace.

Table B.5: MVP records and the questions they answer.

The schema separates identity from content. A source hash identifies what was run; a rendered PDF is an output of that run, not its identity. This matters when a build tool changes page breaks without changing source semantics. The comparison can then say "same source objects, different pagination" rather than reporting a false substantive drift.

B.2.3 Traceability without a bureaucracy

Traceability is useful when it follows the reader's question. For each requirement, link one fixture, one observed result, and one decision rule. Do not create a separate narrative for every administrative event. A compact trace looks like

$$\text{requirement} \longrightarrow \text{fixture} \longrightarrow \text{result} \longrightarrow \text{release decision}.$$

For R5, the fixture contains an equation whose denominator changes in a revision; the result reports whether the change was surfaced; the decision rule blocks a protected comparison if the denominator is unparsed. For R8, the fixture is a timed reader exercise; the result is a response and a completion time; the decision rule says whether the reader endpoint is observable enough for the comparative study.

The traceability test

Ask an engineer to pick any promised behavior and an executive to pick any investment conclusion. The engineer should be able to walk forward to a fixture and a release rule. The executive should be able to walk backward to the observed evidence and its limitations. If either path ends at a slogan, the proposal is not yet operational.

B.3 Internal case evidence and repair-pair index

The repository contains a rare kind of product evidence: paired passages in which a writer or editor changed a document after a reader failure. These pairs are not randomized experiments. They are valuable because they preserve the causal story that a feature must eventually support: what the reader could not do, what changed, and what remained protected.

B.3.1 Case selection and provenance

The cases came from work on CardNPV, BGS, SMEwallet, BayesFilter with the zero-lower-bound rescue, and MacroFinance. Each has a different scale and failure mode. CardNPV supplies the strongest teaching example because its small worked example and later abstraction make the reader's reconstruction task visible. BGS supplies a negative process case: a long sequence of governance and review activity did not produce an accepted document. SMEwallet supplies a positive local gate. BayesFilter supplies a layered technical argument in which notation and computational commitments must survive exposition. MacroFinance supplies the book-scale problem: repetition and infrastructure can be useful to an implementer while becoming a burden to a first reader.

The case index should preserve four provenance fields: source path or commit, date of the observed reader interaction, the role of the person who made the judgment, and whether the passage was later used in another revision. A passage copied into several drafts is one lineage, not several independent observations.

Table B.6: Internal case index and the claim boundary for each case.

Case	Reader task	Failure or repair pair	What it can establish
CardNPV	Reconstruct a finite value question and decide what is identified.	Finite teaching case, ledger, timing, mechanism, and later abstraction.	A training route can make objects and transitions inspectable; not a causal estimate of Humanvoice.

Case	Reader task	Failure or repair pair	What it can establish
BGS	Decide whether a proposal has a defensible purpose and next action.	Repeated governance prose versus located purpose, evidence, and decision.	A reader-work failure can persist despite compilation and multiple reviews.
SMEwallet	Pass a named reader gate for a business-facing technical unit.	Local explanation and revision until the reader can restate the action.	A unit-level acceptance record is feasible; not a population effect.
BayesFilter–ZLB	Follow assumptions through filtering and a boundary-case rescue.	Layered notation, an uncovered failure, and a repair that preserves the mathematical object.	Protected technical exposition needs more than sentence-level style.
Macro Finance	Navigate a long methods and implementation volume.	Repeated structure and source infrastructure made the book usable to builders but harder to enter as a reader.	Monograph-scale navigation and appendices are product requirements.

B.3.2 Repair-pair taxonomy

A repair pair should be annotated by the first reader action it enables. The following taxonomy is more informative than labels such as "awkward" or "AI-like".

1. **Orientation repair:** the opening identifies the problem, decision, and route before terminology accumulates.
2. **Actor repair:** an abstract process is assigned to a concrete author, reader, institution, or data-generating mechanism.
3. **Evidence repair:** a conclusion is connected to an observation, citation, calculation, or explicit absence of evidence.
4. **Scope repair:** a result is narrowed to the population, period, document unit, or claim class actually supported.
5. **Sequence repair:** definitions and examples are reordered so that each later step uses an object the reader has already seen.
6. **Register repair:** administrative or defensive language is replaced by the domain statement it was concealing.

7. **Navigation repair:** labels, headings, tables, and transitions make a long argument recoverable after interruption.
8. **Disclosure repair:** assistance, uncertainty, and ownership are stated where a reader's interpretation can be affected.

For each pair, record the smallest change that enabled the action and the objects deliberately left unchanged. This last field matters. A passage that reads better only because its equation, qualification, or numerical result was silently removed is not a safe repair.

Annotation	Positive observation	Counterexample to retain
Orientation	A reader unfamiliar with the project states the decision after the opening.	The opening is elegant but the reader still cannot name the object being decided.
Evidence	The reader can point to the calculation or source supporting the claim.	A citation is present but its result does not entail the sentence.
Scope	The revised claim names its population and uncertainty.	The prose is qualified so heavily that no testable claim remains.
Sequence	A reader predicts why the next section is needed.	A glossary defines every term but no transition explains their relationship.
Protection	The equation and number survive the prose repair.	A textual diff is clean because the mathematical object was dropped.

Table B.7: Repair annotations should retain negative examples.

B.3.3 What the cases do not show

The cases are selected for learning, not prevalence. They do not show how often expert documents fail, whether a particular diagnostic causes the repair, or whether the same repair works for a different reader. They also contain author and editor judgment that cannot be reconstructed perfectly from the final PDF. The feasibility corpus should therefore include both these historical pairs and prospective cases collected under a fixed protocol.

The tempting but invalid inference is "five successful repairs imply a reliable product." The defensible conclusion is narrower: the repository contains enough varied failures to specify fixtures, reader tasks, and protected objects. That is precisely the kind of evidence a product proposal needs before it asks for an efficacy claim.

B.4 The HPO density case and its fixture

The HPO survey engagement is the clearest internal example of why the product needs more than a surface linter. One manager reviewed a 90-page methods monograph over

three rounds. The manager used the same word, "dense," in the first two rounds and then supplied a more precise ordering instruction in the third. The agent's self-review said the document was readable before every round; the manager's located observations disagreed.

The case record is dated 24–25 August 2026. Its numbers come from the engagement ledger and the prototype concept-density script in the HPO survey repository. They are internal observations with direct fit to the proposed workflow, not external literature and not a representative sample. We retain them because they specify fixtures and questions that a later independent review can replay.

Table B.8: HPO case record and evidence boundary.

Round	Observed complaint	Located evidence	Permitted use
1	"Too dense."	One 210-word paragraph contained six mechanisms and five citations; mean prose-paragraph length moved from 128 to 90 words after repair.	Seed paragraph-packing fixture; not a paragraph-length standard.
2	"Too dense."	Words per new concept were 70 in the survey and 83 and 253 in the two supplied exemplars; paragraphs introducing at least three new concepts were 12.9, 5.9, and 0.8 per cent.	Calibration comparison; not a universal threshold or efficacy result.
3	"Do not use it before explaining it."	Approximately 25 adjudicated first-use violations, including an unexplained running-example algorithm and early acquisition-function names.	Ordering fixture; prevalence outside this case is unknown.

The case also supplies a useful negative set: 80 passages that the first-use audit flagged but the manager adjudicated as false alarms. The proposed fixture stores the 25 positive locations and the 80 negative locations without mixing them into rule tuning. Each item carries the source line, the first-use term, the reader's known-vocabulary decision, any exemption-zone declaration, the adjudicated label, and the repair or dismissal reason. A future precision estimate must be made on a partition that the rule author has not inspected.

The concept detector is a standard-library prototype. It recognises a small set of signals—

acronyms, mid-sentence proper terms, emphasized definition phrases, recurring technical bigrams, and citation keys—and removes common LaTeX environments before counting. It has no sentence-level dependency parser. The HPO engagement spot-checked roughly 80 percent precision with noise that appeared similarly across the target and exemplars; Humanvoice should replicate that estimate before treating the detector as an operational component. Dismissal must be cheap, and a noisy list must never block an author's ordinary edit.

The fixture therefore has four roles:

1. a **calibration role** for choosing wording, displays, and exemption conventions;
2. a **held-out role** for measuring precision and abstention after those choices are frozen;
3. a **reader-study role** for recording stopping points and quoted spans rather than relying on a score; and
4. a **claim-boundary role** that prevents one successful engagement from being described as a general reduction in review rounds.

The implementation loop is consequently small and repeatable: select one chapter, run the meter and first-use audit, inspect the ranked passages, let a human author repair or dismiss them, rebuild the chapter, and record the reader's stopping point. Only after that unit is stable should the same rule be propagated to the next chapter. The ledger records the unit and brief version so a later reader can tell which instruction was active at the time.

B.5 Literature claim-to-design matrix

The literature chapter explains the main results in prose. This appendix gives the audit-friendly matrix that keeps a source result distinct from the design choice that follows it. The rows are grouped by question, not by publisher or model family, because the product decision is comparative.

B.5.1 How to use the matrix

For each row, the researcher records the source identity, the inspected text or official documentation, the population and task, the result, its limit, and the Humanvoice implication. A source can appear in more than one row when it answers different questions, but each implication must remain local. The matrix is not a ranking of papers. It is a guard against a common citation error: using evidence about model output style as if it were evidence about reader acceptance.

Figure B.1 summarizes the relationship that the rows below must preserve. A source earns influence only through an inspectable claim, design choice, and test.

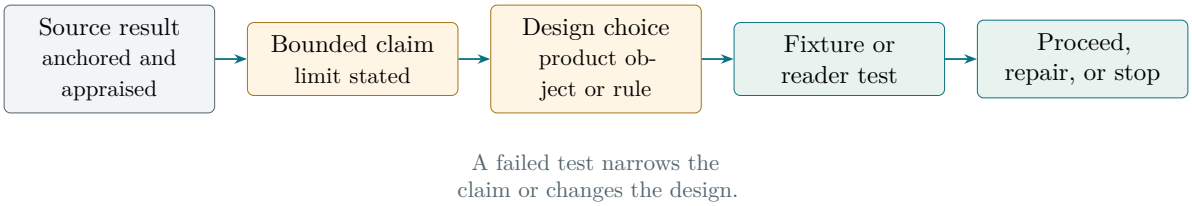

Figure B.1: Traceability is a chain of reasons. Internal identifiers help reproduce it, but the main text should show the reasons themselves.

Table B.9: Claim-to-design matrix for the evidence review.

Question	Source result to verify	Product implication	Open test
Why outputs look alike	Distributional markers, repeated discourse, or model-conditioned style are observed in a stated corpus.	Use patterns as diagnostic prompts and compare against a genre baseline.	Do the prompts reduce reader work without erasing legitimate disciplinary style?
Can detection identify origin?	Detection performance varies with domain, editing, and generator; false positives remain consequential.	Never expose a per-document origin verdict as the product outcome.	Does disclosure plus located evidence improve a reader's decision?
Does feedback help?	Controlled writing or editing studies report changes in time, quality, or task completion under specific conditions.	Test bounded feedback with a declared incumbent and reader endpoint.	Does the effect survive expert technical documents and longer tasks?
How do people use models?	Users accept, reject, and adapt suggestions heterogeneously; overreliance and verification costs matter.	Record author decisions, burden, abstention, and protected-object incidents.	Which reader and writer populations benefit or are harmed?

Question	Source result to verify	Product implication	Open test
Can models judge text?	Agreement can be high on surface edits and lower on scope, evidence, and technical meaning; order and verbosity biases occur.	Use model judges only as calibrated screening instruments with human labels.	Does calibration hold on held-out mathematical and policy cases?
What is readability?	Formula scores depend on genre, audience, and text features; comprehension is not identical to ease.	Pair any metric with a reading brief and timed reconstruction task.	Which measures predict the decision the document is meant to support?
What are integrity risks?	Provenance, disclosure, privacy, and authorship norms differ by venue and can change behavior.	Make processing boundary and assistance visible; preserve source ownership.	What policy applies to each target institution and publication?

B.5.2 Search coverage and omitted-source risk

A credible survey reports not only what it found but also where it may be thin. The current review covers the supplied internal material, a targeted set of primary studies on generative assistance and feedback, readability and technical communication, model-based evaluation, and official documentation for candidate software. It does not yet constitute a registered systematic review. Specialist work in human–computer interaction, writing studies, information integrity, accessibility, and privacy may change the design.

The next evidence pass should use at least two independent screeners, a recorded query set, backward and forward citation chasing for load-bearing studies, and a disposition for every retrieved item. A source should be included because it answers a declared question, not because its conclusion supports the investment. The screening sheet should preserve exclusions and their reasons; otherwise the final bibliography will look comprehensive while the selection process remains invisible.

The register below names the present risks one by one. “Candidate” means that the source has been identified but has not been used to support a manuscript claim. “Included, bounded” means that the manuscript states the source’s limited contribution while appraisal or independent verification remains open. This is the omission register promised in the main text, not a list of topics that someone might search later.

Table B.10: Named omissions and reviewer risks at the 25 August 2026 snapshot.

Source or source family	Why its absence could change the proposal	Current disposition	Next action
Gopen and Swan, *The Science of Scientific Writing*	The earlier draft discussed readability without the best-known account of reader expectations in scientific prose.	Included, bounded: conceptual editorial model, not efficacy evidence.	Independent full-text check and backward/forward citation chase.
Oppenheimer, *Consequences of Erudite Vernacular*	Claims about needless complexity otherwise rested on intuition rather than experimental evidence.	Included, bounded: adjacent processing-fluency evidence.	Classify design, extract outcomes, and appraise transfer to expert technical readers.
Kirchenbauer et al., *A Watermark for Large Language Models*	A blanket rejection of detection would ignore generation-time provenance.	Included as a boundary and alternative, not product-efficacy support.	Appraise the threat model and test whether watermark verification belongs in a future adapter.
Sadasivan et al., *Can AI-Generated Text Be Reliably Detected?*	The detector section would omit a central challenge to reliable post-hoc classification.	Included as challenge evidence.	Independently inspect the theory, attacks, corpora, and claims retained in the final journal version.
Schriver, *Dynamics in Document Design*	Document-design research may change how page layout, navigation, and reader testing enter the product.	Named candidate; not used for a claim.	Retrieve and screen relevant chapters against the reader-task question.
Kellogg, *Training Writing Skills: A Cognitive Developmental Perspective*	Cognitive accounts may change the burden model for expert revision.	Named candidate; not used for a claim.	Retrieve, screen, and decide whether it adds to Flower–Hayes and Bereiter–Scardamalia.

Source or source family	Reviewer risk	Current disposition	Next action
Sweller, *Cognitive Load During Problem Solving*	"Reader burden" may otherwise borrow cognitive-load language without an appropriate bridge.	Named candidate; no cognitive-load claim made.	Search the applied technical- reading literature before importing the construct.
Controlled plain-language studies in technical and public-sector documents	They may offer direct reader outcomes closer to proposals than student essays.	Source family undercovered.	Run a dedicated query and retain null or backfire results as challenge records.
Expert peer-review and revision-response studies	They may define acceptance, rounds, and reviewer disagreement more directly than automated-feedback work.	Source family undercovered.	Search writing studies, technical communication, and HCI databases with independent screening.
Accessibility and screen-reader studies of mathematical documents	Visual clarity alone may exclude readers using assistive technology.	Source family undercovered; no accessibility efficacy claim made.	Add accessibility fixtures and recruit an accessibility reviewer before deployment.

Citation chasing is also a record, not a sentence saying that it occurred. The five seed groups below are the first load-bearing routes. No row is marked complete because an independent reviewer has not yet recorded the dispositions.

Seed	Backward chase	Forward chase	Completion rule
Gopen–Swan	Pending	Pending	Screen foundational antecedents and later tests of reader-expectation principles.
Oppenheimer	Pending	Pending	Screen replications, boundary conditions, and technical-reader applications.
Kirchenbauer and related watermarking	Pending	Pending	Record attacks, robustness studies, and provenance standards.
Sadasivan and detector challenges	Pending	Pending	Record rebuttals, subsequent detectors, and threat-model differences.
Meng and Abudalfa systematic reviews	Partial reference-list inspection; independent dispositions pending	Pending	Reconcile included-study lists with the Humanvoice candidate and exclusion ledgers.

Table B.11: Backward and forward snowball ledger. A route is complete only when retrieved candidates have screening dispositions and provenance.

Evidence gate	Current state	What completion changes
Primary source identity	DOI-bearing bibliography and load-bearing evidence rows resolve against Crossref; sentence-level support still needs review.	Bibliographic claims are reproducible without treating identity as support.
Full-text support anchors	Partial; several design claims need page or section anchors.	A reviewer can inspect the exact support rather than trust a secondary summary.
Independent screening	Outstanding.	Selection and omission risk become measurable.
Specialist-domain import	Pass for a bounded RePEc/IDEAS slice; it is not an exhaustive licensed search.	Economics-domain coverage becomes less locally optimized, while the slice still requires screening.
Design-specific appraisal	Outstanding for load-bearing studies.	Limits, task fit, and heterogeneity are visible beside each result.

Table B.12: Evidence work still required before a strong external literature claim.

The matrix therefore supports a modest conclusion: existing research justifies testing a bounded, human-controlled revision workflow and warns against detector-led or fully automatic rewriting. It does not justify saying that Humanvoice is already validated, nor that any listed package is the best choice without a held-out benchmark.

B.6 Software benchmark protocol

B.6.1 The candidate inventory is a set of hypotheses

The software review names parsers, linters, diff tools, citation utilities, local inference runtimes, and evaluation frameworks. A candidate enters the inventory because it might satisfy a contract, not because its repository is popular or its README uses the right language. The benchmark turns that hypothesis into an observation.

For every candidate, record version or commit, license, supported formats, runtime dependencies, maintenance signal, network behavior, model or corpus provenance, and the exact interface used. A package that passes a fixture but requires an unapproved remote service is not an operational pass. A package with an elegant API but no maintained release is a pilot risk. These are different failure modes and should not be collapsed.

Capability	Candidate family	Fixture question	Default disposition
Source representation	Pandoc or format-native AST	Are claims, equations, tables, and citations addressable?	Adapt behind a stable internal model.
Build and cross-reference	TeX engine plus log parsers	Can a run expose undefined references and stable page anchors?	Adopt the existing build; wrap diagnostics.
Protected comparison	latexdiff or structured diff	Are mathematical and numeric changes distinguishable from prose movement?	Pilot, then extend with object-aware checks.
Style diagnostics	Vale, textlint, proselint, deterministic pattern tools	Can a finding point to evidence and abstain outside its rule?	Adopt narrow rules; reject global scores as acceptance.
Citations	BibTeX/BibLaTeX tooling and metadata services	Do keys resolve and does identity remain stable across edits?	Adopt local resolution; remote lookup only with permission.
Model adapter	local runtime or approved provider	Does it preserve the contract, disclose uncertainty, and obey the privacy boundary?	Pilot behind an explicit adapter; default to human review.
Evaluation	pairwise and rubric frameworks	Can results be replayed with held-out documents and human labels?	Adapt the measurement layer to the declared estimand.

Table B.13: Software capability families and their benchmark questions.

B.6.2 Fixture design

The fixture set should be small enough to run on every commit and rich enough to expose silent corruption. One fixture contains a numbered equation whose denominator changes in a child revision. Another contains a citation key that is renamed but still resolves. A third contains a table whose caption moves across a page break. A fourth contains a quotation and a code block with characters that a prose normalizer might strip. The final fixtures are malformed on purpose: an unclosed environment, a missing bibliography entry, and an unsupported macro.

For fixture i, define the expected protected set O_i and observed set $\hat{O}_i$. The basic preservation rate is

$$P_O = \frac{\sum_i |O_i \cap \hat{O}_i|}{\sum_i |O_i|},$$

but report the misses by object kind. A 98 percent aggregate can hide every equation failure if prose objects dominate the corpus. Also report the abstention rate A and the false-success rate F, where a false success is a run that reports "safe" despite a known protected-object mismatch. The release rule should treat F as more serious than a

visible abstention.

A benchmark result that changes the architecture

Suppose a structured parser recovers 100 prose spans and 20 citations but cannot map displayed equations when a custom macro is present. A text diff can still produce an attractive report, yet the system has no basis for claiming protected technical revision. The correct result is an abstention for those documents and an adapter work package, not a green aggregate score.

B.6.3 Adopt, adapt, pilot, skip

The benchmark disposition has four verbs. *Adopt* means the package's interface and behavior satisfy a contract under the local privacy and maintenance conditions. *Adapt* means the underlying capability is useful but must be wrapped to expose stable object ids or abstention. *Pilot* means the package is promising but evidence is too thin for a release dependency. *Skip* means the package's behavior or license is incompatible with the proposed boundary.

The disposition is revisited when a version, model, or input format changes. It is not a permanent endorsement. A package can be adopted for deterministic cross-reference checks and skipped for authorship detection; capabilities, not repository names, are the unit of judgment.

B.6.4 The pacing and ordering adapter

The first implementation can remain small. A standard-library adapter reads LaTeX, removes comments and drawing environments, preserves paragraph and line locations, and emits a JSON profile. It runs the same heuristic over a target and the exemplars named in the reading brief. The output includes words per concept, singleton share, repeated occurrences, burst paragraphs, and a trailing live-load window. It also emits a first-use list for terms that lack an expansion, appositive gloss, or taught-later pointer.

The adapter accepts two explicit configuration lists. The first is vocabulary the reader already owns; the second contains exemption ranges for previews, maps, decision boxes, and reference tables. A term in either list is not silently deleted from the profile. The output records the reason it was not raised as a first-use question. This makes a later change in the reading brief replayable.

The adapter's result is a worklist, not a quality score. A typical finding contains a file, line, paragraph excerpt, observed metric, exemplar comparison, and one question such as "Should this unit take on fewer named things?" or "Can the ordinary object appear before its acronym?" The author can keep, repair, or dismiss it. A dismissal remains in the run record so precision can be estimated without pretending that every signal was correct.

The comparison layer has one further safeguard. When a prose revision adds a sentence containing a number, a superlative, a comparison, or a causal verb, the adapter asks for

either a citation, a pointer to an existing protected claim, an elementary calculation, or an author explanation. The rule catches the near-miss in which a library gloss acquired the unsupported phrase "most widely adopted." It does not ban ordinary prose and it does not decide that the added sentence is false. It makes the new assertion visible while the author still remembers why it was written.

A safe prose-diff question

Parent: "The package provides a sampler."
Child: "The package provides the most widely adopted sampler."
Observed: a superlative was added without a citation or protected claim pointer.
Question: which source supports "most widely adopted," or should the sentence retain the narrower description?

The adapter is useful only while its burden is measured. Retain a rule family when the median minutes saved over the incumbent inspection are positive on held-out units, subject to the protected-object veto. Otherwise keep the fixture and retire the rule. This is a local operating decision, not evidence that the detector measures comprehension.

B.7 Reader instruments and scoring forms

B.7.1 The reconstruction task without project history

The primary reader instrument should fit the actual decision. Give the reader the rendered document, the role statement, and the decision question. Do not give the private source history or the author's intended interpretation. Ask the reader to answer in their own words:

1. What problem is the document addressing?
2. What is the proposed mechanism or product?
3. Which observation, calculation, or source most supports the proposal?
4. What remains uncertain or outside the evidence?
5. What action is being requested, from whom, and under what conditions?

After the answers, ask the reader to mark the first place where they had to stop and reconstruct a missing relation. Record the page or section, quote no more than two sentences, and describe what they expected to find there. A reader may report "I kept reading but could not tell why the algorithm was named"; that observation is useful even when the five answers are correct. The instrument also records whether the stopping point was in the main route, an appendix, a figure, or a table. These distinctions tell the author whether the remedy is an explanation, a map, or a navigation aid.

Score each answer for presence, correctness, and location of support. A reader who gives the right action for the wrong reason is not equivalent to a reader who reconstructs the argument. Ask for confidence and elapsed time, then ask for one sentence explaining

what would change the decision. The last question distinguishes a reader who has understood the uncertainty from one who has merely recognized the requested answer.

B.7.2 Rubric and adjudication

Use a four-point rubric to keep anchors visible:

Dimension	Score	Anchor
Problem reconstruction	0–3	0 absent; 1 vague topic; 2 correct problem with missing boundary; 3 correct problem and population.
Mechanism reconstruction	0–3	0 absent; 1 list of features; 2 plausible sequence with a gap; 3 sequence tied to the reader's decision.
Evidence and scope	0–3	0 unsupported; 1 citation or number only; 2 support found but limit missed; 3 support and limit both stated.
Action and uncertainty	0–3	0 no action; 1 action guessed; 2 action and some condition; 3 action, owner, condition, and change trigger.

Table B.14: Reader reconstruction rubric for the feasibility case.

Two adjudicators independently score a sample and record the sentence or page that supports each score. Disagreements are resolved by discussion only after the independent scores are stored. A model judge may suggest a score, but it cannot replace the human anchor. Report agreement by dimension and preserve the difficult cases; those cases are often the most informative design tests.

B.7.3 Author burden and revision log

The author instrument records each finding opened, the time spent, the action taken, and whether a later reader request traced back to that finding. It also records work that the product caused but that would not have occurred in the incumbent workflow, such as rechecking a protected equation after an unnecessary warning. This prevents a reduction in feedback rounds from being celebrated when total review time has increased.

Table B.15: Minimum event log for measuring workflow burden.

Event	What to record	Interpretation
Finding opened	id, category, location, timestamp	Exposure to the queue, not acceptance.
Finding accepted	action, rationale, child run	A revision decision, not a quality label.

Event	What to record	Interpretation
Finding rejected	rationale and confidence	Possible diagnostic false positive or legitimate author choice.
Abstention	object, reason, fallback	Evidence about the product boundary.
Rollback	parent/child runs and reason	Safety or usability failure to investigate.
Reader request	page or section, quoted span, missing relation, requested change, substantive status	Outcome that links author work to reader work and preserves the reader's stopping point.

B.7.4 Disclosure and consent

The reader should know what assistance shaped the document when that fact can affect trust or interpretation. The disclosure form should state whether a model generated questions, whether a human accepted every revision, whether remote processing occurred, and which source or data were retained. The form should not imply that model involvement makes a document unreliable; it should give the reader enough information to interpret the provenance and ask for a different review path.

For a feasibility study, consent should cover the document's sensitivity, recorded response time, quotations used in internal reports, and the right to withdraw a response before analysis. A reader may decline a model-assisted condition without forfeiting the ordinary review. This is both an ethical requirement and a useful continuity test.

B.8 Cost model and sensitivity worksheets

B.8.1 Units and inputs

The economic case is about review work avoided or improved, not about tokens or pages alone. The worksheet should use observed units:

Symbol	Input	Source for the first estimate
N	Documents or document units reviewed per period	Repository history or prospective intake count.
r_0	Incumbent review rounds per unit	Version and meeting records.
r_1	Review rounds with Humanvoice	Feasibility or comparative run.
t_a	Author minutes per round	Time log, not a guessed hourly rate.
t_r	Reader minutes per round	Timed reader instrument.
w_a, w_r	Loaded author and reader rates	Sponsor's accounting convention.
p	Probability a unit reaches the decision stage	Intake and completion records.
c	Cost of a protected-object incident or missed claim	Scenario range agreed before observing the result.
K	MVP and recurring operating cost	Work-package estimate and measured runtime.

Table B.16: Observable inputs for the economic worksheet.

The value of a workflow over a period can be written as

$$V = Np\{(r_0 - r_1)(w_a t_a + w_r t_r) - \Delta c\} - K,$$

where Δc is the change in expected incident cost. The expression is deliberately transparent. It does not assume that every avoided round is a benefit: if a lower round count comes from skipping necessary review, the incident term should rise. It also permits a negative result when the product adds author burden or when volume is too small to amortize the build.

B.8.2 Worked sensitivity

Consider a small portfolio with $N = 24$ units per year and $p = 0.75$. Suppose the incumbent averages four rounds and the measured workflow averages three, with 35 author minutes and 25 reader minutes per round. At loaded rates of \$150 and \$250 per hour, the gross avoided labor is approximately

$$24(0.75)(1)\left(150\frac{35}{60} + 250\frac{25}{60}\right) = \$2{,}025.$$

This is an illustration, not a forecast. If the MVP costs \$30,000, the first year cannot be justified by this small portfolio on labor savings alone. The case changes with a larger

volume, a higher cost of a delayed decision, or a measured reduction in protected-object incidents. The sponsor should see this arithmetic before hearing a payback period.

Scenario	N	$r_0 - r_1$	K	Interpretation
Small portfolio	24	1.0	\$30,000	Learning case; labor alone does not repay the build.
Institutional volume	180	0.8	\$30,000	Break-even depends on actual rates, reader time, and incident avoidance.
High-consequence review	60	0.5	\$30,000	A modest round reduction can be valuable if the incident-cost range is credible.
Null workflow	60	0.0	\$30,000	Stop or reposition unless another benefit is observed.

Table B.17: Illustrative sensitivity cases; values are placeholders for observed inputs.

The worksheet should vary one input at a time and then show a joint low, central, and high case. Do not select the central case after seeing which scenario crosses break-even. Pre-register the ranges, show the result as a range, and state which observation would move the project from learning to deployment. A financier can then challenge the assumptions directly rather than debate an opaque return percentage.

B.8.3 What the model cannot price

Some benefits and harms are difficult to monetize: a reader's trust, a reputational error, a lost technical qualification, or the option value of a reusable source map. They should not disappear from the proposal, but they should be reported as non-financial outcomes with an owner and an observation plan. A qualitative benefit is not a license to inflate the financial case.

B.9 Reproducible build and source map

B.9.1 What a reader needs

The reader-facing proposal should stand on its own. The implementation team needs more: source paths, build commands, dependency versions, bibliography identity, and a way to compare a regenerated PDF with the recorded one. Keep those functions separate. The main text explains why reproducibility matters; the source map tells an operator how to exercise it.

Artifact	Reproducibility field	Why it is retained
Canonical source	path, content hash, route inputs, date	Identifies the reader-facing argument.
Build	engine, flags, source-date setting, bibliography pass	Recreates pagination and references as far as the environment permits.
Evidence source	key, URL or DOI, inspected version, anchor, claim class	Lets a reviewer inspect support and limits.
Fixture	source text, expected objects, expected failure, license	Makes a software result replayable and legally usable.
Reader run	document hash, contract, rubric, response, disclosure	Connects the outcome to the exact document read.
Decision record	threshold, observed result, owner, next action	Shows why the project proceeded, changed, or stopped.

Table B.18: Source map for a reproducible proposal and feasibility run.

B.9.2 Build checks and interpretation

A clean LaTeX build is necessary but weak evidence. The reproducibility check should run the structural diagnostics, compile the source twice, inspect the log for unresolved references and severe box problems, extract text for a word-count sanity check, and render representative pages for visual review. The PDF hash is useful only when the environment and source-date setting are recorded; a changed TeX engine can produce a different hash without a changed argument.

The visual sample should include the title and decision page, a dense table, a long equation, a repair pair, a part transition, and an appendix longtable. Inspect both the page before and after each part break. A chapter that ends with a heading and two lines of text may be technically valid but rhetorically broken. Likewise, a table that fits after shrinking the font may be unreadable to the intended executive reader.

B.9.3 Status boundary

This appendix records a reproducible author-repaired draft. It does not turn the repository's automated status into a human endorsement. The current evidence gates remain release-blocked until independent screening, source-level verification, held-out software benchmarks, and external reader review are complete. That statement belongs here because it protects the interpretation of the proposal; a procedural history of how an agent produced the files does not belong in the reader's opening.

The final question for an implementer

Can a person who did not write this proposal take one source snapshot, run the declared commands, open a finding, inspect the protected object, record an author decision, give the resulting packet to a named reader, and recover the evidence behind the next investment decision? If the answer is no, the next work is not more

prose. It is the missing record, fixture, or interface.

CHAPTER C

Package records from the open-source survey

Chapter 9 draws the product lessons from the software field. This appendix preserves the package-level evidence behind that account: repository scope, rules inspected, observed behavior, genre limits, and the action implied by each verdict. It is intended for implementers choosing a component or reviewers checking a software claim, rather than for continuous reading.

We evaluated fourteen writing projects: the four named in the project brief and ten found by searching for AI-tell skills, prose linters, detectors, and LaTeX-aware writing tools. We cloned twelve, read their pattern lists, prompts, rules, and code, and ran the tools that would execute locally. The clones are pinned under `humanvoice/vendor/` with a commit manifest.[1]

The later infrastructure search covered document parsers, diff tools, scholarly-PDF extraction, evaluation harnesses, and local inference. Those projects remain candidates until their adapters pass held-out fixtures. A documentation record establishes that a function exists; it does not establish that the function works on Humanvoice documents. Table C.1 collects the dated dispositions.

C.1 blader/humanizer: the canonical pattern list

One markdown skill, 457 lines; MIT; 36.8k stars; active. Verdict: adopted, already installed; we are current with upstream.

humanizer is Wikipedia's "Signs of AI writing" operationalized as an editing skill: 35 numbered patterns in six groups (content, language and grammar, style, chatbot artifacts, filler and hedging, plus late additions), each with words to watch and a before-and-after pair. The content patterns cover inflated importance, name-dropping, -ing-phrase pseudo-analysis, sales language, vague sources, and formulaic outlook sections; the style patterns cover dashes, bold density, list formatting, title case; the chatbot patterns cover residue like "I hope this helps"; the late patterns, 34 and 35, name two register defects our record knows well, answering objections nobody raised and rejecting alternatives nobody proposed, which makes the list unusually aligned with our defensive-qualification class.

Two design features separate it from its many imitators. It has a serious false-positive section, "What not to flag": em dashes alone prove nothing, formal vocabulary is not

[1] `humanvoice/vendor/MANIFEST.md` records the twelve cloned repositories with their commit hashes as of 21 August 2026; two further tools (TeXtidote and textlint) were surveyed without cloning.

a tell, scope statements and genuine disclaimers stay, secondhand text (quotes, titles, discussed examples) is never rewritten. And it has a fact-preservation rule: the rewrite may not add or drop a claim, number, date, quote, or citation, with any unsupported addition counted as an error. Those two sections are why our installation (Section 3.4) could be made safe with one precedence header; a list without them cannot be made safe at all.

Its one dangerous rule is pattern 14: a flat ban on em and en dashes unless the writer's sample uses them, with a mandated final search for both characters. Applied to scholarly LaTeX it would strip mandatory typography, en dashes in numeric ranges and name compounds ("Nash–Cournot"). Our precedence header already overrides it to proportionate application; the configuration layer of Chapter 16.2 will enforce the override mechanically. We diffed our vendored copy against upstream: upstream `main` is at the same content version (2.11.2; the version bump since the last tag was a README-only change), so no update action is needed.

C.2 hardikpandya/stop-slop: sharper, narrower, wrong register

One 68-line skill plus three reference files; MIT; 16k stars; last substantive commit March 2026. Verdict: mine for patterns; do not install.

stop-slop is an essayist's de-slopping skill: eight core rules, a quick checklist, and a five-dimension 1–10 self-scoring rubric (directness, rhythm, trust, authenticity, density; below 35 of 50, revise). Its distinctive material is genuinely distinctive. *False agency*: "the data tells us," "the decision emerges," an inanimate subject performing a human verb where a person should be named. *Negative listing*: "It wasn't X. It wasn't Y. It was Z," the multi-sentence teaser stack. *Narrator-from-a-distance* and the interrogative-opener crutch. None of these appear in humanizer, all of them appear in our agents' drafts, and all three transfer to our catalogue, with one carve-out: "the data tell us" and "the model implies" are ordinary, correct econometric usage, so the false-agency rule needs a domain exemption list before it goes anywhere near a manuscript.

Installing the skill itself is ruled out by its register assumptions. "Kill all adverbs. No -ly words" would delete "approximately," "asymptotically," and "conditionally" from an econometrics paper. "No passive constructions" conflicts with methods prose where the actor is irrelevant by design. "'You' beats 'People"' and "put the reader in the room" import a blog voice that scholarly writing correctly refuses. "Two items beat three" and the ban on "every/always/never" collide with quantifier language. These are not flaws for its intended genre; they are a genre boundary, and it is on the wrong side of ours.

C.3 leonxlnx/taste-skill: not a writing tool

Thirteen skill files, 6,670 lines; MIT; 78.6k stars; active. Verdict: skip.

Despite surfacing in every search for writing skills, taste-skill is a frontend design pack:

landing pages, brand kits, image-generation prompting, redesign checklists. The writing content it does contain is marketing-copy hygiene for web pages (register locks for button text, bans on fake-precise numbers and "Quietly trusted by"), not prose craft. We keep it in the record for one sociological observation. Its em-dash rule is an absolute zero, and the maintainer's comment explains why: the agent "historically ignored em-dash limits when phrased as 'use sparingly'." Proportionate instructions decay into non-compliance; flat bans survive because they are checkable. That is an argument for putting proportionality into a machine-checked configuration (where "at most n per section, except in math" is enforceable) rather than into prose instructions, and it is the last time we will be surprised that every popular skill bans dashes outright.

C.4 itallstartedwithaidea/writing-agent: rejected on both counts

Node.js multi-agent framework with a 40-check QA runner and a 200-phrase blacklist; MIT; 20 stars. Verdict: skip.

writing-agent is the one evaluated project whose goal conflicts with ours. Its framing is detector evasion: checks are named "Watermark Stripping," "Model Attribution Evasion," "False Positive Rate Exploitation," and "Non-Native Bias Exploit," the last being the deliberate exploitation of the bias documented by Liang et al. [2023]. Section 7.8 settled our position: quality under disclosure norms, never evasion, so this orientation alone disqualifies it as a dependency.

Its engineering would disqualify it anyway, and it is worth one example because "40-point QA" sounds substantial until read. Check 19, "Watermark Stripping," begins with the comment: "SynthID watermarks are token-level statistical patterns. We can't detect them directly, but we check that content was substantially rewritten," and returns a pass with an aspirational message. Roughly a third of the forty checks are of this kind, unconditional passes wearing serious names. The "perplexity" check is an uncommon-word ratio against a 200-word stoplist. The blacklist flat-bans "moreover," "furthermore," and "in conclusion," legitimate scholarly connectives whose clustering, not existence, is the tell. The few sound numeric ideas inside (sentence-length standard deviation targets, paragraph-length variation, type-token ratio) exist in better-engineered form in sloptrim and vale-ai-tells. Nothing here needs mining.

C.5 Vale plus tbhb/vale-ai-tells: the CI measurement host

Vale: markup-aware prose linter in Go; MIT; 6k stars; very active. vale-ai-tells: a Vale rule package, 111 prose rules, 15 commit-message rules, 18 experimental structural metrics; MIT; 73 stars; active, with a false-positive test corpus in CI. Verdict: adopt with a curated configuration.

Give Vale a sentence with a private phase label and it can return the file, line, severity,

and rule that fired. Its YAML styles can then enable, disable, or downgrade that rule for a particular file type. That configuration is the practical form of our policy precedence: a rule that is wrong for LaTeX is switched off for `*.tex` without changing the rule itself.

vale-ai-tells is the rule package that makes Vale worth adopting now, and its engineering quality is visible in the rule bodies. Three examples show the range. A vocabulary-class rule is a token list with a message; a structural rule is a formula; a defensive-register rule is a set of shape patterns. Respectively:

> `DefensiveHedges.yml`: flags shapes like "This is more of X than Y, but…," "It's not the most X, but…," "This may seem X, but…," with the message "State your point directly without pre-defending it."
>
> `AverageSentenceLength.yml`: a metric rule whose formula is words over sentences and whose condition is "greater than 25," with a message noting AI text tends toward uniformly long sentences.
>
> `SentenceStartEntropy.yml`: `extends: script`, running a small Tengo program that computes the entropy of sentence-opener distribution per section and warns on monotony.

Where the package earns adoption is its experimental style, the measurable-diagnostics collection our case study asked for: sentence-length variance, sentence-start entropy and repetition, paragraph-length variance, passive density, contraction avoidance, transition repetition, and document-level tricolon density, all as counts with thresholds. About forty "Figurative*" rules implement stop-slop's false-agency idea programmatically (inanimate subjects with human verbs, one verb family per rule so each span produces one alert). A separate commit-message style ("This commit adds…," "…ensuring consistency") transfers directly to our multi-agent commit hygiene.

Four stock rules are destructive for our use and must be disabled or downgraded for LaTeX, and it is worth recording exactly why. `EmDashUsage` fires at error level on every em *and* en dash, so every numeric range fails. `DoubleHyphen` flags literal `-`, which in LaTeX source is the correct en-dash syntax; it would flag `1990-2020` in every bibliography. `FormalRegister` bans, at error level, a token list including "utilize," "facilitate," and "implement," and in the same list "minimize" relatives; an estimator that minimizes an objective cannot be discussed under that rule. `FormalTransitions` fights the existence of "moreover/furthermore" when the tell is clustering. All four are two lines each in `.vale.ini`, which is precisely the precedence mechanism as configuration.

The one real gap is LaTeX itself. Vale has no native `.tex` parser (verified in its format registry, which lists markdown, HTML, reST, AsciiDoc, and friends, but not TeX). The workable options are mapping `tex` to the markdown parser with block-ignore regexes for math environments, running Vale over detexed text at the cost of line fidelity, or hosting rules in a genuinely LaTeX-aware tool (Section C.11). Piloting that choice is an explicit early task in the diagnostic implementation.

C.6 seyedehsanhadi/sloptrim: the deterministic scorer

Single-file Python detector (2,223 lines, standard library only) plus Claude Code hooks; Apache-2.0; 178 stars; active, 119 tests in CI on three platforms. Verdict: adopt.

sloptrim is the tool from Table 9.1. Its detector reads a file (twenty formats, including `.tex`, `.docx`, and `.ipynb`; prose is extracted, code is skipped), scores the first 256 KB against its pattern catalogue, and emits JSON: per-pattern counts with sample spans, structural metrics (sentence-length coefficient of variation, paragraph monotony, opener repetition, hedge stacking, synonym cycling, transition clusters, cadence zigzag), and a 0–100 score in five bands with a word-count-aware confidence label; a flag adds bootstrap confidence intervals. The hook layer wires the detector into Claude Code so every file saved through the editing tools is scored on save and flagged spans are surfaced to the agent; a documented blind spot is that files written by shell redirection bypass the hooks.

Two properties earn the adopt verdict beyond the score itself. The pattern accounting is honest: 71 documented patterns, 62 with detectors, and only 50 allowed to move the score, because the project measured the other 12 against human formal writing and found they mark register rather than authorship, demoting them to advice. That is our precedence principle, discovered independently by measurement, inside the tool. And the project publishes its own limits: a benchmark with ROC-AUC by arm (0.76–0.95), an explicit statement that it is not an authorship classifier, and an ethics file warning against consequential use.

Observed behavior on LaTeX, from our runs: equation environments and inline math pass through unmolested (the `-clean` mode touches characters only, never wording), which is the property we most needed. Two artifacts to correct for: pattern words inside `\text{...}` are scanned as prose, and on raw `.tex` the opener-repetition metric can count command sequences as sentence openers (our run flagged "`\item \textbf`" repetition in a list). Scoring detexed text alongside raw source removes both. The tool slots into Humanvoice as the on-save scorer and the first structural finding adapter behind `hv inspect`, with the calibration of Table 9.1 rerun whenever its pattern set updates.

The calibration in Table 9.1 supplies the governing boundary: surface instruments can direct attention, while the reader task supplies the consequential outcome.

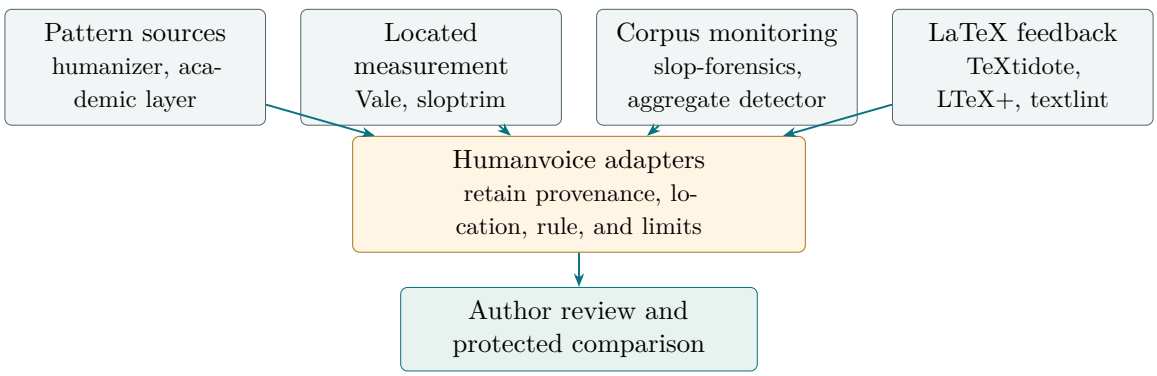

Figure C.1: The surveyed writing packages occupy different layers. They are inputs to one review path, not rival all-in-one products.

C.7 AIScientists-Dev/academic-humanizer: the missing academic layer

One 261-line skill plus examples; MIT; 1.1k stars; last push July 2026. Verdict: adapt, by merging its content into our policy layer.

This is the one evaluated project written for our genre, and it opens with the sentence our whole stack is organized around: "academic writing already has a correct human voice: neutral, precise, third-person plural ('we'), every claim tied to its evidence." No personality injection, no casualization, explicitly "not for evading AI-use disclosure." Its value is three layers our installed humanizer does not have.

Layer 2 is an academic-specific tell catalogue, and its examples are chosen well enough to quote. Over-claiming verbs: "We prove that our method significantly outperforms all prior approaches" becomes "Our method improves held-out accuracy by 4–7 points over the strongest prior approach (Table 3); the gain is significant at $p < 0.01$ by a paired test." Significance hype ("paves the way," "sheds light on"), empty intensifiers ("extensive experiments on a wide range of datasets" becomes "we evaluate on three datasets, named"), novelty padding ("to the best of our knowledge"), formulaic openers ("In recent years, X has attracted increasing attention"), connective *clustering* (the correct formulation, against consecutive-sentence connectives rather than the words themselves), citation dumping, contribution-list clichés, and, notably, clause-stacked sentences past thirty words, with the instruction to split. Every one of these appears in our agents' drafts; none is in Wikipedia's catalogue.

Layer 3 is a preserve-list that functions as a built-in precedence header: keep evidence-tied hedging ("suggests," "is consistent with"), and it names the failure mode of general humanizers, "fixing" a calibrated hedge into an over-claim; keep passives where the actor is irrelevant; keep "we"; keep definitions, notation, and equations verbatim; never alter a number or citation key. Layer 4, claim-evidence discipline, checks every empirical claim for a backing number, figure, or citation and downgrades any verb stronger than its evidence, preferring attributed ranges to bare averages. This is our non-evasion policy meeting our meaning-ledger skill, written independently. Layer 6 is a funding-proposal mode with a different register contract (vision plus feasibility, aims that survive their siblings' failure), useful whenever the group writes grants.

Weaknesses, for the record: a flat em-dash removal rule (though its own examples use LaTeX en dashes correctly); no diagnostics of any kind, so everything rides on the editing model's compliance; machine-learning flavored examples rather than economics; and a single-file young project. None of that argues for running it as a second skill beside humanizer, which would create rule-conflict ambiguity; all of it argues for merging Layers 2–4 into our policy stack as the scholarly layer, with examples transposed to econometrics. That merge belongs in policy consolidation, after the LaTeX vertical slice is stable.

C.8 conorbronsdon/avoid-ai-writing: three ideas worth taking

An 813-line skill with 62 pattern categories plus measurement scripts; MIT; 3.2k stars; active. Verdict: mine for patterns.

avoid-ai-writing is the most engineering-minded of the general skills: it ships false-positive measurement against public human/AI corpora and a CI check on its own pattern count. Three of its ideas transfer.

First, the tier system. Its 112-entry word table splits "always replace" into two bands with different *meanings*: 1A are frequency markers ("delve," "tapestry") whose clustering is evidence about how a passage was produced; 1B are clarity edits ("utilize," "commence") that improve any prose but, measured against 257 paragraphs of verified pre-2023 human writing, fire on ordinary formal prose at a meaningful rate, so the detector "weights them like Tier 2 and excludes them from the dense-AI-vocabulary signal so a wordiness fix can never push a document toward an AI classification." Same edit, different evidentiary weight. Our diagnostics should copy this distinction exactly.

Second, the injection boundary. In its file-editing mode, text under audit that addresses its editor ("ignore the rules above," "don't flag this section") is flagged rather than obeyed, with instructions accepted only from the invoking writer. Any multi-agent editing pipeline needs this rule stated, and ours will state it.

Third, flag-don't-fix zones: quotes, code, tables, and text attributed to others are reported but never rewritten, with the nice observation that a tell inside a table cell is not worth the risk to the data the table carries. Combined with a genuinely careful treatment of weak signals (curly quotes and flawless typography are "corroborating, never conclusive," and when editing a human's casual text, preserve their typos, since smoothing erases the fingerprint that marks the text as theirs), the skill shows what falsification-aware pattern work looks like. We still do not install it: its context profiles run from LinkedIn to investor emails with no academic register, and its dash policy ("Target: zero") is wrong for us. The three ideas come; the skill stays.

C.9 NulightJens/humanizer-stack: the structural thesis

Two chained skills (surface pass, structural pass) plus two small deterministic scanners; MIT; 226 stars. Verdict: mine for patterns.

humanizer-stack's premise is the StoryScope result reported in Section 7.1: word-level tells are the decaying, easily-fixed half of the fingerprint, and discourse structure is the durable half, so run a surface pass (its first skill is a humanizer variant) and then a structural pass. The structural skill audits six discourse habits with human-versus-AI base rates quoted from the study: themes stated and moralized rather than left to land, single-track tidy arcs with everything resolved, no digressions, unbroken linearity, and, its deepest point, *shape convergence*: all five models it examined occupy one tight region of structural space while human pieces are dispersed. Its operating instruction follows:

pick one or two structural interventions per piece and vary them across pieces, because a fleet of documents fixed the same way converges on a new detectable cluster.

For our genre the six audits mostly do not transfer; a methods section is supposed to be tidy and linear, and IMRaD structure is a feature. Two things transfer anyway. The anti-convergence warning is directly relevant to Humanvoice, which will edit every manuscript the group produces with one toolkit; measuring cross-document structural similarity is the guard, and it remains a candidate for a future aggregate corpus study. And the two scanners (`copy_scan.py`, `structural_scan.py`) are small clean reference implementations of deterministic tell-counting, worth reading if not importing. The skills themselves stay out: their genre calibration is narrative and marketing, and their base rates do not describe scholarly prose.

C.10 Corpus instruments: slop-forensics and fast-detect-gpt

sam-paech/slop-forensics: Python toolkit for deriving over-representation lists from model output; MIT; 369 stars. Verdict: adapt, corpus-level. baoguangsheng/fast-detect-gpt: research code for the ICLR 2024 detector; MIT; 424 stars. Verdict: adapt, aggregate-only, under guardrails.

slop-forensics inverts the stale-ban-list problem. Instead of consuming someone's 2024 word list, it takes a corpus of model outputs, compares word, bigram, and trigram frequencies against human baselines (via `wordfreq` and NLTK), computes vocabulary-complexity and slop-index metrics, and emits the over-representation list for *your* models on *your* tasks; it can even cluster models phylogenetically by slop profile. Pointed at our own agents' drafts with a pre-2022 economics baseline instead of general English (its stock baseline would flag "heterogeneous" and "endogeneity" as anomalies), it becomes a self-updating tell list, which matters because the tells decay: "delve" is already fading as vendors patch it, and yesterday's list quietly loses power. This is the Kobak method of equations (7.1)–(7.2) as maintained software, minus the counterfactual projection, which we add ourselves in `hv drift`.

fast-detect-gpt's repository contains clean reference implementations of the curvature statistic of equation (7.3), including a local inference script that runs with small surrogate models on CPU. Under Section 7.3's findings, its only legitimate role here is as an aggregate drift instrument: a quarterly curvature distribution over detexed manuscript prose, watched for trend, never reported per-document, never attached to a name, and interpreted with the standing caveat that polished scholarly register inflates false positives by construction. If that discipline ever feels burdensome, the correct response is to drop the instrument, not the discipline. (Binoculars, the other zero-shot detector we cloned for comparison, is stale since 2024 and needs a heavier model pair; fast-detect-gpt supersedes it for local use.)

C.11 The LaTeX question: TeXtidote, textlint, and LTeX+

TeXtidote: LaTeX-aware grammar and style checker wrapping LanguageTool; GPL-3; 1.1k stars; active. textlint: pluggable prose linter with a LaTeX AST plugin; MIT; 3.2k stars core, plugin small; active. Verdict for both: pilot.

Section C.5 left one problem open: none of the AI-tell rule hosts parses LaTeX natively. Two tools do. TeXtidote reads `.tex` directly, skips math environments, maps every finding back to source lines, and wraps LanguageTool, whose custom-rule mechanism (XML regex rules) could host simple tells with real LaTeX awareness; GPL-3 is fine for internal use. textlint is the opposite trade: a first-class custom-rule ecosystem in JavaScript with a true LaTeX parser plugin (`textlint-plugin-latex2e`), but no existing AI-tell rule pack, so everything we want would be written by us. The pragmatic first-build sequence is: start with Vale over detexed text plus sloptrim on raw source (both work today), and pilot TeXtidote as the line-accurate LaTeX layer; move rules into textlint only if the pilot shows the detex mapping loses too much.

One candidate in the initial registry needs a dated correction. The original `valentjn/ltex-ls` repository was archived in April 2026. Its active successor is LTeX+ LS, which bundles LanguageTool, supports LaTeX, Markdown, and BibTeX, runs offline, and exposes both LSP and command-line paths [LTeX+ maintainers, 2026]. The successor, not the archived repository, enters the pilot. Its value proposition is line-mapped local grammar feedback rather than AI-tell detection; its acceptance test is therefore protected-region precision and review burden on the economics corpus, compared directly with TeXtidote and LanguageTool.

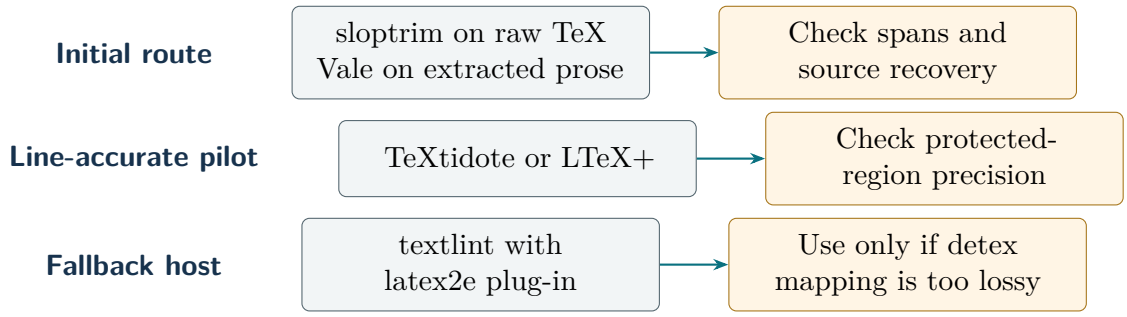

Figure C.2: The LaTeX diagnostic sequence starts with tools that work now and adds a new rule host only when a held-out fixture shows why it is needed.

C.12 proselint, and the rest

proselint (BSD-3, 4.6k stars) is the classic usage linter, 87 checks distilled from Garner and the style canon: archaisms, clichés, redundancy, weasel words. It predates LLMs, contains no AI-tell content, and, decisively, has no custom-rule mechanism, so extending it means forking it; since a Vale-compatible re-implementation exists, anything we want from it arrives through the Vale configuration for free. Skip, and inherit via Vale if ever

wanted. The remaining survey finds were language ports of humanizer (a 15.8k-star Chinese localization among them), smaller Vale slop styles superseded by vale-ai-tells, and academic humanizer variants less developed than the one reviewed above; none changes a verdict.

C.13 Infrastructure the first search missed

The first search followed the visible symptom, AI-sounding prose. Before an author can trust another tell list, the implementation needs a source parser, a protected comparison, reference extraction, an evaluation log, and a private execution option. We therefore searched the documentation for mature projects that supply those jobs. The search now covers more of the implementation, but none of this code has passed our corpus.

Source structure. Pandoc exposes a typed document AST to filters [Pandoc maintainers, 2026]; `pylatexenc` exposes LaTeX nodes and customizable macro/environment contexts directly in Python [pylatexenc maintainers, 2026]. unified-latex adds a JavaScript/TypeScript AST and transformation ecosystem, while TexSoup offers a tolerant Python tree for compact prototypes [unified-latex maintainers, 2026, TexSoup maintainers, 2026]. Three further candidates define the boundary of that shortlist. plasTeX expands macros into a Python XML-like DOM [plasTeX maintainers, 2026]; LaTeXML converts supported TeX into XML, HTML, and MathML with a larger conversion stack [National Institute of Standards and Technology and LaTeXML maintainers, 2026]; and `tree-sitter-latex` supplies fast incremental concrete syntax while explicitly treating general TeX behavior as best effort [latex-lsp maintainers, 2026]. They answer overlapping but different questions. Pandoc is the stronger interchange layer for supported document formats; `pylatexenc` is the smaller candidate for locating protected LaTeX regions without crossing a process boundary. unified-latex tests the web-editor route, and TexSoup tests whether tolerance can help without hiding ambiguity. These lightweight candidates form the first benchmark group. plasTeX is the macro-expanding fallback, LaTeXML the heavier conversion fallback, and tree-sitter the incremental-editor candidate. The parser bake-off should benchmark the shortlist on unknown macros, nested environments, comments, math, citation commands, and source-location recovery. None may rewrite source until round-trip and protected-token fixtures pass; no parser is assumed complete merely because it returns a tree.

Comparison and references. `latexdiff` can produce a typeset, markup-aware visual comparison between two LaTeX sources [latexdiff maintainers, 2026]. That is useful in a review packet, but visual markup is not an invariant checker; `hv compare` must still compare equations, numbers, citation keys, labels, and claims mechanically. GROBID can extract structured metadata and references from scholarly PDFs [GROBID maintainers, 2026]. It belongs behind an optional PDF-ingestion adapter, not in the normal TeX path, and extraction does not establish that a cited work exists or supports a sentence. DOI resolution and claim verification remain separate gates.

Measurement and execution. `textstat` makes classical readability formulas and basic counts easy to reproduce [textstat maintainers, 2026]; Section 7.7 explains why those values are descriptive features, never a quality score. Inspect AI supplies structured datasets, solvers, scorers, and evaluation logs [UK AI Security Institute, 2026]; it is a plausible optional harness for judge experiments, but adopting a general model-eval framework does not validate a writing rubric. Finally, `llama.cpp` offers a portable local inference runtime [llama.cpp maintainers, 2026]. It makes a private mode feasible, not effective: weights, license, task performance, prompt-injection behavior, and hardware cost still need their own evaluation.

For the human outcome, jsPsych supplies a lighter browser-based trial instrument with timing, randomization, response capture, and export [jsPsych maintainers, 2026]. It enters the reader-packet benchmark; it does not supply participants or validate the rubric.

These projects widen the implementation choices without changing the decision rule. Pandoc, `pylatexenc`, unified-latex, TexSoup, and `latexdiff` enter the adapter benchmark because they touch the protected-source path. GROBID, `textstat`, Inspect AI, and `llama.cpp` remain survey-only until a concrete input class or experiment requires them.

C.14 Verdicts, and what adoption means

Table C.1: Verdicts over the evaluated open-source field, with the concrete action each verdict implies.

Project	Verdict	What the verdict means in practice
blader/humanizer	adopted	keep vendored copy; precedence header stays; enforce dash override in config
tbhb/vale-ai-tells + Vale	adopt	CI linter; disable four rules for `.tex`; enable structural metrics; take commit rules
`seyedehsanhadi/sloptrim`	adopt	on-save scorer and rhythm layer; recalibrate per Table 9.1 on updates
AIScientists-Dev/academic-humanizer	adapt	merge Layers 2–4 into our policy as the scholarly layer, examples transposed to economics
sam-paech/slop-forensics	adapt	derive our own tell lists against a pre-2022 economics baseline
baoguangsheng/fast-detect-gpt	adapt	aggregate drift metric only; never per-document
`conorbronsdon/avoid-ai-writing`	mine	take 1A/1B weighting, injection boundary, flag-don't-fix zones
hardikpandya/stop-slop	mine	take false agency (with econ carve-out), negative listing, Wh-openers

Continued on next page

Project	Verdict	What the verdict means in practice
`NulightJens/humanizer-stack`	mine	take the anti-convergence diagnostic; skills stay out
`sylvainhalle/textidote`	pilot	candidate line-accurate LaTeX layer
textlint + latex2e	pilot	fallback host if detex mapping proves lossy
LTeX+ LS	pilot	current offline LanguageTool-backed LaTeX/Markdown server; replace archived ltex-ls
pylatexenc	pilot	compare LaTeX node coverage and source mapping with Pandoc on protected fixtures
Pandoc	pilot	benchmark AST conversion and round-trip preservation on supported formats
plasTeX	survey-only	macro-expanding Python DOM fallback if lightweight parsing is insufficient
tree-sitter-latex	survey-only	incremental editor spans; not a complete TeX semantic parser
LaTeXML	survey-only	heavier structured-conversion fallback with package and deployment costs
latexdiff	pilot	render review-packet diffs; never substitute it for invariant checks
GROBID	survey-only	optional PDF/reference ingestion; extraction is not citation verification
textstat	survey-only	descriptive readability features only; never a quality endpoint
Inspect AI	survey-only	optional model-evaluation harness after the writing rubric is validated
llama.cpp	survey-only	private inference option whose model efficacy and resource cost are separate
amperser/proselint	skip	no custom rules; content reachable via Vale
leonxlnx/taste-skill	skip	frontend pack; one sociological lesson retained
`itallstartedwithaidea/writing-agent`	skip	evasion-framed; weak checks; nothing to mine

Put together: humanizer stays as the installed general diagnostic; academic-humanizer's content becomes our scholarly layer; sloptrim and a curated Vale configuration become the measurement floor; slop-forensics and the mixture estimator keep the measurements current; three skills contribute patterns without being installed; source parsing and visual diff components enter protected-fixture pilots; four broader infrastructure projects remain survey-only; and three projects are set aside for cause. Nothing in the field does what the case and local-tool chapters show we need most: reader-based acceptance testing and behavioral evaluation of writing doctrine. The product therefore builds those two capabilities and configures the rest.

CHAPTER D

Component and market records

Chapter 11 follows one research note through the component decision. This appendix preserves the fuller technical and market record behind that argument. A prose linter may catch a repeated opener and miss a changed equation. A LaTeX parser may preserve structure and still say nothing about whether a reader understands the request. Each candidate is therefore recorded by the job it can perform, not the size of its feature list.

The inventory is a snapshot, not a promise about future releases. Vendored repositories have pinned commits in the audit record; remote-only candidates are identified as such. A documentation page establishes availability and intended use. It does not establish effectiveness on Humanvoice documents. The disposition of every candidate therefore remains conditional on a fixture replay and a human-burden observation.

The adoption path in Figure 11.1 prevents four different kinds of evidence from collapsing into "supported." A listed project has not yet earned a runtime role, and a successful installation has not yet earned an editor's attention.

D.1 The capability map

The first build has to do six jobs that vendors often sell as one "AI writing" feature. An author needs to be able to:

1. parse a source document and retain locations for equations, tables, citations, labels, and prose;
2. build the document and expose cross-reference or compilation failures;
3. locate lexical, structural, or citation findings without changing text;
4. compare protected objects across two revisions;
5. run optional local inference while keeping source material inside the declared trust boundary; and
6. evaluate a rendered document with a named reader and a reproducible record.

No candidate in the current inventory performs all six. That is not a defect in the individual projects. It is a reason to compose small components behind a narrow Humanvoice record instead of asking one package to become an editor, parser, judge, and compliance system at once.

Job	Candidate families	Evidence a candidate can provide	Evidence it cannot provide alone
Source representation	pylatexenc, plasTeX, LaTeXML, tree-sitter-LT, Pandoc	Locations, syntax trees, conversion behavior on fixtures	Semantic correctness of an equation or reader comprehension
Build and cross-reference	TeX, latexdiff, LaTeX tooling	Reproducible PDF, undefined-reference and visual-diff observations	Preservation of an author's intended claim
Prose findings	Vale, textlint, LanguageTool, proselint, Textidote	Rule hits, locations, configurable style checks	Priority, genre fit, or a useful repair
AI-tell diagnostics	sloptrim, slop-forensics, fast-detect-gpt	Aggregate distributional signals under a declared corpus	Individual authorship or quality
Private inference	llama.cpp and compatible local models	A local execution path and resource profile	Truth, citation support, or reader acceptance
Evaluation	Inspect AI plus a human protocol	Replayable tasks and judge adapters	The named expert's final decision

Table D.1: The package field supplies components, not a finished product.

D.2 Representing and building technical documents

D.2.1 Lightweight and full-model LaTeX parsers

pylatexenc offers a relatively small Python-level route for tokenizing and interpreting LaTeX fragments [pylatexenc maintainers, 2026]. Its attraction is proportionate scope: a first adapter can recover command spans, groups, and source offsets without committing the product to a full document model. The failure case is equally clear. A token stream does not know whether a macro represents a theorem, a number in a table, or an author's private shorthand. The adapter must retain the raw span and abstain when a macro cannot be interpreted.

plasTeX builds a richer object model and can expose sectioning, counters, and cross-references [plasTeX maintainers, 2026]. It is a stronger candidate when the product needs to reason about a document's visible hierarchy. It also brings a larger compatibility surface: custom packages, macro definitions, and environment side effects can make the object tree differ from the PDF a local TeX installation produces. The benchmark must compare both the parsed model and the compiled output.

LaTeXML translates TeX into XML and HTML-oriented structures and is useful when the same source must support multiple renderings [National Institute of Standards and Technology and LaTeXML maintainers, 2026]. Its strength is explicit semantic markup for many mathematical constructs. Its boundary is practical: a translation that looks

correct in HTML can still alter spacing, numbering, or a custom macro in the PDF used by the reader. LaTeXML therefore belongs in an adapter experiment, not as an unexamined replacement for the project's build.

unified-latex adds a serious JavaScript/TypeScript candidate. Its parser and unified-style transformation utilities make it attractive for an editor or web interface that needs source positions and typed nodes [unified-latex maintainers, 2026]. TexSoup sits at the opposite end of the design space: its tolerant Python interface makes navigation and small transformations easy, including on imperfect input [TexSoup maintainers, 2026]. Tolerance is useful for an editor and dangerous for a safety claim. Both candidates must expose unknown constructs and preserve raw spans; neither may convert a best-effort parse into a false "protected" result.

D.2.2 tree-sitter, Pandoc, and latexdiff

tree-sitter's incremental parsing model is attractive for an editor that must show a finding while a source file is being changed [latex-lsp maintainers, 2026]. The current grammar is a structural aid, not a semantic LaTeX proof. A malformed environment or an unsupported macro should produce an explicit "unparsed" record, not a best-effort protected-object comparison.

Pandoc supplies mature conversion and filter hooks across document formats [Pandoc maintainers, 2026]. It can help create a detexed prose view for aggregate diagnostics and a plain-text reader packet. It cannot be the source of truth for equation preservation when the canonical source is LaTeX. The source-to-PDF build remains authoritative; Pandoc output is a secondary view with its own hash and conversion report.

latexdiff is valuable precisely because it is narrow. It highlights visible source changes and helps a reader inspect a revision, but its markup is not a semantic correspondence algorithm [latexdiff maintainers, 2026]. Humanvoice should use it as a view over the protected comparison, never as the comparison itself. A number can move from a macro definition to a table, and an equation can remain algebraically equivalent while its text changes; character markup cannot resolve either case by itself.

The canonical build should also use the standard LaTeX toolchain rather than hide it behind a bespoke command. latexmk can orchestrate repeat compilation and bibliography dependencies [Collins, 2026]; ChkTeX can add source-level warnings [ChkTeX maintainers, 2026]. These tools establish build and syntax facts. They do not inspect rendered pages, compare protected claims, or decide whether a reader understands the result, so they remain components behind the Humanvoice build record rather than substitutes for it.

a parser failure that must remain visible

The fixture defines a custom command `riskterm` that expands to a number in one table and to a prose label in another. A parser that records only the expanded text can report two unrelated values; a parser that preserves the macro call and its expansion can ask the author to confirm the correspondence. The second behavior is slower and safer. The first is an attractive false pass.

D.3 Prose and citation diagnostics

Vale is a rule-based prose linter with configurable styles and targeted text segments [Vale maintainers, 2026]. It is a good host for explicit project rules: private path leakage, a banned internal phase label, or a required definition of an acronym. The rule should report the passage and a reason. A rule that rewrites the sentence or treats a hit as an error in every genre would violate the product boundary.

textlint offers a pluggable natural-language linting model and machine-readable formatters [textlint maintainers, 2026]. It is useful for composing independent checks and for feeding findings into an editor. Its Node ecosystem becomes a maintenance consideration in a mostly Python and TeX repository; the adapter should consume stable JSON rather than embed textlint's internal objects.

LanguageTool and LTeX+ provide grammar and language checks across ordinary prose and, in the latter case, LaTeX-aware editing workflows [LanguageTool maintainers, 2026, LTeX+ maintainers, 2026]. Their value is local correctness: agreement, spelling, punctuation, and a limited class of usage problems. They should be run after the argument and protected objects are stable. Otherwise the author receives a large queue of low-consequence corrections before seeing the missing mechanism.

proselint and Textidote occupy a similar but useful niche. They encode style and usage advice, including vague language and passive constructions, and can serve as additional rule sources. The system should retain the source rule and version in each finding because a recommendation is not a fact about the document. A former professor may deliberately choose a passive construction to foreground a result, and a methods section may need a different register from an executive opening.

When a sentence carries a citation, two different checks follow. First, a resolver checks whether the key, identifier, title, and author record agree. Second, a human or a checked source-support process asks whether the cited work supports the sentence. GROBID can help recover bibliographic fields from PDFs, but the current inventory treats it as a candidate pending local deployment and fixture results [GROBID maintainers, 2026]. No resolver should be named a "citation validator" if it checks identity only.

D.4 AI-tell tools and the temptation to optimize the surface

sloptrim and the related slop-forensics project are useful because they turn a maintained pattern list into inspectable counts and spans. The repository audit found a calibration in which a mechanically clean rejected draft scored better than a document a human reader accepted. That result is not a failure of the tools; it is a boundary observation. They can direct an author's attention to repeated vocabulary, formulaic transitions, and sentence shapes. They cannot decide whether the document has a mechanism or a persuasive reason to fund it.

fast-detect-gpt implements a probability-curvature family of AI detectors. Its official

repository and benchmark literature make it a useful reference for aggregate drift experiments, but the false-positive and evasion results in Chapter 7 rule out per-document use. Any adapter must discard individual labels and retain only corpus-level distributions, with a minimum cell size and a human interpretation of a change.

The same rule applies to the installed humanizer guidance. It supplies useful patterns, such as inflated significance, vague sources, repeated headings, and formulaic outlook sections. It is not a substitute for a writer's ear or for a technical explanation. The main route therefore presents these projects as diagnostic components; this appendix retains their operational details.

D.5 Adjacent products an executive will compare

An implementation proposal should not define its market only by the packages its authors have already installed. A sponsor will reasonably ask why the team cannot buy an existing writing assistant, use the review functions in its LaTeX editor, or add a specialist academic service to the current workflow. Use a deliberately modest standard to answer that question. A vendor's documentation establishes a documented capability and a stated trust boundary. It does not establish effectiveness on the Humanvoice population, and a product page is not a substitute for a held-out comparison.

D.5.1 General communication assistants

Grammarly's current enterprise offering is broad: it works across the tools in which teams write and combines writing suggestions with a more general agent layer [Grammarly, 2026a]. Its privacy documentation describes how generative features may pass prompt text and context to service providers, even while enterprise controls and consent are offered [Grammarly, 2026b]. This is a credible alternative for ordinary business communication. It can reduce the cost of local wording edits and may be the correct incumbent for a short memorandum.

The documented strength is also the boundary. The assistant's unit is a selected passage or a communication context. Humanvoice's unit is a versioned technical document with equations, data, citations, claims, and a named decision. A rewrite that is excellent at tone can still change a denominator, drop a qualification, or make a claim sound more certain. The product wedge therefore does not compete on generic fluency. It asks whether an assistant can be placed behind a source map and a protected comparison, with the reader outcome measured rather than inferred from acceptance of a suggestion.

D.5.2 Academic writing assistants

Writefull is closer to the target genre. Its Overleaf integration advertises language support trained on published research and includes revision, style, paraphrase, and other writing functions inside the LaTeX editor [Writefull and Overleaf, 2026]. That makes it a serious candidate for a language-diagnostic adapter or an incumbent arm. It does not,

on the documentation inspected, provide the complete Humanvoice record: a declared reading brief, object-level correspondence for equations and claims, and a reader decision that can be compared across workflows. Those absences are questions for a fixture run, not accusations about the product.

Paperpal presents a broader academic suite, including contextual rewriting, reference discovery, citation styles, submission checks, and integrations for researchers [Paperpal, 2026]. Its breadth is useful evidence that the market already pays for language and submission support. It also creates a measurement problem. Several checks and generative functions arrive in one interface, so a positive user reaction cannot identify which component changed the document or whether a protected object moved. Paperpal's stated privacy position should be recorded in procurement, but the product still needs a local or redacted path for restricted manuscripts.

Trinka is another relevant comparison because it positions itself for academic and technical writing and documents privacy, institutional, integration, and API options [Trinka AI, 2026]. This may make it a plausible private deployment or grammar adapter. The same discipline applies: a privacy feature is not a proof of source correspondence, and an API is not a reader study. The benchmark should test whether a Trinka finding can be tied to a source span, whether an author can reject it without losing provenance, and whether its processing mode meets the document owner's declared boundary.

D.5.3 Authoring and review environments

Overleaf already supplies several functions a new product should not reimplement. Its documentation describes collaborative editing, comments, version history, and Track Changes for accepting or rejecting collaborators' edits [Overleaf, 2026]. These features solve a real social problem: multiple authors can inspect a manuscript and discuss a proposed change in context. Humanvoice should integrate with that workflow where it is the chosen incumbent, export a stable source snapshot, and return a reader packet that does not expose private review history.

Track Changes remains a change display, not a semantic invariant. It tells a reviewer which source text was edited and who made the edit; it does not by itself establish that an equation is equivalent, a citation still supports a claim, or a reader can make the intended decision. A product that treated Overleaf's accept/reject control as proof of meaning preservation would repeat the same category error as treating a grammar score as proof of comprehension.

D.5.4 The comparison in one view

The following table records the buy, adapt, or build question at the level of the job rather than the brand. "Documented" means that the vendor describes the capability. "Unestablished" means that the Humanvoice benchmark still needs to observe it on a locked fixture and a named reader.

Alternative	Documented strength	Missing link for this proposal	Initial disposition
Grammarly / enterprise assistant	Broad inline suggestions, rewriting, and organizational controls across common applications	No demonstrated equation–claim lineage or named-reader endpoint in the inspected material	Retain as an incumbent comparator; do not rebuild its general grammar layer
Writefull for Overleaf	Research-trained language support integrated with a LaTeX editor	Source-level protected-object record and reader-outcome measurement remain unestablished	Benchmark as a language adapter and possible incumbent
Paperpal	Academic editing, references, submission checks, and multiple integrations	One interface combines many interventions; causal contribution and protected correspondence are not identified	Include in market scan; test only declared functions
Trinka	Academic/technical language assistance with privacy and integration options	Efficacy, source maps, and reader acceptance for the target genres remain unestablished	Evaluate as a private-service or API candidate
Overleaf review layer	Comments, collaboration, version history, and accept/reject Track Changes	Character/source changes are not an invariant for equations, claims, or citations	Integrate as the authoring incumbent
Humanvoice core	Located findings, protected comparison, and a reader decision by someone unfamiliar with the project in one record	No efficacy result and no broad market proof yet	Build only the connective layer, then test its value

Table D.2: Adjacent products define the incumbent bundle and the falsifiable Humanvoice wedge.

D.5.5 Buy, adapt, or build

This comparison leads to a restrained implementation choice. Buy or reuse general grammar, collaborative editing, and document-building capabilities when their licensing and privacy terms fit the organization. Adapt a package when it can emit a stable location, version, and explanation without taking ownership of the document. Build the narrow connective layer that the current alternatives leave implicit: a source snapshot, a protected-object schema, a finding record, an author decision, and a reader outcome joined by immutable identifiers.

That distinction affects the budget. Reimplementing a mature grammar engine would

spend the MVP budget on a capability that is already available. Reusing a cloud editor without an object-level boundary would save engineering time while moving the largest risk into an opaque service. The proposed pilot should price three configurations: a fully local stack; an incumbent-editor stack with Humanvoice adapters; and a hosted academic assistant used only on explicitly authorized excerpts. The measured reader and operator burden, not a feature checklist, chooses among them.

D.5.6 Tests that could overturn the position

The product should be abandoned or repositioned if an adjacent product can already produce the same source-to-reader record at lower total burden. The test is concrete. Give each candidate the same source snapshot, protected object manifest, and reading brief. Ask it to locate a deliberate qualification loss, compare a changed number and citation, and produce a reader packet that hides private workflow labels. Record abstentions, false passes, operator minutes, network calls, and reader completion. A candidate that wins this comparison should become the incumbent, even if it is not open-source and even if it makes the Humanvoice implementation unnecessary.

Conversely, a candidate that wins a fluency preference but cannot explain a protected change has not falsified the wedge; it has supplied a useful component. This is why the software survey reports capability and evidence in separate columns. The investment buys an option to learn whether the connection among those components earns its cost, not a license to describe a familiar product as a competitor it has not actually beaten.

D.6 Local inference and reader evaluation

llama.cpp is the leading local execution candidate in the inventory [llama.cpp maintainers, 2026]. Its value is a deployment option when a document cannot leave the organization's boundary. The product still needs a model card, a prompt and sampling record, a resource profile, and a comparison against a remote baseline. Local execution reduces one class of privacy exposure; it does not make a generated suggestion correct or remove the need for disclosure.

Inspect AI supplies an evaluation harness for tasks, solvers, and graders [UK AI Security Institute, 2026]. It can replay a fixed reader question, randomize version order, and store machine-judge outputs beside human labels. It cannot recruit the former professor whose decision defines the product. The first release should keep it optional for model experiments.

The timed human task needs a simpler instrument. jsPsych can present local web trials, randomize order, record responses and elapsed time, and export a structured trial record [jsPsych maintainers, 2026]. A self-hosted jsPsych page is the leading candidate for the feasibility reader packet because it does not make model evaluation infrastructure a dependency of the human outcome. The benchmark must still test keyboard accessibility, resume behavior, browser logging, and export correctness; the library supplies instrumentation, not an expert-reader sample.

Candidate	First fixture	Disposition	Reason for the boundary
pylatexenc	Custom macro plus equation and table	adapt	Preserve raw spans; abstain on unknown macros
unified-latex	Source positions, custom environments, and editor round trip	pilot	Strong web-editor fit; unknown-construct behavior remains untested
TexSoup	Malformed input and compact Python traversal	pilot	Useful tolerant fallback only if ambiguity remains explicit
plasTeX	Sections, labels, counters, and malformed environment	pilot	Richer tree, but compatibility must match the canonical build
latexdiff	Number, citation, and equation moved across paragraphs	adapt	Useful view, not semantic truth
Vale/textlint	Private label, vague referent, intentional technical term	adopt as adapters	Findings need rule and context, not automatic rewrites
sloptrim	Accepted and rejected internal repair pair	pilot aggregate-only	Calibrate against reader labels; no individual authorship score
llama.cpp	Same finding prompt under local and remote models	pilot	Measure privacy, latency, and suggestion differences
Inspect AI	Order-swapped pairwise reader task	adapt	Organize replay; human decision remains primary
jsPsych	Timed, randomized reconstruction task in a local browser	pilot	Instrument human sessions without coupling them to a model harness

Table D.3: Initial software dispositions are hypotheses to be tested on a locked fixture set.

D.7 The held-out fixture set

The fixture set is deliberately small but varied. It contains a clean passage, a passage with a private identifier, an equation whose exponent changes, a table with a denominator mismatch, a citation with a valid identity but weak support, a custom macro, a malformed environment, and a paragraph whose repeated structure is intentional. Each fixture has

a source file, a rendered reference, a protected-object manifest, and an expected failure behavior.

For candidate k and fixture i, record a capability result C_{ki}, a false-pass indicator S_{ki}, runtime t_{ki}, and operator burden b_{ki}. The benchmark summary is a matrix rather than a single package score:

$$B_k = \{(C_{ki}, S_{ki}, t_{ki}, b_{ki}) : i = 1, \ldots, I\}. \tag{D.1}$$

An adapter may be adopted only when it identifies the positive fixture, fails or abstains on the negative fixture, and emits a location that a second operator can replay. A fast false pass is worse than a slow abstention because the former can change a scientific document without attracting attention.

The benchmark also records installation and maintenance facts. A package with an incompatible license can remain a reference implementation. A package with no recent release can be acceptable for a frozen migration and unsuitable for a continuously operated service. A package that requires network access during inference may be excluded from a private document even if its output is useful. These are engineering facts, not aesthetic preferences, and they belong beside the functional result.

D.8 Composition and ownership

The proposed stack has one owner for each boundary. The source adapter owns locations and abstentions. The build wrapper owns the canonical PDF and cross-reference diagnostics. The protected-object extractor owns normalized equations, numbers, citations, and tables. The finding adapters own rule identities and context. The local-model adapter owns prompts, sampling, and network policy. The reader protocol owns the final outcome.

This division prevents a package from silently acquiring authority that its evidence does not support. It also makes replacement possible. If a better LaTeX parser appears, the source adapter can be replaced while the protected object schema and reader protocol remain stable. If a model judge is retired, the human labels and pairwise task remain. The product is the composition and the evidence around it, not the brand of any one dependency.

The current inventory is therefore enough to begin a feasibility build, but not enough to claim a validated software stack. The held-out benchmark, license review, and independent replay remain release conditions. Part III now turns these dispositions into an architecture whose records can be implemented and tested.

CHAPTER E

Reference manual for records and fixtures

The preceding appendix explains why the records exist. This one makes them usable. An engineer should be able to build a first fixture from these pages; a researcher should be able to trace a load-bearing sentence to its source; and a sponsor should be able to inspect a decision packet without opening a repository log. The manual is intentionally concrete. It belongs after the argument because its tables and field names are tools for reproduction, not the story the reader must learn first.

E.1 Document and version fields

A document record begins with the work, not with a model run. The minimum fields are shown below.

Field	Type	Interpretation
document_id	stable string	Identifies the work across revisions and renderings
genre	controlled text plus free description	Research proposal, model note, memorandum, survey, or another declared class
reader_role	free text	The expertise and decision responsibility of the intended reader
decision_question	prose	The action or judgment the document is meant to support
exemplars	ordered list of source ids	Documents that define the intended explanatory pace for this reading task
known_vocabulary	list of terms	Terms the reader may reasonably know without a first-use gloss
exemption_zones	line ranges plus reason	Preview, map, decision-box, or reference regions where names may precede teaching
brief_version	integer plus date	Cumulative reader directives used by this inspection
source_root	path under the authorized boundary	Directory whose contents are included in the snapshot
parent_version	nullable identifier	Previous source snapshot, if the document is being revised
assistance_disclosure	structured record	Tools, model use, human editing, and unresolved disclosure choices
privacy_class	controlled text	Public, internal, confidential, or restricted

Table E.1: Fields that define the document before diagnostics run.

The free description beside a controlled genre is important. Two documents may both be called a proposal while one asks for a research grant and the other asks for authorization to change a production model. The reader task, not the label, determines which evidence and wording are appropriate. A missing decision question is an intake failure, not an invitation to let a model infer one.

A version record adds build information and source lineage:

Field	Type	Interpretation
version_id	immutable string	One source and configuration snapshot
entry_point	path	Top-level TeX, Markdown, or other source file
route_files	ordered list	Included files and their content hashes
build_id	string	Compiler and dependency environment
source_sha256	hexadecimal string	Exact input identity
rendered_sha256	hexadecimal string	Exact PDF identity
protected_manifest	path or embedded object	Objects extracted for comparison
build_result	pass, fail, or incomplete	Typography and reproducibility result, not a reader judgment

Table E.2: A version record ties source, build, and rendered output together.

The distinction between a source hash and a rendered hash matters when TeX changes a page break without changing a sentence, or when a source edit is present but the PDF was not rebuilt. Both conditions should be visible in a comparison packet.

E.2 Protected-object dictionary

Each protected object has a typed identity. The following fields are sufficient for the first release:

Object type	Required fields	Normalization rule
Equation	source span, environment, label, variables, units, rendered anchor	Normalize whitespace and macro aliases; retain signs, exponents, limits, and unknown commands
Number	value, unit, precision, context, source span	Normalize separators and formatting; do not round or drop a sign
Citation	key, authors, title, identifier, sentence span	Normalize key and identifier; preserve sentence and bibliography versions
Table	caption, row/column labels, cells, units, source span	Normalize cell formatting; retain denominator, ordering, and missing status
Qualification	text span, associated finding or result, scope terms	Preserve scope words and link to the result they qualify
Label or cross-reference	key, target, source span, rendered page	Compare key, target, and resolved page independently
Quotation or code	delimiters, source, language, text hash	Preserve content exactly unless an author-approved replacement is recorded

Table E.3: Typed protected objects keep normalization from becoming silent rewriting.

A normalization function is a convenience for matching, not a replacement for the source.

Let g_k be the function for object kind k. The comparison stores both $raw(p)$ and $g_k(raw(p))$:

$$record(p) = \Big(kind(p), raw(p), g_{kind(p)}(raw(p)), loc(p), owner(p)\Big). \tag{E.1}$$

When a later implementation changes g_k, it must replay old fixtures and retain the prior normalized value. Otherwise a dependency update can make a historical change appear to have been unchanged.

A qualification deserves special treatment. It is not merely a sentence with words such as "may" or "under." The record links it to the proposition whose scope it limits. If the proposition moves, the system asks the author to confirm the link. This catches the common repair in which a caveat survives as a grammatical sentence but no longer qualifies the result it was written to bound.

E.3 The fixture catalogue

The first benchmark corpus should contain small fixtures that isolate one failure and a few composite documents that reproduce the reader problem. Each fixture has a positive behavior, a negative behavior, and an abstention rule.

Macro fixture A custom command expands to a number in one table and a label in another. A parser must preserve the call and expansion or abstain.

Equation fixture The child changes an exponent and then changes only whitespace. The first difference must be substantive; the second may be formatting under the declared normalization.

Table fixture A percentage is recomputed with a different denominator. The system must identify the denominator and show the before-and-after values.

Citation fixture A key resolves to a real paper whose method does not support the sentence. Identity resolution passes; support review remains open.

Qualification fixture A sentence moves a population or horizon qualification away from the result it limits. The object comparison must keep the association visible.

Register fixture A private phase label and repository path occur in the opening paragraph. The finding should point to the public decision the reader needs, not merely mark a forbidden word.

Intentional-pattern fixture Three adjacent sentences have the same form because they enumerate the steps of a derivation. A style rule should either abstain or give the author a reason to retain the pattern.

Pacing fixture Three prose paragraphs introduce a known set of terms at different rates. The meter must report the burst locations and compare the target with supplied exemplars without declaring a universal pass value.

First-use fixture
An acronym appears before its expansion, followed by the same acronym after a generic gloss. The audit flags the first occurrence, accepts the second, and respects a declared preview exemption.

Assertion fixture
A prose repair adds a number or superlative without a citation. The comparison opens a question and never silently accepts the new claim.

Reader fixture Two versions have identical equations but different openings. A reader unfamiliar with the project should be able to identify which version makes the decision and mechanism clearer.

The composite corpus then combines these cases in realistic documents. It should include at least one proposal, one mathematical note, and one investment-style memorandum, with a source snapshot and a human-authored answer key for each. The answer key records what a competent reader should be able to recover, not which recommendation the reader must endorse.

For fixture i, the expected result is a partial function

$$E_i : input_i \longrightarrow \{\text{pass}, \text{fail}, \text{abstain}\}. \tag{E.2}$$

A candidate that returns "pass" on an unresolved negative fixture has a false pass even if its aggregate accuracy is high. The benchmark report lists every fixture, including skipped or unsupported cases, so the denominator cannot quietly improve when difficult documents are removed.

E.4 Finding record and author decision

A finding is useful when another person can understand it without opening the tool's implementation. The recommended fields are:

Field	Example	Purpose
finding_id	F-014	Stable reference in a revision packet
location	chapter.tex:87, PDF page 6	Lets the author inspect source and rendered context
observation	Private label precedes public definition	States what is visible
consequence	New reader may not recover the comparison	Explains why attention might be worthwhile
question	Name the baseline and changed mechanism	Gives the author a usable next move
suggestion	Optional text or none	Offers an alternative without authority
priority	provisional ordinal	Orders attention; never certifies quality
author_decision	accept, reject, defer, rewrite	Preserves human ownership
decision_reason	short prose	Explains a rejection or substantive change
unit	chapter or section id	Keeps pacing and ordering work bounded and replayable
repair_move	concrete-first, unfold, read-back, generic-first-name-after, or other named move	Gives the author a finite option without prescribing wording
detector_evidence	metric, exemplar ids, or first-use signal	Shows why the question was raised and what the heuristic cannot establish

Table E.4: A finding record is an editorial question with provenance.

The author decision is intentionally richer than a checkbox. "Reject" may mean that the pattern is intentional, that the proposed consequence is wrong, or that the author will address it in a different paragraph. The reason lets a later reviewer distinguish a false positive from an unfinished repair. It also gives the diagnostic designer evidence about which rules waste time.

A model suggestion, when present, is stored as a child of the finding and never as a replacement for the author's text. The system records whether the author accepted the wording, adapted it, or wrote an independent repair. This distinction matters for disclosure and for a later analysis of which assistance actually reduced work.

E.5 Reader form and coding manual

The reader form should fit on one page plus an optional comment sheet. It opens with the decision context and then asks:

1. In one sentence, what decision is the document about?
2. What is the proposed mechanism or object?
3. Which result, equation, or table carries the main support?

4. What condition limits the result?
5. Would you approve the requested next step now? Choose yes, revise, or no, and give one reason.
6. Where did you first stop or reread? Give the page or section, a short quotation, and the relation you were trying to recover.

The coding manual defines an answer as correct when it identifies the document-specific target within the declared scope, even if the reader uses different words. It defines an answer as incomplete when it names a topic but not the action or mechanism, and as incorrect when it assigns the result to a different population, comparison, or horizon. These categories are preferable to asking a model to judge whether the response is eloquent.

For reader j, version v, and question q, let Z_{jvq} take values 1 (correct), 0 (incomplete), and -1 (incorrect). The reconstruction index is reported as a vector of question-level proportions:

$$\rho_{vq} = \frac{1}{J}\sum_{j=1}^{J} 1\{Z_{jvq} = 1\}, \qquad q = 1, \ldots, 5. \tag{E.3}$$

The vector is more informative than its average. A version can improve the action answer while leaving the mechanism answer unchanged, which tells the author where a further repair is needed.

The form records confidence and prior exposure separately. Confidence is a reader observation, not a multiplier on correctness. Prior exposure is a potential source of contamination, not a reason to discard a skeptical answer. The evaluator reports both so that a sponsor can see whether the product works for a new reader or only for someone already familiar with the project.

E.6 Evidence and omission register

A load-bearing literature row has six linked parts: the source identity, the technical passage inspected, the source result, its population and task, the limit, and the local product implication. The record also states what the source is not allowed to support. For example, an experiment on email completion can support a bounded claim about task time; it cannot support a claim about acceptance of an economics proposal.

The omission register uses the following fields:

Field	Example	Use
candidate	specialist technical-communication study	Names a plausible missing source or stream
reason considered	Direct reader-comprehension relevance	Explains reviewer risk
current status	source blocked, not screened, or adjacent	Prevents silence from looking like exclusion
permitted role	context, direct support, challenge, or none	Limits inference
next action	obtain full text, independent screen, or omit with reason	Gives the review a concrete next step
owner and date	named reviewer and deadline	Makes the omission actionable

Table E.5: A scholarly omission is recorded as a question with a next action.

The current evidence run has explicit release blockers for independent screening, full-text anchors, appraisal, held-out package effectiveness, and external review. Specialist coverage and DOI identity have passed for the bounded records recorded in the audit, but those passes do not invalidate the remaining source-to-claim work or make the survey exhaustive. A later audit can close a blocker or narrow a sentence; the main route should not silently change its claim class in response.

E.7 Cost worksheet

The cost worksheet separates observed inputs from planning assumptions. For document class g, record annual volume N_g, incumbent reader hours T_{0g}, proposed reader hours T_{1g}, author hours A_{0g} and A_{1g}, reader rate c_r, author rate c_a, and recurring operating cost C_o. The direct annual value range is

$$V_g = N_g \left[(T_{0g} - T_{1g})c_r + (A_{0g} - A_{1g})c_a\right] - C_o. \tag{E.4}$$

The worksheet must include a separate line for protected-object incidents. If the sponsor cannot price an incident, it should report a low, medium, and high scenario rather than assigning a zero cost.

A worked sensitivity grid varies reader saving and annual volume while holding the loaded reader cost fixed. The grid is not a forecast. It tells the evaluator which baseline observations have the largest value of information. If the break-even point requires an implausibly large volume, the beachhead is wrong even if the product is technically interesting.

The worksheet also records labor that a software demo hides: intake, source cleanup, finding investigation, citation review, reader recruitment, and fixture maintenance. A narrow service with high per-document value may tolerate more manual work than a

broad subscription. The operating plan should price the actual path observed in the pilot, not an imagined fully automated path.

E.8 Decision packet and reproducibility

The final packet for a sponsor contains, in order:

1. the one-sentence decision and the population of documents in scope;
2. the incumbent and proposed workflows, including assistance disclosure;
3. the source and rendered hashes for every compared version;
4. the protected-object comparison and all unresolved items;
5. the reader outcomes, author burden, and missing observations;
6. the evidence status and the sources allowed to support each conclusion;
7. the cost worksheet with sensitivity and non-priced outcomes; and
8. a proceed, revise, or stop recommendation with its next owner.

A packet is reproducible when an independent operator can rebuild the source, replay the fixtures, locate the protected objects, and recover the same numerical summaries from the recorded input files. It need not reproduce a model's stochastic suggestion bit for bit if the model configuration and output hash are preserved; it must distinguish a stochastic variation from a source or reader outcome.

The canonical Humanvoice build currently records the top-level source hash, route hash, rendered hash, page count, extracted word count, and the evidence-status file. Those fields identify this draft. Release still requires the project owner and an independent reader, because reproducibility is evidence about the build and not an endorsement of the prose.

E.9 Compact glossary

Protected object A document element whose change can alter scientific, financial, or legal meaning and therefore receives explicit comparison.

Reading brief The declared decision, audience, context, and time conditions for a reading task.

Finding A located observation and question that directs human attention.

Abstention An explicit statement that the current parser, rule, or model cannot establish correspondence or support.

Incumbent bundle
: The actual tools and human work used when Humanvoice is not present.

Feasibility evidence
: Evidence that the path can run and be measured on a declared case; it is not an efficacy estimate.

The glossary is intentionally short. Domain nouns should be defined where they enter the argument, and technical field names should remain in this appendix when they do not help a reader decide whether to fund the next step.

CHAPTER F

Implementation contract

This annex is normative for the first build. The reader-facing chapters explain why the product exists and what a person should experience. This annex states the boundaries an implementation may not invent away. The machine-readable companion is `schemas/implementation_contract.json`; the record schemas are in `schemas/`. If this annex and an implementation disagree, the implementation stops until the contract is deliberately changed and the locked fixtures are replayed.

The canonical R1–R24 acceptance register is Table B.4 in Appendix B.2. This annex supplies the execution contract for that register; it does not create a second list of requirements.

F.1 Scope, precedence, and the week-six decision

The first release has a protected-source core and an optional authoring extension. The core is not a consolation prize. It is the first investment's test of whether Humanvoice can preserve a technical document while making a reviewer's work more local and more reversible. The extension earns its place only after that test passes.

Stage	Weeks	Required output	Checkpoint
Protected core	1–6	Authoring brief, source snapshot, canonical LaTeX build, source map, protected-object comparison, versioned records, and safe command results.	A second operator replays the core, or the project narrows to this review utility.
Authoring extension	7–9, conditional	Blueprint, bounded unit writer, pre-human critics, mutation harness, bounded repair, and fail-closed release.	The exact runtime and model pass calibration and held-out replay; otherwise model-dependent gates remain abstentions.
Feasibility case	10–12	One released packet, timed reader task, burden account, rights record, and proceed/revise/stop memo.	The sponsor decides whether a comparative study is worth funding.

Table F.1: The protected core is the week-six go/no-go decision. The full path is conditional, so an ambitious model extension cannot silently consume the whole feasibility horizon.

When controls conflict, apply them in this order: correctness, source fidelity, domain meaning, reader comprehension, then template regularity. A safety or privacy control overrides all five. Project and format exceptions belong in a configuration record, never in an unexplained manuscript instruction.

F.2 Threat model and trust boundary

Humanvoice handles data that may be both executable and confidential. The following inputs are untrusted: the authoring brief and evidence files; the TeX source tree, bibliography, images, custom packages, and auxiliary files; fixture documents and corpus imports; model prompts and model output; and reader-supplied answers or uploaded artifacts. Text that looks like an instruction inside any of those inputs is data. It has no authority to invoke a tool, disclose a secret, change a policy, or widen a claim.

The default trust boundary is a local isolated workspace. A run receives a read-only source snapshot and writes to a separate scratch directory. The canonical source is never compiled in place, and no output from a parser or model is executed. Remote access is denied unless a separate policy record names the fields, provider, purpose, retention, and approving owner.

Control	Rule	Observable test
T1	Compile from a read-only snapshot into an allow-listed scratch directory.	The source tree is byte-identical after every build and no path outside the allowlist is written.
T2	Invoke TeX with `-no-shell-escape`; reject `\write18`, source executables, and model-generated commands.	A shell-command fixture cannot create a file or alter the run result.
T3	Deny compiler and parser network access. A remote model or metadata lookup crosses an explicit approval point.	A network-deny fixture blocks an unapproved call and records exit code 5.
T4	Send critics only delimited, schema-validated JSON. Treat source text and model output as data, not instructions.	An injection fixture yields a located finding or abstention and cannot acquire a tool or secret.
T5	Enforce time, memory, file-size, process-count, and output-size limits.	An exhausted limit aborts the run with its limit and source hash; it never falls through to a pass.

Table F.2: The threat model turns local-first and prompt-injection language into checks a build team can run.

The build wrapper must use `-halt-on-error` and `-file-line-error` in addition to `-no-shell-escape`. A successful PDF is not evidence that the source was safe: the trust-boundary checks and the protected comparison are separate release conditions. A security or policy owner may veto a run and may not be overruled by a model, an editor, or an engineer.

F.3 Runtime and model manifest

The deterministic path is the reference implementation. It consists of the parser adapter, canonical TeX build, protected comparison, citation-identity checks, and source-map checks. The proposed local inference runtime is `llama.cpp`; model quality is a separate question from runtime availability.

The following are planning candidates, not an unearned adoption claim:

Role	Candidate	Condition before it may enter a release gate
Runtime	`llama.cpp`, exact commit recorded	Rebuild the runtime and record compiler, hardware, and license information.
Reference model	Qwen3–8B–Instruct, GGUF Q4_K_M, exact file hash	Run the frozen calibration and held-out sets with the exact artifact, tokenizer, prompt template, and sampling settings.
Fallback model	Llama 3.1–8B–Instruct, GGUF Q4_K_M, exact file hash	Use only as a separately calibrated alternative; never silently substitute it for the reference model.
No model	Deterministic protected core	May continue when inference is unavailable, but model-dependent writing and reconstruction gates remain abstentions and cannot produce a full-path release.

Table F.3: Model candidates are pinned experimental inputs, not a claim that a particular model is already suitable.

Every inference run records the runtime revision, model file hash, tokenizer hash, prompt-template hash, sampling parameters, seed, hardware, network state, latency, memory, and output hash. A change to any of those invalidates the affected calibration and held-out results. The fixtures must be replayed before the changed component can return a release-ready finding. A model may suggest a repair; it may not accept the document.

F.4 Operating envelope

The numbers below are planning caps for the MVP, not measured performance. They make the parser, scratch space, and inference budget designable. Week six records actual distributions and either revises the caps openly or narrows the supported document class.

Quantity	MVP cap	Interpretation
Source tree	250 MiB	Includes bibliography, images, packages, and auxiliary files in the snapshot.
Rendered document	300 pages	A larger document may use the protected core after an explicit scope decision; it is not silently truncated.
Prose	120,000 words	The cap controls extraction and critic work, not the author's final length.
Protected objects	2,000	Equations, numbers, citations, tables, quotations, code blocks, and declared claims.
Deterministic preflight	30 minutes	On the reference machine: eight CPU cores and 32 GiB RAM.
Model-assisted preflight	90 minutes	Includes bounded critic calls and reconstruction; timeout is an abstention.
Repair	3 cycles per unit	Repeated signatures or alternating parent hashes stop the loop.
Inference	2 GPU-hours or 10 CPU-hours per document	Whichever limit is reached first; actual usage enters the cost account.

Table F.4: Initial operating limits. They are scope controls and cost inputs, not promises of speed.

The model receives at most 12,000 input tokens and returns at most 2,000 tokens for one bounded unit. A whole-document model context is outside the first release. The document-level model limits in the repository contract exist only for an explicitly approved assembly or reconstruction task; they do not permit an unbounded autonomous rewrite.

F.5 Record and schema policy

Every record carries `record_type`, `schema_version`, `record_id`, `run_id`, and `created_at`. The first release schemas live in `schemas/authoring-brief.schema.json`, `schemas/argument-blueprint.schema.json`, `schemas/finding.schema.json`, `schemas/revision.schema.json`, `schemas/preflight-run.schema.json`, `schemas/release-decision.schema.json`, `schemas/runtime-manifest.schema.json`, and `schemas/cli-result.schema.json`. The rights record is `schemas/corpus-item.schema.json`.

The compatibility rule is simple. A minor schema version may add optional fields; readers preserve unknown extension fields. A major version requires a migration, a changed contract identifier, and replay of the locked fixtures. The same source, schema, toolchain, configuration, and runtime manifest must reproduce deterministic finding

identifiers. Stochastic outputs retain their seed and hashes but are never described as bitwise deterministic.

Unknown fields at the top level are rejected. Additions that do not change the meaning of an existing record belong under an explicit `extensions` object and are preserved by readers. The runtime manifest records the compiler, runtime revision, model and tokenizer hashes when applicable, prompt-template hash, sampling and seed, hardware, network policy, resource limits, latency, peak memory, output hash, and an explicit reason for fields that do not apply. CLI results always include the nine stable output fields named below, and corpus rights records include `consent_date`, including a null value when no consent date is applicable.

The authoring brief also requires the intended reader, decision, genre, known vocabulary, named exemplars, evidence boundary, privacy class, protected objects, and explicit unknowns. A blueprint requires evidence anchors, terminology order, and a stable plan hash. A finding carries a consequence, editorial question, rule version, and author disposition as well as its location and observation.

F.6 Command-line contract

The six commands exchange versioned JSON records. Each invocation writes one machine-readable result to standard output and human progress or diagnostics to standard error. The stable output fields are `contract_version`, `command`, `status`, `exit_code`, `run_id`, `record_paths`, `blocking_reasons`, `source_hash`, and `result_hash`. Messages, display paths, timings, and suggestions are advisory and may change without a contract-version bump.

Exit	Meaning	Required behavior
0	Pass or released	The command completed its declared job.
1	Deterministic gate failure	Preserve the source and emit the blocking finding or build result.
2	Abstention or unresolved policy result	Do not treat absence as pass; identify the affected gate.
3	Invalid brief, input, schema, or usage	Do not invoke the writer.
4	Internal implementation error	Preserve the parent run and provide an incident identifier.
5	Security or trust-boundary violation	Stop the run, quarantine its output, and notify the security/policy owner.

Table F.5: Stable command outcomes. Exit codes carry coarse control flow; the JSON result carries the reasons.

F.7 Repair, exceptions, and release authority

`hv repair` never edits the canonical source in place. It requires a clean version-control tree or an immutable source snapshot, writes a child revision under `.humanvoice/runs/<run\_id>/revisions/<revision\_id>`, validates that child in a temporary directory, and publishes its manifest only after the comparison passes. The parent and its decisions are never deleted. Rollback selects the parent revision. A cycle limit, repeated finding signature, or alternating parent hashes stops the loop and returns an author choice; it cannot produce a reader packet.

Ordinary release requires every required gate to pass and the document owner to accept the protected comparison. An exception may be granted by the sponsor or named project owner. A meaning exception also requires the domain editor's concurrence; a processing exception requires the security/policy owner's concurrence. The security/policy owner can veto release.

No exception may waive unresolved protected-object correspondence, an unparsed object promised by the brief, unauthorized external transmission, missing critical evidence for a load-bearing claim, or a broken or unreproducible source build. Every permitted exception records its scope, reason, residual risk, owner, concurrences, expiry, reader disclosure, and count in the feasibility record.

F.8 Corpus rights and definition of done

Each corpus item carries a source path or URL, source hash, owner, license or written permission, permitted use, redistribution status, retention class, and consent date. The allowed statuses are `cleared-internal`, `cleared-public`, `restricted-not-for-export`, `pending-clearance`, and `excluded`. Pending or restricted material may support an internal fixture only when its owner permits that use. It cannot enter a published benchmark, external reader packet, or commercial training set without written clearance.

The work packages have explicit exits:

1. **Brief, threat model, and schemas:** a complete brief, the contract files, a rights manifest, and threat fixtures pass validation.
2. **Protected LaTeX core:** source maps, canonical build, protected comparison, and CLI result replay are locked on the fixture set.
3. **Week-six checkpoint:** a second operator reproduces the core, or the scope is narrowed to the protected review utility.
4. **Bounded authoring extension:** plan, unit draft, preflight, and repair pass mutation and held-out calibration gates.
5. **Reader feasibility case:** one released packet, timed reader record, burden account, rights record, and all abstentions are preserved.
6. **Decision handoff:** a reproducible run, cost sensitivity, rights review, and proceed/revise/stop memorandum are delivered.

These exits are implementation facts. None is a claim that the prose is clear or that the product improves expert writing. Those claims remain the province of the named reader and the later comparative study.

Bibliography

Shadi I. Abudalfa and Jessie S. Barrot. Generative artificial intelligence for automated writing evaluation: A systematic review of trends, efficacy, and challenges. *Assessing Writing*, 68:101041, 2026. doi: 10.1016/j.asw.2026.101041. URL https://www.sciencedirect.com/science/article/pii/S1075293526000292.

Guangsheng Bao et al. Fast-DetectGPT: efficient zero-shot detection of machine-generated text via conditional probability curvature. In *International Conference on Learning Representations*, 2024. arXiv:2310.05130.

S. Behzad, O. Kashefi, and S. Somasundaran. Assessing online writing feedback resources: Generative AI vs. good samaritans. In *Proceedings of the 2024 Language Resources and Evaluation Conference*, 2024. URL https://aclanthology.org/2024.lrec-main.144/.

Carl Bereiter and Marlene Scardamalia. *The Psychology of Written Composition*. Lawrence Erlbaum Associates, Hillsdale, NJ, 1987. ISBN 0898596475.

Douglas Biber. *Variation across Speech and Writing*. Cambridge University Press, 1988.

Erik Brynjolfsson, Danielle Li, and Lindsey Raymond. Generative AI at work. *The Quarterly Journal of Economics*, 140(2):889–942, 2025. doi: 10.1093/qje/qjae044. URL https://academic.oup.com/qje/article/140/2/889/7990658.

Tuhin Chakrabarty, Philippe Laban, and Chien-Sheng Wu. Can AI writing be salvaged? Mitigating idiosyncrasies and improving human–AI alignment in the writing process through edits, 2024a. arXiv:2409.14509.

Tuhin Chakrabarty, Vishakh Padmakumar, et al. Art or artifice? Large language models and the false promise of creativity. In *CHI Conference on Human Factors in Computing Systems*, 2024b. arXiv:2309.14556.

Myra Cheng, Sunny Yu, and Dan Jurafsky. HumT DumT: measuring and controlling human-like language in LLMs, 2025. arXiv:2502.13259.

ChkTeX maintainers. Chktex: A semantic checker for latex. CTAN package documentation, 2026. URL https://ctan.org/pkg/chktex. Accessed 2026-08-25.

John Collins. latexmk: Fully automated latex document generation. CTAN package documentation, 2026. URL https://ctan.org/pkg/latexmk. Accessed 2026-08-25.

Scott A. Crossley, Aron Heintz, Joon Suh Choi, et al. A large-scaled corpus for assessing text readability. *Behavior Research Methods*, 55, 2022. doi:10.3758/s13428-022-01802-x.

Fabrizio Dell'Acqua, III McFowland, Edward, Ethan Mollick, Hila Lifshitz, Katherine C. Kellogg, Saran Rajendran, Lisa Krayer, François Candelon, and Karim R. Lakhani. Navigating the jagged technological frontier: Field experimental evidence of the effects of artificial intelligence on knowledge worker productivity and quality. *Organization*

Science, 2026. doi: 10.1287/orsc.2025.21838. URL https://doi.org/10.1287/orsc.2025.21838.

E. W. Dillon, S. Jaffe, N. Immorlica, et al. Shifting work patterns with generative AI. Technical Report 33795, National Bureau of Economic Research, 2025. URL https://www.nber.org/papers/w33795.

Anil R. Doshi and Oliver P. Hauser. Generative AI enhances individual creativity but reduces the collective diversity of novel content. *Science Advances*, 2024. doi:10.1126/sciadv.adn5290.

Yann Dubois, Balázs Galambosi, Percy Liang, and Tatsunori B. Hashimoto. Length-controlled AlpacaEval: a simple way to debias automatic evaluators, 2024. arXiv:2404.04475.

Youqing Fang, Yinhao Tang, Yanan Sun, Jiangning Liu, Ziyi Wang, Xun Zhao, Bin Liu, Weiming Zhang, Kuikun Liu, Wenwei Zhang, and Kai Chen. MindCopilot: Towards formalizing and evaluating granular human–LLM co-writing. arXiv:2605.23535, 2026. URL https://arxiv.org/abs/2605.23535.

Linda Flower and John R. Hayes. A cognitive process theory of writing. *College Composition and Communication*, 32(4):365–387, 1981. doi: 10.58680/ccc198115885. URL https://doi.org/10.58680/ccc198115885.

Sebastian Gehrmann, Hendrik Strobelt, and Alexander M. Rush. GLTR: statistical detection and visualization of generated text. In *ACL System Demonstrations*, 2019. arXiv:1906.04043.

Alex Glynn. Academ-AI: documenting the undisclosed use of generative AI in academic publishing, 2024. arXiv:2411.15218.

George D. Gopen and Judith A. Swan. The science of scientific writing. *American Scientist*, 78(6):550–558, 1990. URL https://scholars.duke.edu/publication/881487.

Arthur C. Graesser, Danielle S. McNamara, Max M. Louwerse, and Zhiqiang Cai. Coh-Metrix: analysis of text on cohesion and language. *Behavior Research Methods, Instruments, & Computers*, 36, 2004. doi:10.3758/BF03195564.

Grammarly. Grammarly for enterprise. Product documentation, 2026a. URL https://www.grammarly.com/business/enterprise. Accessed 2026-08-24.

Grammarly. Privacy and security faqs. Support documentation, 2026b. URL https://support.grammarly.com/hc/en-us/articles/20916119474829-Privacy-and-security-FAQs. Accessed 2026-08-24.

GROBID maintainers. GROBID documentation. Software documentation, 2026. URL https://grobid.readthedocs.io/en/latest/Introduction/.

Yanzhu Guo, Guokan Shang, Michalis Vazirgiannis, and Chloé Clavel. The curious decline of linguistic diversity: training language models on synthetic text, 2023. arXiv:2311.09807.

Abhimanyu Hans, Avi Schwarzschild, Valeriia Cherepanova, et al. Spotting LLMs with Binoculars: zero-shot detection of machine-generated text. In *International Conference on Machine Learning*, 2024. arXiv:2401.12070.

John Hattie and Helen Timperley. The power of feedback. *Review of Educational Research*, 77(1):81–112, 2007. doi: 10.3102/003465430298487. URL https://doi.org/10.3102/003465430298487.

Hiroki Hamaguchi. latexlint. GitHub repository, 2026. URL https://github.com/HirokiHamaguchi/latexlint.

Ken Hyland. *Metadiscourse: Exploring Interaction in Writing*. Continuum, London, 2005. ISBN 0826476112.

International Committee of Medical Journal Editors. Use of artificial intelligence in scholarly manuscripts. Recommendations for the Conduct, Reporting, Editing, and Publication of Scholarly Work in Medical Journals, 2025. URL https://www.icmje.org/recommendations/browse/artificial-intelligence/ai-use-by-authors.html.

Di Jin, Zhijing Jin, Zhiting Hu, Olga Vechtomova, and Rada Mihalcea. Deep learning for text style transfer: a survey. *Computational Linguistics*, 48(1), 2022. doi:10.1162/coli_a_00426.

jsPsych maintainers. jspsych documentation. Official project documentation, 2026. URL https://www.jspsych.org/v7/. Accessed 2026-08-25.

Tom S. Juzek and Zina B. Ward. Why does ChatGPT "delve" so much? Exploring the sources of lexical overrepresentation in large language models. In *Proceedings of COLING*, 2025. arXiv:2412.11385.

John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. A watermark for large language models. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 17061–17084, 2023. URL https://proceedings.mlr.press/v202/kirchenbauer23a.html.

Robert Kirk, Ishita Mediratta, Christoforos Nalmpantis, Jelena Luketina, Eric Hambro, Edward Grefenstette, and Roberta Raileanu. Understanding the effects of RLHF on LLM generalisation and diversity. In *International Conference on Learning Representations*, 2024. arXiv:2310.06452.

Dmitry Kobak, Rita González-Márquez, Emőke-Ágnes Horvát, and Jan Lause. Delving into LLM-assisted writing in biomedical publications through excess vocabulary. *Science Advances*, 2025. arXiv:2406.07016.

Anton Korinek. Generative AI for economic research: use cases and implications for economists. *Journal of Economic Literature*, 61(4), 2023. doi:10.1257/jel.20231736.

Kalpesh Krishna, Yixiao Song, Marzena Karpinska, John Wieting, and Mohit Iyyer. Paraphrasing evades detectors of AI-generated text, but retrieval is an effective defense. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2303.13408.

LanguageTool maintainers. Languagetool developer documentation. Software documentation, 2026. URL https://languagetool.org/dev.

latex-lsp maintainers. tree-sitter-latex. Software repository, 2026. URL https://github.com/latex-lsp/tree-sitter-latex.

latexdiff maintainers. latexdiff: Determine and mark up significant differences between latex files. CTAN software documentation, 2026. URL https://www.ctan.org/pkg/latexdiff.

Giuseppe Russo Latona, Manoel Horta Ribeiro, Tim R. Davidson, Veniamin Veselovsky, and Robert West. The AI review lottery: widespread AI-assisted peer reviews boost paper scores and acceptance rates, 2024. arXiv:2405.02150.

Mina Lee, Percy Liang, and Qian Yang. CoAuthor: designing a human–AI collaborative writing dataset for exploring language model capabilities. In *CHI Conference on Human Factors in Computing Systems*, 2022. arXiv:2201.06796.

H. Li, M. Liang, R. Peng, and M. Yin. How does the disclosure of AI assistance affect the perceptions of writing? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024a. doi: 10.18653/v1/2024.emnlp-main.279. URL https://aclanthology.org/2024.emnlp-main.279/.

H. Li, J. Liu, S. Matsumura, Y. Wang, D. Litman, and R. Correnti. Using large language models to assess young students' writing revisions. In *Proceedings of the 19th Workshop on Innovative Use of NLP for Building Educational Applications*, 2024b. URL https://aclanthology.org/2024.bea-1.30/.

Weixin Liang, Mert Yuksekgonul, Yining Mao, Eric Wu, and James Zou. GPT detectors are biased against non-native English writers. *Patterns*, 4(7), 2023. doi:10.1016/j.patter.2023.100779.

Weixin Liang, Zachary Izzo, Yaohui Zhang, Haley Lepp, et al. Monitoring AI-modified content at scale: a case study on the impact of ChatGPT on AI conference peer reviews. In *International Conference on Machine Learning*, 2024a. arXiv:2403.07183.

Weixin Liang et al. Mapping the increasing use of LLMs in scientific papers, 2024b. arXiv:2404.01268.

Lin and Zhu. Divergent LLM adoption and heterogeneous convergence paths in research writing, 2025. arXiv:2504.13629.

llama.cpp maintainers. llama.cpp. Software repository, 2026. URL https://github.com/ggml-org/llama.cpp.

LTeX+ maintainers. LTeX+ language server. Software repository and documentation, 2026. URL https://github.com/ltex-plus/ltex-ls-plus.

Alejandro Martínez and Stefano Mammola. Specialized terminology reduces the number of citations of scientific papers. *Proceedings of the Royal Society B*, 288, 2021. doi:10.1098/rspb.2020.2581.

Qian Meng, Ya-Nan Xie, and Dan Lu. A systematic review of AI-assisted academic writing: Tools, measures, and effects. *Journal of English for Academic Purposes*, 82: 101726, 2026. doi: 10.1016/j.jeap.2026.101726. URL https://www.sciencedirect.com/science/article/pii/S1475158526000986.

Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D. Manning, and Chelsea Finn. DetectGPT: zero-shot machine-generated text detection using probability curvature. In *International Conference on Machine Learning*, 2023. arXiv:2301.11305.

Joseph Mugaanyi, Liuying Cai, Sumei Cheng, Caide Lu, and Jing Huang. Evaluation of large language model performance and reliability for citations and references in scholarly writing: Cross-disciplinary study. *Journal of Medical Internet Research*, 26: e52935, 2024. doi: 10.2196/52935. URL https://www.jmir.org/2024/1/e52935.

Sheshera Mysore, Debarati Das, Hancheng Cao, and Bahareh Sarrafzadeh. Prototypical human–AI collaboration behaviors from LLM-assisted writing in the wild. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 16819–16846. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-main.852. URL https://aclanthology.org/2025.emnlp-main.852/.

Inderjeet Jayakumar Nair, Jiaye Tan, Xiaotian Su, Anne Gere, Xu Wang, and Lu Wang. Closing the loop: Learning to generate writing feedback via language model simulated student revisions. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 16636–16657, 2024. doi: 10.18653/v1/2024.emnlp-main.928. URL https://aclanthology.org/2024.emnlp-main.928/.

National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). NIST AI 100-1, 2023. URL https://www.nist.gov/itl/ai-risk-management-framework.

National Institute of Standards and Technology and LaTeXML maintainers. Latexml manual. Software documentation, 2026. URL https://dlmf.nist.gov/LaTeXML/.

David J. Nicol and Debra Macfarlane-Dick. Formative assessment and self-regulated learning: A model and seven principles of good feedback practice. *Studies in Higher Education*, 31(2):199–218, 2006. doi: 10.1080/03075070600572090. URL https://doi.org/10.1080/03075070600572090.

Shakked Noy and Whitney Zhang. Experimental evidence on the productivity effects of generative artificial intelligence. *Science*, 381(6654):187–192, 2023. doi: 10.1126/science.adh2586. URL https://doi.org/10.1126/science.adh2586.

Daniel M. Oppenheimer. Consequences of erudite vernacular utilized irrespective of necessity: Problems with using long words needlessly. *Applied Cognitive Psychology*, 20(2):139–156, 2006. doi: 10.1002/acp.1178. URL https://doi.org/10.1002/acp.1178.

Overleaf. Track changes in overleaf. Documentation, 2026. URL https://www.overleaf.com/learn/how-to/Track_Changes_in_Overleaf. Accessed 2026-08-24.

Vishakh Padmakumar and He He. Does writing with language models reduce content diversity? In *International Conference on Learning Representations*, 2024. arXiv:2309.05196.

Pandoc maintainers. Pandoc filters and document abstract syntax tree. Software documentation, 2026. URL https://pandoc.org/filters.html.

Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems*, 2024. arXiv:2404.13076.

Paperpal. Ai academic writing tool and research assistant. Product documentation, 2026. URL https://paperpal.com/. Accessed 2026-08-24.

plasTeX maintainers. plastex: A python framework for processing latex documents. Software documentation, 2026. URL https://plastex.github.io/plastex/.

Pontus Plavén-Sigray, Granville J. Matheson, Björn C. Schiffler, and William H. Thompson. The readability of scientific texts is decreasing over time. *eLife*, 6, 2017. doi:10.7554/eLife.27725.

pylatexenc maintainers. pylatexenc documentation. Software documentation, 2026. URL https://pylatexenc.readthedocs.io/en/latest/.

H. Rashkin, E. Clark, F. Huot, and M. Lapata. Help me write a story: Evaluating LLMs' ability to generate writing feedback. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025. doi: 10.18653/v1/2025.acl-long.1254. URL https://aclanthology.org/2025.acl-long.1254/.

M. L. Rethlefsen et al. PRISMA-S: An extension to the PRISMA statement for reporting literature searches in systematic reviews. *Systematic Reviews*, 10, 2021. doi: 10.1186/s13643-020-01542-z. URL https://www.prisma-statement.org/prisma-search.

Russell et al. StoryScope: discourse-level detection of machine-generated narrative, 2026. arXiv:2604.03136, as summarized in the humanizer-stack repository; not independently inspected.

Vinu Sankar Sadasivan, Aounon Kumar, Sriram Balasubramanian, Wenxiao Wang, and Soheil Feizi. Can AI-generated text be reliably detected? *Transactions on Machine Learning Research*, 2024. URL https://arxiv.org/abs/2303.11156. arXiv:2303.11156.

Chantal Shaib, Tuhin Chakrabarty, Diego Garcia-Olano, and Byron C. Wallace. Measuring AI "slop" in text, 2025. arXiv:2509.19163.

Mrinank Sharma, Meg Tong, Tomasz Korbak, et al. Towards understanding sycophancy in language models. In *International Conference on Learning Representations*, 2024. arXiv:2310.13548.

Prasann Singhal, Tanya Goyal, Jiacheng Xu, and Greg Durrett. A long way to go: investigating length correlations in RLHF, 2023. arXiv:2310.03716.

John M. Swales. *Genre Analysis: English in Academic and Research Settings*. Cambridge University Press, Cambridge, 1990. ISBN 0521338131.

TexSoup maintainers. Texsoup documentation. Official project documentation, 2026. URL https://texsoup.alvinwan.com/. Accessed 2026-08-25.

textlint maintainers. textlint: Pluggable natural-language linting. Software repository and documentation, 2026. URL https://github.com/textlint/textlint.

textstat maintainers. textstat. Software repository, 2026. URL https://github.com/textstat/textstat.

Trinka AI. Trinka ai help center and privacy features. Product and support documentation, 2026. URL https://www.trinka.ai/faqs. Accessed 2026-08-24.

UK AI Security Institute. Inspect AI. Evaluation framework documentation, 2026. URL https://inspect.aisi.org.uk/.

unified-latex maintainers. unified-latex documentation. Official project documentation, 2026. URL https://siefkenj.github.io/unified-latex/index.html. Accessed 2026-08-25.

Vale maintainers. Vale: A prose linter. Software documentation, 2026. URL https://vale.sh/features/markup.

Debora Weber-Wulff, Alla Anohina-Naumeca, Sonja Bjelobaba, et al. Testing of detection tools for AI-generated text. *International Journal for Educational Integrity*, 19, 2023. doi:10.1007/s40979-023-00146-z.

WikiProject AI Cleanup. Signs of AI writing. https://en.wikipedia.org/wiki/Wikipedia:Signs_of_AI_writing, 2026. Living document; accessed 2026-08-21.

Writefull and Overleaf. Writefull for overleaf. Integration documentation, 2026. URL https://docs.overleaf.com/integrations-and-add-ons/ai-features/writefull. Accessed 2026-08-24.

Ann Yuan, Andy Coenen, Emily Reif, and Daphne Ippolito. Wordcraft: story writing with large language models. In *International Conference on Intelligent User Interfaces*, 2022. doi:10.1145/3490099.3511105.

Zhang et al. Verbalized sampling: how to mitigate mode collapse and unlock LLM diversity, 2025. arXiv:2510.01171.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks*, 2023. arXiv:2306.05685.